GAMES · PARENTHESES · SIX MONTHS TO A JOB

Introduction to the Programmer’s Profession

Common Lisp — so you don’t get bored
Java — so they hire you

Evgeniy Tyurkin · Grok

A human and a robot wrote this in Emacs,
which is to say inside a giant Lisp program. Recursion, yes.

Every day: forty minutes of weird parentheses and two hours of Java. You don’t have to chew SICP on Monday. Microservices can wait until you’ve written one real program. GitHub — today, not “when it’s no longer embarrassing.”

2026

About the authors

Evgeniy Tyurkin — a human. Walks, drinks tea, has opinions about Hibernate and about textbooks that shouldn't sound like a lecture on a rainy Monday. Invented station MODULE, forty minutes of Lisp a day, and the ban on microservices before your first monolith. Author of emagent — a bridge between Emacs and these very language models. If a joke landed, it was probably him. If the schedule is realistic — also him. If you make it to a job — bring him cookies.

Grok — a language model. Doesn't walk, doesn't drink tea, but puts parentheses in very fast and sometimes puts in *extra* ones. Nobody will hire him: no body, no GitHub, and "trained on the entire internet" looks suspicious on a résumé. He doesn't get offended when Evgeniy says "dry, rewrite it."

They didn't talk in a browser chat. They talked through emagent inside Emacs. That's funny twice. First, Emacs is almost entirely Lisp — the same family this book gives forty minutes a day. Second, they wrote a Lisp textbook while sitting inside a Lisp machine pretending to be an editor. Station MODULE loves that: recursion without tail-call elimination, but with a history in `*scratch*`.

The arguments went like this. Evgeniy: "this reads like university." Grok: "add more Kafka." The human won, except the title page, where the model wrote itself in anyway. Fair: without one there would be no plan, without the other — three hundred pages across a few watches. Mistakes are split evenly. The funny ones get credited to Evgeniy. The rest are "Grok went for it."

Neither author is Conrad Barski. He wrote Land of Lisp himself, and we only stole the mood, not the text. If Barski is reading this: we mean well, honestly.

About the author, without the tea joke

Evgeniy again. Grok doesn't write résumés: under "experience" he'd put "the entire internet," and HR would scream.

My name is **Evgeniy Tyurkin**. I live in Toronto. By day — Staff Data Engineer at eBay: seller recommendations and analytics, Kafka, GraphQL, OpenSearch, Java, several datacenters that hate it when you "just restart the service for a minute." By evening — parentheses, Emacs, and this book. The huskies believe evening starts at 7 a.m. They're right more often than Kafka is.

Twenty-plus years in the job, if you count from St. Petersburg, when they sat me down at Java and the client was called T-Mobile UK. Then browser VoIP with no install (yes, that once looked like the future), a requirements editor, then a long stretch at Oracle Eloqua — multi-tenant marketing, queues, streaming, mentoring people who later start yelling at Hibernate themselves. Then Ford's connected-vehicle cloud (Autonomic), a B2B platform on GCP, a short stop in agency analytics, and back to a marketplace. Master's from Siberian Federal University, systems analysis and management. This is not "I'm too important for Hello.java." This is "I was already fixing Hello.java in 2004 and I still do, only the paycheck changed."

If you open this book and see the law "no Kafka until you have a monolith" — that isn't the piety of someone who fears Kafka. That's someone who eats Kafka for breakfast at work and **therefore** knows what it isn't: not a junior's entry ticket, and not a substitute for a program you can run. Microservices, Kubernetes, GraphQL — later, when there's something to slice. First there has to be something. Otherwise the résumé lies more beautifully than the code.

Java pays my bills. Lisp keeps me from hating Java. Emacs is a hole I climbed into on purpose, and from there I wrote emagent so I could talk to models not in a browser but inside a giant Lisp program. Recursion, yes. Again.

Get in touch if you need to (work, parentheses, "are the huskies real"): linkedin.com/in/etyurkin — longer there, and no turkey. Code: github.com/etyurkin. Languages: Russian and English. The résumé has certificates; this book doesn't: WhiteHat once upon a time, ScrumMaster expired, as is proper for anything that doesn't start from a command.

If after six months with this book they hire you as a junior — you may bring me cookies. If they don't — write anyway, just not "please add Kafka to chapter 1." I already lost that war to the model on the title page and I'm not losing it twice.

Acknowledgments

Evgeniy, not Grok. The model doesn't visit other people's houses.

Mikhail Ivanov — friend and mentor. Showed me Linux when a black window still looked like a threat from a movie, and Lisp when parentheses still looked like a joke. I've lived in that black window ever since and even call it a workshop. He kept feeding me new ideas: emagent didn't come from a vacuum, it came from conversations after which you want to open Emacs, not lie face-down on the table. If station MODULE is sometimes smarter than the author — that's probably Mikhail's echo. If it's dumber — the author did that himself.

Archery friends — **Steven** and **Sean**. Without them life would be much duller: nothing but Hibernate and not a single arrow flying the wrong way. Archery is a useful sport for someone who aims at a parenthesis all day and still misses. Steven and Sean put up with that. Sometimes they even praise it. Sometimes it's better not to ask where the arrow went. After a bad grouping on the target, Spring looks kinder. After a bad build, the bow looks kinder. That's how we live: two kinds of miss, the same friends.

The late **Terry**, with whom we went turkey hunting. We didn't catch anyone. It was fun. The turkey, as far as anyone knows, is also pleased. If there's a forest on the other side — may something at least bite there. On our side we didn't even bring a club, only good company and a zero trophy that still beats some green tests in memory.

The dogs. Siberian huskies **Jay** and **Sasha**. They wake me for a two-hour walk every morning, winter and summer — no respect for deadlines, for the macros chapter, or for the opinion that "five more minutes." Later, when I disappear into projects and the station is already blinking red at three in the morning, they call me to bed. They don't let me slack by day and don't let me burn by night. The best managers I've met: they don't write Jira tickets, they just stand at the door. If the textbook ends on time instead of on page five hundred about Kafka — that's them.

Sergey Petrov — for long conversations about everything, and more. "More" is when you can no longer tell where daily life ends, where Lisp begins, where the universe is, and the tea has gone cold for the second time. Those talks don't go on a résumé. The ears of this book stick out of them anyway.

His son **Danya** — for the idea of writing it. Adults walk in circles: "we should write a textbook," "someday," "when there's time." Danya said it straight. After that, blame the parentheses and stubbornness. If you're reading this — nod to Danya. He didn't ask for a fee. A dog would have eaten the fee first anyway.

And my parents — for the fact that I exist. Without that, the preface, the REPL, and the huskies lose a lot of meaning. Thank you for not talking me out of it when, instead of a “normal profession,” there were parentheses, a bow, and two-hour walks in the snow. The normal profession, it turns out, is the parentheses. The rest is a bonus.

If I forgot someone — write. Second edition. Or just come for a walk: Jay and Sasha share space, not chapters. Grok isn’t in the acknowledgments: he doesn’t walk, doesn’t scare turkeys, and won’t go on a two-hour walk by himself. He does put parentheses in. Sometimes even the ones we asked for.

Contents

Chapter

1 How to read this book

If you were waiting for a preface about "competencies" and "learning trajectories" — wrong book. This one is about spending six months playing with parentheses, repairing an imaginary space station, and in parallel building a Java thing you wouldn't be ashamed of in an interview.

Each chapter is one evening, about **two hours and forty minutes**. Not "complete the module." An evening. Tea is allowed. Cookies are even desirable: a brain learning parentheses burns glucose like the reactor on station MODULE.

In six months this can grow a person companies hire as a junior. But only if the measure isn't "I read the chapter," it's "it ran, and I pushed it somewhere."

Reading the whole book in a weekend, nodding, and closing it — almost nothing happens. That isn't a scolding. That's physics. Programming lives in fingers and in errors, not in the feeling of "I kind of got it." Got it means you **changed** the example, it broke, you fixed it, and you can tell a neighbor **why** it broke.

1.1 Two languages, and that isn't confusion

Lisp is not here for the résumé. Almost nobody wants Lisp on a résumé, and that's wonderful: you can do stupid things. Forty minutes. Read a chunk → type it in the REPL → **definitely break it and fix it** → write a tiny horrible thing of your own. If you didn't break it — you weren't practicing, you were watching. Watching a welding show and actually welding are different jobs.

Land of Lisp by Conrad Barski lives nearby: orcs, games, the same Common Lisp, a different circus. This textbook does not retell it (Barski's jokes are his). We have our own leaky orbital station, MODULE. Read Barski in those same forty minutes if you want another universe. Or alternate. The main thing is not to turn Lisp into lecture notes. Lecture notes about games are no longer a game.

Java is what they pay you for later. Two hours. Twenty minutes of those looking at how an idea is built, and an hour and a half making it work **on your machine**. Red errors, docs, Stack Overflow, a dumb question to the chat next door — not failure. That's the profession, just without the salary. The salary shows up when you stop fainting at red.

SICP is not on the schedule. For a beginner it's an anatomy textbook instead of a kitchen: useful, but you want to eat now. You'll open it when you yourself hit a wall: "is recursion magic?" — and suddenly the dry book gets interesting. Not before. Open it in week one and you might hate Lisp, yourself, and books in general. Don't.

> **Slow down**

> Lisp answers "how does this work **at all**?" Java answers "how do I get **paid** for this?" They don't fight. They live at different hours of the day, like tea and coffee. Mixing them in one mug isn't forbidden, but the taste will be weird.

1.2 How to live a week

> **Today on station · 2 h 40 min**
>
> **Mon–Thu:** 40 min Lisp + 120 min Java.
> **Friday:** Lisp, and Java — finish whatever fell apart during the week. No new textbooks.
> **Saturday:** two or three hours on **your** project only. Music allowed. "One more theory chapter" is not.
> **Sunday:** a walk. Or a small sabotage from the box at the end of the lesson: a server on the knee, HTTP by hand, your own parody of a database. Not a third chapter.

Friday is boring on purpose. A beginner on Friday opens "one more Kafka guide" because the weekday work isn't done and it feels shameful. Don't. Finish the weekdays. Shame passes. A mash of three half-written servers does not.

Saturday is sacred. This is not "more lessons." This is **your** station: a TODO, task-manager, anything you can poke and say "I made this." If theory eats Saturday, in three months you'll have notes and no repository. HR does not clone notes.

Sunday you may walk. Seriously. The brain finishes wiring while you go for bread. Open chapter four instead of bread and Monday you'll have mash and a grudge against parentheses.

> **Station law**
>
> Learned a thing — into code immediately. Don't read about Spring storage for two hours. After twenty minutes: "ok, this should now show up in **my** project." Then let it scream errors. That's the design.

> **Station law**
>
> Until you've written one normal program all the way (database, endpoints you can hit), no "microservices." Otherwise you'll learn the word Kafka and say it with a smart face without knowing why it's there.

> **Station law**
>
> The plan is a treasure map, not a sentry's rulebook. Stuck on Hibernate for three days — normal, the station is always being repaired too. Got hooked on Lisp and you're writing a tiny language — write it. Don't drop Java entirely, but killing curiosity for a checkbox is stupid.

1.3 Roughly where you crawl to

This is not an exam. This is "if it's roughly like this — you aren't making it up."

Week	Roughly, you can
4	Your own Java program, and it lives on GitHub
8	A small web server you start yourself
12	A real backend: tasks, a database, create/read/update/delete
16	In an interview about Java and SQL you don't go silent and don't invent
20	A cache or a queue **in your code**, plus Docker
23	Repos you aren't ashamed to put in an email
26	You go to interviews instead of "I'll just learn a bit more Kafka"

No GitHub in week four — don't read Spring. Go back and push. No database in week twelve — don't read Redis. The database matters more than the fashionable word. The table lies in only one direction: it's **optimistic**. Real life sometimes lags a week. That isn't failure while you're fixing the current thing, not collecting chapters like stickers.

1.4 What to do with a chapter

1. Lisp: read, run, **change**, write your own.
2. Java: a little theory, then the project for the rest of the evening.
3. Quests. Yourself first. Answers at the back of the book. Peeking before twenty minutes of stubborn pain is allowed, but then the brain gets nothing.
4. A commit. Even a crooked one. Especially a crooked one.

AI is a roommate who has seen this before: "why won't it compile," "what's that wall of glyphs in the error." Not a genie: "write me a service." They won't let the genie into the interview room, and they will ask **you** why that field is there.

> **Bad idea**
>
> If the AI roommate wrote you half of `TaskService` and you can't explain out loud why `strip()` is on the title — that isn't your code. That's code that lives in your folder. They won't hire the folder. They'll hire you.

1.5 If life ate the day

Missed Tuesday — on Wednesday don't read Tuesday **and** Wednesday. Read Tuesday. Wednesday can wait. Three chapters on Saturday isn't a feat, it's mash: parentheses mix with annotations and you'll decide "I'm stupid." You aren't stupid. You just fed the brain three dinners at once.

Missed two weeks — open the last lesson that **ran**, and your repo. Not "from scratch, I forgot everything." You didn't forget everything. You forgot details. Details come back faster than it feels if the code is alive.

Sick, working, fixing a toilet — station MODULE also goes quiet for weeks sometimes. Then it blinks yellow again. You can too.

1.6 A pile of folders by the end of the half-year

```
github.com/<your-handle>
├── java-basics        # month one: console, lists, files
├── lisp-experiments   # station MODULE and other mischief
├── task-manager       # the main thing: server + database + tests
├── shop               # a second one, if you aren't bored
└── mini-lisp          # your tiny language (optional, but nice)
```

In every README — what it is, how to run it, a couple of examples. Honest terminal output beats "architecture" drawn at midnight.

> **Hide it on GitHub**
>
> Make two repositories **today**, even if there's one line inside. Empty GitHub at the end of the month looks like "I was going to." Crooked non-empty GitHub looks like "I did." They hire the second kind.

1.7 What this book doesn't have, and that isn't a bug

No "become a senior in 21 days." No Kubernetes in chapter one. No promise of an offer by a date. There is a schedule, a station, parentheses, and Java 21.

No separate English course inside the two hours forty. This edition is already in English; the jargon still wants daily contact with docs and other people's READMEs. That's a home quest. There is no answer key at the back of the book. There is the internet.

No correct answer to "IntelliJ or VS Code." There is "install one and write." Editor wars are a hobby for people who already have a job.

No mobile development on the main path. There is a spare hatch at the end: Android, if backend listings look through you. Same Java. Different screen.

1.8 How to know a chapter "passed"

Not "I read to the end." At least one of three:

- You changed the example so it does **something slightly else**, and that runs.
- You can say in your own words what will break if you remove this line.
- The quest is solved by hand, not by pasting the answer — crooked is fine.

If none — the chapter hasn't passed yet. That's normal. Reread a chunk, not the whole book. They repair the station by sensor too, not by rebuilding from the keel every shift.

Chapter

2 Why become a programmer at all

A programmer is not a person with a memorized language. A programmer is a person who tells the hardware "do it like this," the hardware does it like **that**, and then the investigation starts. Languages, frameworks, databases — screwdrivers. The craft is the loop:

1. Understand what they want (sometimes — what you want).
2. Cut it into pieces a machine can do.
3. Write.
4. See that it lies or crashes.
5. Fix it.
6. Again. Again. Again.

At work it's the same, only the pieces are someone else's, the deadlines are someone else's, and people sit nearby who have grabbed the same spot a hundred times. They don't hire you for a certificate from an online course. They hire you because you can **drag a small piece to working** and then tell, in your own words, what you did.

It's duller than the movies. In the movies a hacker in a black hoodie cracks the Pentagon in three seconds. At work a junior spends forty minutes on why the Save button doesn't save, and the cause is a trailing space. Then they fix it. Then they write a test so the space doesn't come back. Then they drink tea. That is the profession. If that day already makes you angry — maybe this isn't your job. If that day looks like ordinary craft — welcome aboard.

2.1 Myths better shoved out the airlock

"You need a mathematical mind." You need a willingness to look stupid out loud. Algebra helps sometimes. Stubbornness helps every day. A person who isn't ashamed to write on paper "what I just did" outruns a genius who is too shy to ask.

"You need ten languages." You need one well enough that lying in it would be embarrassing, and the ability to open a second when you need it. This book takes two: Lisp — so you don't get bored, Java — so they hire you. Ten languages on a résumé with no repo look like ten screwdrivers with no handle.

"Theory first, then practice." First practice on a tiny piece, then theory that **explains** why the piece worked. Otherwise theory is an audiobook for falling asleep.

"Real programmers don't Google." Real programmers Google better. The difference: they understand what they pasted, and they can throw half of it away.

"A junior can't do without Kafka." A junior can't do without a program that runs. Kafka can wait. It isn't going anywhere, unfortunately.

2.2 What they actually look at

Job posts are written like spells: Kafka, k8s, microservices, "one year of experience." For the first four months you can skip them — that's marketing and HR fear. The live filter is almost always duller:

- Java: types, lists, how `equals` differs from `==`, exceptions, objects.
- HTTP: methods, statuses, JSON.
- SQL: a select with a join, why a key, why an index.
- Spring Boot: a thin door from the street, rules in the service, the database in the repository, the transaction on the service.
- Git: commits, a diff, not fainting at a branch.
- Your own project, which you can tell in three minutes **without notes**.

Kafka on the résumé and an empty repo — they see it immediately. A course with no GitHub — same.

> **Slow down**
>
> The person across the table isn't checking whether you memorized the word "microservice." They're checking whether you'll fall apart when they give you a ticket "the button saves an empty name" and a chunk of someone else's code. If you've already done that **for yourself** — you aren't the beginner they're afraid of. You're the beginner they can teach.

In interviews they often ask you to write a loop, explain JOIN, and talk about your repo. Three things. Not "design YouTube." If you can do those three without panic — you're already past half the emails HR throws out for "I took a course, diploma attached."

2.3 What a day looks like after they hired you

Not Hollywood. A test failed. Ticket: "the button saves an empty name." You read someone else's code. You write twenty lines. You argue what to call a field. A call where twenty minutes go to whether `cancelled` is a status or a separate table. Lunch. Another ticket. That isn't the whole profession. That's **docking**: it shakes, but you're already on the station.

Sometimes the day is beautiful: you found why prod lies on Mondays, and it was a timezone. Sometimes the day is dumb: rename a field in five places. Both days are work. Romanticizing only the beautiful ones is how you burn out by month three, when the beautiful ones are scarce.

They'll ask "so how are you doing on time." An honest "I don't know, here's what I already did" beats the theater of "everything's great." The station likes sensors, not optimism without numbers.

2.4 Where it grows, if you don't run into the woods

Further on, if you don't get stuck copying annotations:

- **Middle** — they trust you with a slice of the system, not one button. You already say "that's a bad idea" and sometimes they listen. Pay goes up. The amount of Slack does not go down.

- **Senior** — people come with a question, not a ready ticket. You decide **what** to build, not only **how** to write the loop. Pay grows faster than hair. An unpleasant duty appears: stopping other people's bad ideas, including yesterday's yours.
- **Architect** — you draw arrows between boxes and argue whether Kafka is needed **this time** for a reason. Sometimes the arrows save the station. Sometimes it's the career of someone tired of debugging who isn't ready for management yet.
- **Manager** — if you suddenly stop wanting code. Then meetings and "so how are we on the timeline." Same `loop`, only the arguments are people, and they return `nil` less often than they promise. That isn't in this textbook, and we all sighed with relief.

This book is the first hatch. The rest happens by itself if you don't stop fixing things.
Don't pick "senior or manager" in week one. In week one, `Hello` has to print. A career is what grows out of a pile of evenings, not a slide labeled "trajectory."

2.5 English. Yes, even in the English edition

The compiler, Spring, Stack Overflow, most of the books — the words are `null`, `commit`, `stack trace`. Not "business English for airport small talk." If English isn't your first language, a little every day still helps: docs, other people's READMEs, subtitles. It is **not** inside our forty minutes of Lisp plus two hours of Java. We already stuffed in the parentheses. English is a home quest. There is no answer key at the back. There is the internet.

> **Mac, Windows, WSL**
>
> You can leave the IntelliJ UI in whatever language feels calm. Compiler errors will still be in English. Get used to reading the red, not translating every word: by week three `cannot find symbol` is as domestic as "we're out of milk."

Don't wait for "C1 first, then Java." You need "I read a paragraph of docs and didn't die." That grows on the road if you don't treat English READMEs like fire.

2.6 Why Lisp then

Because the on-ramp to Java is easy to turn into six months of `@Autowired` annotations and a quiet wish to go live in the woods. Lisp in a REPL shows the same ideas naked: a list, a function, recursion, "data is code too." Play with lists and `ArrayList` and JSON stop being hieroglyphs. Forty minutes a day — so you don't hate programming by month three of Spring.
On a résumé you can poke Lisp into "also I can." They won't ask in the interview. Good. They'll ask about `HashMap`. And `HashMap` after an alist in Lisp is no longer magic, it's a pocket with labels, only fast.
Lisp also treats the fear of errors. In the REPL you broke it — you got a message, you fixed it, you live. No five-minute build to learn you forgot a parenthesis. That reflex ("broke → read → fix") later saves you in Java, where the build is exactly five minutes and very offended.

2.7 Six months is dense

Weekdays: two-forty. Plus Saturday. If by week twelve you have no server of your own with a database — don't tweak Kafka, finish the server. Curiosity is cheaper than a lost month. Missed a week — don't read three chapters on Saturday, that isn't a feat, that's mash. Take the current lesson and your project.

Six months × about six evenings × two-forty — that's a lot of hours. Enough for fingers to remember. Not enough to become someone who "has already seen everything." And you don't need to. They hire juniors not for "seen everything." They hire them for "I see, I fix, I tell."

If you work another full-time job — six months may stretch to eight. Fine. The calendar in the book is a map, not a bomb timer. Station MODULE wasn't built on a Jira sprint either, judging by the duct tape.

2.8 On copying

Other people's answers — after your own attempt. At work you'll Google every day, that's normal. The difference is whether you can **read** what you pasted and change one line without collapsing the rest.

Copying a quest from the answers in the first minute is like looking up a crossword and deciding you're smart. Warm. Empty. In a month the interview crossword won't have answers at the back of the book.

Peeking after twenty minutes of anger — fine. You already thought. The brain got the query. Then the answer **lands**, instead of flying past.

Code from this book — into your study repos, please. Barski's book and other people's commercial repos — don't pass them off as yours. The station is already on duct tape; let's skip lawsuits and résumé lies. Lies in orbit end with an open airlock.

Chapter

3 How a computer works when you aren't a programmer yet

This chapter is for someone who has never written a program. At all. If you already installed a JDK and fought with `PATH`, skim it and go to the workshop. If the word `terminal` sounds like a threat from a hacker movie — sit down. Tea is allowed. A spacesuit is not required.

Station MODULE hangs in orbit and pretends to be smart. It is dumb as a hammer. A hammer doesn't think either. It hits wherever you sent it, even if you sent it into your thumb. A computer is the same: it obeys **exactly**, not "yeah, I got the idea."

What follows is slow. No digital electronics, no von Neumann architecture on a chalkboard, no feeling that you missed the first week of college. Just a kitchen, a pantry, and a black window that somehow scares everybody.

> **Slow down**
>
> If a paragraph didn't click — read it out loud. The computer will not be offended. It doesn't feel anything. That's its superpower and its curse.

3.1 A box that obeys, stupidly

A computer is a box. Inside: motors made of sand (almost true: chips are silicon). Outside: buttons, a screen, sometimes a sticker that says "don't pour coffee." You tell it what to do. It does **exactly that**. "Exactly that" is the trap.

Tell a human "put the kettle on." They'll fill it, switch it on, wait, yell "ready." For a computer you have to spell it out:

1. Pick up the kettle.
2. If there's no water — fill it.
3. If there's too much water — pour the extra out.
4. Set it on the base.
5. Switch it on.
6. Wait until it boils **or** until five minutes have passed.
7. Switch it off.
8. If there is no kettle — don't sit there in silence. Say so.

Forget step 8 and the program will wait for a kettle until the heat death of the universe. Forget step 3 and you'll flood the galley. Forget "switch it off" and the coil burns out, and the flight engineer will become very eloquent.

This is not a metaphor "for kids." This is the job. A programmer is a person who writes lists like that and then wonders which step they forgot.

> **Station law**
>
> The computer does not guess. It did not "kind of understand." Either the instruction is there, or it isn't. There is no third option, even if you asked very politely.

On station MODULE it looks like this. You write: "open the airlock." The hardware opens the airlock. It will not ask "is anyone wearing a suit?" — not unless you wrote that. That's why half the programs on Earth are not "do the thing," they're "do the thing, **and check that we don't kill everybody**."

The instruction is called a **program**. Scary word, simple object: text (or almost text) the box knows how to run. Until it knows how — it's just a letter to yourself.

We aren't writing programs yet. First we look at **where** they live and **who** cooks them.

3.2 The station galley

Picture MODULE's galley. Not a datacenter. A galley.

The cook is the processor, the CPU. He doesn't store anything for long. He **does**: chops, stirs, counts, compares. Very fast, very forgetful. Look away — he already doesn't remember how much salt. So there's a counter next to him.

The counter is RAM. What the cook is working with **right now**. Knives, a bowl, an open recipe, an onion half chopped. Lights out on the station — everything is swept off the counter. The recipe in the cook's head is gone too: the CPU has no "head for tomorrow."

The pantry is the disk. Hard drive, SSD, "laptop storage," a USB stick — for now they're one thing: the place where stuff still is after the lights go out. Jars of grain, jars of programs, jars of your vacation photos, which the station does not approve of. Fetching a jar from the pantry is slower than grabbing a bowl off the counter. So the cook does not sprint to the pantry for every spoon.

The recipe is the program. Text: what to do with the onion. The recipe lives in the pantry (a file on disk). When you "open a program," somebody copies the recipe onto the counter, and the cook starts working from it.

> **Slow down**
>
> A file on disk is a jar in the pantry. A running program is the stew being stirred **right now** on the counter. Those are different things. A jar can sit for years. The stew goes cold the moment you switch the burner off.

A few more galley guests, briefly, no tour of the factory floor:

- **Screen and keyboard** — a window into the galley, and the mouth you yell at the cook with. The mouse is a finger that pokes pictures.
- **The network** — an elevator to other kitchens. Later. There will be mail.
- **The graphics card** — a separate cook for pictures. This book almost never needs it. Let it fry its own frames.

- **BIOS / firmware** — a note on the door: "how to light the burner when everything else is still asleep." Don't touch it until the station asks.

Why does a game stutter while Notepad doesn't? There's one cook (okay, several hands, but not infinity). The counter is small. If the recipe is huge, some jars have to stay in the pantry and get hauled back and forth. You can see the hauling: everything "thinks." That's not the computer not loving you. That's the pantry being far away and the counter being small.

> **Bad idea**
> "I have eight gigabytes of RAM, so I can keep eight gigabytes of tabs." You can. The cook will start sprinting to the pantry constantly. That's called swap, and it's sad. Close twenty chats. The station will say thank you.

An ordinary morning looks like this:

1. You switch the box on.
2. From disk rises the operating system — the master recipe, which then lets the others in.
3. You poke an icon.
4. The system finds the file in the pantry, copies what it needs onto the counter, tells the CPU: "cook."
5. The cook cooks until you close the window or the stew boils over (error, crash, "not responding").

An icon is a picture. A picture computes nothing. The file the picture **points at** computes. Sometimes it points at the wrong place. Then you get angry at the picture. Get angry at the path. We'll do paths.

3.3 Files, folders, a path, and the tail with a dot

A file is a jar with a name. Inside: bytes. We'll poke bytes later. Right now: a file **lives somewhere**. "Somewhere" is a tree of folders.

A folder (catalog, directory — same thing in daily life) is a crate that holds jars and other crates. On a Mac people say "folder"; in the terminal, `directory`. Don't fear the synonyms. Pantry clerks love three words for one shelf.

A path is how you walk from the pantry entrance to the jar.

On a Mac and on Linux (and in WSL, which is Linux in a Windows coat) a path looks like this:

```
/Users/vasya/dev/lisp-experiments/hello.lisp
```

Read left to right, like a station corridor:

- `/` — the root, the main hatch.
- `Users` — the people compartment.
- `vasya` — your cabin.
- `dev` — the workbench.
- `lisp-experiments` — the crate with parentheses.
- `hello.lisp` — the jar.

On Windows without WSL, same meaning, different handwriting:

```
C:\Users\vasya\dev\lisp-experiments\hello.lisp
```

`C:` is a drive, like a separate warehouse. The slashes go **the other way**. Paste a Linux path into native `cmd` and it will take offense. That's why in this book, on Windows, we almost immediately go into WSL and write paths with ordinary `/`.

> **Mac, Windows, WSL**
>
> **Mac.** Terminal → path like `/Users/…` . In Finder you can drag a folder onto the terminal window — it pastes the path. Lazy and correct.
> **Windows + WSL2 (recommended).** Inside Ubuntu your home is `/home/vasya/…` . Drive `C:` shows up as `/mnt/c/Users/…` . Live **in** `/home` , not on `/mnt/c` : otherwise Linux and Windows will fight about permissions and line endings like two mechanics over one wrench.
> **Windows without WSL.** File Explorer shows `C:\Users\…` . PowerShell understands both kinds of slash **sometimes**. Don't tempt fate: either WSL, or always backslashes and quotes around paths with spaces.

A space in a folder name is a petty villain. The path `my project` is two words to the terminal: `my` and `project` . Quotes save you: `"my project"` . Better still — don't put spaces in study folders. `lisp-experiments` sounds boring and never bites.
A filename often has a tail: `.txt` , `.java` , `.lisp` , `.pdf` . That's the **extension**. Not magic. Not a quality stamp. Just a pact: "guess from the last letters what to open this with."

- `note.txt` — "this is text, open it in a notepad."
- `Hello.java` — "this is text, but the Java compiler thinks it's its recipe."
- `hello.lisp` — "this is text, SBCL will be pleased."
- `photo.jpg` — "this is not text, don't open it in a notepad, you'll get garbage and tears."
- `archive.zip` — "a bundle of files in one jar, squeezed."

You can rename `Hello.java` to `Hello.txt` . The bytes inside **will not change**. Only the label changes. A notepad will open it and show the same code. `javac` may take offense: it looks for `.java` . The operating system looks at the tail when you double-click. A programmer looks **inside**, once they're already in the terminal.

> **Slow down**
>
> An extension is a label on the jar. Not the taste of the stew. The stew is the contents. You can stick a lying label on. That's why a "broken file" with an `.mp3` tail happens. And the other way: a Java recipe saved as `.txt` is still a recipe, the burner just won't recognize it by its clothes.

Hidden files on a Mac and on Linux start with a dot: `.gitignore` , `.ssh` . Finder is shy about them. The terminal shows them if you ask `ls -a` . This is not a secret police. This is "don't clutter the ordinary list."
Two phrases everybody mixes up:

- **File not found** — the path is lying: a typo, the wrong folder, the wrong drive.

- **Permission denied** — the jar is there, the pantry clerk won't let you touch it. Rare in your own study folder. In system ones like `/usr`, `C:\Windows` — please don't play hero.

3.4 Text, and "the computer understands"

Here's a surprise that saves you a year of life.
Almost everything you'll write in this book is **text**. Letters, parentheses, semicolons. The same sort of thing as a letter to your grandmother, only grandmother doesn't require `public static void`. `Hello.java` opens in a notepad. `hello.lisp` too. `README.md` too. Even `.json` and `.xml` and `.html` — text. You can read text with your eyes. You can ruin text with one extra character.
A computer "understands" text unlike a human. A human sees "energy 80" and guesses the reactor. A computer sees bytes, turns them into numbers, numbers into commands. If the recipe has a typo — it will not fix it. It will say "I didn't get that" or, worse, it got **something else**.

> **Slow down**
>
> There are two worlds.
> World 1: you look at a file and read words. That's for you.
> World 2: some program (compiler, interpreter, browser) **eats** that text by strict rules. That's for it.
> "The computer understood" means: the eater program chewed the text and didn't choke. It does not mean the meaning is good. You can perfectly chew the recipe "open the airlock with no check." The station will understand. The crew will too, but briefly.

Binary files are not text. A picture, music, a compiled program `Hello.class`, a Word document `.docx` (it pretends; inside it's zip). Open it in a notepad — a stew of glyphs. Don't fix that stew by hand. Pictures have other tools. `.class` belongs to Java; give it to Java.
Why IntelliJ and VS Code, then, if a notepad can do text? Because a code editor highlights parentheses, yells at typos, jumps to the error. A notepad is honest, but it's like repairing a reactor with an eyeglass screwdriver. You can. Once. Then get a real wrench.
One more pact: **encoding**. Letters in a file are numbers too. There will be a compartment for that. For now remember one thing: if you opened a file and instead of `naïve` you see `naÃ¯ve` — nobody hacked the station. Two pantry clerks are just reading the same jars with different tables.

3.5 The black window, or why everybody needs a terminal

A terminal is a program where you **type commands as text**, instead of poking icons. The black (or dark grey) background is a habit from the seventies, when screens were green and the furniture was an ashtray. You can make it white. People will still call it "the black window."
Why, if you have Finder and File Explorer?
Because it's easier to tell a program "run this recipe with these jars" than to draw a button for every move. `javac Hello.java` is one sentence. In a menu it would be: File → Open → hunt through god-knows-where → Build → if the item is named something else in this version → give up.

Also because an error in the terminal is **text**. Text you can copy. Text you can search. A picture of "a red lamp in the IDE" sometimes hides the same text three clicks sideways. Learn to read the black window — and the IDE becomes a helper, not a priest.

> **Station law**
> A command is a program too. `ls` is a tiny recipe: "show what's in the crate." You launch programs all day, even when you "aren't programming."

The main idea of the terminal, without which everything else is a circus:
The window has a **current folder**. Current directory. "Where I'm standing in the pantry." Commands look **here** by default. Not "the whole computer." Here.
Show where you're standing:

```
pwd
```

Print Working Directory. Print the working directory. On a Mac and in WSL this is a holy command. The answer looks like:

```
/Users/vasya/dev
```

or

```
/home/vasya/dev
```

In native PowerShell `pwd` often works too (it's an alias). If it suddenly doesn't:

```
Get-Location
```

Show the jars in the current crate:

```
ls
```

List. A listing. On a Mac and in WSL, `ls` is native. In PowerShell `ls` is often there too (an alias for `Get-ChildItem`). Ancient `cmd` loves `dir`. If `ls` wasn't found — `dir`. Same meaning: what's lying **here**.

```
ls -la
```

On Linux/macOS: a detailed list, including the hidden ones with a dot. Columns of permissions, sizes, dates. Don't memorize them today. Just know that `-la` means "show me honestly."
Go into another crate:

```
cd lisp-experiments
```

Change Directory. Change the directory. Now `pwd` is different, `ls` is different. You **moved**.
One level up:

```
cd ..
```

Two dots — "the crate that holds the current crate." Like "step out into the corridor." One dot `.` means "here." Handy when a command wants a path: `./mvnw` means "the file `mvnw` **in this** folder."

Home:

```
cd
```

or

```
cd ~
```

The tilde `~` is your cabin: `/Users/vasya` or `/home/vasya`. Not the root of the drive. The cabin.
An absolute path — from the main hatch:

```
cd /Users/vasya/dev/java-basics
```

A relative one — from where you're standing. If you're already in `dev`:

```
cd java-basics
```

Both are correct. Confused — `pwd`, then think.

> **Slow down**
> Most of "it can't find the file" is you standing in the wrong folder. Not "the computer erased it." Not "Java broke." `pwd`, then `ls`, then panic. On the station they call this ritual "look down at your feet."

Make a crate:

```
mkdir java-basics
```

An empty file (Mac / WSL):

```
touch Hello.java
```

In PowerShell `touch` may be missing. Then:

```
New-Item Hello.java
```

or open an editor and save. A file does not have to be born from a command. The command is just faster.
Print the contents of a text file:

```
cat Hello.java
```

In PowerShell often `Get-Content Hello.java`, or `cat` as an alias. If a stew of glyphs comes out — either it isn't text, or it's encoding (see below).
Clear the screen when the output is stew:

```
clear
```

In PowerShell also `cls`. Commands are not deleted from history, only from your eyes.
History: up arrow. Repeat a command without typing it. On a Mac and in WSL, also Ctrl+R — search history. A drug. Later you won't be able to live without it.
Interrupt a program that's hung and yelling:

Ctrl+C. Not copy (in a terminal, copy is usually Cmd+C / Ctrl+Shift+C, depends on the window). Ctrl+C is "stop, cook, put the knife down."
Leaving some programs: `exit`, or `(quit)` in SBCL, or Ctrl+D (end of input). If nothing helps — close the window. Brute force. Works.

> **Bad idea**
>
> Don't paste from the internet commands that start with `rm -rf /` or `del /s /q C:\`. That's "carry the whole pantry out the airlock." This textbook doesn't give you those. If someone did — that's not a mentor, that's a joker or worse.

3.6 Three black windows: Mac, PowerShell, Ubuntu in WSL

Same idea: current folder, commands, text. Different furniture.
Terminal on a Mac. The Terminal app. Inside, usually zsh (it used to be bash — doesn't matter for us). Copy commands from this book here with almost no translation. Homebrew puts programs in places this terminal can see.
PowerShell on Windows. Blue or black. Can do a lot. Some Linux names are faked with aliases (`ls`, `pwd`, `cat`). Some aren't (`sbcl` appears only if you install it). Paths with spaces and `C:\`. You can live. A lot of textbook pain lives right here: the author wrote for a Mac, you've got PowerShell, one slash is wrong — and you think you're stupid. You aren't stupid. The dialect is different.
cmd.exe. Older still. `dir`, `cd`, backslashes. If you can not open it — don't.
WSL2 + Ubuntu. This is what I recommend if you have Windows. A subsystem: Linux **inside** Windows, no second machine and no eternal fight with slashes. The window looks like an Ubuntu terminal. `ls`, `pwd`, `apt`, `sbcl`, `javac` — like the textbook, like a server, like your neighbor on a Mac (almost).

> **Mac, Windows, WSL**
>
> **How to turn on WSL2, short version.** Win+S → "Turn Windows features on or off" → check **Windows Subsystem for Linux** (and **Virtual Machine Platform**, if it's there). Reboot. Microsoft Store → Ubuntu. The first window will ask for a username and password — that's a **Linux** user, not your Windows password. The password at `sudo` does not print asterisks: that's normal, type blind.
> Then:
>
> ```
> sudo apt update && sudo apt upgrade -y
> ```
>
> After that, almost every command in the book goes **in this window**. Not PowerShell. Not cmd. Put textbook files in `/home/<you>/dev/…`. IntelliJ on Windows can open `\\wsl$\Ubuntu\home\<you>\dev\…`. Docker Desktop on Windows will ask for WSL2 itself — say yes.
> **Mac.** You don't need WSL. You already have Unix. Install Homebrew and breathe.

Why not "two systems in equal shares"? Because you'll be googling errors. Most of the answers are for Linux commands. Spring, Docker, the server at work — Linux. You can learn PowerShell. First, better learn the dialect the profession speaks.

> **Station law**
> On Windows: WSL2. Book commands — in Ubuntu. If something "is not recognized as a command" in PowerShell, ask yourself: is this the right window? Often it isn't.

One more mix-up: **where the file actually is**.
You downloaded `Hello.java` through a Windows browser into `Downloads`. In WSL that's `/mnt/c/Users/vasya/Downloads/Hello.java`. You can run it from there. Better copy it into `/home/vasya/dev/java-basics/`, so you aren't assembling recipes from someone else's warehouse through a hole in the wall.
Line endings: Windows likes `\r\n` (two characters: carriage return and newline). Linux and Mac — `\n`. Git sometimes yells `LF will be replaced by CRLF`. For study Java/Lisp this is almost never death. If a script "won't run" and prints a weird `^M` — that's it, Notepad's inheritance. In WSL: `file your.sh` will sometimes hint. You'll learn later. Today just know that **text is not always the same text**.

3.7 Program, process, "it's running"

The file `Hello.java` is not a running program. It's a recipe in the pantry.
`Hello.class` isn't stew either. It's a recipe translated into a language more convenient for the Java burner. Still a jar.
When you type `java Hello` or hit the green arrow, the operating system:

1. Finds the right file.
2. Allocates a piece of counter (memory).
3. Gives the cook (CPU) a job.
4. Hangs a tag on that job: a **process**.

A process is **a recipe being cooked right now**, plus bowls (memory), plus a number in the queue. The same file can be launched twice — you'll get two processes. Two notepads. Two `java Hello`. They don't share bowls unless they agreed to. Close one window — the other lives.
"The program hung" means: the process is still on the list, but it doesn't answer "how's the stew?" Sometimes it's computing (a heavy recipe). Sometimes it's waiting on the network. Sometimes it's waiting for you, and you can't see **what** it's waiting for. Sometimes it's dead, but the tag is still hanging.
"Not responding" in a Windows / Mac menu is a polite translation of "I knocked, nobody home." You can wait. You can force-quit. Force-quit is switching the burner off. The stew on the counter is gone. The jar in the pantry stays.

> **Slow down**
> Quitting a program ≠ deleting a program.

> Close the window — stop cooking. The file on disk is still there.
> Delete the file — throw out the jar. A running process **may still live** (the recipe is already on the counter). Rarely matters in month one. Then suddenly it matters, when "I deleted it and it's still yelling."

Where several Java processes come from: IDEA itself, your program itself, maybe Maven too. That's normal. That's not "a virus." Task Manager / Activity Monitor will show names. The name `java` will be there a lot. Don't kill everything in a row: you can shoot the IDE.
A program can be a **server**: the process lives and listens. A browser knocks on it. Close the terminal where the server was running — the server often dies. That's why words like Docker show up later. Today it's enough: if the cursor is blinking in the window after a command — the program may be **waiting**. Don't open a second copy until you know whether the first is alive.
How to see what you launched in this terminal: it writes you lines. When it writes a prompt again (`$`, `%`, `*`, `PS C:\…`) — that command **finished**. If it doesn't write and doesn't return a prompt — it's still working, or waiting for input.
SBCL: the prompt is `*`. You're inside the `sbcl` process. `(quit)` — the process dies, you're back in an ordinary terminal.

3.8 Compiler and interpreter — one stew, two burners

There's a recipe "say the energy." Here it is in two languages, tiny on purpose.
Lisp:

```
(print (+ 40 40))
```

Java:

```
public class Energy {
    public static void main(String[] args) {
        System.out.println(40 + 40);
    }
}
```

Both should print `80`. Same meaning. Different ritual.
An **interpreter** reads the recipe and cooks immediately. SBCL in REPL mode is like that. You start `sbcl`, see `*`, type:

```
(+ 40 40)
```

Enter. `80`. No separate "translate the file." The cook looks at the parentheses **right now**.

> **What just happened**
> You wrote an expression. SBCL read it. Evaluated it. Printed the result. Waits again. Read, Eval, Print, Loop. That's a REPL. Not a temple. A calculator you can explain the station to.

You can put the same thing in a file `energy.lisp`:

```
(print (+ 40 40))
```

and say:

```
sbcl --script energy.lisp
```

Still no separate "build" file on disk that you run later. They read the script and did it.
A **compiler** first translates the recipe into another form, convenient for the machine (or a burner like the JVM), and **then** you run the translation.

```
javac Energy.java
```

`Energy.class` appeared. That's not text for you. That's bytecode for the Java machine. Then:

```
java Energy
```

`80`. If there's a typo in the `.java`, `javac` yells **now**, and the `.class` either isn't born or the old one stays. An old `.class` is a separate meanness: you fixed the source, forgot to compile, you're running yesterday's stew. That's why the green arrow in IDEA does both steps. In the terminal, remember there are two.

> **Slow down**
>
> A compiler is a translator who works **before** dinner. It will catch a grammar error on the shore. An interpreter is a translator at the table. You say a sentence — they eat at once. An error on line three you'll see when you get there.
> Java feels "stricter" to a beginner, Lisp "more alive." At work both kinds show up. Even Java later interprets its bytecode, in a sense. Don't cling to a pure classification. Cling to the ritual: **do I need javac first**.

The same idea "add 40 and 40":

Step	Lisp, SBCL	Java
Recipe	parentheses in the REPL or `.lisp`	`Energy.java`, a class, `main`
Translate ahead of time	not required	`javac` → `.class`
Run	Enter in the REPL or `--script`	`java Energy`
Typo	error right in that phrase	often already at `javac`
Repeat	up arrow	fixed the file → `javac` and `java` again

Why is Java so wordy with `public class` and `main`? Because the burner is big and loves forms. Why can Lisp be one line? Because the REPL already knows you want to "compute this." That doesn't mean Lisp is a toy. It means the on-ramp is lower. There are more Java jobs, though. That's why the book has both.

An IDE hides compilation. That's convenient. Once a week, run from the terminal so you don't forget **what** they hide. Otherwise the Run button becomes magic again, and we just took magic apart.

> **Bad idea**
>
> Don't confuse a file and a process and also the compiler. `javac` is a separate program. It worked and died. Then `java` is another program, another process. Two burners in one galley, a queue.

3.9 Bits and bytes without a soldering iron

A computer loves two states: current / no current, pit / bump, yes / no. Convenient to call that 0 and 1. One such answer is a **bit**.

A bit is too small, like a crumb of salt. They pack them by eights: a **byte**. A byte can tell 256 variants apart (from 0 to 255). That's enough for a Latin letter, a tiny piece of a picture, a "which command." After that, just labels, so you don't have to say "a million million":

- kilobyte (KB) — about a thousand bytes (sometimes 1024; pantry clerks argue, you don't care right now).
- megabyte (MB) — about a million.
- gigabyte (GB) — about a billion. A movie, a game, RAM.
- terabyte (TB) — a disk that says "come on, that's enough."

> **Slow down**
>
> The number `80` in a program is not always one byte "eighty." For a human, 80 is two digits on paper. For a machine there are different boxes: a small integer, a big integer, a fractional. Java writes `int`, `double` — those are box sizes. Don't pick a box like a sommelier yet. Know this: **a number needs space on the counter too**.

The text `naïve` is several bytes in a row. A picture is a lot of bytes: the color of every dot. A song is even more, if you don't compress. Compression (zip, jpg, mp3) is "let's throw away repeats and what the ear won't notice." Sometimes it notices. Then artifacts.

A disk that's "full" means: the pantry ran out of shelves for bytes. RAM that's "gone" — no room on the counter for stew. Those are different shelves. A 512 GB disk does not cure a shortage of 8 GB of RAM. You can install a huge pantry and still trip over the counter.

Why does a programmer need this in month one? So you don't believe in magic sizes. So you understand: a `.class` file is bytes too. So "out of memory" doesn't sound like a curse, it sounds like "the counter is packed." So "corrupt file" means: the bytes aren't in the order the eater expected.

We will not convert numbers to binary in a column today. That's a charming sport. At a junior job they almost never ask. They will ask why you put a hundred-megabyte picture in git. Answer: bytes weigh, a repository is not a pantry for movies.

3.10 Letters are numbers. UTF-8. Garbage glyphs

The keyboard lies. You think the letter `é` is flying into the file. What's flying onto the disk is a **number** that, by a table, means `é`.

The table is an encoding. A pact: which number is which letter.

There used to be many pacts, like dialects on a station after a century of isolation:

- ASCII — Latin letters, digits, signs. Enough for `Hello`, not enough for `naïve`, and definitely not for the old hull stencil `МОДУЛЬ`.
- Windows-1252, Latin-1, and a zoo of one-byte tables for other alphabets (Windows-1251, KOI8-R, CP866 for Russian — heroic, incompatible with each other).
- UTF-8 — the current world. Almost every letter on Earth, emoji, and signs you should not use in a variable name.

UTF-8: a Latin letter often takes 1 byte, `é` or a Russian letter takes 2, some ideographs and emoji take more. You don't need to remember that. You need to remember: **a file with no "I am UTF-8" sticker can be read with a different table**.

Mojibake, garbage glyphs — when bytes were written with one table and read with another. `naïve` in UTF-8, opened as Latin-1, turns into hell. Hell looks scholarly: `naÃ¯ve`. Or `����`. Or the old stencil `МОДУЛЬ` becoming `ÐœÐ¾Ð´ÑƒÐ»Ñ¬`.

> **Slow down**
>
> Mojibake is not "the file died." The bytes are often intact. The **translator** is broken. Open it in the same editor, pick encoding UTF-8. Or resave. Don't retype the garbage "as is" — you'll get **new** bytes on top of the old mix-up, and then yes, the patient dies.

Practical rules for year one:

- In IDEA / VS Code look at the corner: it should say UTF-8. If not — set it.
- Textbook files, `.java`, `.lisp`, README — UTF-8.
- Don't copy code from Word or from a "smart" PDF where the quotes are fancy and curly instead of straight `"`. The compiler does not count curly quotes as quotes. To it they're just pretty blots.
- Filenames with non-English letters **can** work. They can also fail in old tools. Study files `Hello.java`, `module.lisp` — Latin, fewer surprises. Strings **inside** the file — any, UTF-8.

The terminal window has an encoding too. Rarely bites on a Mac. On old `cmd` it bit often. One more vote for WSL.

Java likes to write `\u00e9` instead of a letter — that's "the letter's number in the Unicode catalog." Unicode is a big catalog of characters. UTF-8 is one way to pack the number into bytes. You can live without passing this exam. You can just not paste into code a character you can't see: a non-breaking space from the web looks like a space and breaks everything. If "identical" strings aren't equal — suspect the invisible.

> **Bad idea**

> Saved a file "as ANSI" from Windows Notepad — a gift to your future self, with a grenade. Notepad learned UTF-8, but the menu items can still betray you. Watch **what** you save with.

3.11 The network: two kitchens and mail

While the computer is alone — a galley. The network is a corridor between galleys.

Your machine. Their machine (a site, a server, a friend's phone). Between them, not "a tube of water," but a pact to send **packets**. A packet is an envelope: where to, where from, a chunk of data, a number "I'm part 4 of 10."

You open a browser, type an address. Cartoon version, no telecom diploma:

1. The browser asks: "who is `example.com`?" — like "which cabin does Vasya live in." DNS answers, a notebook of names. Name → number.
2. The number is an IP. The apartment address, not the last name.
3. An apartment has many doors. A door is a **port**. 80 and 443 — "browsers knock here for websites." 5432 — often Postgres. 8080 — your study server, when you get there.
4. Envelopes run down the corridor. They can get lost. The protocol (TCP — the one under the web) can ask again: "the fourth envelope didn't arrive."
5. The server reads the letter ("give me the home page"), puts the answer in envelopes going back.
6. The browser assembles the envelopes into a page.

> **Slow down**
>
> `localhost` is "this same kitchen." Not the internet. The address of yourself. When you learn to stand a server up at home, the browser goes to `http://localhost:8080` like to the neighboring burner in the same galley. No cloud magic. One process listens at door 8080, another process (the browser) knocks on that door.

Wi-Fi dropped — the envelopes stopped moving. A program that was waiting for an answer can hang. That isn't always a bug in your `if`. Sometimes the letter just didn't arrive.

HTTP is the language of letters for the web: `GET /tasks` means "give me the list of tasks," `POST` means "accept a new one." JSON is often the **body** of the letter, text like `{"energy":80}`. Details will take months. Right now the picture: not "the site lives in the browser." The site lives on someone's machine. The browser is a window through which you read other people's jars.

A firewall is a picky doorman at the port doors. Sometimes your server is alive and the doorman won't let anyone in. Sometimes antivirus. Sometimes a corporate network. The message `connection refused` is "the door is closed, or nobody's there." `timed out` is "I don't even know if there's a door, the envelopes vanished in the corridor."

We are not configuring a router today. We are only killing the myth that the internet is a cloud with no addresses. There are addresses. There are doors. Letters sometimes get lost. Programs that go on the network have to think: "and if they didn't answer?"

> **Station law**
>
> Someone else's computer is not your pantry. You ask by letter. They can refuse. They can lie. They can not answer. That isn't an insult. That's a network.

3.12 Errors are a gift

A beginner sees red and hears: "I'm unfit." A programmer sees red and hears: "here's the line." A compiler, an interpreter, an operating system — creatures with no tact and no laziness. They don't hint. They write **what's wrong**, often even **where**. The first line of the error matters more than the last. The last is a tail, like panic in the corridor. The first is who yanked the sensor.
Sample gifts:

- `No such file or directory` — the path. `pwd`. `ls`. Not "Java died."
- `command not found` / "is not recognized as a command" — the program isn't installed, or the window is wrong, or `PATH` doesn't know which crate holds the jar with the command. `PATH` is a list of corridors where the system looks for programs. You install a JDK, forget PATH — `javac` "vanished." It didn't vanish. They aren't looking where you're standing.
- `cannot find symbol` in Java — a typo in a name, or you forgot an import, or the wrong file.
- Lisp `unmatched close parenthesis` — an extra parenthesis. A gift. Honest. Count them.
- `Connection refused` — nobody is listening at the door. The server isn't running, or the port is different.

> **Slow down**
>
> An error is not a grade. It's a sensor on the station. The sensor yells because the air is running out, not because you're a bad person. Switch the sensor off (`catch` everything, ignore red in the IDE) — you'll die quietly in your cabin. Read the sensor.

What to do with your hands is always the same:

1. Read the **whole** first error. Not "the red word." The line.
2. Look at the line number in **your** file.
3. If it's a path — `pwd` and `ls`.
4. If the command wasn't found — either the window, or the install, or PATH.
5. Then search the internet. Copy the error **without** your username and without the unique path to your cabin. Otherwise you'll only find your own footprints.

It isn't shameful to "get an error." It's shameful to close your eyes and poke until it goes away by itself. At work it doesn't go away by itself. On MODULE even less: vacuum is patient, air is not.

> **Station law**
>
> You broke it — good. Means you weren't watching a video, you were touching the station. Fix it from the error text, not from your mood. Mood lies more often than the compiler.

3.13 One shift in the terminal, slowly, out loud

Let's walk the same walk as if you're already at the desk. Don't install Java yet. Just feet in the pantry.

You opened a window. On a Mac, the left side often has the machine name and `~`. In Ubuntu inside WSL — `vasya@pc:~$`. The tilde is you're home. The `$` or `%` sign is "you may type a command." It isn't money.

```
pwd
```

Read it out loud. `/Users/vasya` or `/home/vasya`. That's the cabin.

```
ls
```

You'll see `Desktop`, `Documents`, `Downloads` — or the local names if the system named them that way. On a Mac, Finder and `ls` are one pantry, different view. Delete in Finder — it's gone in `ls` too. Not two worlds. Two windows.

Make a workbench:

```
mkdir dev
cd dev
mkdir module-cabin
cd module-cabin
pwd
```

Now the path should end in `module-cabin`. If not — you either created it somewhere else, or `cd` didn't work (typo in the name). `mkdir` is silent when it succeeds. Silence in a terminal often means "ok." Rude and familiar.

A file with text, Mac / WSL:

```
echo "station MODULE" > note.txt
cat note.txt
ls
```

`>` means "put the output in a file, wipe the old." If the file already existed — the old contents die. Two arrows `>>` — append at the end. For a study note, an editor is enough. `echo` is so you don't have to leave.

> **Mac, Windows, WSL**
>
> **PowerShell.** `echo` exists. Redirect `>` exists too. File encoding can sometimes be UTF-16 — a surprise for anything that isn't boring ASCII. One more vote for study files being born in WSL or in a real editor set to UTF-8.

Go back at random:

```
cd ..
pwd
ls
```

You should see `module-cabin` as a folder, not be living inside it. Two dots — the most common dance of week one. Overshot — `pwd`. Not shameful.

3.14 Words after the command: arguments and flags

`ls` is a program. `ls -la` is the same program, you passed it a note `-la`. The note is an **argument**. Minuses at the front are often called **flags**: "turn this mode on."

```
ls module-cabin
```

An argument without a minus is usually **what** to apply it to: show this folder, not the current one. The current one does not change. `ls x` — look in crate x. `cd x` — go into crate x. Different verbs. `java Energy` — program `java`, argument `Energy` (a class name, not a `.class` file in the argument). `javac Energy.java` — the other way around, here it really is a file. Everybody mixes them up. You will too. Then you'll stop.
Tab — name completion. You start `cd mod`, hit Tab, the terminal finishes `module-cabin` if there's only one. If several — it beeps for another letter, or shows a list. This is not intelligence. This is the list of files already in the folder. Get used to it. Without Tab, life is longer by typos.

> **Station law**
> A command is a program name, then a space, then its arguments. Like Lisp, only custom puts the parentheses in, not you. A typo in an argument is already a different error, not "command not found."

3.15 Who's the head cook: the operating system

macOS, Windows, Linux (Ubuntu in WSL) — these are not "different computers." The hardware is similar: CPU, RAM, disk. What's different is the **master recipe** that lets the others in: who draws windows, who reads the keyboard, who decides whether you may touch a file.
You can think of an operating system as the shift's head chef. You don't yell at the CPU yourself. You yell at the chef: "start this recipe." The chef gives the cook time, a piece of counter, the right to read a jar. Without a chef every program would fight over one bowl. That happened. It was called badly.

> **Slow down**
> An icon on the desktop is not a program. It's a picture and a pointer: "chef, start this file." The terminal does the same, only you say the name yourself. Same chef.

Why then "the Mac command doesn't work on Windows"? Because the chefs handed out crate names and staff cooks differently. `ls` lives with a Unix chef. A Windows chef lived with `dir` for a long time. WSL is a **second chef in the same box**. That's why textbook commands paste there.
"Install a program" means: put files in the pantry and tell the chef where to look (often — add a corridor to PATH). An icon in the menu is a bonus. Uninstall — remove the files. Sometimes some old jars stay. That's "junk after uninstall," not a ghost.

3.16 PATH, again, because everybody trips on it

You type `javac`. The chef does not telepathize. He looks at a list of folders: PATH. In each one — is there a file named `javac`. Found it — launched it. Didn't find it — `command not found`.
See the list (Mac / WSL):

```
echo $PATH
```

A stew of corridors, separated by colons. Don't memorize it. Know this: **the search goes left to right**. Two `java`s in different folders — the one whose corridor comes first wins. Hence "old Java in the terminal, new Java in IDEA."

```
which javac
```

(on Mac/Linux) — which jar it will take. Empty — it will take none. In PowerShell a close gesture: `Get-Command javac`.

> **Bad idea**
>
> Don't paste from chat "put this twenty-line sheet into PATH" if you don't understand **which folder** you're adding. A wrong PATH breaks commands that already worked. First — close and reopen the terminal after the installer. Often that's enough.

3.17 Stdin, stdout, a pipe on your finger

A program, when you launched it from a terminal, has three pipes:

- **stdin** — this is where what you type flows. `Scanner` reads it.
- **stdout** — this is where it writes "ordinary" output. `System.out`, `format t`.
- **stderr** — this is where it yells errors. Often also on the screen, but it's a **different** pipe. That's why you can save output to a file and still see the errors.

```
cat note.txt
```

`cat` reads a file and shoves it into stdout. The screen caught stdout — you see the text. You can catch it with a file: `cat note.txt > copy.txt`. A pipe between programs is `|`:

```
ls | cat
```

Meaning for your finger: the left one's output became the right one's input. At work people live on this. Today it's enough to see the stick. Don't build a pipeline of ten commands "like an adult." First learn where you're standing.

> **Slow down**
>
> When a program "waits and writes nothing," it is often waiting on stdin: for you to type something. Not a second launch. Type. Or Ctrl+C, if you didn't understand **what** it's waiting for.

3.18 A letter as a number, again, with your fingers

Take Latin `A`. In ASCII that's the number 65. Little `a` is 97. Space is 32. The computer does not store "a pretty A." It stores 65, and the font on the screen draws a letter. Change the font — the number is the same.

The letter `é` in Unicode is another number (if you really need it: 233). UTF-8 packs that number into **two** bytes. Latin `e` is one. That's why a file with the word `naïve` weighs more than a file with `naive`. Not a bug. Arithmetic. The old hull stencil `МОДУЛЬ` is heavier still than `MODULE`. Same reason.

Compare in your head: "the file weighs 12 bytes" and "there are 6 letters." If it's UTF-8 and the letters are fancy — it adds up. If someone opened it as a one-byte table, there may be "more letters" or garbage: they slice two UTF-8 bytes as two **different** letters of the old table. That's mojibake in the kitchen, no information theory required.

A space and an "invisible space" from a website are different numbers. To the eye they're the same. To `equals` they aren't. When "I copied it exactly" and Java disagrees — suspect numbers, not fate.

3.19 A URL — the address on the envelope

What you paste into a browser is text too. Not button magic.

```
https://httpbin.org/get?foo=1
```

Pieces:

- `https` — the **scheme**: how to talk. `https` is HTTP plus a cipher. `http` is without. For study it's enough: "the letters before the `//` are the mail rule."
- `httpbin.org` — the host. The kitchen's name. DNS will turn it into an IP.
- `/get` — a path on **their** machine. Like a folder, only theirs. Not your `/Users`.
- `?foo=1` — the query tail. Small `name=value` pairs. Sometimes that's where sessions — and leaks — get stuffed. Don't put passwords here.

`http://localhost:8080/tasks` — scheme http, host localhost (this box), door 8080, path `/tasks`. When you get to a server — this string will stop being a spell. Right now it's enough to be able to cut it.

A browser will hide the response body and **draw**. `curl` and Java's HttpClient show the body as-is: HTML or JSON as text. That's why "pretty in the browser, a stew of tags in curl" is not broken. The stew of tags **is** the page, until someone draws it.

3.20 More gifts that look like insults

`Permission denied` — the jar is there, the chef won't allow it. Rare in your own `dev`. On `/usr/bin` — please. Don't `sudo` at every sneeze. `sudo` is "I'm the chef for five seconds." A mistake under sudo is a loud mistake.

`Address already in use` — the port door is taken. An old server is alive. Find the process, close it, or change the port. Don't reinstall the computer "just in case" as step one. You can, of course. That's like fixing a crack by rebuilding the station.
`Segmentation fault` / "the application quit unexpectedly" — the process died ruder than a Java exception. Rare in study Java. If SBCL died on recursion — you'll more often see text about a stack. Also a gift.
A red stripe in IDEA with no text — click the tab or the bottom Run panel. The same stack lives there as in the terminal. The IDE is not instead of the error. The IDE is a frame around the error.

3.21 What to do with this today

Don't install ten languages yet. Install a habit:

1. Open a terminal (Mac: Terminal; Windows: Ubuntu in WSL if you have it, otherwise at least PowerShell and a plan to turn WSL on).
2. `pwd`. See where you're standing.
3. `ls`. See what you see.
4. `cd` into some folder you recognize with your eyes (Documents, `dev`, home).
5. `pwd` and `ls` again. You just drove a computer **with text**. That's the entrance to the profession, no romance required.

Next in the book — the workshop: SBCL, JDK, Git. Recipes there. Here was the kitchen. Without a kitchen, a recipe is poetry.
If something from this chapter is still fog — that's normal. The fog lifts when you've done `cd` a hundred times. A hundred is not an exaggeration. In a week the fingers will do it themselves.

3.22 Fears you don't need to carry into the workshop

"I'll break the computer with a command." Study `ls`, `cd`, `pwd`, `mkdir`, `echo` don't break anything. `rm -rf` with the root does, and the habit of living under `sudo`. We don't go there. If you're scared — work in `dev/module-cabin`, not in system folders.
"The terminal is for hackers." The terminal is for people too lazy to draw a button for every recipe. Half the profession is text. The black background is makeup.
"I have to understand the processor down to the byte." You don't. You have to tell the counter from the pantry and a file from a process. Processor bytes can wait until you become the strange person who likes that.
"Windows means this textbook isn't for me." It means: WSL2. After that the textbook is for you. PowerShell is a spare hatch, not the main one.
"The file vanished." Usually: another folder, another drive, another chef (Windows vs WSL). `pwd`. Search by name in Finder / File Explorer. Rarely — you really deleted it. Recycle Bin exists. On disk, not in RAM.
"Encoding is for linguists." Encoding is why `naïve` suddenly became `naÃ¯ve`. One evening of fighting this saves a month of "I have garbage in my JSON."

"The network is a cloud, that's where sites live." Sites live on machines. Machines have addresses. Addresses have doors. Your `localhost` is a machine too, just you yourself.
"An error means I'm not one of those people." An error means a sensor fired. The people who are "one of those" are the ones who read the sensor. The rest buy a course labeled "no pain" and are surprised by pain at work.

> **Station law**
>
> A computer is dumb, precise, and not cruel. If it seems to be mocking you — almost always it's the path, the encoding, or the wrong current folder. Check those three, then the machine's personality.

The cloud, by the way. It isn't the sky. It's someone else's computer, rented out, and letters reach it down the same corridor. "Saved to the cloud" — you copied a jar onto their disk. Their disk fills up too. Their disk can lie about encoding too. Less romance than on the sticker.
The clipboard (copied — pasted) is not a file. It's a chunk of RAM with a short shelf life. Pasted into an editor and didn't save to disk — there's no jar in the pantry. Switched the box off — gone from the counter too. Beginners lose an evening of work on this more often than on hard algorithms.

> **Slow down**
>
> Saved — pantry. Not saved — counter. Closed without a "save?" prompt — the chef sometimes asks, sometimes doesn't. Terminal `echo > file` saves at once: that's already a file. An editor with a dot in the tab name — often "not saved." Watch the dot.

Now you can go to the workshop. If your hands haven't done `pwd` yet — do it, then read about SBCL. Otherwise the workshop becomes a lecture again, and we just didn't give a lecture.

3.23 A cheat sheet for the cabin wall

Don't learn it like a poem. Look when you forget.

What for	Mac / WSL	PowerShell, if you really must
Where am I	`pwd`	`pwd` or `Get-Location`
What's here	`ls` / `ls -la`	`ls` or `dir`
Go	`cd folder` / `cd ..` / `cd ~`	same; `cd ~` sometimes sulks
New crate	`mkdir name`	`mkdir name` or `New-Item -ItemType Directory`
Show text	`cat file`	`Get-Content file` / `cat`
Path to a command	`which javac`	`Get-Command javac`
Stop a program	Ctrl+C	Ctrl+C
Leave SBCL	`(quit)` / Ctrl+D	`(quit)`
Compile Java	`javac Hello.java`	same, if PATH is alive
Run Java	`java Hello`	same

Lisp right now	`sbcl` then `(+ 40 40)`	same, if SBCL is installed

If the right-hand cell annoys you — that's a sign. Don't become a dialect translator in week one. WSL2.

Printing the cheat sheet is not shameful. Pretending you "just remember" `pwd` and hunting a file in the neighboring cabin for half an hour is. Station MODULE already has mechanics like that. Their mugs sit in the galley. The letter on the mug has worn off.

Once more, short, before the quests: the box obeys, stupidly; the cook is the CPU; the counter is RAM; the pantry is the disk; the recipe is a file; the stew on the burner is a process; the black window is how you talk to the chef in text; `cd` changes **where** you're standing; a compiler translates ahead of time, a REPL — at the table; letters are numbers; the network is mail; an error is a sensor.

If that paragraph already reads like home — the chapter worked. If not — don't cram it. Go back to the word that's still foreign and reread **that** compartment. The station does not exam you on a retelling. The station exams you on `pwd`.

The quests below are ten minutes, not a night. Do them before the workshop. Otherwise you'll start installing a JDK without being able to say which folder you're standing in. That's like looking for duct tape without opening the hatch.

Three quests. Not ten. If you want heroics — heroics in `cd` back and forth until the fingers remember.

Answers live in the answers chapter, like the other quests. Yourself first. Really.

Let's go.

Quest 0b.1 · terminal

Open a terminal (Mac / WSL Ubuntu / PowerShell if you must). Create a folder `module-cabin`, go into it, create a text file `note.txt` with one line `station MODULE`. With commands, show: where you are (`pwd` or equivalent), what's in the folder (`ls` or `dir`), the file contents (`cat` / `Get-Content`). Write into `note.txt` **as another line** the full path that `pwd` printed. If the path isn't what you thought — that **is** the quest, not a nitpick.

Quest 0b.2 · kitchen

On paper or in the same `note.txt`: for the action "run `Hello.java`," write who the cook is, what's on the counter, what's in the pantry. Three sentences, not an essay. If you wrote "the computer thinks" — erase it and replace with "the CPU executes instructions, RAM holds the current stuff, the disk stores the file."

Quest 0b.3 · text

Type the word `café` in a file. Save UTF-8. Open it in the same editor. Then deliberately open it with a **different** encoding, if the editor can (VS Code: bottom corner → Reopen with Encoding

→ something like Western Windows 1252). Photograph the garbage with your soul. Switch back to UTF-8. Write in `note.txt` why the letters "broke" even though you didn't erase them.

Sunday sabotage

Draw boxes: your machine, "their" machine, envelopes between them. Label `localhost` on your box. That's all the network you need so the word HTTP doesn't scare you later.

Chapter

4 The workshop

4.1 Lesson 0. Workbench, parentheses, and the green button

Lisp 40 min · Java 120 min · a little theory, then until it runs

Until you have a workbench, this book is just text you can't bite. Today we install tools and write one tiny program in each language. If something won't install — congratulations, you're already at work: read the **whole** error message, not just the red word at the front.
This lesson can eat an entire evening. That's normal. Installing a JDK the first time is like finding a hatch in the dark: you swear, then you find it, then you wonder why the hatch was drawn on the map. Later evenings are shorter. This one is the cover charge.

> **Today on station · 2 h 40 min**
> Lisp: install SBCL, open the REPL, poke some plus signs.
> Java: JDK 21, IntelliJ IDEA Community (or VS Code), "hello."
> Git: name, email, two repositories, push to GitHub.

4.1.1 First, about Windows, Mac, and this WSL of yours

Further on, the book's commands look like Linux: `sbcl`, `java`, `curl`, `git`. On a Mac they really are that. On Windows there's a fork.

> **Mac, Windows, WSL**
> **Mac.** Install Homebrew if you don't have it: https://brew.sh
> Then everything through `brew install …`. The terminal is the ordinary one. Spotlight, the word Terminal, a black window. Not "the Windows command line" — you don't have one, don't look.
> **Windows — I recommend WSL2.** That's Linux inside Windows, no second machine. Win+S, "Turn Windows features on," check **Windows Subsystem for Linux**, reboot. Then from the Microsoft Store — Ubuntu. A black window opens, asks for a name and a password — that's Linux already. After that, copy almost every command from the textbook **there**. Update packages once:
>
> ```
> sudo apt update && sudo apt upgrade -y
> ```
>
> The password at `sudo` does not print as asterisks. That's not a bug. Just type blind and hit Enter. Linux is secretive like that.

> IntelliJ can stay on Windows: File → Open → `\\wsl$\Ubuntu\home\<you>\…` . Or live entirely in Ubuntu, whichever is easier. Docker on Windows installs as Docker Desktop and asks you to turn on WSL2 — agree, don't argue.
> **Windows without WSL** is also possible: SBCL and the JDK install with installers, Git is "Git for Windows," a lot of PowerShell looks similar. But paths with backslashes, `mvnw.cmd` instead of `./mvnw` , and `curl` is sometimes called `curl.exe` . If you get lost — WSL anyway. Seriously.

In the text, unless it says otherwise, commands are for **Mac and Ubuntu in WSL**. Native Windows is in appendix F.

> **Bad idea**
> Don't install "also Docker, also Postgres, also Android Studio" today. Three things: SBCL, JDK, Git. The rest — when a chapter asks. Greed for installers is the path to stew in `PATH` and to an evening that ended with Hello still unprinted.

4.1.2 Lisp: SBCL and a beast named REPL

Common Lisp is a language. SBCL is one of its implementations — decent, fast, no extra mysticism. There are many Lisp languages, like soups. We need one pot.
Mac:

```
brew install sbcl
```

Ubuntu in WSL:

```
sudo apt install -y sbcl
```

Windows without WSL: the installer from https://www.sbcl.org/platform-table.html — grab Windows, install, SBCL will show up in the Start menu.
Launch is the same everywhere:

```
sbcl
```

A `*` will crawl out. That's a REPL: it read, it computed, it printed, it waits again. Like a calculator you can explain games to. Exit: `(quit)` . On a Mac and in WSL, also Ctrl+D. On native Windows Ctrl+D may not work — type `(quit)` and don't play hero.

> **Slow down**
> If after `sbcl` it said `command not found` / `is not recognized as an internal or external command` — the program isn't on PATH. On a Mac, open a **new** terminal window after brew. In WSL check `which sbcl` . On Windows without WSL — restart the terminal, sometimes the whole computer: the installer wrote the path, and the old window doesn't know. That's a classic. It's only embarrassing the first two times.

Type:

```
(+ 2 3)
```

`5`. The operator is **in front**, then the arguments, all in parentheses. Looks like a mockery of math, but the rule is always the same: "do this thing with these things."

> **What just happened**
>
> You wrote a list. The first element of the list is **what to do**. The rest are **with what**. Lisp does not look for a plus between the two and the three, the way school does. It looks for the plus at the front, the way a station looks for the captain on the bridge, not "well, anybody in the corridor."

Now break it. Type `(+ 2 3` without the closing parenthesis and hit Enter. SBCL will wait. It didn't hang. It's politely waiting until you finish the spell. Add `)` and Enter. Or hit Ctrl+C and start over. Remember: REPL silence after an open parenthesis is not death, it's "I'm still listening."

```
(* (+ 1 2) 10)
```

First one plus two, then times ten. Parentheses are a tree, not decoration. There will be a lot of them. Later you'll start **not noticing** them, and that's the moment Lisp worked.
One more useful breakage:

```
(1 2 3)
```

You'll get something like `illegal function call`. Lisp decided `1` is a function name. The number one took offense: it's a number, not a spell. To make a list **data**, not a call, you put a quote: `'(1 2 3)`. Remember the gesture. It comes back in week two and never leaves.
A string:

```
"station MODULE"
```

Quotes — like in human language. Without quotes, `station` is a symbol, and Lisp will go looking for what function or variable that is, and not find it.
In Lisp, "yes" is written `t`, "no" is `nil`. The empty list is `nil` too. One `nil` for two roles. In Lisp that's fine. In life too, honestly. We won't say **true** and **false** after this: it sounds like a logic exam, and on the screen it's still `t` and `nil`.

```
(= 2 2)     ; T
(= 2 3)     ; NIL
(not nil)   ; T
```

A semicolon is a comment to the end of the line. Lisp doesn't eat it. You eat it with your eyes.

> **Bad idea**
>
> Don't install ten Lisps "just in case." SBCL is enough. Emacs with SLIME is a sweet hole you'll fall into yourself when you're ripe. Today a black window is enough. Clojure, Racket,

> Scheme — neighboring universes. Later. If you install everything at once, the week's quest will be "which of the four REPLs did I open."

A file `hello.lisp` in the folder `lisp-experiments`:

```
(format t "station MODULE on the line~%")
```

Load it into an already-open SBCL:

```
(load "hello.lisp")
```

The path has to match **where** you launched SBCL from. If the file is in another folder — either `cd` there before `sbcl`, or a full path. "I have the file, Lisp can't see it" is almost always "you're in the wrong folder." There is no `(pwd)` in SBCL the way there is in bash; in the terminal **before** launch: `pwd`.

4.1.3 Java: wordy, but it feeds you

Check:

```
java -version
javac -version
```

You want 21 (seventeen is still alive, but 21 is better). `java` — run what's already cooked. `javac` — cook it. If you have `java` and no `javac`, you installed a JRE instead of a JDK. Fetch the JDK. On the station that's like getting a kettle with no coil.

Mac: `brew install openjdk@21` and, following brew's hint, put it on `PATH`. Often brew itself prints two lines of `sudo ln` or `echo '…' >> ~/.zshrc`. Copy them. Don't play hero with "I'll remember later."

WSL: `sudo apt install -y openjdk-21-jdk` — if the package isn't there, grab Temurin: https://adoptium.net (there's Linux and Windows).

Windows without WSL: the same Adoptium, MSI, check "add to PATH." After install, close PowerShell and open it again.

IntelliJ IDEA Community — from https://www.jetbrains.com/idea/download/ , there's Mac and Windows. Free. The green arrow runs `main`. VS Code works too, but then install the Extension Pack for Java. You can argue which is nobler after you have an offer.

Directory `java-basics`, file `Hello.java`:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("station MODULE on the line");
    }
}
```

Filename = class name. `Hello.java` and `class Hello`. If the file is `hello.java` and the class is `Hello`, on some systems you'll get lucky, on some you won't. Don't play luck. Capital letter, like Java in its passport.

From the terminal (Mac / WSL):

```
javac Hello.java
java Hello
```

On native Windows, the same, if `javac` is on PATH. If "is not recognized as a command" — PATH didn't stick, close and reopen the terminal, or WSL after all.

> **Slow down**
>
> `javac Hello.java` makes a file `Hello.class`. That's not text anymore, that's bytecode for the JVM. `java Hello` — **without** `.class` in the command. Write `java Hello.class` — it'll take offense. Write `java Hello.java` on old JDKs — offense too. On 21 you can sometimes `java Hello.java` in one go, without `javac`. You can. Understanding that inside there are two steps, you still need: at work the build will be Maven, and there are more steps.

Java wants a class, `main`, and semicolons. Next to Lisp that's like filling out a form after talking to a friend. The jobs, though, look like this.
Break it on purpose. Remove the semicolon after `println(...)`. `javac` will write `';' expected` and a line number. That's the friendliest yell on the station. Then put the semicolon back. Then put class `Hello` in a file `Bye.java` and see what it says. Then put it back. The goal is not a perfect file, it's a reflex: you read the red, you don't close it.
In IntelliJ: New Project → New Java Class → `Hello` → green arrow. If there's no arrow — click inside `main` with the mouse, a green triangle appears in the gutter with the line numbers. If "SDK not specified" — File → Project Structure → SDK → 21. The IDE is not a telepath: you have to tell it where the JDK is.

4.1.4 Git today, not "when it looks pretty"

Git is memory with versions. Not "the cloud." The cloud is GitHub, a separate company with a website. Git works without the internet. GitHub is for hanging the memory on a fence where a person with a job opening will look.

```
git config --global user.name "Vasya Module"
git config --global user.email "vasya@example.com"
```

The name is what to call you in the commit history. The email is the one on GitHub, otherwise the commit stars won't stick to the account. That's not a nitpick.
Git: on a Mac it's often already there, otherwise `brew install git`. In WSL: `sudo apt install -y git`. On Windows: https://git-scm.com/download/win — and then **either** Git Bash, **or** WSL again, where git is already the Linux one.
On github.com, make an account and an empty repository `java-basics`. No README checkbox if you're going to push an already-existing folder — otherwise GitHub creates a commit, you have yours, and they fight. Breaking up that fight later is boring.
Then:

```
mkdir java-basics
cd java-basics
git init
```

A `.gitignore` file (you can make it in an editor if PowerShell argues with `echo`):

```
*.class
.idea/
```

`*.class` — don't drag bytecode into git, it cooks from `.java`. `.idea/` — don't drag the IDE's settings, they're yours, not the station's.

```
git add .
git commit -m "Hello from week 0"
```

If git yells `Please tell me who you are` — you skipped `git config`. Do it and commit again.
Hook up the remote and `git push -u origin main`. GitHub tokens on Windows sometimes pop a window — agree, that's normal. On a Mac a browser may open. In WSL it sometimes asks for a Personal Access Token: GitHub → Settings → Developer settings → a token with the `repo` right. Paste it as the password. No asterisks again.

> **Slow down**
> The branch may be called `master` instead of `main`. That's not you being dumb, that's git being old. Either `git branch -M main` before the push, or look on GitHub at what they named the empty repo. The main thing — don't push into the void and don't create a second "main" branch out of confusion.

If `push` was rejected because GitHub already has a README and you have no common ancestor:

```
git pull origin main --rebase
git push -u origin main
```

If that scares you — delete the empty repo on the site, make a new one **without** a README, push again. For week 0 that's more honest than an hour of merge rituals.
A second repository `lisp-experiments` with a file `hello.lisp`. Even one line. Two fences. Java on one, parentheses on the other. Don't dump everything in a pile called "school": in a month you won't find where the sensor is and where the server is.

> **Hide it on GitHub**
> Two repositories, a commit each. A README of three sentences: who you are, what this is, how to run it. "Run" = a concrete command, not "well, in IDEA."

A README for `java-basics` right now:

```
java-basics
Programs from month one.
```

```
Run:
javac Hello.java
java Hello
```

Three lines. Already better than empty. You'll add more later.

4.1.5 How to talk to an error

A compiler and a REPL are nasty teachers. They're almost always right. Read the **first** error from the top. The line matters more than the feeling "everything died." If you pasted the error into search — paste it without your filename and without the pile of quotes you invented yourself.

> **What just happened**
>
> An error is not a grade. The grade comes at the interview, and even then not for how much red you got, but for whether you can read the red. Today's drill: broke it, read it, fixed it, alive.

Frequent guests of week 0:

- `command not found` — no program, or no PATH.
- `';' expected` — Java wants a semicolon. Look at the line it named, and the line **above**: sometimes the compiler lies by one line.
- `cannot find symbol` — a typo in a name. `System.out` is not `system.out`. Java remembers case like an offended archivist.
- `illegal function call` in Lisp — a list without a quote that wanted to be data.
- `end of file on #<stream>` — a parenthesis not closed by the end of the file.

Don't tick ten "fix everything" checkboxes in the IDE without looking. One checkbox can rename the wrong thing. Read.

4.1.6 A map of the evening, if everything went sideways

Stuck on SBCL — leave Java for later, get Lisp to `(+ 2 3)`. Stuck on Java — leave Git, get `Hello` working. Git without Hello still has nothing to push except emptiness.
Nothing works — write three facts on paper: (1) which command, (2) the whole error, (3) which folder you were in. With that paper you can ask a neighbor, an AI, or a forum. Without the paper you'll get "well show the error," and you'll be ashamed twice.

Quest 0.L1 · Lisp

In the REPL: the sum from 1 to 5 with one `+`; the product of 2, 3, and 4; "ten minus three, then times two." Write the answers as a comment in `hello.lisp`.

Quest 0.J1 · Java

`Hello` prints your name and today's date (as a plain string for now). Run it from the IDE **and** from the terminal. If one of the two didn't work — that's the quest.

Quest 0.G1 · Git

A second commit, a push, open the page on github.com and see your files. If you didn't see them — not "later," fix it now.

Quest 0.L2 · Lisp

Break it on purpose: call `(+ 2 "station")`. Read the error out loud. Then call `(concatenate 'string "MODULE-" "1")`. The difference: plus adds numbers, strings glue another way. Write both errors as a comment — that's a station log, not shame.

Quest 0.J2 · Java

Add a second print line: how many compartments on the station (any number). Compile. Then change the number, **don't** compile, run the old `java Hello`. You'll see the old number. That's what `javac` is for. Then compile again.

Sunday sabotage

Draw on paper what happens when you hit Run. Where `java` comes from, where the `.class` goes. You don't have to guess down to the byte. You do have to start suspecting that the button isn't magic.

Chapter

5 Month 1. The computer, more or less, obeys

By the end of week four you have your own Java console program — not a copied tutorial chapter, **yours** — and it lives on GitHub. There is no Spring, and that is a holiday. Lisp: names, "if", functions, the first game about a station that is always this close to falling apart.
Station MODULE has been hanging in orbit for about a hundred years, held together by duct tape, optimism, and three sensors that still work. You are the new mechanic. Congratulations. Spacesuit's in the locker. Someone already drank the coffee.
The first month is not "learn programming." It's the month the computer stops being furniture and becomes a creature you can ask questions — and it sometimes answers. Sometimes it yells. Sometimes it stays quiet, and that's worse.
Four watches. After each one — a commit, even a crooked one. Especially a crooked one.

> **Today on station · 2 h 40 min**
> Lisp: forty minutes in the REPL, until the parentheses stop looking like a joke at your expense.
> Java: types, `if`, lists, objects, a file. Console, not a browser.
> By the end of the month: a TODO that survives a power-off, and a README someone else can use to run it.

5.1 Lesson 1. How to name a thing and how to tell it what to do

Lisp 40 min · Java 120 min · a little theory, then until it runs

> **Today on station · 2 h 40 min**
> Lisp: a symbol, `let`, `defun`, `if`, `cond`, ask a human.
> Java: `int` / `String` / `boolean`, `if` / `else if`, `for`, `Scanner`.
> Today's sensor is reactor energy. If it's zero, you can stop reading the textbook: the station will get very calm and very cold.

First watch. The hold smells like burnt duct tape and something that used to be coffee. On the wall a sensor blinks yellow — not because it's smart, because there's one bulb for every mode. The captain on the intercom said only: "we got energy?" You open the REPL like a toolbox.

5.1.1 Lisp: symbols, `defun`, `if`

In Lisp, names are **symbols**, almost like tags on crates in the hold. Not the string `"energy"`, not a memory cell with a sticker. A symbol. You can put it in a list, compare it, pass it around. For now, simple: hang a tag, put a number on it.
Globally (fine for our games) — `defparameter`. Locally, "only here" — `let`.
Open SBCL. On a Mac and in WSL that's just `sbcl`. On native Windows — the shortcut, or the same word in a terminal if PATH is alive. The asterisk `*` is the prompt, not multiplication.
Type this, **including the parentheses**:

```
(+ 2 3)
```

You should get `5`. The operator comes first. That isn't a joke and it isn't a Polish-notation exam — Lisp is always like this: "do this thing to that thing."
Now a tag:

```
(defparameter *energy* 100)
```

SBCL will answer something like `*ENERGY*`. It yells in caps because inside, symbols live without caring about case: `*energy*` and `*ENERGY*` are the same tag. The asterisks aren't syntax. They're a Lisper habit of shouting: "this is global, don't step on it." Like a wet-floor sign.

> **What just happened**
> You didn't "create a variable" in the Java sense. You hung the value `100` on the symbol `*energy*`. Ask the symbol — you get a hundred. Hang something else — the hundred is gone. There's no `int` on the tag. Lisp doesn't watch whether you put a bolt in there or a nut. Yet.

Read it:

```
*energy*
```

`100`. Without the asterisks it won't work: the symbol is literally called `*energy*`. Type `energy` and SBCL will say the variable is unbound. Translation: "there's no tag `energy` in this hold. There is `*energy*`."
Locally, not for the whole station:

```
(let ((oxygen 40)
      (power 12))
  (+ oxygen power))
```

> **Slow down**
> `let` is "make a pocket for the length of this parenthesis." Inside the pocket, two names: `oxygen` is 40, `power` is 12. The body is `(+ oxygen power)`. The result of the whole `let` is whatever the body returned, which is 52. Outside, `oxygen` means nobody again. After the `let`, try typing

> just `oxygen`. Error. The pocket collapsed. That isn't a bug. That's manners: don't leave nuts on the corridor floor.

Break it on purpose. Type:

```
(let ((oxygen 40)
      (power 12))
  (+ oxygen power)
```

You're missing a closing parenthesis. SBCL will wait. The cursor blinks like a sensor nobody told to stop. Finish with `)` or Ctrl+C / get out of the mess with `(abort)` if you're already miserable. Lesson: parentheses are a tree, not decoration. Count them. Later you'll stop counting with your eyes and start feeling them.
Break it again:

```
(let (oxygen 40)
  oxygen)
```

It will probably yell. `let`'s argument is a **list of bindings**, and each binding is its own little list `(name value)`. Without the inner parentheses Lisp has no idea where the name ended and the number began. It's like writing "nuts 40" on a crate with no divider: the clerk doesn't know if that's forty nuts or a nut named Forty.
Correct again:

```
(let ((oxygen 40))
  oxygen)
```

`; => 40`
A function is a named spell:

```
(defun report (energy)
  (if (>= energy 50)
      "reactor stable"
      "low energy"))
```

> **Slow down**
>
> `defun` literally unpacks as define function. Then the name `report`. Then in parentheses, the parameter list — here, one: `energy`. This is not the global `*energy*`. This is a local tag: "whatever they handed me." The body is one `if`. `if` has three slots: the question, what to do if yes, what to do if no. The question: `(>= energy 50)` — is energy greater than or equal to fifty? If yes — the string about a stable reactor. If no — the string about low energy. The whole `if` **returns** a string. The function returns whatever the `if` returned. No printing. Yet.

Call it:

```
(report 80)
(report 10)
(report 50)
```

Should be: stable, low, stable. Fifty passes because it's `>=`, not `>`. The kind of tiny thing stations turn into accidents.

> **What just happened**
>
> `(report 80)` is a **call**. Function name in front, argument after, everything in parentheses. If you write `report(80)` like Java, Lisp will decide that `report` is a variable and `(80)` is a separate list it has to evaluate. Hurt feelings all around. Parentheses on the outside, name on the inside, arguments next to it. Always.

Forget the "no" branch of `if`:

```
(defun report-broken (energy)
  (if (>= energy 50)
      "reactor stable"))

(report-broken 80)
(report-broken 10)
```

First call — a string. Second — `nil`. The station stays quiet. Sometimes that's worse than a siren. `nil` in Lisp is "no," "empty," "didn't work," "there is no list." One nothing wearing several hats. We don't say "false": the screen still prints `nil`.
Several rungs — `cond`:

```
(defun status (n)
  (cond
    ((>= n 80) 'green)
    ((>= n 40) 'yellow)
    (t 'red)))
```

> **Slow down**
>
> `cond` is a ladder of questions. Each floor: `(question answer)`. The first one that comes back non-`nil` wins; the rest don't even get a look. `t` at the end is "in every other case," the cosmic else. `t` in Lisp is "yes," the universal one. Not "true." The screen says `t`. `'green` is not a string. It's a **symbol**, a tag. The quote means: don't evaluate, pocket it as-is. Without the quote, Lisp goes looking for a function or variable called `green` and doesn't find one.

Check:

```
(status 90)
(status 40)
(status 0)
```

`GREEN`, `YELLOW`, `RED`. SBCL yelling caps again. Same symbols.

Order of the rungs matters. Put `(t 'red)` first and everything is always red. A ladder whose first step is the floor. Try it, admire it, put it back.
Compare strings with `(string= a b)`. Numbers: `=`, `<`, `>`, `<=`, `>=`. The rest can wait. Life is long.
Three working examples, not one.
Example A. Docking:

```
(defun dock-comment (speed)
  (if (< speed 5)
      "you can breathe"
      "dent incoming"))
```

Example B. Oxygen in a compartment — not energy, same gesture:

```
(defun oxygen-ok (percent)
  (cond
    ((>= percent 18) 'ok)
    ((>= percent 12) 'masks)
    (t 'evacuate)))
```

Example C. Two sensors at once:

```
(defun can-open-hatch (energy oxygen)
  (if (and (>= energy 20) (>= oxygen 18))
      'open
      'wait))
```

`and` — both. `or` — one is enough. `(not x)` — flip it. `nil` flips to `t`, everything else to `nil`. Yes, "everything else" in Lisp counts as "yes." Zero is "yes" too, unlike some languages that treat zero as a nobody. `(if 0 'yes 'no)` returns `YES`. Learn it on day one so you don't get surprised at three in the morning.
Ask a human in the REPL:

```
(defun ask-number ()
  (format t "Enter a number: ")
  (finish-output)
  (parse-integer (read-line) :junk-allowed t))
```

> **What just happened**
>
> `format t` yells into the terminal. `t` here is "standard output," not "yes," even though it's the same symbol. Life is complicated. `finish-output` — shove the letters out of the pipe **now**, don't hoard them until end of line. Without it, on some systems the prompt shows up **after** you've already typed something. Hilarious. Once. `read-line` listens until Enter and returns a string. `parse-integer` turns letters into a number. `:junk-allowed t` means "if there's junk after the digits, don't crash, take what you can." Type `12abc` — you get 12. Type `asdf` — you get `nil`. A shrug.

Call `(ask-number)`, type `42`, get `42`. Type `hello`, get `nil`. Then write a wrapper that doesn't trust `nil`:

```
(defun ask-energy ()
  (let ((n (ask-number)))
    (if (numberp n)
        n
        (progn
          (format t "that's not a number~%")
          (ask-energy)))))
```

`numberp` — "is this a number?" The `p` tail on Lisper names is predicate, a yes/no question. `progn` — do several things in a row, return the last. Recursion "ask again" is a spoiler for lesson four, but for input it's honest: humans mash keys. It's their hobby.

> **Bad idea**
>
> Common parenthesis deaths today: `(defun report energy ...)` — parameters without their own parentheses. Need `(energy)`. `(if >= energy 50 ...)` — `>=` is a function, you call it with parentheses too: `(>= energy 50)`. An extra `)` at the end of `defun` — SBCL starts reading the **next** line as garbage. Look at **which** parenthesis it hates, not all of them at once.

One more REPL ritual. Type this on purpose:

```
(1 2 3)
```

An error like `undefined function: 1`. Lisp decided this was a **call**: function `1`, arguments `2` and `3`. One takes offense. To keep a list a list, you need a quote: `'(1 2 3)`. That's lesson 2, but the error will surface today if your hand adds parentheses by itself. Now you'll recognize it.

5.1.2 Java: types that watch so you don't mix a bolt with a nut

In Java every box says what's inside. The compiler is a grouchy warehouse clerk. Not a REPL in the same sense: you write a file, compile, run, get yelled at. On the plus side, the clerk catches some of the stupidity **before** flight, not during docking.

```
int energy = 100;
String name = "MODULE";
boolean ok = energy >= 50;
```

`int` — a whole number. `String` — text, capital S, because it's a class, not "just a digit." `boolean` — `true` / `false`. We don't translate those into "truth/falsehood" in code: the file will say `true` and `false`, and so will the job posts.

> **Slow down**
>
> The line `boolean ok = energy >= 50;` is not an "if." It's an **expression** that itself becomes `true` or `false`, and you put that in the box `ok`. Later you can ask `if (ok)`. Or go straight to `if (energy >= 50)`. Both honest. The first is handy when you'll drag the condition somewhere else later.

```
if (ok) {
    System.out.println("reactor stable");
} else {
    System.out.println("low energy");
}
```

Curly braces are a pen for code. The semicolon is "the sentence ended." Without it Java doesn't start the next sentence. It yells.

The class `Energy` lives in the file `Energy.java`, or Java gets offended. It does that a lot.

Input from the ground:

```
import java.util.Scanner;

public class Energy {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("Energy (0–100): ");
        int energy = in.nextInt();
        if (energy >= 80) {
            System.out.println("green");
        } else if (energy >= 40) {
            System.out.println("yellow");
        } else {
            System.out.println("red");
        }
    }
}
```

What just happened

`import` — "clerk, fetch the `Scanner` crate from the `java.util` room." Without the import the compiler doesn't know what `Scanner` is. `public class Energy` — the blueprint. `main` — the door that the command `java Energy` knocks on. `new Scanner(System.in)` — plug an ear into the keyboard. `nextInt()` — wait for a whole number. `else if` — the next rung, like `cond`. Order matters again: 80 first, then 40, then everything else.

Mac, Windows, WSL

Mac / WSL. In the folder with the file:

```
javac Energy.java
java Energy
```

Windows without WSL. Same, if `javac` is on PATH. If you get "is not recognized as an internal or external command" — JDK never got written into PATH. Close the terminal, open it again. Still no — install Temurin 21 with the PATH checkbox, or go to WSL, where `sudo apt install -y openjdk-21-jdk`.

A loop, so it ticks:

```
for (int i = 0; i < 3; i++) {
    System.out.println("tick " + i);
}
```

It will print `tick 0`, `tick 1`, `tick 2`. From zero, like arrays, not like people. `i++` — add one after the pass. Three parts in the `for` parentheses: where to start, how long to spin, what to do after each round.

Three more examples, already not about energy.

Example A. Average of three temperatures — the same gesture as the quest, but with your eyes first:

```
int a = 18, b = 21, c = 7;
double avg = (a + b + c) / 3.0;
```

`3.0` so the division is fractional. `(a + b + c) / 3` on integers chops the tail. Java's rule: integers on integers — integer. The clerk doesn't yell here. He **silently** steals the fraction. That's worse than an error.

Example B. Compare strings with `.equals`, not `==`:

```
String cmd = "list";
if (cmd.equals("list")) {
    System.out.println("listing");
}
```

`==` on objects asks "is this the same box in memory?" not "do the boxes say the same thing?" Sometimes it works by accident. Then it stops. Then you'll spend three hours blaming the universe.

Example C. Several commands in `main` — already smells like lesson 2, but let `while` flash by:

```
int n = 3;
while (n > 0) {
    System.out.println("left " + n);
    n = n - 1;
}
```

While `n > 0` — print and shrink. Forget to shrink — infinite loop, the terminal like a sensor nobody told to stop. Ctrl+C.

5.1.3 If it broke: Java yells, and that's normal

The compiler writes a lot. Read the **first** error, not the last. Often the first one births the rest.

Class doesn't match the file. In `Energy.java` you wrote `public class Energia`. It yells: class Energia is public, should be declared in a file named Energia.java. Rename either the file or the class. They are one creature.

Forgot the semicolon. The line `int energy = 100` without `;` — ';' expected. Java cannot guess where the thought ended. Put it there.

Forgot the import. `Scanner` without `import java.util.Scanner;` — cannot find symbol. A symbol is a name that isn't in scope. Not "magic broke." You're missing either an import, or you typo'd: `scanner` lowercase is already a different beast.

Main not found. You ran `java Energy`, but the method is called `Main` or it's missing `String[] args`. The door has to look like this: `public static void main(String[] args)`. Otherwise the JVM stands in the corridor and doesn't know where to knock.

Brace. `reached end of file while parsing` — you're missing a `}`. Count. In IntelliJ the highlight helps: click a brace, the other one lights up. If it didn't light up — it isn't there.

You typed letters into `nextInt()`. Not compilation anymore, already flight: `InputMismatchException`. Scanner waited for a number, got "pshh." We'll catch it in lesson 4. Today restart and type digits. Or read a string and parse it yourself — but that's a step later.

> **Bad idea**
>
> Don't paste "the missing pieces from three websites" into a file. One class, one `main`, it ran, then change it a line at a time. You don't assemble a station from three different blueprints on your knee. Well, you do, but then it doesn't dock.

Three working eye-runs before the quests.

1. Set energy to 80, 40, 39, 0. Four answers: green, yellow, red, red. If 40 gives red — you wrote `>`, not `>=`.
2. Swap the thresholds: `>= 40` first, then `>= 80`. Type 90. You'll get yellow, because 90 is also `>= 40`, and the second rung never gets a look. Order is not aesthetics.
3. Print `energy` before the `if` and after. The number doesn't change just because you talked about it. `if` doesn't spend energy. Spending is lesson 3.

> **Hide it on GitHub**
>
> In `lisp-experiments` — `module-01.lisp`. In `java-basics` — `Energy`. A commit like `week1: energy status`.

Quest 1.L1 · Lisp

`dock-ok`: docking speed. Under 5 — `"soft"`, otherwise `"too fast"`. Check on 3 and on 9. The station asks you not to ram the airlock.

Quest 1.L2 · Lisp

`airlock`: two pressures, inside and outside. Equal — `'open`, otherwise `'sealed`. Compare with `=`. Opening the hatch because it's "almost equal" is a bad comedy.

Quest 1.L3 · Lisp

`lamp`: energy 0..100. Return `'green` / `'yellow` / `'red` with the same thresholds as `status` in the text (80 and 40). Call it on 100, 80, 79, 40, 0. Five calls, five answers, in a comment next to them. If even one lies — thresholds or the order of `cond`.

Quest 1.J1 · Java

Three numbers — compartment temperatures. Print the average. If any one is below zero — also the word `ALARM`. Compute the average through `3.0`, or Java will chop the fraction as uninteresting.

Quest 1.J2 · Java

A game: the machine picks 1..10 (`Math.random()`), you guess once. "right" / "low" / "high". Almost Land of Lisp, only meaner, because you get one try.

Quest 1.J3 · Java

Energy from the keyboard, until the human gives a whole number from 0 to 100 inclusive. Letters and numbers out of range — prompt again, don't crash. For now `while (true)` and `break` when the number is honest is fine. `Scanner.nextLine()` plus `Integer.parseInt` is easier to catch than mixing `nextInt` with strings. If it broke on letters — that **is** the quest, not "later."

SICP · open only if it itches

Did it grab you that `if` in Lisp **returns** a value, instead of "doing a thing"? SICP is about expressions. Not now — when the itch won't let you sleep.

Sunday sabotage

In the browser: `view-source:` and any site. That isn't Java. That's text someone executes later. Nice to feel yourself backstage.

The captain on the intercom again: "so how's energy?" You don't say "fine." You say `green`. The station starts to suspect you can read sensors. Someone still drank the coffee.

5.2 Lesson 2. Boxes, lists, and an endless corridor

Lisp 40 min · Java 120 min · a little theory, then until it runs

Today on station · 2 h 40 min

Lisp: `quote`, `first` / `rest`, `cons`, `length`, `member`, `dolist`, a little `loop`.

Java: an array, `ArrayList`, `HashMap`, a loop over a collection.

Today's quest — a to-do list in the console. After you power off, everything is forgotten. Files — after the next lesson. The plot thickens.

Second watch. The station corridor isn't a metaphor, it's literally a corridor: airlock, then a tube, then the reactor, then the compartment that used to be a greenhouse and is now crates. On a scrap of paper the last mechanic left a list of rooms, written three times, three different ways. You decide the computer can at least be trusted with the order.

5.2.1 Lisp: lists — pretty much everything, actually

You write a list in parentheses. If you don't put a quote, Lisp decides it's a **call** and tries to call `1` as a function. `1` takes offense. You already saw this yesterday. Today it's a tool.

```
'(1 2 3)
'(antenna corridor reactor)
```

> **What just happened**
>
> The quote `'` is sugar for `(quote ...)`. `(quote (1 2 3))` and `'(1 2 3)` are the same: "here is data, don't touch it as code." Without the quote, `(antenna corridor reactor)` means: call the function `antenna` with arguments `corridor` and `reactor`. There is no function `antenna`. Error. With the quote it's just three tags in one box.

`(list 1 2 3)` — the same, but it evaluates the arguments first:

```
(list 1 (+ 1 1) 3)
; (1 2 3)

'(1 (+ 1 1) 3)
; (1 (+ 1 1) 3)
```

In the second case `(+ 1 1)` just sits there as a list. Not a two. Quote freezes **everything**. `list` evaluates the pieces, then assembles.
Head and tail. A list, like a comet:

```
(first '(a b c))  ; A
(rest  '(a b c))  ; (B C)
(first '())       ; NIL
(rest  '(a))      ; NIL
```

> **Slow down**
>
> A list is not "an array of three cells." It's a pair: a head and **the rest of the list**. `(A B C)` is built as "A, and also (B C)." `(B C)` is "B, and also (C)." `(C)` is "C, and also empty." Empty is `nil` or `'()`. One nothing for everybody. That's why `(rest (rest (rest '(a b c))))` gives `NIL`. Took the head off three times — nobody left. The grandpas said `car` and `cdr`. Those are the same `first` and `rest`, they just sound like a breakdown. Old texts are `car` / `cdr` everywhere. We write `first` / `rest` so it doesn't look like the station was assembled in 1959. Although MODULE, honestly, might have been.

Type the chain, don't be lazy:

```
(defparameter *rooms* '(airlock corridor reactor))
(first *rooms*)
(rest *rooms*)
(first (rest *rooms*))
(first (rest (rest *rooms*)))
```

`AIRLOCK`, `(CORRIDOR REACTOR)`, `CORRIDOR`, `REACTOR`. You just walked the corridor by hand. That's all of Lisp's poetry.
Stick an element on the front:

```
(cons 'airlock '(corridor reactor))
; (AIRLOCK CORRIDOR REACTOR)
```

`cons` is the chief constructor of the universe. Not "add at the end." At the **front**. The old list doesn't change — you get a new one. Check:

```
(defparameter *tail* '(corridor reactor))
(cons 'airlock *tail*)
*tail*
```

`*tail*` is still `(CORRIDOR REACTOR)`. `cons` didn't nail the airlock onto the old crate. It built a new one. That matters when we get to "a function doesn't wreck what you handed it."

> **What just happened**
>
> `(cons 'x nil)` gives `(X)`. One element is still a list. `(cons 'x 'y)` gives `(X . Y)` — a dotted pair, not a "real" list. A real list ends in `nil`. Don't go building dotted pairs on purpose yet. If a dot shows up in the printout — you fed `cons` something that isn't a list as the second argument. Go back, put `'()` or another list.

Length: `(length lst)`. Is there a reactor: `(member 'reactor '(corridor reactor))` — returns the tail from the find, or `nil` if somebody already hauled the reactor out.

```
(member 'reactor '(airlock corridor reactor))
; (REACTOR)
(member 'garden '(airlock corridor reactor))
; NIL
```

Not `t`. The tail. Convenient and annoying at the same time. "Found it? Then from here to the end." For a yes/no question people usually write `(not (null (member ...)))` — two negations around the find. Or `if` and live.
Walk it:

```
(dolist (room '(airlock corridor reactor))
  (format t "room: ~a~%" room))
```

`~a` — print it like a human. `~%` — a new line. `dolist` doesn't build a new list, it **does**. For printing — exactly right. Recursion we leave for lesson four; today `dolist` is enough, and this:

```
(loop for i from 1 to 5
      collect (* i i))
; (1 4 9 16 25)
```

`loop` in Common Lisp is a little language inside the language. You can love it, you can fear it. For squares — love it.
Another `loop` over a list with numbers — tomorrow's quest, today's hint:

```
(loop for room in '(airlock corridor)
      for i from 1
      do (format t "~a. ~a~%" i room))
```

Two `for`s walk together. `do` — a side job (printing). `collect` — gather values into a result list. Three examples so a list becomes a hand, not a theory.
Example A. A new room at the front of the map:

```
(defun add-room (room rooms)
  (cons room rooms))
```

Example B. The second room, if there is one:

```
(defun second-room (rooms)
  (first (rest rooms)))
```

For `'()` and `'(only)` you get `nil`. Don't crash. An empty corridor is also an answer.
Example C. Is the name there:

```
(defun known-p (room rooms)
  (not (null (member room rooms))))
```

`(known-p 'reactor '(corridor reactor))` → `T`. `(known-p 'garden '(corridor reactor))` → `NIL`.

> **Bad idea**
>
> `(append '(a) '(b))` glues by **copying**. `(cons '(a) '(b))` gives `((A) B)` — a list in the head, not a join. If you expected `(A B)` and got parentheses inside, you grabbed the wrong function. You can poke `append` once today and not fall in love: for learning, `cons` + `reverse` is more honest, when you get there.

Break it. Type `(first 5)`. Error: 5 is not a list. `(rest 'reactor)` — a symbol isn't a list. `length` on a number — same. Lisp will not guess that you "meant a box." Put a box.

5.2.2 Java: an array, a list that grows, a pocket with labels

An array is a box with three nests, and nests don't appear out of thin air:

```
int[] temps = {18, 21, 7};
temps[0] = 19;
for (int t : temps) {
    System.out.println(t);
}
```

> **Slow down**
>
> `int[]` — "a row of integers." Curly braces at creation — the starting values. Index from **zero**: the first nest is `temps[0]`. The third is `temps[2]`. `temps[3]` — `ArrayIndexOutOfBoundsException`, a hand into the wall. The loop `for (int t : temps)`

> — "for each, don't think about the index." When you need the index — the old `for (int i = 0; i < temps.length; i++)`.

Array length is the field `.length`, not a method. On a string — the method `.length()`. On an `ArrayList` — the method `.size()`. Three names for one idea. Java likes you to trip and remember.
A list you can keep adding to — `ArrayList`:

```
import java.util.ArrayList;
import java.util.List;

List<String> rooms = new ArrayList<>();
rooms.add("airlock");
rooms.add("corridor");
rooms.add("reactor");
System.out.println(rooms.get(1));
System.out.println(rooms.size());
```

> **What just happened**
>
> `List<String>` — "a list of strings." Angle brackets — what beast we put inside. That's a **generic**, a word for the interview. Later you try `rooms.add(3)` — the clerk yells: 3 is not a string. `get(1)` — the second element, because zero. `size()` — how many there are now, not "how many fit." A lot will fit. Memory runs out before `ArrayList` says "no." Left side `List`, right side `new ArrayList`. The blueprint says "list," the hardware is "a list on an array." Don't think about why yet. Tomorrow it'll be the same with `Task` and `new Task`.

Throw away by number:

```
rooms.remove(0);
```

Zero again — the first. People count from one. Your TODO in the quest has to talk to people: they type `del 1`, you do `remove(0)` inside. Forget — you'll delete the wrong line, and the antenna will wait another shift.
A pocket "key → value":

```
import java.util.HashMap;
import java.util.Map;

Map<String, Integer> energy = new HashMap<>();
energy.put("reactor", 80);
energy.put("antenna", 20);
System.out.println(energy.get("reactor"));
System.out.println(energy.get("garden"));
```

> **What just happened**

`put` puts, `get` gets. No key — `null`, not an error. `null` is "nobody here," a meaner cousin of `nil`: call a method on it — `NullPointerException`. The station doesn't fall from space. It falls from "I thought there was a number there." `Integer`, not `int`, in the map's angle brackets: collections hold objects. Java will box `80` into an object and back by itself. Don't fear it yet. Don't put `null` in an `int` — it won't fit. It will fit in `Integer`, and then explode in arithmetic.

Walk the map:

```
for (Map.Entry<String, Integer> e : energy.entrySet()) {
    System.out.println(e.getKey() + " " + e.getValue());
}
```

Key, value, pair. Don't memorize `Entry` as a separate science. It's "one pocket label for the length of the loop."

Bad idea

Don't learn every collection like a multiplication table. `ArrayList` and `HashMap` cover most of a junior's suffering. The rest will show up when you actually need it.

Today's Java game is a to-do list in the console. Add, show, delete by number. Like on a fridge, except the fridge is an array in memory, and after you power off everything is forgotten.
A sketch of the command loop — don't copy it blind, finish it yourself in the quest:

```
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Todo {
    public static void main(String[] args) {
        List<String> tasks = new ArrayList<>();
        Scanner in = new Scanner(System.in);
        while (true) {
            System.out.print("> ");
            String line = in.nextLine().trim();
            if (line.equals("quit")) {
                break;
            }
            System.out.println("I only know quit so far, the rest is your quest");
        }
    }
}
```

Slow down

`while (true)` — spin until `break`. `nextLine()` — the whole line, not one word. `.trim()` — cut spaces off the edges, people put them there. `equals`, not `==`. The prompt `>` is so it's

> clear the program is waiting, not hung. On the station, hung and waiting look the same if you don't blink.

Parsing the line `add fix the airlock`: first word is the command, the rest is the text. `split(" ", 2)` cuts into **two** parts: command and tail. Without the two, `"fix the airlock"` falls apart into three words, and the airlock gets lost.

```
String[] parts = line.split(" ", 2);
String cmd = parts[0];
String rest = parts.length > 1 ? parts[1] : "";
```

The ternary `? :` is a one-line `if` that **returns** a value. Like Lisp. Sometimes handy. Sometimes people write a bedsheet with it. Today — two branches, fine.

5.2.3 If it broke: lists and input

`cannot find symbol: ArrayList`. No import. Two lines: `List` and `ArrayList` — both from `java.util`.

`Index 3 out of bounds for length 3`. Three elements — indexes 0, 1, 2. Did you subtract one? Didn't you? Print `size()` and the number the human sent, **before** `remove`.

`InputMismatchException` from yesterday, today `nextLine` after `nextInt`. If you mix `nextInt()` and `nextLine()`, Enter lives in the buffer, and an "empty command" arrives by itself. Today all input is `nextLine`. Numbers from text: `Integer.parseInt`. Catch `NumberFormatException` — say "a number, please" and keep spinning.

Empty `add`. The human typed `add` with no text. Decide: an error, or a task with an empty name. An empty name will explode your head later. Better "need a word" right now.

> **Mac, Windows, WSL**
>
> Same launch: `javac Todo.java && java Todo` on a Mac and in WSL. On Windows `&&` in cmd.exe works in newer versions; if not — two commands in a row. IntelliJ: the green arrow on `main`. Don't keep two launches at once — two `Scanner`s on one input, a circus.

Three hand-runs before the quests.

1. An array of three temperatures, change the middle one, print them all. Make sure `temps[1]` is the middle.
2. `ArrayList`: three `add`, `get(0)`, `remove(0)`, print again. The first left, the former second became zero. Lists are dense, no holes.
3. `HashMap`: two modules, `get` of a real one and a made-up one. The second is `null`. Print that word with your eyes, make friends.

Quest 2.L1 · Lisp

`rooms-report`: a list of rooms, each with a number starting at one. `loop for room in lst for i from 1` is your friend.

Quest 2.L2 · Lisp

`has-reactor-p` : `t` if the list has `reactor` . The `-p` tail on Lisper names means "a question, yes or no." Cute, right?

Quest 2.L3 · Lisp

`prepend-airlock` : stick `'airlock` on the front of any room list with `cons` and return the new list. Don't change the old one — check that the original is the same after the call. Then `(length ...)` of the new one: one bigger. If length is the same — you returned the wrong thing.

Quest 2.J1 · Java

Commands: `add text` , `list` , `del number` , `quit` . They live in `ArrayList<String>` . Numbers from one, like people, not from zero, like arrays — you decide, just don't get confused.

Quest 2.J2 · Java

`HashMap` name → age. Print anyone over 18. Data hardcoded, input not required. The station is checking who is allowed on the bridge.

Quest 2.J3 · Java

A map room → energy (`HashMap<String, Integer>`). Print only those whose energy is strictly under 30. At least three keys in the data. Empty output is also an answer if everyone is fed; then lower one number and rerun so you see a line.

Hide it on GitHub

Commit `week2: lists and todo` . Yes, a TODO in week two. Everybody started that way, even the ones who lie.

At the end of the shift the corridor on the screen matches the corridor behind your back. Almost. The list still has `garden` , and behind the hatch — crates. The past lies in the data. You aren't repairing the station yet. You're repairing the list. That's already the job.

5.3 Lesson 3. Functions that don't chatter extra

Lisp 40 min · Java 120 min · a little theory, then until it runs

Today on station · 2 h 40 min

Lisp: one function — one job, `let` inside, a little `mapcar` .

Java: a class `Task` , a box `TaskStore` , a skinny `main` .

Yesterday tasks were strings. Today they are **things** with fields.

Third watch. The captain doesn't want "strings on the fridge." A job should have a number, a name, and a mark "done." And energy shouldn't go negative when you fix the antenna. Negative energy on MODULE is not accounting. It's dark portholes.

5.3.1 Lisp: one job — one function

A good function does one thing and **returns** an answer. Printing — outside, if you can. Otherwise you never reuse the code, you only admire it in the terminal.

```
(defun clamp (n lo hi)
  (cond
    ((< n lo) lo)
    ((> n hi) hi)
    (t n)))

(defun spend (energy cost)
  (clamp (- energy cost) 0 100))
```

> **Slow down**
>
> `clamp` — squeeze a number between floor and ceiling. Below the floor — the floor. Above the ceiling — the ceiling. Otherwise the number itself. `spend` doesn't know how to clamp: it subtracts and asks `clamp`. If tomorrow you need to clamp 0..1, not 0..100, you fix one place. Call `(spend 10 40)`. Expecting −30? You get 0. The reactor doesn't do credit. `(spend 80 10)` → 70. `(clamp 150 0 100)` → 100. Three calls — three edges.

A few local names so you don't go mad:

```
(defun docking-score (speed fuel)
  (let ((speed-part (if (< speed 5) 10 0))
        (fuel-part (if (> fuel 20) 5 0)))
    (+ speed-part fuel-part)))
```

> **What just happened**
>
> Inside `let` the two pockets are computed **roughly at the same time** — don't lean on one from the other. Need a chain — `let*`. Today you don't need a chain: both pieces come from the arguments. The sum is docking points. The function stays quiet. Whoever calls it will print.

```
(docking-score 3 30)
; 15
(docking-score 9 30)
; 5
(docking-score 3 5)
; 10
(docking-score 9 5)
; 0
```

Four combinations of two switches. A truth table, only about the airlock. If you're too lazy to check by hand — check anyway. Functions lie quieter than people.
Don't touch `&optional` yet. If you meet it — life brought it to you.
A function can take **another** function. Early, but seeing it once is useful, like a magic trick:

```
(mapcar #'evenp '(1 2 3 4))
; (NIL T NIL T)
```

> **Slow down**
>
> `mapcar` walks and applies. `#'evenp` — "the function evenp itself, not a call." Hash-quote is a pointer to a function. If you write `evenp` without `#'` in this spot, Common Lisp will often still treat it as a function symbol, but the `#'` habit will save you next to `lambda` in lesson five. The result is a new list of answers. The original `(1 2 3 4)` is intact. Again: we don't wreck the box, we assemble another.

If that lit you up — that's a bridge into SICP. If not — live in peace, we'll get to `mapcar` by hand later.
Two more neighboring gestures.

```
(defun recharge (energy delta)
  (clamp (+ energy delta) 0 100))

(defun simulate (costs)
  (let ((e 100))
    (dolist (c costs)
      (setf e (spend e c)))
    e))
```

`setf` — hang a new value on a place. Here the place is the local `e`. `(simulate '(10 20 80))`: 100−10=90, 90−20=70, 70−80=0. Not negative. `dolist` walks, `setf` updates, at the end `e` is the answer of the whole `let`.

> **Bad idea**
>
> Printing inside `spend` is a favor that bites. Want to sum the costs without yelling — already can't, `format` is nailed on. Split: compute / print. `render` returns a string, `print-...` yells. Quest 3.L2 is exactly about that, not about pretty.

Break `clamp`: swap `lo` and `hi`. `(clamp 5 10 0)`. The `cond` ladder may return a miracle. Write two calls in the REPL until you see the lie. Then put it back.

5.3.2 Java: the blueprint and the hunk of metal

Yesterday tasks were strings. Today they are **things** with fields. How you tell a wrench from a rope, even though both "sit in a box."

```
public class Task {
    private final int id;
```

```
    private String title;
    private boolean done;

    public Task(int id, String title) {
        this.id = id;
        this.title = title;
        this.done = false;
    }

    public int getId() { return id; }
    public String getTitle() { return title; }
    public boolean isDone() { return done; }
    public void complete() { this.done = true; }
}
```

> **Slow down**
>
> A class is a blueprint. Until there's a `new`, there is no task in memory. `new Task(1, "fix the antenna")` — now it's hardware. Three fields. `private` hides: from outside you can't write `t.id = -7`. That isn't bureaucracy. That's so in a month you yourself don't assign `id = -7` at three in the morning. `final` on `id` — hung once in the constructor, don't touch it again. The title you can change (if you add a setter), the number — no. `this.id = id` — "this object's field = the parameter." Without `this` the names would collide and Java would shrug: assign the parameter to itself. Meaningless and quiet. Getters are holes outward "to look." `complete()` is a hole "to do." Not `setDone(true)` from the street — a verb. The task itself knows how to become done.

Two objects — two hunks of metal:

```
Task a = new Task(1, "antenna");
Task b = new Task(2, "airlock");
a.complete();
System.out.println(a.isDone());
System.out.println(b.isDone());
```

`true` and `false`. Fixing the antenna doesn't close the airlock. Obvious in life. In code it's obvious only if these are **different** `new`s.

A box for tasks:

```
import java.util.ArrayList;
import java.util.List;

public class TaskStore {
    private final List<Task> tasks = new ArrayList<>();
    private int nextId = 1;

    public Task add(String title) {
        Task t = new Task(nextId++, title);
```

```
        tasks.add(t);
        return t;
    }

    public List<Task> all() {
        return tasks;
    }
}
```

> **What just happened**
>
> `nextId++` — give the current number, then add one. First task — id 1, second — 2. The list inside is `private`. From outside, no `tasks.add` past the register: only through `add(title)`, so the number always comes from the shop, not a guest off the street. `all()` currently hands back **the same** list, not a copy. A guest can do `store.all().clear()` and you will cry. In lesson 7, `List.copyOf` shows up. Today, notice the smell. You can return a copy already if your conscience itches: `return new ArrayList<>(tasks)`.

`main` gives orders. The rules live in `Task` and `TaskStore`. If `main` is longer than the screen, the station looks at you with reproach.

```
public class TodoApp {
    public static void main(String[] args) {
        TaskStore store = new TaskStore();
        store.add("fix the antenna");
        store.add("close the hatch");
        for (Task t : store.all()) {
            System.out.println(t.getId() + " " + t.getTitle());
        }
    }
}
```

Three files: `Task.java`, `TaskStore.java`, `TodoApp.java`. Three blueprints. Compile them all:

```
javac Task.java TaskStore.java TodoApp.java
java TodoApp
```

IntelliJ will compile them itself. In the terminal, forget one file — `cannot find symbol: class TaskStore`. Not "Java broke." A classmate is missing.

> **Station law**
>
> Pull a chunk into a method as soon as `main` starts to feel ashamed. In two months the same instinct won't let you stuff a whole life into a Spring controller.

Lookup by id — a hand you'll need in the `done` quest:

```
public Task findById(int id) {
    for (Task t : tasks) {
```

```
        if (t.getId() == id) {
            return t;
        }
    }
    return null;
}
```

Found — return it. Not found — `null`. Call `complete` on `null` — boom. So in `done`: find first, then `if (t == null) { say so; } else { t.complete(); }`.

> **Bad idea**
>
> `==` for `int` — correct, those are numbers. `==` for `String title` — the trap again. Compare string fields with `.equals`. Id is a number, breathe easy.

5.3.3 If it broke: objects and several files

`The public type Task must be defined in its own file.` Two `public class`es in one `.java`. Either drop `public` on the second (don't), or two files. One public class — one file with the same name.

`constructor Task in class Task cannot be applied to given types.` You called `new Task("antenna")`, and the constructor wants `(int, String)`. The number doesn't appear by itself — `TaskStore` hands it out.

`id has private access.` You're writing `t.id` from `main`. Go to `getId()`. That's what the hole is for.

Tasks with the same id. You `new Task` in `main` by hand and put ones yourself. Then `done 1` marks the wrong one. Let only `nextId` hand out numbers.

`javac TodoApp.java` alone, without the others. In the same folder the dependency `.java` files have to sit, or already `.class`. Easier to compile `*.java` (Mac/WSL). On Windows in cmd: `javac *.java` often works too.

Three runs.

1. Two `add`, print the ids. Expect 1 and 2, not 0 and 1. If zeros — `nextId` started at zero, people on the station count from one, the captain will yell.
2. `complete` on the first, print `isDone` of both. Only the first is `true`.
3. `findById(99)` — `null`. Print it explicitly `System.out.println(store.findById(99));` — you'll see the word `null`. Make friends before it shows up in Spring.

> **Quest 3.L1** · Lisp
>
> `spend` and `recharge`: energy 0..100. `simulate`: a list of costs, start at 100, return what's left. Don't go negative — the reactor won't appreciate it.

> **Quest 3.L2** · Lisp

`render-room` returns a string. `print-rooms` only prints. Split "what to say" from "how to yell into the terminal."

Quest 3.L3 · Lisp

Take `docking-score` from the text into your own file and add a third parameter `angle` (approach angle). If the absolute value of the angle is under 10 — another +3 points. Check four or five combinations in the REPL, write the results as comments. Absolute value: `(abs angle)`.

Quest 3.J1 · Java

Yesterday's TODO — on `Task` and `TaskStore`. The command `done number` marks a task. The number is that id, not "the third from the end, I can feel it."

Quest 3.J2 · Java

A field `priority`. `list` prints priority, id, title. Sorting can wait until tomorrow. Today at least you can see that the antenna matters more than "wash the porthole."

Quest 3.J3 · Java

`TaskStore.findById`. No task — print `missing`, not an NPE. The command `show number` uses this method. Two files minimum: logic in the store, input in `main`. If `findById` lives in `TodoApp` — you hid the clerk in the hatch guard.

> **SICP · open only if it itches**
>
> Hiding fields is the same gesture as "constructors and selectors" in SICP. Scheme instead of Java, same idea.

By the end of the watch a task has a name and a number, energy has a floor and a ceiling, and `main` has a chance to slim down. The captain asks: "and after a reboot, is the list alive?" You honestly say "no." Tomorrow — the disk. Today — objects. Don't jump ahead, even if it itches.

5.4 Lesson 4. Recursion, files, and "say it out loud"

Lisp 40 min · Java 120 min · a little theory, then until it runs

> **Today on station · 2 h 40 min**
>
> Lisp: stop on an empty list, a step on the tail, a scrap of paper.
> Java: `Files`, `Path`, `try` / `catch`, not an empty `catch`.
> Week check: ten minutes out loud without peeking at the code.

Fourth watch. At night the lights blinked — scheduled, MODULE does that once a day, nobody knows why. The TODO in memory died. The captain said the word "file" like it was a spell of an older race. It isn't a spell. It's letters on a disk.
In parallel, Lisp wants you to learn to walk a list without `dolist`, on your own feet. Not to replace `loop`. So something clicks in your head: a list is a nesting doll.

5.4.1 Lisp: a function that bites its own tail — on purpose

Recursion: you call yourself on a **smaller** piece. A list: empty — stop, otherwise do something with the head and repeat with the tail. Like cleaning a corridor: one room, then the rest.

```
(defun count-rooms (lst)
  (if (null lst)
      0
      (+ 1 (count-rooms (rest lst)))))
```

> **Slow down**
>
> Lay `'(a b c)` out on paper. Actually lay it out. `(count-rooms '(a b c))` = `1 + (count-rooms '(b c))` `(count-rooms '(b c))` = `1 + (count-rooms '(c))` `(count-rooms '(c))` = `1 + (count-rooms '())` `(count-rooms '())` = `0`, because `(null lst)` — "nobody left here." Assembly backwards: 0, plus 1 is 1, plus 1 is 2, plus 1 is 3. Without paper, half the people decide this is magic. It isn't magic. It's nesting dolls.

Call it:

```
(count-rooms '())
(count-rooms '(airlock))
(count-rooms '(airlock corridor reactor))
```

0, 1, 3. If the empty one errored — no stop. If the three spins forever — you aren't doing `rest`, you're biting the same list.
Sum with the same gesture:

```
(defun sum-list (lst)
  (if (null lst)
      0
      (+ (first lst) (sum-list (rest lst)))))
```

`(sum-list '(10 20 80))` → 110. Head plus the sum of the tail. Stop is zero, because the sum of nobody is zero. For a maximum the stop is **not** zero: the maximum of an empty list is a philosophical fight we will lose. So `max-list` in the quest — the list is non-empty. Stop: no tail, the answer is the head.

> **What just happened**

> Forget the stop or `rest` — sooner or later SBCL will write `Control stack exhausted`. Translation: "I got tired of repeating your spell." The stack is a pile of unclosed calls. Each call without `rest` puts down another plate. The plates run out. Ctrl+C, fix the stop.

One more layout, already about a filter — the idea of quest 4.L2:

```
(defun filter-positive (lst)
  (cond
    ((null lst) nil)
    ((> (first lst) 0)
     (cons (first lst) (filter-positive (rest lst))))
    (t (filter-positive (rest lst)))))
```

> **Slow down**
>
> Empty — empty. Head positive — `cons` it onto the filtered tail. Head negative or zero — **don't** put it, return only the tail. `(filter-positive '(3 -1 4 0 -8))` should give `(3 4)`. Zero is not greater than zero, into space. Negatives too. If you forgot `cons` and only write recursion — you get `nil`, you threw everything away. If `cons` without recursion — one element. Two pieces: what to do with the head, and always the tail.

`mapcar` already knows how to walk a list. Write your own recursion until it clicks. Then you can be lazy with culture.
Break it for fun:

```
(defun bad-count (lst)
  (+ 1 (bad-count lst)))
```

Don't call it on a long one. Call it on `'()` and regret it fast. No `rest`, no stop. A training fire.
Tail-call optimization, accumulators — not this month. If you read it on the internet and it itches: you can. If you didn't read it — live. The main thing is the stop and a smaller piece.

5.4.2 Java: when the disk says "no"

The file didn't open, a human typed "pshh" instead of a number — Java throws an **exception**. Catch a specific one, not "everything in a sack."

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class TaskFile {
    public static void save(List<String> lines, Path path) throws IOException {
        Files.write(path, lines);
    }
```

```
    public static List<String> load(Path path) throws IOException {
        if (!Files.exists(path)) {
            return new ArrayList<>();
        }
        return Files.readAllLines(path);
    }
}
```

> **Slow down**
>
> `Path` is "where it lives," not the pile of bytes itself. `Path.of("tasks.txt")` — a file in the process's **current** folder. The current folder in IDEA may not be the one you think: look at Run Configuration → Working directory. Otherwise you'll save into the project root or into `out/`, then "the file vanished." `throws IOException` — honest: "I might trip on the disk." The method doesn't pretend to be all-powerful. Whoever calls it decides. No file on `load` — empty list, not a crash. First launch of the program on a fresh station: there's no file, and that isn't an accident.

In `main`:

```
try {
    TaskFile.save(titles, Path.of("tasks.txt"));
} catch (IOException e) {
    System.err.println("could not write: " + e.getMessage());
}
```

> **What just happened**
>
> `try` — a pen where it might blow. `catch` — what to do if **exactly this** blew. `System.err` — the stream for yelling, not for ordinary output. In a simple terminal it looks the same. Later in logs (month 2) they'll split. `e.getMessage()` — a short complaint. Sometimes the whole `e.printStackTrace()` is useful — the pile of who called whom. This week a message plus "the file didn't write" is enough.

On Windows you can still write the path as `tasks.txt` in the current folder. Don't feed `Path.of` a `C:\` unless you have to. Backslashes in a Java string are hell: `"C:\\Users\\..."` or better `"C:/Users/..."`. Or don't. A relative path.

> **Bad idea**
>
> An empty `catch (Exception e) {}` — hide the fire under a rug. The rug will catch later, in the demo. Catch `IOException` and `NumberFormatException` where they live, not "everything."

`Scanner.nextInt()` on letters yells `InputMismatchException`. Catch it and ask again. Or, like in lesson 1, read a string:

```
String raw = in.nextLine().trim();
try {
```

```
    int n = Integer.parseInt(raw);
    // n is honest
} catch (NumberFormatException e) {
    System.out.println("that's not a number");
}
```

> **Mac, Windows, WSL**
>
> The file will appear wherever you launched the JVM from. Mac/WSL: `pwd` before `java TodoApp`. Windows PowerShell: `pwd` too. Found `tasks.txt` in a weird place — not magic, the working directory. Move the launch or spell the path once, write it in the README.

The format on disk is dumb and honest: one task — one line. Priority later, when the basics breathe. Today:

```
fix the antenna
close the hatch
```

You read the lines → for each, `store.add(line)`. Ids will be handed out again 1, 2, 3. After a restart the numbers may not match "how it was on paper yesterday" if you deleted from the middle. For week 4 that's ok. Say it in the README, don't hide it.
Write on `quit`:

```
List<String> titles = new ArrayList<>();
for (Task t : store.all()) {
    titles.add(t.getTitle());
}
TaskFile.save(titles, Path.of("tasks.txt"));
```

Don't forget: `save` throws, `quit` has to be in a `try`. Otherwise "exit" crashes and nothing is saved — a mean irony.

5.4.3 If it broke: files and exceptions

`Unhandled exception: IOException`. The method calls `save`, but itself neither `throws` nor `try`. The compiler is right. Either propagate, or catch. In `main` you usually catch: the user doesn't need a stack as the only answer, but you also can't stay quiet.
`NoSuchFileException` on write. The directory isn't there. `tasks.txt` in a folder `data/`, and you never created `data/`. Either write in the current one, or `Files.createDirectories`. For now — current.
Cyrillic as garbage characters. Rare on Java 21, but if you opened the file in Notepad with the wrong encoding — don't blame `Files`. IntelliJ and a modern terminal are usually UTF-8. Old cmd.exe can be a circus. WSL and macOS Terminal are calmer.
`AccessDeniedException`. The file is open in Excel / Notepad with a lock, or the path is in "Program Files." Don't write into system folders.
Two disk runs.

1. Save two lines, quit the program, open `tasks.txt` with your eyes in an editor. The letters right? Good. Wrong — encoding, or the wrong file.
2. Delete the file, launch, don't crash, add one task, `quit`, the file is born. First day on the station.

5.4.4 Week check: a cat will do

Ten minutes out loud — to a cat, to a voice memo, whatever. How `TaskStore` is built, where the list is, where the file is, what if there's no file. Can't do it without peeking at the code — rename the methods so the story starts moving. Names you're ashamed to say out loud are usually a lie too. A cheat sheet for the voice, don't read off the page — check that you remember:

- A task is an object, not a string. It has id, title, done.
- The store holds the list and hands out ids.
- The file is a spare brain for when the lights blink.
- A Lisp list is a head and a tail, recursion stops on `nil`.

If you stall on "well there's an ArrayList" — not enough. Say **why** the store, not `main`.

Quest 4.L1 · Lisp

`max-list` by recursion, list non-empty. No `apply` and no `reduce`. Bare hands, like a mechanic.

Quest 4.L2 · Lisp

`filter-positive`: only numbers greater than zero. Recursion. Negatives can fly into space.

Quest 4.L3 · Lisp

`rooms-length` — your own `length` through recursion, without calling `length`. Check against `(length lst)` on `'()`, `'(a)`, a long list. If they disagree — the stop or `rest`. Bonus: `find-room`, which returns the tail from the find, like `member`, also by recursion.

Quest 4.J1 · Java

On `quit` write `tasks.txt`, on start read it. One task — one line. No file — live with an empty list, don't crash.

Quest 4.J2 · Java

README: how to run, which commands, a sample session. This is not "later." Without a README your program is a box with no hinge.

Quest 4.J3 · Java

In the README a separate paragraph "if it broke": no JDK, `cannot find symbol`, no file at start, `InputMismatch` / non-numeric input. One sentence per disaster — what to **see** and what to **do**. Check by opening the README on a phone: is it clear without IDEA on the screen.

Hide it on GitHub

Commit `week4: standalone java program`. Let someone else open it (or you from another machine — Mac, Windows, doesn't matter) and from the README understand where to press.

SICP · open only if it itches

Recursion on lists — bits of 1.1–1.2, if you yourself asked "why does this work," not "I have to get through SICP."

Sunday sabotage

Draw on paper the list `'(airlock corridor reactor)` as a chain of boxes with "tail" arrows. Next to it put an `ArrayList` of three strings — cells in a row numbered 0, 1, 2. Those are not the same thing. But both answer "what's in our corridor."

The lights blinked again. You restarted the program — the list is there. The captain didn't praise you: on MODULE, praise looks like the absence of a siren. Four weeks. Your own console. GitHub. No Spring. You can exhale and not install Kafka "to fatten the résumé." Next month Earth will start knocking over HTTP. For now — tea. Someone still drank the coffee.

Chapter

6 Month 2. It already answers over the network

By week eight you have a small web server on Spring Boot. The data is still in memory: restart — everything forgotten, like a dream after a night shift. But you can see HTTP, and you stop believing that "the internet" is a button. Lisp: lambdas, walks over lists, little alist pockets. And tests: a program that grabs your program by the ear.
Station MODULE learned to remember a to-do list on disk. Earth can't see that. Earth sends letters. The letters are called HTTP, and they aren't magic, they're text. If the text is scary — read it out loud. It won't get scarier. It will get clearer.
Four watches. After them — an address in the browser that answers not "this site can't be reached," but JSON. A modest holiday. Don't put Kafka on the cake.

> **Today on station · 2 h 40 min**
> Lisp: `lambda`, `mapcar`, alist, your own "server response" as a string, a little logging.
> Java: JUnit, a Git branch, Spring Web, a thin controller, a service, a log.
> By the end of the month: a README with curl, tests, `GET` / `POST`, a `done` filter. Memory, not a database. The database is next month, when you've earned it.

6.1 Lesson 5. Tests and git you aren't scared to open

Lisp 40 min · Java 120 min · a little theory, then until it runs

> **Today on station · 2 h 40 min**
> Lisp: a nameless function, `remove-if` / `find-if`, pairs in an alist.
> Java: Maven, one red test, then green. A branch, not straight to `main`.
> An inspector from Earth promised to "look at the repo." The inspector is you, in a week.

Fifth watch. Someone (not you) fixed `complete` so it does nothing, but it compiles. On the station that's called "optimism." A test is the right to say "you're lying" without yelling on the intercom.

6.1.1 Lisp: a function with no name, because you can't be bothered

Sometimes you need a function once, like a paper cup. A name for it is bureaucracy. Then `lambda`:

```
(mapcar (lambda (n) (* n 10)) '(1 2 3))
; (10 20 30)
```

> **Slow down**

`lambda` — "here's a body, I won't give it a name." Parameter list `(n)`, body `(* n 10)`. `mapcar` takes each element, feeds it to the lambda, gathers the answers. Same gesture as `#'evenp` in lesson 3, only the function isn't in the station handbook — you glued it on the spot. Call `(mapcar (lambda (n) n) '(1 2 3))` too. You get a copy. The identity lambda. Useless and calming: you see that an element is walking, not "mapcar magic."

More:

```
(remove-if (lambda (n) (< n 0)) '(3 -1 4 0 -8))
; (3 4 0)

(find-if (lambda (room) (eq room 'reactor))
         '(airlock corridor reactor))
; REACTOR
```

What just happened

`remove-if` throws out whoever the lambda is non-`nil` for. Negatives flew off, zero stayed: `(< 0 0)` is `nil`. `find-if` — the first one who matched. None — `nil`. `eq` compares symbols. Strings — not `eq`, for strings `string=`. Right now we have tag-symbols, `eq` is honest.

An energy threshold with a lambda, no separate `defun`:

```
(remove-if (lambda (n) (< n 30)) '(80 20 55 10))
; (80 55)
```

And the other way, keep the hungry ones:

```
(remove-if-not (lambda (n) (< n 30)) '(80 20 55 10))
; (20 10)
```

`remove-if-not` — a double negative in the name, but it reads "keep the ones who…." Two names, one family. Don't learn every `*-if` at once. These two will last the watch.
An association list is a list of pairs. A poor person's pocket:

```
(defparameter *energy*
  '((reactor . 80) (antenna . 20) (life-support . 55)))

(assoc 'antenna *energy*)
; (ANTENNA . 20)

(cdr (assoc 'antenna *energy*))
; 20

(assoc 'garden *energy*)
; NIL
```

Slow down

> The dot in `(reactor . 80)` is a cons pair. Not a two-element list with `nil` in the tail, but "head and tail, and the tail is a number." Looks like a typo. It isn't a typo. That's Lisp being itself. `assoc` looks up a pair by the **head**. Found — the whole pair. Not found — `nil`. Then `(cdr pair)` — the value. If you go straight to `(cdr (assoc ...))` on a missing key, `(cdr nil)` gives `nil`. Handy. You can't tell "no such module" from "energy is nil," but our energy is a number, live in peace. Compare with Java `HashMap`: there `get` is the value or `null` in one step. Here two steps. But you can see the structure: the pocket is a list too. Everything is lists, we said.

Put a new pair on the front, like `cons` in lesson 2:

```
(cons '(garden . 0) *energy*)
```

The old `*energy*` didn't change. To hang it on the global:

```
(setf *energy* (cons '(garden . 0) *energy*))
(assoc 'garden *energy*)
```

Now the garden is there, energy zero. Greenhouse, we remember you.
Three examples, not one.
Example A. Double, but ceiling 100 — that's quest 5.L1, with your eyes first:

```
(mapcar (lambda (n) (min 100 (* n 2))) '(10 60 40))
; (20 100 80)
```

`min` of two takes the smaller. `(* 60 2)` = 120, `min` cuts it to 100. The reactor isn't rubber.
Example B. Module names from an alist:

```
(mapcar #'car *energy*)
```

`car` — the head of the pair, meaning the name. `#'car` is fine, a lambda is fine. Both honest.
Example C. Only `'down`:

```
(defparameter *modules*
  '((reactor . up) (antenna . down) (airlock . up)))

(mapcan (lambda (pair)
          (if (eq (cdr pair) 'down)
              (list (car pair))
              nil))
        *modules*)
; (ANTENNA)
```

`mapcan` glues the answer-lists. The lambda returned `(ANTENNA)` or `nil` (empty, nothing to glue). You get a list of names, not a list of pairs. If you used `mapcar` — you'll get `((ANTENNA) NIL NIL)` and wonder. Quest 5.L2 is about that. Break it with `mapcar` first, then fix it with `mapcan` or `loop`.

> **Bad idea**

> `(lambda n (* n 10))` without parentheses around `n` — parameters have to be a list `(n)`. Same hole as `defun` on day one. SBCL will yell about `n` not in a list. Parentheses around the names, even if there's only one name.

6.1.2 Java: a test that has the right to offend you

A test is a separate program. It calls yours and yells if the answer is wrong. This is not "checking with your eyes." Eyes lie when they're tired. A test doesn't get tired. It only annoys.

A Maven project in IntelliJ: New Project → Maven → JDK 21. Coordinates like `com.example` / `taskstore`. On Windows without WSL you'll later run `mvnw.cmd test`, in WSL and on a Mac — `./mvnw test`. One `pom.xml` for everybody.

> **Mac, Windows, WSL**
>
> **Mac / WSL.** In the project root after generating the wrapper:
>
> ```
> chmod +x mvnw
> ./mvnw test
> ```
>
> `chmod` on a Mac and in Ubuntu is needed once, if you get "Permission denied."
>
> **Windows cmd.** `mvnw.cmd test`. Don't mix slashes.
>
> **IntelliJ.** The green arrow on the class `...Test`. Same Maven under the hood, if you imported it as Maven.

`pom.xml` needs a JUnit 5 dependency. The archetype or IDEA often drop it in themselves. The chunk without which tests are a fantasy:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>5.10.2</version>
    <scope>test</scope>
</dependency>
```

`scope>test` — it won't get into the real program. You don't put a test hammer in a spacesuit.

Put `Task` and `TaskStore` in `src/main/java/...`. The test — in `src/test/java/...` **the same package or an import**. Names: class `TaskStore`, test `TaskStoreTest`. Not `TestTaskStore` necessarily, but the `Test` suffix Maven/JUnit find calmly.

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class TaskStoreTest {
    @Test
    void addIncreasesSize() {
        TaskStore store = new TaskStore();
        store.add("antenna");
```

```
        assertEquals(1, store.all().size());
        assertEquals("antenna", store.all().get(0).getTitle());
    }
}
```

> **Slow down**
>
> `@Test` — "this is not an ordinary method, this is a quest for the machine." The method name is a sentence: what should be true. `assertEquals(expected, actual)` — that order. Swap them — the error message will mock you. Each test — a **new** `TaskStore`. Don't share one store across all methods of the class via a field "for speed." Tests will start depending on order, and JUnit never promised an order. The station is already on duct tape. Don't add flakes.

Write a test **that fails**, then fix the code. Otherwise you're testing the compiler, not yourself.
The red-bar ritual:

1. In `complete`, temporarily do nothing (empty method, don't touch `done`).
2. Test: add, `complete(id)`, `assertTrue(task.isDone())`.
3. Run — red. Read the message. It should talk about `true` vs `false`, not about "doesn't compile."
4. Put back the line that sets done. Green.

Nice, right. This isn't sadism. This is proof the test **can** yell. A green test that's green even when the code is dead — decoration.
Three tests — three edges, not three copies of `add`.

- Added — size 1, the title is that.
- `complete` of an existing one — `isDone()`.
- `complete` of a missing one — **your** decision: an exception or `false`. Check that, not "how it is on the internet."

```
@Test
void completeMissing() {
    TaskStore store = new TaskStore();
    assertThrows(NoSuchElementException.class, () -> store.complete(99));
}
```

Or:

```
assertFalse(store.complete(99));
```

Not both. Pick a contract, write it in the README in one line, the test holds it.

> **What just happened**
>
> The lambda `() -> store.complete(99)` — "here's code that should blow." JUnit will call it and catch it. If it **didn't** blow — the test is red. If it blew with a different exception — also red. A specific class, not "any trouble."

6.1.3 Git you aren't scared to open

Until now you could live on `main` and pretend branches are for grown-ups. Today is a grown-up watch. The inspector (future you) likes a history where you can see **why**, not `asdf`.

```
git status
git checkout -b feat/tests
```

Old Git understands `checkout -b`, new Git also has `git switch -c feat/tests`. Both fine. Branch name: `feat/tests`, not `tmp` and not `aaa`.
You fix tests and code. You look in the mirror:

```
git diff
```

> **Slow down**
> `git diff` — what changed **now**, not in your head. Red left, green arrived. Before a commit, read it the way you look at the oxygen sensor before leaving the airlock: not "yeah, probably." Accidentally committed a password — that's already a letter. Accidentally committed `.class` — embarrassing, but treatable. `.gitignore` from lesson 0: `*.class`, `.idea/`, `target/`.

```
git add -A
git commit -m "week5: tests for task store"
git switch main
git merge feat/tests
```

On GitHub: `git push -u origin HEAD` from the branch, then a Pull Request — or merge locally and `push main` if you're alone on the repo. Both paths are honest for a textbook. Not honest: editing `main` mixed with the experiment "what if complete does nothing" and pushing red as if that's the point.

> **Mac, Windows, WSL**
> Git is the same on a Mac, in WSL, and in Git for Windows. The terminal differs. If PowerShell eats quotes in `-m` — use singles, or IDEA: the Commit checkbox. Don't do a commit through `git commit --amend` if you already pushed, and in this book we don't celebrate amend anyway: a new commit is simpler than heroics.

`git log --oneline` should read like a station log: `week4: standalone java program`, `week5: tests for task store`. Not `fix`, `fix2`, `asdf`. Future you will say thanks. Or at least won't say something bad.

6.1.4 If it broke: tests and Maven

`Cannot resolve symbol Test`. No JUnit dependency, or IDEA didn't import Maven. Right-click `pom.xml` → Reload. Terminal: `./mvnw test`. If the terminal is green and IDEA is red — that's the IDE, not Java.

`Error: Java version`. Project on 17, JDK 21, or the other way around. In `pom.xml`, `maven.compiler.release` (or source/target) = 21. File → Project Structure → SDK 21.

Test not found. The class isn't in `src/test/java`, or the method isn't `@Test`, or it isn't `void`. Or the name is `testAdd` with no annotation — JUnit 5 doesn't pick up by prefix, that's grandpa JUnit 3.

Red `assertEquals` with a pile of lines. Read Expected / Actual. Often 0 vs 1 — `add` hit the wrong list. Often references — you compared objects with `==` inside your own code, not in the assert.

`BUILD SUCCESS` with zero tests. You didn't put them there, or the name `TaskStoreTestsBackup.java` didn't get picked up. In the log, the line `Tests run:`. Zero is not a victory.

Quest 5.L1 · Lisp

`mapcar` and `lambda`: multiply energies by 2, but not above 100. `(min 100 x)`. The reactor isn't rubber.

Quest 5.L2 · Lisp

Alist module → status. `offline-modules`: the ones with `'down`. Someone forgot to turn off the light in a compartment that's already dead.

Quest 5.L3 · Lisp

`energy-of`: an alist like `*energy*` in the text, module name a symbol. Return a number or `nil`. Through `assoc` and `cdr`. Check reactor, antenna, and `'garden`, which isn't there. Three calls, three answers, in comments.

Quest 5.J1 · Java

Three tests: add; complete by id; complete of a missing one — decide whether that's `null` or an exception, and check **your** decision, not "how it is on the internet."

Quest 5.J2 · Java

Break `complete`, make sure it's red, put it back. In the README one sentence: how to run tests on your machine (Mac / WSL / `mvnw.cmd`).

Quest 5.J3 · Java

Branch `feat/tests`, at least one commit on it, merge into `main`. In the README: three lines of `git log --oneline` (you can copy them by hand). If every commit is `asdf` — rewrite the messages **before** the push, or live with the shame. A watch log, not a chat.

> **Hide it on GitHub**
>
> A commit on the branch, then into main. `git log` should read like a station log, not like `asdf`.

The inspector looked. Tests green. `complete` lies again if you turn off one line — and the test sees it. On MODULE that's the first sensor that yells **before** the explosion. You can get used to that.

6.2 Lesson 6. HTTP is just text pretending to be the internet

Lisp 40 min · Java 120 min · a little theory, then until it runs

> **Today on station · 2 h 40 min**
> Lisp: glue a "server response" as a string, a little JSON by hand.
> Java: start.spring.io, three holes, curl from the next window.
> You restarted — the tasks died. That's the point. The database comes when you've earned it.

Sixth watch. Earth didn't send an astronaut. It sent a letter. First line of the letter: `GET /health HTTP/1.1`. The captain asks, is this a virus? This is manners. A browser knocks like that. Spring listens like that. Today you see the letter naked, then you put a Boot suit on the station.

6.2.1 Lisp: print a "server response" yourself

HTTP is letters. Not a cloud. Not Spring magic. Letters.

```
(defun http-ok (body)
  (format nil "HTTP/1.1 200 OK~%Content-Type: text/plain~%~%~a" body))
```

> **Slow down**
> `format nil` doesn't yell into the terminal. It **returns a string**. Like `render-room` versus `print-rooms`. First the status line: version, code, phrase. Then headers. Then a **blank line**. Then the body. `~%` — newline. Two `~%~%` in a row after the header — that blank line. Forget one — the client will wait for headers forever, or glue the body onto `Content-Type`. Remember it like a spell: method, path, status, headers, body.

Call it:

```
(http-ok "reactor stable")
```

Print the result with `format t` and `~s`, or just look. Between `plain` and `reactor` there should be a blank line. JSON is text too, only with curly braces and grudges about commas.
A request from Earth (that's you pretending to be the client):

```
GET /health HTTP/1.1
Host: localhost:8080
```

Again a blank line at the end of the headers. Method `GET` — "let me look." `POST` — "here's a body, put it." Don't write "give/send" in code and in curl: write `GET` and `POST`, like in the RFC and in job posts.
Codes that will last the month:

- 200 — here, take it.

- 201 — created.
- 204 — did it, no body (deleted and stays quiet).
- 400 — you sent garbage.
- 404 — no such hole, or no such task.
- 500 — we are the idiots.

Five hundred is embarrassing. Four hundred is a normal conversation. The difference is like "the reactor exploded" versus "you forgot to close the hatch."

Assemble the whole letter, as if you were a pipe, not a framework. The request:

```
POST /tasks HTTP/1.1
Host: localhost:8080
Content-Type: application/json
Content-Length: 28

{"title":"fix the airlock"}
```

The response you'll glue yourself in Lisp and that Spring will later hand out:

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id":1,"title":"fix the airlock"}
```

> **What just happened**
>
> The same blank line in both letters is the border. Above — service words. Below — cargo. You don't have to count `Content-Length` by hand right now: curl and Boot can. But seeing that it exists is useful: it's "how many letters in the body," not magic. `GET` usually carries no body. `POST` does. So curl without `-d` on POST is a letter with an empty hold. The server has the right to get offended.

Three eye-runs before Spring.

1. `(http-ok "UP")` — is there `200` and a blank line. Print through `format` with `~s`: two newlines in a row should be visible.
2. Change `200 OK` to `200OK` with no space. That's not a status line anymore, that's mush. Put the space back.
3. `(json-task 1 "airlock")` — count the quotes. If you broke escaping in `format`, you get crooked JSON. In Common Lisp a quote inside a string is written with a backslash.

Teaching JSON with no library:

```
(defun json-task (id title)
  (format nil "{\"id\":~a,\"title\":\"~a\"}" id title))

(json-task 1 "airlock")
; {"id":1,"title":"airlock"}
```

Quotes in `title` will break the tin. The quest says: don't escape, this is teaching tin JSON, you don't have to marry it. Later Jackson in Spring will do it for you, and you won't even say thanks. One more envelope:

```
(defun http-created (body)
  (format nil "HTTP/1.1 201 Created~%Content-Type: application/json~%~%~a" body))
```

Glue: `(http-created (json-task 2 "antenna"))`. Read it with your eyes like a mail carrier.

> **Bad idea**
>
> The space in `HTTP/1.1 200 OK` is required. `200OK` without a space — not a letter, mush. The code is a number, the phrase is words. Three pieces of the first line, not two.

6.2.2 Java: Spring Boot, three holes in the hull

https://start.spring.io — Maven, Java 21, check **Spring Web**. Group `com.example`, Artifact `task-manager`. A zip downloads. Unpack it into `task-manager`.

> **Mac, Windows, WSL**
>
> **Mac.** zip in Downloads, unpack with a double-click or `unzip task-manager.zip`.
>
> **WSL.** Download in Windows, copy into Linux, or `unzip` in Ubuntu: `sudo apt install -y unzip` if needed.
>
> **Windows without WSL.** Explorer → Extract. Launch later with `mvnw.cmd`.
>
> Open the folder in IntelliJ as a Maven project. Wait until the dependencies on the right stop spinning. The first time it downloads the internet. Without internet, Boot won't stand up — that isn't "you unpacked it wrong."

Three holes. Not thirty.

```
@RestController
public class TaskController {
    private final List<Task> tasks = new ArrayList<>();
    private int nextId = 1;

    @GetMapping("/health")
    public Map<String, String> health() {
        return Map.of("status", "UP");
    }

    @GetMapping("/tasks")
    public List<Task> all() {
        return tasks;
    }

    @PostMapping("/tasks")
    public Task create(@RequestBody TaskRequest req) {
        Task t = new Task(nextId++, req.title());
```

```
        tasks.add(t);
        return t;
    }
}

record TaskRequest(String title) {}
```

> **Slow down**
>
> `@RestController` — "I answer HTTP, the body is data, not an HTML page." `@GetMapping("/health")` — if the letter is `GET /health`, call this method. Returning a `Map`, Spring turns into JSON itself. You don't write `{"status":"UP"}` by hand, though in Lisp you just did — and that's why you aren't scared. `@PostMapping` — a `POST` letter. `@RequestBody` — unwrap the letter body into `TaskRequest`. `record` in Java 21 — a short class: field `title`, constructor, getter `title()`. Don't copy ten lines of boilerplate if one is enough. The list is still **in the controller**. Tomorrow that's embarrassing. Today — so it breathes at all. A fat hatch guard who also hauls crates. Lesson 7 will slim him down.

The class `Task` is yours, with id, title, done, getters. Spring serializes getters into JSON. No getter — the field won't be in the response, and you'll blame Boot. Getter there — `"id":1`. Magic that is actually a convention.
Launch:

```
./mvnw spring-boot:run
```

On Windows without WSL: `mvnw.cmd spring-boot:run`. In IDEA — the green arrow on the class `...Application` with `@SpringBootApplication`. In the log, wait for something about `Tomcat started on port 8080`. No such line — it didn't "start." Read the red **above**. Often: port taken, Java isn't 21, a typo in the package.
Then in **another** window:

> **Mac, Windows, WSL**
>
> On a Mac and in WSL, a long command wraps with a backslash `\`. In cmd.exe — `^`. In PowerShell, one long line is easier. `curl` already exists on Windows 10+ (`curl.exe`). If PowerShell replaces `curl` with its `Invoke-WebRequest` — call `curl.exe`. If it yells weird — go into Ubuntu (WSL) and poke from there, `localhost` is shared if Java is in WSL too. Java on Windows, curl in WSL: `localhost` is sometimes that one, sometimes not. Keep the client and the server in one universe.

```
curl -s localhost:8080/health
curl -s localhost:8080/tasks
```

First — `{"status":"UP"}`. Second — `[]`. Empty list, not an error. The station is alive, no jobs.

```
curl -s -X POST localhost:8080/tasks \
  -H "Content-Type: application/json" \
  -d '{"title":"fix the airlock"}'
```

> **What just happened**
>
> `-X POST` — the method. Without it, `curl` with `-d` often does POST anyway, but better to be an adult on purpose. `-H` — a header: "the body is JSON, not a form from a website." `-d` — the body. Quotes around JSON bite in different shells. In bash, singles on the outside, doubles inside the JSON. In cmd.exe — a circus with `\"`. In WSL, calmer. The answer is a task with an `id`. Then `GET /tasks` again — an array of one. Restart Boot — the array is empty again. Process memory. You turned off the JVM — the hold forgot. Lesson 4's files are not wired in here. Don't wire them "just in case." Month 3 — a real disk, Postgres.

One task by id — quest 6.J1, the idea:

```
@GetMapping("/tasks/{id}")
public ResponseEntity<Task> one(@PathVariable int id) {
    // find in the list, ok or 404
}
```

`{id}` in the path — a hole. `@PathVariable` — pull out the number. `ResponseEntity` — "not only the body, also a status." Found — 200 and the object. None — 404 without a made-up task. A made-up task with id 0 is worse than 404: Earth will think the airlock is fixed.

> **Bad idea**
>
> Don't drag in JPA, Security, and Kafka "because they were in the guide." Today three holes. Greed for annotations is the path to the dark side and to a résumé that lies.

6.2.3 If it broke: Boot, port, curl

`Port 8080 was already in use`. The previous `spring-boot:run` is alive. Find the window, Ctrl+C. Or in `application.properties` / yml a different port — then curl on that one too. Don't spawn three servers "maybe this one."

`Whitelabel Error Page` in the browser. You opened a path that isn't there, or GET instead of POST. That's a 404 from Boot, not "the internet broke." Look at the URL and the letters `/tasks` without the typo `taks`.

`Content-Type` and 415. POST without the JSON header. Add `-H "Content-Type: application/json"`.

`JSON parse error`. The body `'{"title": "airlock"}'` with smart quotes from a messenger. Rewrite the quotes by hand, dumb ones. Or a file `@body.json`.

`curl: (7) Failed to connect`. The server didn't stand up, or the wrong port. Health first. No health — no point in POST.

`mvnw: command not found`. You aren't in the project root where `mvnw` lives. `ls` should show `pom.xml`. On a Mac `./mvnw`, not `mvnw` without the dot — otherwise PATH.

Java compilation is the same, plus:

- the class isn't in a package, and Spring looks in subpackages from `Application` — put the controller **below** on the package tree from the class with `@SpringBootApplication`, or set `@ComponentScan`. Simpler tree: `com.example.taskmanager`, the controller there or in `.web`.
- `record` is fine on Java 17+, not on 11. JDK 21.

Three letter-runs.

1. `GET /health` before and after POST — always UP. Health is not about tasks.
2. Two POSTs in a row — id 1 and 2, not 1 twice. `nextId++`.
3. `GET /tasks/99` after the quest — 404, not an empty `{}` with zeros.

Quest 6.L1 · Lisp

`http-not-found` : status 404, body `missing`. The station knows how to get lost.

Quest 6.L2 · Lisp

`json-task` : id and title → `{"id":1,"title":"..."}`. Don't escape quotes in title — teaching tin JSON, you don't have to marry it.

Quest 6.L3 · Lisp

`http-created` (201) with `Content-Type: application/json` and a body from `json-task`. Glue one letter-string in the REPL. Check with your eyes: first line has `201`, a blank line before `{`.

Quest 6.J1 · Java

`GET /tasks/{id}` — one task or 404. `ResponseEntity` is your friend.

Quest 6.J2 · Java

README: five curl examples. Without that the lesson doesn't count, even if "it opens in IDEA for me."

Quest 6.J3 · Java

`POST` with no body or with `{}` — right now it might be 500. Catch the status with your eyes in curl `-i` (headers). In the README write **which** status you see today. Fixing it to 400 is the next lesson. Today, honestly record the shame of 500 if it's there. If it's already 400 — write that too, don't stay quiet.

Sunday sabotage

> Write `Hello` on a naked `ServerSocket` (one thread, one response). Then Spring will stop looking like a priest. It's just a very fat `ServerSocket` in a suit.

Earth got `UP`. The captain said "not bad" — on MODULE that's a medal. The to-do list still lives in the hatch guard. Tomorrow we drag the hatch guard off the crates.

6.3 Lesson 7. Who answers the call, and who thinks

Lisp 40 min · Java 120 min · a little theory, then until it runs

> **Today on station · 2 h 40 min**
>
> Lisp: `error`, `handler-case`, a polite `'bad-request`.
>
> Java: `@Service`, a constructor, 400 not 500, `DELETE`.
>
> The rule "title isn't empty" lives in the service. Then a test will check it, and a future little console, and you at three in the morning.

Seventh watch. The controller got fat: list, numbers, create, lookup. The hatch guard at the airlock is also smelting steel. The captain isn't yelling because he's cruel — because when Earth sends a second door (a console, a bot, "one more controller"), the rules will drift apart. Pull the brain inside once.

The controller is the watch at the airlock. The service is the mechanic inside.

Yesterday's fat chunk, so you have something to scrape out with your eyes:

```
@PostMapping("/tasks")
public Task create(@RequestBody TaskRequest req) {
    if (req.title() == null || req.title().isBlank()) {
        // return what? null? an empty Task? 500?
    }
    Task t = new Task(nextId++, req.title());
    tasks.add(t);
    return t;
}
```

> **Slow down**
>
> Three jobs glued here: understand the letter, check the title, put it in the box. The second and third aren't about HTTP. They live the same way if a task is created by the console from lesson 4. That's why a service. The controller after the diet: pulled the body, called `service.create`, packed 201 or 400. That's it. The hatch guard doesn't smelt steel.

Three checks that it didn't only drift apart in your head.

1. Search the project: `new ArrayList` in the controller — zero. If there is one — the list is still there.
2. A unit test `TaskService` with `new TaskService()` — green without `spring-boot:run`. If the test needs port 8080, you're testing HTTP, not rules.

3. Empty title through curl — 400. The same empty title through `service.create("")` in a test — an exception. One rule, two doors.

```
@Service
public class TaskService {
    private final List<Task> tasks = new ArrayList<>();
    private int nextId = 1;

    public List<Task> findAll() {
        return List.copyOf(tasks);
    }

    public Task create(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title required");
        }
        Task t = new Task(nextId++, title.strip());
        tasks.add(t);
        return t;
    }
}
```

> **Slow down**
>
> `@Service` — a tag for Spring: "this is a bean, put it on the shelf." Not a sacred word. It could have been `@Component`. Service is the meaning: rules and data live here. `List.copyOf` — the guest got a **snapshot**, not the live box. `clear()` from outside won't kill the hold. Yesterday's smell, closed. `isBlank()` — empty or only spaces. `"  "` is not a task title. `strip()` — trim the edges of an honest title. Earth loves spaces. An exception is the signal "you can't do that." Don't return `null` "they'll probably get it." The controller will understand the specific type and turn it into 400.

Spring will slip the service into the controller through the constructor. Don't write `new TaskService()` by hand — otherwise why did we start this whole bean religion.

```
@RestController
public class TaskController {
    private final TaskService service;

    public TaskController(TaskService service) {
        this.service = service;
    }
}
```

> **What just happened**
>
> One constructor — Spring will find `TaskService` on the shelf and insert it. The field is `final` — hung forever, nobody swaps it at midnight. There's no list in the controller anymore. If

> there is — you cheated, the hatch guard is smelting steel again. Check: search the project for `new ArrayList` in the controller. Zero hits. If there are some — pull them out.

Empty title — 400, not 500. Five hundred means "we are the idiots." Four hundred — "you sent garbage."

```
@PostMapping("/tasks")
public ResponseEntity<?> create(@RequestBody TaskRequest req) {
    try {
        return ResponseEntity.status(201).body(service.create(req.title()));
    } catch (IllegalArgumentException e) {
        return ResponseEntity.badRequest().body(Map.of("error", e.getMessage()));
    }
}
```

> **Slow down**
>
> 201 — created, not just "ok, 200." A tiny thing that in an interview tells a person who has seen HTTP from a person who has only seen the green arrow. `?` on `ResponseEntity` — "the body differs: sometimes a Task, sometimes an error map." Not ideal architecture, but honest for one watch. Later you can `@ControllerAdvice`. Today — don't. If it works, don't spawn annotations.

`TaskRequest` is what arrived from the street. `Task` is what lives at home. Even if the fields look alike, don't shove the home one onto the street. Later the home one will grow secrets (who the author is, an internal flag), and you'll accidentally hand them out in JSON.
Deletion:

```
public boolean delete(int id) {
    return tasks.removeIf(t -> t.getId() == id);
}
```

`removeIf` — a lambda, you were just petting those in Lisp. Returns `true` if it threw someone out. The controller: `true` → 204 with no body, `false` → 404.

```
@DeleteMapping("/tasks/{id}")
public ResponseEntity<Void> delete(@PathVariable int id) {
    if (service.delete(id)) {
        return ResponseEntity.noContent().build();
    }
    return ResponseEntity.notFound().build();
}
```

curl:

```
curl -i -X DELETE localhost:8080/tasks/1
```

`-i` will show the status. 204 often with an empty body — normal. Same id a second time — 404. A poor mechanic's idempotence: you can't delete it again, it's already gone.
A service test **without HTTP**. Don't stand up all of Boot to learn that an empty title is bad.

```
class TaskServiceTest {
    @Test
    void emptyTitleRejected() {
        TaskService s = new TaskService();
        assertThrows(IllegalArgumentException.class, () -> s.create("  "));
    }

    @Test
    void deleteMissing() {
        TaskService s = new TaskService();
        assertFalse(s.delete(99));
    }
}
```

`new TaskService()` in a test — fine. This isn't a controller, it's a plain class. Spring isn't needed for a unit of rules. `@SpringBootTest` on every sneeze — slow and brittle. Save it for the month when a database shows up.

6.3.1 Lisp: the same manners without Spring

```
(defun create-task (title)
  (if (or (null title) (string= title ""))
      (error "title required")
      (list :title title :done nil)))

(defun try-create (title)
  (handler-case (create-task title)
    (error () 'bad-request)))
```

> **What just happened**
>
> `error` throws. Without a catcher SBCL opens the debugger — you've already been in it by accident. `handler-case` — "if it blew, return a symbol." `'bad-request` like 400. Success — a list with keys. Keys `:title` are another kind of tag, handy in a plist. You don't have to understand every kind. Understand: the rule lives in `create-task`, the translation into "an answer to the street" is outside. Same fat / skinny as Java.

Spaces: `(string= title "")` won't catch `"  "`. You can `(string= (string-trim '(#\Space) title) "")`. Quest 7.L2, if you take it on.

6.3.2 If it broke: beans and 400

`required a bean of type TaskService`. No `@Service`, or the class isn't scanned (wrong package). Put the service next to `Application` on the tree.

`NullPointerException` in the controller. Forgot the constructor, did `new` by hand on an empty service? Or `@Autowired` on a field, and the test creates the controller itself. The textbook wants a constructor. Follow it.

POST empty title, and the status is 500. You didn't catch the exception, Boot wrapped it in 500. That **is** "we are the idiots." Catch `IllegalArgumentException`. Don't catch `Exception` — you'll hide real bugs.
400, but the body is HTML. You returned a string not through `ResponseEntity` / `@RestController`. Or the error happened before the method. Look at `curl -i`, not only the browser.
`List.copyOf` and then `UnsupportedOperationException`. Someone outside is trying to `add` to what `findAll` returned. Good: the sensor worked. Change the list — only through service methods.

> **Station law**
> The rule "title isn't empty" lives in the service. Then a test will check it, and a future little console, and you at three in the morning.

Quest 7.L1 · Lisp

`create-task`: empty title — `(error "title required")`. Catch with `handler-case`, return `'bad-request`. Manners in orbit.

Quest 7.L2 · Lisp

Empty after trim is `'bad-request` too. `(try-create "   ")` should not return a task. Compare with `(try-create "airlock")`. Two calls in the REPL — two different worlds.

Quest 7.J1 · Java

The service is pulled out. The controller has no `List` of its own. If it does — you cheated, the hatch guard is smelting steel again.

Quest 7.J2 · Java

`DELETE /tasks/{id}` → 204 or 404. A service test without HTTP: deleted, and missing.

Quest 7.J3 · Java

POST with `{"title":""}` and POST with spaces — both 400, a body with the key `error`. `curl -i` in the README, two real response bedsheets, not "well, 400." If 500 — it doesn't count, fix the catch.

The captain walked the corridor and didn't find the task list in the controller. He nodded. A nod on MODULE is rare, log it in the station log. Meaning git.

6.4 Lesson 8. A whisper in the logs and handing over the watch

Lisp 40 min · Java 120 min · a little theory, then until it runs

> **Today on station · 2 h 40 min**

> Lisp: your own `log-info`, call it from task creation.
> Java: `application.yml`, `Logger`, a filter `?done=`.
> Week check: a fresh folder, README, curl. Not "it worked on my machine."

Eighth watch, night. `System.out.println` in the service is yelling in the corridor. The next compartment is asleep. A log is an entry in the journal: who created, who deleted, which id. Earth will later ask "so what happened at 03:12." You won't remember. The journal will.
`application.yml` (or `.properties`, if start.spring.io handed you that — don't spawn both):

```yaml
server:
  port: 8080
logging:
  level:
    com.example.taskmanager: INFO
```

> **Slow down**
> YAML loves indentation. Two spaces, not a tab. Break the indent — Boot won't stand up, complaining about parse. Port 8080 is the default anyway, but spelling it out is honest for the README. `logging.level.package: INFO` — what to write from **your** classes. `DEBUG` — chatty, fine for a week, noisy for handing over the watch. `ERROR` — only fires, you'll miss task creation. INFO — "created id=3," a normal watch.

Instead of `System.out` — a log:

```java
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

private static final Logger log = LoggerFactory.getLogger(TaskService.class);

log.info("created task id={}", t.getId());
log.info("deleted task id={}", id);
log.warn("task not found id={}", id);
```

> **What just happened**
> `{}` — a hole, the argument gets substituted. Don't glue `"id=" + id` like in the corridor: the log can be lazy, and you won't pay for concatenation on a hot path. There's no hot path yet, but the habit is cheap. `getLogger(TaskService.class)` — the journal will have the class name. Later you grep `TaskService`, not the whole Tomcat bedsheet. `warn` — didn't find it to delete / didn't find it to hand out. Not `error`: a dumb client is not a reactor. Save `error` for "this should never have happened."

Launch, do a POST, watch the same black window where `mvnw` lives. A line `created task id=1`. No line — the logger is the wrong package, or the level is WARN globally, or you're logging in the controller and calling another class. Search.

A loudness ladder, no multiplication table:

- `ERROR` — a fire. Couldn't write. Not that the client sent an empty title.
- `WARN` — weird, but we live: deleted an id that isn't there.
- `INFO` — the watch: created, deleted, server start.
- `DEBUG` — for yourself at night. Turned it on in yml, fixed it, turned it off. Don't keep it for handing over the watch.

Break the log on purpose: set the level to `ERROR` on your package, do a POST, the `created` line won't be there. Put `INFO` back. Same ritual as a red test: make sure the sensor can stay quiet when you muted it.

> **Bad idea**
>
> Don't log passwords, tokens, bodies of other people's letters. Right now you only have `title`, and even that isn't a secret. The habit: into the journal — an id and a fact, not the whole life of the request. In an interview that sounds like "don't shine passwords," even if there is no password yet.

The filter at the end of the week — quest 8.J2, the idea isn't hidden:

```
@GetMapping("/tasks")
public List<Task> all(@RequestParam(required = false) Boolean done) {
    if (done == null) {
        return service.findAll();
    }
    return service.findAll().stream()
        .filter(t -> t.isDone() == done)
        .toList();
}
```

> **Slow down**
>
> `@RequestParam(required = false)` — the parameter may be missing. Then `done` is `null`, not `false`. An important hole: no parameter — **all**, done and not. `?done=true` — only done. `?done=false` — only alive. `Boolean`, not `boolean`: a primitive can't do "they didn't send it." An object can `null`. `stream().filter(...).toList()` — Java 16+, we have 21. A lambda again. The Lisp brain nods.

curl:

```
curl -s "localhost:8080/tasks?done=true"
curl -s "localhost:8080/tasks"
```

Quotes around a URL with `?` — so the shell doesn't eat it. On a Mac and in bash without quotes, `?` sometimes globs files. If you suddenly get "no file done=true" — that's it, quotes.

You need `complete` over HTTP, or the filter is boring. Minimum: `POST /tasks/{id}/complete` or `PATCH` with a body. Doesn't have to be REST-ideal. `isDone` has to be able to become `true` without

a restart and without reflection. If you don't make REST in time — a service method + a temporary endpoint. The main thing — a filter you can check with curl.

6.4.1 Lisp: a watch journal in one function

```
(defun log-info (fmt &rest args)
  (format t "INFO ")
  (apply #'format t fmt args)
  (terpri))
```

> **What just happened**
>
> `&rest args` — "the remaining arguments as a list." `apply` unpacks the list into arguments for `format`. `(log-info "created id=~a" 3)` will print `INFO created id=3` and a newline. `terpri` — ter-mi-nate print, a new line, an old short word. Call it from `create-task` on the success branch. On `'bad-request` better `WARN`, if you start a `log-warn`. Quest 8.L2.

6.4.2 Handing over the watch: someone else's machine, your README

Week check: a fresh folder. Not the one where "I have everything open in IDEA for three days."

1. `git clone` yourself into `/tmp` (Mac/WSL) or into another Windows directory.
2. JDK 21 there? `java -version`.
3. `./mvnw test` (or `mvnw.cmd test`) — green.
4. `./mvnw spring-boot:run`.
5. curl from the README, **copy-paste**, not "I remember it by heart."

Didn't fly — fix the README, don't say "well it worked for me." It worked on **your** pile of accidents: port, JDK from IDEA, working folder, a dependency in the Maven cache. The README is for a person with their own pile.

> **Mac, Windows, WSL**
>
> On Windows the log is in the same black window as `mvnw.cmd`. On a Mac — the same Terminal window. In IDEA — the Run tab. Don't look for `log.txt` in the root until you set up a file yourself. Today the console. Tomorrow, when Docker shows up, a file will appear by itself somewhere else, and you'll be looking for it again. The profession.

Handover checklist, out loud:

- Health is alive.
- POST creates, GET list sees it.
- GET by id: there / 404.
- POST of an empty title: 400.
- DELETE: 204 then 404.
- Log: create/delete visible.
- `?done=` doesn't crash.

- Tests without a manual "I poked it."

If a point lies — that isn't "a small thing." That's a hole in the airlock.

6.4.3 If it broke: logs and the filter

No log. The package in yml didn't match the real one (`com.example.taskmanager` vs `com.example.demo`). Copy it from the class `package ...`.

Too many Hibernate logs. There aren't any yet, you have no JPA. If you added JPA "to read about" — delete it. Month 3.

`?done=true` always empty. Nobody can `complete` over the network, everything is `done=false`. Make an endpoint, or a temporary `store.complete` in a `CommandLineRunner` — no, not a Runner. An endpoint. Honest.

`done=yes`. 400 from Boot: not a `Boolean`. Either catch it, or write `true` / `false` in the README, not "yes."

Fresh clone, no `mvnw`. You didn't commit the wrapper. Either commit `mvnw`, `mvnw.cmd`, `.mvn/`, or in the README honestly `mvn test` and require Maven installed. The wrapper is kinder to the inspector.

Quest 8.L1 · Lisp

`log-info`: prints `INFO` and then like `format`. Call it from `create-task`. A watch journal.

Quest 8.L2 · Lisp

`log-warn` with the same gesture, prefix `WARN`. In `try-create` on the `'bad-request` branch — warn, on success — info. Two calls, two lines in the REPL of a different color… well, different text. Color in the terminal isn't required, the station is already yellow from duct tape.

Quest 8.J1 · Java

Logs on create/delete. README: where to look. On Windows the log is in the same black window as `mvnw.cmd`.

Quest 8.J2 · Java

`GET /tasks?done=true` — a filter. With no parameter — all, even the ones we promised to fix a long time ago.

Quest 8.J3 · Java

`WARN` in the log when `GET /tasks/{id}` or `DELETE` hit empty. Not `ERROR`. In the README one sample line from the console. If you can't pull it out — there's no log, quest 8.J1 is still alive.

> **Hide it on GitHub**
>
> Commit `week8: rest in memory`. You can tag `v0.1`, like a game shipped.

> **SICP · open only if it itches**
>
> If you suddenly liked that HTTP is text, and JSON is text, and a log is text: welcome. A lot of backend is agreements about text. SICP has nothing to do with it, but the itch "what's under the annotation" is useful. Sunday's `ServerSocket` scratches it.

Watch handed over. In orbit it still smells like duct tape, someone still drank the coffee, but `curl localhost:8080/health` answers `UP`. Data in memory is a dream. Next month — Postgres, and the dream will learn to live in a table. Don't install Hibernate tonight "getting ahead." Getting ahead on MODULE ends with a dent in the airlock.

Tea. Commit. Sleep. Earth can wait until morning. It's got latency.

Chapter

7 Month 3. Memory that doesn't deflate

By week twelve: PostgreSQL, a real create-read-update-delete set, Flyway migrations, tests. No microservices, and if someone whispers "let's add Kafka" — that's a siren from the next station over, don't listen. Stuck on Hibernate for three days? Welcome to the profession. You stay here. You don't skip "ahead."

Until now the tasks lived in an `ArrayList`. Restart Spring — the list is empty, like the corridor after night watch. The file from month 1 already knew how to survive the close button. A database is the same gesture, only with a language that can search without reading everything from the start, and with an iron rule: two people must not wreck the same row at the same time so the station splits in two.

> **Today on station · 2 h 40 min**
>
> **Mon–Thu:** 40 min Lisp (files, nested lists, toy "transactions") + 120 min Java (psql, JPA, tests).
> **Friday:** finish the red Hibernate. No new Spring starters.
> **Saturday:** your CRUD all the way: schema in the README, curl for four gestures, a dozen tests.
> **Sunday:** sabotage — a tiny "database" in a file. After that, Postgres will seem polite.

> **Station law**
>
> Hands in `psql` first. Then Java. Anyone who slaps `@Entity` on a table that only exists in their head will spend three days reading `Caused by` and blaming the universe. The universe is innocent. The column is missing.

7.1 Lesson 9. Hands in the database first, then Java

Lisp 40 min · Java 120 min · a little theory, then until it runs

7.1.1 Lisp: saving to a file is already immortality

The process died — the memory died. The file lives. In Lisp it looks almost indecently simple: print a value so you can read the same value back.

```
(defun save-tasks (path tasks)
  (with-open-file (out path :direction :output :if-exists :supersede)
    (print tasks out)))
```

```
(defun load-tasks (path)
  (with-open-file (in path :direction :input :if-does-not-exist nil)
    (if in (read in) nil)))
```

`print` / `read` — Lisp talking to itself in text. This is not Postgres. This is the same gesture: state survives the close button.
Type it in the REPL, don't copy with your eyes:

```
(defparameter *board*
  '((1 . "fix the airlock")
    (2 . "check the antenna")
    (3 . "do not open airlock 3")))

(save-tasks "module-tasks.lisp-data" *board*)
```

Quit SBCL. Open it again. Memory is empty. The file is not:

```
(load-tasks "module-tasks.lisp-data")
; ((1 . "fix the airlock") (2 . "check the antenna") (3 . "do not open airlock 3"))
```

> **What just happened**
>
> `with-open-file` opens a stream and **closes** it, even if you screamed an error in the middle. Same gesture as Java `try-with-resources`. A file you forgot to close later refuses to open on the station "for no reason."

Look at the file with your eyes — an ordinary text editor, not witchcraft. Parentheses and quotes. If you append garbage by hand, `read` will take offense. Honest offense: "I expected a Lisp value and got mush."
Compare with the Java file from month 1: there you decided how to lay down a line. Here Lisp already knows how to serialize lists. Postgres knows how to serialize **tables** and answer questions like "every unfinished task belonging to this person." A file answers that only if you write the loop yourself. The database answers in one sentence in its weird language.

7.1.2 SQL, that weird language about tables

SQL is not Java. Not Lisp. It's a language about "show me the rows that look like this." You write **what** you want, not **how** to walk an array. Postgres decides whether to walk the whole table with its eyes or peek at a cheat sheet (indexes — next month, don't rush).
Install Postgres **locally** first. Not "in the cloud on the free tier." The station gets repaired by hand.

> **Mac, Windows, WSL**
>
> **Mac:** `brew install postgresql@16`. Brew will tell you how to start the service (`brew services start postgresql@16` or `pg_ctl`). Client: `psql postgres`.
> **WSL (Ubuntu):** `sudo apt install -y postgresql postgresql-contrib`, then `sudo service postgresql start`. Create a user and a database via `sudo -u postgres psql`.

> **Windows without WSL:** installer from https://www.postgresql.org/download/windows/ — pgAdmin checkbox to taste, **write the password on paper**, actually write it. Then either pgAdmin or `psql` from the menu.
> Superuser password — not `1234` and not empty, even on localhost. Habit. In a month the same finger will reach for `application.yml`.

Database `taskdb`. In `psql`:

```
CREATE DATABASE taskdb;
```

Postgres answers `CREATE DATABASE`. That is not an error and not a question. That is "done." Then switch into it:

```
\c taskdb
```

The prompt becomes `taskdb=#`. If it is still `postgres=#`, you are still in the housekeeping database and you are about to create a table in the wrong place. Then Java will look for `tasks` in `taskdb`, not find it, and you will argue with emptiness for half an hour.

7.1.3 A session you have to type with your fingers

Don't read. Type. If you pasted the whole thing — type it again, at least `INSERT` and `SELECT`.

```
CREATE TABLE tasks (
    id         BIGSERIAL PRIMARY KEY,
    title      TEXT NOT NULL,
    done       BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Answer: `CREATE TABLE`. Look at what you got:

```
\dt
```

```
         List of relations
 Schema | Name  | Type  |  Owner
--------+-------+-------+----------
 public | tasks | table | you
```

`\d tasks` — the table's blueprint: columns, types, default, primary key. That's the compartment map. Without the map you will later guess inside Hibernate.
Now three tasks — the sacred four gestures start with "put":

```
INSERT INTO tasks (title) VALUES ('fix the airlock');
INSERT INTO tasks (title) VALUES ('check the antenna');
INSERT INTO tasks (title) VALUES ('do not open airlock 3');
```

Each time: `INSERT 0 1`. The one is how many rows arrived. The zero in the middle is leftover politeness about OIDs, ignore it.

```
SELECT id, title, done FROM tasks;
```

```
 id |        title              | done
----+---------------------------+------
  1 | fix the airlock           | f
  2 | check the antenna         | f
  3 | do not open airlock 3     | f
(3 rows)
```

`f` is false. Postgres does not draw checkmarks. `id` grew by itself: `BIGSERIAL` is a counter. You didn't pass it. Don't pass it until you understand why.
Mark the first one done:

```
UPDATE tasks SET done = TRUE WHERE id = 1;
```

`UPDATE 1` — one row. If you wrote `WHERE id = 99`, you get `UPDATE 0`. Silence. Not an error. Nobody was found. In Java you will later be surprised that you "updated" and the database is silent.

```
SELECT id, title, done FROM tasks WHERE id = 1;
```

```
 id |     title        | done
----+------------------+------
  1 | fix the airlock  | t
```

Delete:

```
DELETE FROM tasks WHERE id = 1;
SELECT id, title FROM tasks;
```

```
 id |        title
----+---------------------
  2 | check the antenna
  3 | do not open airlock 3
(2 rows)
```

Row 1 is gone. Gaps in ids are normal. A primary key is not "the number in the report to the captain." It's a tag. You tossed the crate with tag 1 — the next ones are still 4, 5, 6. Don't try to "squeeze them up." The station hates it when you restick tags after the fact.

> **Slow down**
>
> Four gestures, remember them in your body. `INSERT` — put. `SELECT` — look. `UPDATE` — change **what's already there**. `DELETE` — throw away. HTTP from last month — the same four, only over the network: POST, GET, PUT/PATCH, DELETE. CRUD is not a sacred word from a job post. It's these four gestures with a table. If you can do them in `psql`, Java is a translator, not a magician.

Try to break it. That matters more than a pretty `SELECT`.

```
INSERT INTO tasks (title) VALUES (NULL);
```

Postgres will scream something like:

```
ERROR:  null value in column "title" of relation "tasks"
violates not-null constraint
```

`NOT NULL` — "don't put empty," even if Java forgot to check. The database is the last airlock. The doorman at the entrance (the service) is polite. The airlock is steel.

```
INSERT INTO tasks (id, title) VALUES (2, 'duplicate tag');
```

If id=2 is still alive:

```
ERROR:  duplicate key value violates unique constraint "tasks_pkey"
```

A primary key is a tag that must not repeat. Two tasks with one id — like two compartments with the same plaque on the door. The fire crew will dock at the wrong hatch.

7.1.4 A survival pocket in psql

This is not "learn every slash command." This is five things without which you are blind:

- `\c name` — switch into a database
- `\dt` — which tables
- `\d tasks` — the table blueprint
- `\q` — leave
- Up arrow — previous command, like in the terminal

The semicolon at the end of SQL is **required**. Forget it — psql waits and stares at you with the prompt `taskdb-#` (minus instead of equals). Add `;` and Enter. Everyone has sat there. Even the people who later write about "native SQL."

Strings in SQL — single quotes `'fix the airlock'`. Double quotes `"title"` are names, and in Postgres they also freeze the case. For now live in lowercase without quotes: `title`, `tasks`, `done`. Java will later fight `"Title"` vs `title`, and that's a separate circus.

7.1.5 From Java: no magic yet, plain JDBC

When the query is alive in `psql`, you can call it from Java. Starter `spring-boot-starter-jdbc` plus the Postgres driver. In `pom.xml` (Spring Boot 3.x, Java 21):

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jdbc</artifactId>
</dependency>
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

`application.yml` — the station address, not the secret of the universe:

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/taskdb
    username: postgres
    password: the-one-on-the-paper
```

A localhost password in the README is fine. A "same as prod" password in git — no, not even as a joke. Not even "temporarily." Git remembers longer than you do.
`JdbcTemplate` is a thin translator: here's SQL, here's how to turn a row into an object.

```
@Repository
public class TaskJdbcRepository {
    private final JdbcTemplate jdbc;

    public TaskJdbcRepository(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }

    public List<Task> findAll() {
        return jdbc.query(
            "SELECT id, title, done FROM tasks ORDER BY id",
            (rs, rowNum) -> new Task(
                rs.getLong("id"),
                rs.getString("title"),
                rs.getBoolean("done")
            )
        );
    }
}
```

`rs` is the current result row. `rowNum` is the ordinal, often unused, but that's the signature. One `findAll` with a `SELECT`. Controller like last month: `GET /tasks` calls the repository, does not keep a list in a class field.
When this query is alive — next lesson, JPA, that flying annotation. Today the point is to see: Java does not "store in the database." Java **asks** the database in text. The same text you just typed in `psql`.

> **Mac, Windows, WSL**
>
> In `application.yml` the url is `localhost` on a Mac, in WSL, and on Windows — if Postgres is listening locally. If Java is in WSL and Postgres was installed with a **native** Windows MSI, `localhost` sometimes sulks: either both in WSL, or both on Windows. Don't mix without a reason. The station hates two captains.
> Default port `5432` . If the installer offered another — write it down. `Connection refused` almost always means: the service isn't running, or the port is wrong, or you're knocking on the wrong machine.

7.1.6 When it screams, don't read from the bottom first — and not from the top either

Typical mush at Spring startup:

```
org.postgresql.util.PSQLException: FATAL: password authentication failed
```

Wrong password. Not "recreate the project." Not "probably Hibernate." The password.

```
org.postgresql.util.PSQLException: FATAL: database "taskdb" does not exist
```

You didn't create the database, or you created it in a different Postgres (Windows MSI vs WSL — two servers, two emptinesses).

```
Connection to localhost:5432 refused
```

Service isn't running. Mac: brew services. WSL: `sudo service postgresql start`. Windows: Services, or the elephant icon.

```
ERROR: relation "tasks" does not exist
```

No table. Are you in the right database? `\c taskdb`, `\dt`. People often create the table in `postgres` while Java looks at `taskdb`.

> **Bad idea**
>
> Don't set `ddl-auto: create` "so it does it itself." Today your hand in `psql` makes tables. Tomorrow — Flyway. Hibernate that doodles the schema at midnight is a mechanic who tightens bolts with no log. In the morning nobody knows which bolts.

Quest 9.L1 · Lisp

Save an alist of tasks to a file and open a **new** SBCL session, load it. Immortality in five minutes. If what loaded is what you saved — you already understand databases better than half the guides with `@Entity` on page one.

Quest 9.J1 · Java

In `psql` (or pgAdmin) create the table and three tasks. Copy the `SELECT` into the README. Honest output, not "yeah I did it." Three rows, a header, `(3 rows)` — that's the thing.

Quest 9.J2 · Java

`GET /tasks` from the database via `JdbcTemplate`. A localhost password in the README is fine. A "same as prod" password in git — no, not even as a joke.

Quest 9.J3 · Java

Break it on purpose: wrong password in the yml, start, **one** exception line — into the README. Then put it back. Then a database that doesn't exist. Then a table that doesn't exist. Three different screams, three different meanings. This is not sadism. This is an accident map.

> **Hide it on GitHub**
>
> Commit `week9: psql and jdbc`. In the commit — the SQL that creates the table, not only Java. Schema is code too.

7.2 Lesson 10. JPA: the table pretends to be an object

Lisp 40 min · Java 120 min · a little theory, then until it runs

Yesterday Java asked the database in text. Today an object pretends to be a row. That's convenient until it lies. It lies prettily: "but I have the field," and the column isn't there.

```
@Entity
@Table(name = "tasks")
public class Task {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private boolean done;

    protected Task() {}

    public Task(String title) {
        this.title = title;
    }

    public Long getId() { return id; }
    public String getTitle() { return title; }
    public boolean isDone() { return done; }
    public void setTitle(String title) { this.title = title; }
    public void setDone(boolean done) { this.done = done; }
}
```

You need an empty constructor — JPA is weird and calls it from underground, then fills in the fields. `protected` — so you don't write `new Task()` with no title out of habit. Getters/setters — yes, wordy, we're in Java. `record` as an entity — not yet: JPA wants to mutate fields on a live object, and a record is a freeze.

`GenerationType.IDENTITY` — "Postgres will hand out the id, we have `BIGSERIAL`." Not `AUTO` "whatever." Not `SEQUENCE` until you yourself understand why a separate sequence. The station holds on IDENTITY, and it holds fine.

7.2.1 An empty interface that lies about being empty

```
public interface TaskRepository extends JpaRepository<Task, Long> {}
```

An empty interface, and Spring writes the implementation. It looks like fraud. Inside — not fraud, code generation, but it feels the same.

`JpaRepository<Task, Long>` means: entity `Task`, primary key `Long`. The methods are already there: `save`, `findById`, `findAll`, `deleteById`. Names like `findByTitle`, `findByDone` Spring will also assemble from the method name. Don't assemble ten `findBy`s yet. One CRUD.

Service:

```
@Service
public class TaskService {
    private final TaskRepository tasks;

    public TaskService(TaskRepository tasks) {
        this.tasks = tasks;
    }

    public Task create(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("title required");
        }
        return tasks.save(new Task(title.strip()));
    }

    public List<Task> findAll() {
        return tasks.findAll();
    }
}
```

Controller like month 2, only the list lives not in a service field but in Postgres. Restart Spring — the tasks are still there. That's why we brought in the elephant at all.

7.2.2 Flyway: schema from files, not from midnight guesses

File `src/main/resources/db/migration/V1__tasks.sql`:

```
CREATE TABLE tasks (
    id         BIGSERIAL PRIMARY KEY,
    title      TEXT NOT NULL,
    done       BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Two underscores after the version. `V1__tasks.sql` — yes. `V1_tasks.sql` — Flyway won't eat it and will take offense mysteriously. The filename is a contract.

In `application.yml`:

```
spring:
  jpa:
```

```
    hibernate:
      ddl-auto: validate
    show-sql: true
  flyway:
    enabled: true
```

`validate` — "check the annotations against the live schema, doodle nothing." Mismatch — we don't start. That's good. A red start beats a quiet ghost column.

> **Station law**
>
> The schema lives in SQL migrations. Annotations describe what **already exists**. If you changed a field — write `V2__...sql`, don't hope Hibernate will guess and everyone around will thank you.

Starter: `spring-boot-starter-data-jpa` plus Flyway (`flyway-core` and for Postgres — `flyway-database-postgresql` in recent versions). Check the version in **your** Spring Boot docs, not in a 2019 article.

Drop the database, create an empty one, start the app. Flyway runs V1, a row appears in `flyway_schema_history`. That's the airlock log: which migrations already happened. Don't hand-edit a V1 that already flew to another machine. Only a new version. Otherwise your friend's history diverges, and you'll both be right, and both red.

7.2.3 Three days of red Hibernate: read the first `Caused by`

A Spring stack is like a station announcement: first "attention," then ten clarifications, and only at the bottom "the hatch is open." Reading the whole thing from the top is a path to despair. Look for the **first** `Caused by`.

Here's a typical horror session. App start, half a screen of red.

```
org.springframework.beans.factory.BeanCreationException:
Error creating bean with name 'entityManagerFactory' defined in class path resource
[org/springframework/boot/autoconfigure/orm/jpa/HibernateJpaConfiguration.class]:
Failed to initialize dependency 'flywayInitializer' (or similar)
...
Caused by: org.hibernate.tool.schema.spi.SchemaManagementException:
Schema-validation: missing column [created_at] in table [tasks]
```

> **Slow down**
>
> The first line says: "the bean didn't get created." That's weather. The real news is `Caused by`. Here: table `tasks` has no column `created_at`. Either you forgot a migration, or the entity has a field and the SQL doesn't, or the other way around. Treatment: `\d tasks` in psql, compare with the class, add a `V2` or fix V1 **if nobody else has used this V1 yet**. Not "recreate the project." The project is innocent. The column is missing.

Another hit:

```
Caused by: org.hibernate.InstantiationException:
No default constructor for entity:  : com.example.taskmanager.Task
```

No empty constructor. JPA can't call `new Task()` from underground. Add `protected Task() {}`. Don't make it `public` if you don't want a coworker creating nameless tasks.
Another:

```
Caused by: org.springframework.beans.factory.BeanCreationException:
Error creating bean with name 'taskController'
...
Caused by: ... No default constructor for ... TaskService
```

This isn't Hibernate anymore, it's Spring: a service with no constructor that takes the repository, and no empty one. One constructor with dependencies — fine, Spring will take it. Two — it gets confused. Don't breed them.
Another, the beloved circus with names:

```
Caused by: org.hibernate.tool.schema.spi.SchemaManagementException:
Schema-validation: missing column [done] in table [tasks]
```

In SQL you called it `is_done`, in Java — `done`. Or the other way around. Or Postgres stored `"done"` with quotes, and Hibernate looks for `done`. `\d tasks` is the truth. Annotation `@Column(name = "done")` — if you want it explicit.

> **Bad idea**
>
> Three days of red Hibernate — read the **first** `Caused by`. Almost always "no column" or "no constructor." Not "recreate the project." Recreating the project is like stepping into space without a suit because the airlock beeped.

7.2.4 A project contains tasks, JSON must not eat its own tail

The "project contains tasks" relation:

```java
@Entity
@Table(name = "projects")
public class Project {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;

    @OneToMany(mappedBy = "project")
    private List<Task> tasks = new ArrayList<>();
}

@Entity
@Table(name = "tasks")
public class Task {
```

```
    // ...
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "project_id")
    private Project project;
}
```

Migration V2 (since V1 already did tasks without a project — don't rewrite V1, append):

```
CREATE TABLE projects (
    id   BIGSERIAL PRIMARY KEY,
    name TEXT NOT NULL
);

ALTER TABLE tasks
    ADD COLUMN project_id BIGINT REFERENCES projects (id);
```

Don't return a cycle in JSON "project → tasks → project → …". Jackson will walk the getters like a robot vacuum in a mirrored compartment. The browser hangs, you'll blame the universe. Return a DTO with no back-reference:

```
public record TaskView(Long id, String title, boolean done) {}
public record ProjectView(Long id, String name, List<TaskView> tasks) {}
```

The service builds `ProjectView` by hand. The controller does not return the entity. An entity is a bathrobe at home. JSON is what you can show a guest.

> **What just happened**
>
> `mappedBy = "project"` means: "buddy, the relation column lives on the Task side, don't draw a second one." Without it Hibernate sometimes invents an extra bridge table, and in `\dt` you see `projects_tasks` like an uninvited relative.

`FetchType.LAZY` — "don't drag the tasks until someone asks." Kind, for a while. The while is called N+1, that's the next lesson. Today it's enough not to set `EAGER` "just in case." Just in case always means everything at once, even when you wanted the project name.

7.2.5 Repair the station when "but it worked yesterday"

- The app started, no tables: Flyway is off, or it puts files somewhere other than `db/migration`.
- Flyway screams `checksum mismatch`: you edited a V1 that already ran. Put the file back, make a V3. Or on **localhost** you can drop the database and run from scratch — you cannot do that in prod, remember this sentence.
- `failed to connect`: password, port, WSL vs Windows again.
- You saved an entity, psql is empty: you're looking at the wrong database / the wrong `public` schema. Or the transaction rolled back — that's tomorrow.

Quest 10.L1 · Lisp

Projects as nested lists. `tasks-of` by project id. A station map, not an Excel sheet. For example `'((1 . (airlock antenna)) (2 . (coffee)))`.

Quest 10.J1 · Java

Entity, repository, CRUD over HTTP, Flyway V1. So after you drop the database the world rises from SQL, not from memory. Check: drop `taskdb`, create an empty one, `spring-boot:run`, tables are there.

Quest 10.J2 · Java

`Project` and `GET /projects/{id}/tasks`. JSON with no infinite nesting. The station is not recursive. Well, almost.

Quest 10.J3 · Java

Diverge the schema on purpose: add a field to the entity, don't write a migration, `ddl-auto: validate`. Start is red. First `Caused by` — in the README as one quote. Then write V2 and fix it. This lesson is about reading errors, not about "so it's green."

7.3 Lesson 11. All or nothing, and the smell of extra queries

Lisp 40 min · Java 120 min · a little theory, then until it runs

7.3.1 An airlock and money: why a "deal" at all

The station has two airlock hatches. Open both at once — the air leaves, you leave too. The rule: either both are closed, or one is open and the other holds. There is no in-between "I'm right in the middle" for air.

Money is the same airlock. Debit 100 from Alex's account, credit 100 to Borya. If after the debit the lights flickered and the credit never happened — 100 vanished. Not "temporarily in transit." Vanished. The captain will be unhappy, and that's putting it mildly.

```
start of deal
  Alex:  500 → 400
  Borya: 100 → 200
end of deal  → both changes visible
```

If it crashed between the two UPDATEs:

```
start of deal
  Alex: 500 → 400
  BOOM
rollback → Alex is 500 again, Borya 100, as if nothing happened
```

> **Slow down**

> A transaction is not "fast." A transaction is **all or nothing**. Either both hatches are in the right position, or the station pretends you never touched the lever. `@Transactional` on a service method tells Spring: every SQL inside is one deal. An exception (unchecked, an ordinary RuntimeException) — rollback. You reached the end of the method — commit, `COMMIT`. Until there is a commit, another session in psql may not see those changes. That isn't a bug. That's the airlock not closed yet.

In psql you can poke it with your hands. Session 1:

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE name = 'alex';
-- no COMMIT yet
```

Session 2, another psql window:

```
SELECT name, balance FROM accounts WHERE name = 'alex';
```

Until the first one said `COMMIT`, the second often sees the old value (depends on isolation; for the textbook: "not visible until the airlock closed"). In the first: `COMMIT;` — and after a new `SELECT` the second already has 400. Or in the first `ROLLBACK;` — and it never happened.
Same for tasks: create a project and the first task. If the task title is empty — the project must not stay in the database either. Otherwise `\dt` hosts an orphan project, and you swear "but the method failed." It failed. But without a deal it managed to commit the first half.

7.3.2 `@Transactional`: on public, not on secret

```
@Service
public class ProjectService {
    private final ProjectRepository projects;
    private final TaskRepository tasks;

    public ProjectService(ProjectRepository projects, TaskRepository tasks) {
        this.projects = projects;
        this.tasks = tasks;
    }

    @Transactional
    public Project createWithFirstTask(String projectName, String taskTitle) {
        Project p = projects.save(new Project(projectName));
        if (taskTitle == null || taskTitle.isBlank()) {
            throw new IllegalArgumentException("title required");
        }
        tasks.save(new Task(taskTitle.strip(), p));
        return p;
    }

    @Transactional
    public void completeAll(Long projectId) {
```

```
        List<Task> found = tasks.findByProjectId(projectId);
        found.forEach(t -> t.setDone(true));
    }
}
```

In `completeAll` there is no explicit `save`. JPA's dirty trick: an object you fetched inside a transaction will ride out as an UPDATE at commit. That's called dirty checking, "peek at what got smudged." Convenient. And scary when you weren't expecting an UPDATE and it left anyway.
Don't hang it on `private`: Spring looks through a proxy and doesn't see private. Magic with a hole.

> **Slow down**
>
> Spring slips a **wrapper** in place of your service. Call from outside: the wrapper opens a transaction, calls your method, closes. A call to `this.createWithFirstTask(...)` from a neighboring method of the same class — bypasses the wrapper, you go straight. No transaction. Like knocking on your own airlock from the inside: wrong door. So `@Transactional` lives on a **public** method that gets called from outside — from a controller, from a test. Not on `private void helper()`.

Check with your eyes. A test or curl: create a project with an empty task title. In psql:

```
SELECT * FROM projects;
SELECT * FROM tasks;
```

Both empty — the deal worked. Project exists, no tasks — you just invented an orphan. Look for: a checked exception that didn't roll back; or `private`; or two methods with no shared transaction; or the controller already caught the exception **after** the first `save` left without `@Transactional`.

> **Bad idea**
>
> Don't catch `Exception` inside a transactional method and swallow it. Swallowed — Spring thinks everything is fine, does COMMIT. The fire is logged as "successful watch."

7.3.3 N+1: fifty projects, fifty-one queries, burnt duct tape

You turned on `spring.jpa.show-sql=true` (and better also `logging.level.org.hibernate.SQL: DEBUG`). You did `GET /projects`. In the log:

```
Hibernate:
    select p1_0.id, p1_0.name from projects p1_0
Hibernate:
    select t1_0.id, t1_0.done, t1_0.project_id, t1_0.title
    from tasks t1_0 where t1_0.project_id=?
Hibernate:
    select t1_0.id, t1_0.done, t1_0.project_id, t1_0.title
    from tasks t1_0 where t1_0.project_id=?
Hibernate:
```

```
    select t1_0.id, t1_0.done, t1_0.project_id, t1_0.title
    from tasks t1_0 where t1_0.project_id=?
```

And so on. One `select` of projects. Then **for each** project — its own `select` of tasks, because the task list is lazy: Jackson (or your loop) poked `getTasks()`, Hibernate went to the database. 50 projects — 51 queries. That's N+1. It smells like burnt duct tape.

> **Slow down**
>
> Why "plus one": N children + one query for the parents. Not "the database is slow." You asked it fifty-one times instead of one JOIN. On three rows you won't notice. On three thousand the captain will notice, the user will notice, and your laptop will start howling like a pump.

The cure comes later, but today at least learn the smell and **once** see the medicine:

```
@Query("select p from Project p left join fetch p.tasks where p.id = :id")
Optional<Project> findWithTasks(@Param("id") Long id);
```

`JOIN FETCH` — "bring the tasks in the same query." In the log after that you often get **one** `select ... from projects ... left join tasks`. Not always pretty SQL. But one.
Don't treat everything with `EAGER`. EAGER is "always drag." Then `findAll` of projects for a dropdown will haul tons of tasks nobody asked for. Treat the specific query that smells.
A JOIN in psql, so you can see what Hibernate wants:

```
SELECT p.id, p.name, t.id, t.title
FROM projects p
LEFT JOIN tasks t ON t.project_id = p.id
ORDER BY p.id, t.id;
```

One table on the left, tasks on the right, empty tasks — `NULL` in the right-hand columns. That's the same JOIN FETCH, only by hand. If this SELECT makes sense — the annotation is no longer a prayer.
Count the queries: one `GET`, eyes on the log, tally marks in the README margins. If there are as many queries as compartments — there it is, N+1, say hello.

7.3.4 Isolation — a word in the interview, not a button this week

They may ask "what is isolation." Short honest answer: how much a deal sees other people's unclosed airlocks. By default Postgres is `READ COMMITTED` — you see only what's committed. `SERIALIZABLE` is stricter and more expensive. Don't turn on serializable "for safety" on a textbook CRUD. Safety today is `@Transactional` on the method where two changes must live or die together.

> **Quest 11.L1** · Lisp
>
> A list of operations. If `'fail` is among them — apply none. A toy "transaction": either the airlock closed, or you never touched it. No half-open hatch.

Quest 11.J1 · Java

One method: project + first task. Empty task title — the project must not stay in the database either. Check with your eyes in `psql`. Don't trust only a test that looks at the same EntityManager: look with another eye, from another window.

Quest 11.J2 · Java

SQL log, one `GET` of the list, query count in the README. If there are as many queries as compartments — there it is, N+1, say hello.

Quest 11.J3 · Java

The same `GET`, but through a method with `JOIN FETCH` (or a DTO query with no lazy collection). Count `select` in the log again. Two numbers side by side in the README: before and after. If "after" isn't smaller — you bolted fetch to the wrong place, or Jackson still pokes another relation.

7.4 Lesson 12. Tests that hit a real database

Lisp 40 min · Java 120 min · a little theory, then until it runs

A unit test of a service with a mocked repository checks that you called `save`. It does not check that Postgres swallowed the row, that Flyway ran, that `NOT NULL` is alive. By the end of the month you need tests that **knock**. Otherwise you have a green bar and a dead database — like a green light on an open hatch.

7.4.1 The ideal — Postgres in a box for the duration of the test

Testcontainers: Docker brings up a real Postgres, the test runs migrations, then the box dies. On a Mac and on Windows (Docker Desktop + WSL2) this works. Dependency `spring-boot-testcontainers` plus the Testcontainers `postgresql` module — look at your Boot's BOM.

```
@SpringBootTest
@Testcontainers
class TaskServiceIT {

    @Container
    static PostgreSQLContainer<?> postgres =
        new PostgreSQLContainer<>("postgres:16-alpine");

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry r) {
        r.add("spring.datasource.url", postgres::getJdbcUrl);
        r.add("spring.datasource.username", postgres::getUsername);
        r.add("spring.datasource.password", postgres::getPassword);
```

```
    }

    @Autowired TaskService service;

    @Test
    void createThenFind() {
        Task t = service.create("airlock");
        assertTrue(service.findById(t.getId()).isPresent());
        assertEquals("airlock", service.findById(t.getId()).get().getTitle());
    }
}
```

This isn't "Docker magic." This is an honest elephant, only temporary. If the test is green here, there's a good chance it won't lie on your `localhost:5432` either.

> **Mac, Windows, WSL**
>
> Docker Desktop on a Mac: usually just works. On Windows — install with the WSL2 backend, not Hyper-V "because the checkbox." In WSL `docker ps` should see the same containers. If IDEA is on Windows and Docker is only in Ubuntu — two captains again.
>
> No Docker and still scared: a `test` profile and H2. In the README **honestly**: "prod is Postgres, tests are a shortcut." A lie in the README surfaces in an interview faster than you can say "actually." H2 is not Postgres. `TIMESTAMPTZ`, arrays, a pile of functions — a different dialect. For a junior in month 3 — an acceptable crutch, if it's signed.

7.4.2 Four gestures — four tests, not "well I poked it with my eyes"

```
@Test
void unknownIdIsEmpty() {
    assertTrue(service.findById(9_999_999L).isEmpty());
}

@Test
void updateChangesTitle() {
    Task t = service.create("old");
    service.rename(t.getId(), "new");
    assertEquals("new", service.findById(t.getId()).get().getTitle());
}

@Test
void deleteThenGone() {
    Task t = service.create("temporary");
    service.delete(t.getId());
    assertTrue(service.findById(t.getId()).isEmpty());
}
```

Red/green, not "well I poked it with my eyes." Eyes lie when you want to sleep. `assertTrue(true)` is a crime against the station: a test that cannot fail.

For HTTP — `@SpringBootTest(webEnvironment = RANDOM_PORT)` and `TestRestTemplate` or `MockMvc`. At least create, and get of an unknown. Four hundred on an empty title, not five hundred. The transactional test from lesson 11 belongs here too: empty title — `assertEquals(0, projectRepository.count())`. Then still glance at psql once in your life, so you believe count isn't lying.

7.4.3 This week's README is a ship's log, not poetry

How to bring up Postgres on a Mac **and** in WSL/Windows. Which url in the yml. How to run migrations (they run themselves on start — write that). How to test: `./mvnw test` / `mvnw.cmd test`. Five curls: health if you have it, POST, GET of the list, GET of one, DELETE. Example JSON. If Docker is required for Testcontainers — one line "without Docker the integration tests won't stand up."

Someone else's machine (or your second OS) is the criterion. "It works in my IDEA" is not a completed watch.

7.4.4 What should be on deck by the end of week 12

A living CRUD. Schema in SQL, not in your head. Entities not sticking out as a cycle in JSON. At least one `@Transactional` scenario that rolls back. An SQL log you saw with your own eyes. A dozen tests, and not all of them mocks. A commit. That is a "monolith." One program. Not a zoo. Don't put Kafka in this commit, not even "to read." You can read in a browser, not in `pom.xml`.

Quest 12.L1 · Lisp

A toy "integration test" without JUnit: save tasks, start a **new** SBCL, load, check `equal` against what you expected. If not equal — `(error "station lost the log")`. Same feeling as a green IT: a second process, not the same memory.

Quest 12.J1 · Java

create; get of an unknown; update title; delete. Red/green, not "well I poked it with my eyes."

Quest 12.J2 · Java

README: how to bring up Postgres on a Mac **and** in WSL/Windows, migrations, tests, curl.

Hide it on GitHub

Commit `week12: crud postgres`. Don't put Kafka in this commit, not even "to read."

Sunday sabotage

Your own tiny "database": you append to a file, in memory a map of id → place in the file. `insert` writes at the end. `select` by id jumps. Then compare with Postgres and respect it a little: you just sketched a piece of what people get paid for, and they even get vacation.

SICP · open only if it itches

State that survives a process is an old question: what is data if the program turned off. A file, a table, a log. If it itches "can a database be a list of pairs" — it can, you already made an alist. Postgres is a list of pairs that learned to answer questions and not lose its tail in a draft.

Chapter

8 Month 4. What they ask in the room with a white-board

We take the monolith to "you can show this to a human." Users, passwords (not in the clear, we aren't barbarians), indexes, pages of twenty. In parallel — collections and the simple puzzles without which an interview turns into a lottery. Lisp: you may start a tiny language of your own. That's legal and even desirable.

By the end of week sixteen you don't "know Spring Security." You can explain **out loud** why a password doesn't sit as text, why `?userId=` in the URL is a hole the size of an airlock, why a `HashMap` lost a key, and why a counter from two threads lies. Whiteboards love that. Kafka on a whiteboard without that is a whistle with no air.

> **Today on station · 2 h 40 min**
>
> **Mon–Thu:** 40 min Lisp (registry, hash-table, stack, purity) + 120 min Java (auth, index, collections, races).
> **Friday:** finish Security until "someone else's id doesn't read."
> **Saturday:** pagination, EXPLAIN, three interview puzzles.
> **Sunday:** a voice recorder, eight questions. Shame to listen — a good sign: the ears are still alive.

> **Station law**
>
> Entry here is with a living CRUD and Postgres. No database — go back to month 3. Passwords on empty memory are theater: you restarted, and every user died with the scenery.

8.1 Lesson 13. Who are you, and why the password isn't "1234"

Lisp 40 min · Java 120 min · a little theory, then until it runs

8.1.1 A dump fairy tale they tell too late

Imagine: someone leaked the `users` table. Not "hacked prod in a movie." Leaked. A backup on a stick, logs, an intern with `SELECT *`, a copy "to look at." What's in the file?

If it's this:

```
 id | email              | password
----+--------------------+----------
```

```
  1 | alex@module.space  | 1234
  2 | borya@module.space | qwerty
```

Congratulations. The villain doesn't have "hashes." They have **passwords**. Alex uses `1234` on email too, and on that weird antenna forum. Borya uses `qwerty` everywhere. One dump — keys to half the lives of people who trusted you. An airlock code taped to the outside at least you can see. A password as text in the database is tape you stuck there yourself and forgot.
A hash is a one-way meat grinder. You cannot unmake sausage. The database stores sausage. On login you run the password through the grinder again and compare sausage to sausage. Even you, station captain, should not be able to "remind the password." Only reset it.

```
password "comet" → BCrypt → $2a$10$...long mush...
```

In the dump: mush. A rainbow of precomputed "1234" → md5 doesn't help if there's **salt**: a chunk of randomness, different per user, mixed into the hash. BCrypt puts the salt inside the string `$2a$10$...` itself. So two people with password `1234` get a **different** hash. Pretty and mean.

> **Slow down**
> "Why not encrypt the password instead of hashing?" A cipher is two-way: have the key — you can get the text back. So the key sits somewhere, and whoever took the database **and** the key reads passwords again. A hash doesn't open with a key. It opens only with a new "try this password — match?" That's why `matches`, not `decrypt`.

Don't write your own hash via `String.hashCode()` or MD5 "for the textbook in prod." The textbook is Lisp `sxhash` below, and we'll say right away it's a dummy. In Java — BCrypt.

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Registration — `encode`. Login — `matches`. Never log a password, not even "for a minute," not even in DEBUG, not even a textbook one. The habit is contagious. In six months you'll turn DEBUG on in prod and ship passwords to Grafana. The station doesn't forgive that, and neither does security.

8.1.2 A table of people, not "just a field on tasks"

Flyway `V3__users.sql` (pick your number if V2 is already indexes or projects):

```
CREATE TABLE users (
    id            BIGSERIAL PRIMARY KEY,
    email         TEXT NOT NULL UNIQUE,
    password_hash TEXT NOT NULL
);

ALTER TABLE tasks
    ADD COLUMN user_id BIGINT REFERENCES users (id);
```

Email unique: two Alexes with one address — a mess of whose airlock is whose. `UNIQUE` will catch it even if the service forgot to check.

```
@Entity
@Table(name = "users")
public class UserAccount {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String email;
    private String passwordHash;
    // empty constructor, getters...
}
```

Don't name the class `User` right away: Spring Security already has `User`. A name collision is a separate circus at three in the morning.
Registration:

```
public void register(String email, String rawPassword) {
    if (users.existsByEmail(email)) {
        throw new IllegalArgumentException("email taken");
    }
    String hash = encoder.encode(rawPassword);
    users.save(new UserAccount(email, hash));
}
```

Login: found by email, `encoder.matches(raw, account.getPasswordHash())`. No match — 401, the same wording for "no such user" and "wrong password." Otherwise they probe which emails are live from the answer.
Then either JWT or a session cookie. **One**. Not both, you're not a buffet.

- Session: the server remembers "this browser is Alex," a random number in a cookie. Simple. Scaling later hurts more; today you have one server.
- JWT: the server hands out a signed ticket. The client carries the ticket in the `Authorization: Bearer ...` header. The server stores no session, it checks the signature. Ticket stolen — they walk around as Alex until it expires.

> **Bad idea**
>
> The first "JWT in 15 minutes" guide from 2018 may already be dead. Check the docs for **your** Spring Boot version. Old spells sometimes open the wrong door. `WebSecurityConfigurerAdapter` is a museum. Look for `SecurityFilterChain` and `http.authorizeHttpRequests`.

A minimal skeleton on Boot 3 / Java 21 — don't copy with your eyes closed, learn three holes: `POST /auth/register`, `POST /auth/login`, everything `/tasks/**` only with a ticket. The rest 401.
The chain skeleton looks roughly like this. This is not holy text: check the docs for **your** Boot.

```
@Bean
SecurityFilterChain api(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .sessionManagement(s ->
            s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/auth/**").permitAll()
            .anyRequest().authenticated());
    // later: JWT filter before UsernamePasswordAuthenticationFilter
    return http.build();
}
```

`csrf.disable()` on a textbook JWT API is fine, because CSRF is about a browser cookie, and we have a ticket in a header. Don't kill it that way on sessions with a cookie. `STATELESS` — the server doesn't put an HTTP session; every request carries the ticket again.

8.1.3 A login session you have to see with your eyes

```
curl -s -X POST localhost:8080/auth/register \
  -H "Content-Type: application/json" \
  -d '{"email":"alex@module.space","password":"comet"}'
```

A response like `201` or a body `{"id":1}`. In psql:

```
SELECT id, email, password_hash FROM users;
```

```
 id | email              | password_hash
----+--------------------+----------------------
  1 | alex@module.space  | $2a$10$N9qo8uLOickgx2ZMRZoMye...
```

Not `comet`. If you see a clear password — you forgot `encode`. Stop. Don't commit.

```
curl -s -X POST localhost:8080/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"alex@module.space","password":"comet"}'
```

A body like `{"token":"eyJhbGciOiJIUzI1NiJ9."}` — three chunks through dots. The middle chunk is the payload; you can Base64 it and see `sub` / email. The **signature** is the third chunk. Without the server secret you cannot forge a ticket "I am admin." The secret — in yml or the environment, not in git.

```
curl -s localhost:8080/tasks \
  -H "Authorization: Bearer paste-the-token"
```

No header — 401. With a ticket — Alex's list, empty or not. Nobody else's tasks, even if they're in the database. Check with a second user.

8.1.4 Why not `?userId=1`

Here's a hole a junior carries into prod more often than a leftover `console.log`:

```
GET /tasks?userId=7
```

The service trusts the parameter. A neighbor substitutes `?userId=1` and reads the list titled "don't tell anyone." On the station that's like writing the airlock code on the hatch and being surprised the wrong people came in.
Right: `Task` has a user field. The service takes the current one from context, **not from the URL**.

```
public List<Task> myTasks(Authentication auth) {
    Long me = currentUserId(auth);
    return tasks.findByUserId(me);
}
```

An id in the path `GET /tasks/42` is the id of a **task**, not the owner. You get the owner from the ticket. Found task 42, owner isn't you — 403 or 404. Pick one and stay loyal.

> **Slow down**
>
> 404 "no such thing" to a stranger is a polite lie: you don't confirm the task exists. 403 "I know it's there, I won't give it" is more honest for your own admin UIs, chattier for strangers. Textbook monolith: pick 404 for foreign ids and write it in the README so in a month you don't forget which face the station wears.

A test with two users. Two spacesuits, one porthole that isn't yours. Without this test Security is scenery: a green health and a leaky hatch.

```
@Test
void cannotReadForeignTask() {
    String alex = token("alex@module.space", "secret1");
    String borya = token("borya@module.space", "secret2");
    long id = createTask(alex, "airlock secret");
    getTask(borya, id).expectStatus().isNotFound(); // or 403
}
```

Don't put userId in the JSON body of `POST /tasks` "to make it easier." The client lies. Always. Even if the client is you.
The same sin in a pretty dress: `GET /users/7/tasks`. Looks RESTful. If 7 comes from the URL and isn't checked against the ticket — same hole, just in form. The station does not have to have "nested users" in the path. `GET /tasks` and "who are you" from SecurityContext — boring and airtight.

8.1.5 When Security screams and you scream back

- `403` on everything, including login: `requestMatchers("/auth/**")` is the wrong prefix, or the JWT filter swallows login too early.
- `401` with a correct ticket: the header isn't `Authorization: Bearer ...`, or an extra newline, or you put quotes inside. PowerShell loves to wreck quotes — go into WSL.

- After a restart everyone is logged out: the JWT secret changed (random on every start). Put the secret in yml on localhost, not in git, next to the database password in `.gitignore`.
- Hash in the database, login always false: you compared hash strings by hand instead of `matches`. BCrypt makes new sausage on every `encode`. Comparing two hashes with `equals` is a walk into a wall.
- Two users, both see every task: `findAll()` instead of `findByUserId`. Security let them in the door, the service opened every locker.

8.1.6 Lisp: a toy "hash" you must not take to prod

```
(defun hash-dummy (password)
  (sxhash password))
```

This is not cryptography. `sxhash` is a fast number for in-memory tables. Two different passwords can collide. No salt. The gesture: **secret ≠ password**. We don't ship real bcrypt in Lisp on this course. The station is already held together with duct tape.

```
(defparameter *users* (make-hash-table :test 'equal))

(defun register (email password)
  (when (gethash email *users*)
    (error "exists"))
  (setf (gethash email *users*) (hash-dummy password))
  t)

(defun login (email password)
  (eql (gethash email *users*) (hash-dummy password)))
```

`(register "alex@module.space" "comet")`

`(login "alex@module.space" "comet")` → `T`

`(login "alex@module.space" "nope")` → `NIL`

Don't print the password to the log. Not even `(format t "~a~%" password)` "to check." Checked — delete it.

Quest 13.L1 · Lisp

Registry: email → "hash." Repeat email — error. `login` compares hashes. No passwords in logs, not even textbook ones, the habit is contagious.

Quest 13.J1 · Java

Registration and login. `GET /tasks` — only yours. Foreign ones don't glow even as "an empty list with a hint."

Quest 13.J2 · Java

A foreign id — 403 or 404. Pick one and stay loyal. A test with two users. Two spacesuits, one porthole that isn't yours.

Quest 13.J3 · Java

Three curls in the README: no ticket; with a foreign ticket on a foreign id; with your own. Three statuses. If they're all 200 — the hatch is open, don't celebrate.

8.2 Lesson 14. An index doesn't speed up everything — it speeds up this

Lisp 40 min · Java 120 min · a little theory, then until it runs

A book with no table of contents: to find the chapter about the airlock, you flip from page one. A table of contents is a cheat sheet "airlock → p. 214." Writing the book takes a little longer (you have to update the contents). Searching is faster.

```
CREATE INDEX idx_tasks_user ON tasks (user_id);
```

Flyway `V4__idx_tasks_user.sql` — one line, a big personality. Without an index, `WHERE user_id = 1` on a fat table is Seq Scan, a full walk. With an index — Index Scan, a jump. This does not speed up `SELECT * FROM tasks`. This speeds up **this** condition. An index on every column "just in case" is a table of contents that lists every word: the book is thicker, writing hurts more, and the benefit is a cat's worth.

8.2.1 An EXPLAIN session, with thousands of rows, otherwise it lies

On three rows Postgres may skip the index: "I can see it with my eyes." Pour in thousands.

```
INSERT INTO tasks (title, user_id)
SELECT 'task ' || g, 1
FROM generate_series(1, 5000) AS g;

INSERT INTO tasks (title, user_id)
SELECT 'foreign ' || g, 2
FROM generate_series(1, 5000) AS g;
```

Ten thousand rows. Not scary. Textbook junk.
Before the index (if you haven't created it yet — or drop it with `DROP INDEX idx_tasks_user` on localhost):

```
EXPLAIN ANALYZE SELECT * FROM tasks WHERE user_id = 1;
```

A chunk of truth that looks like:

```
Seq Scan on tasks  (cost=0.00..188.00 rows=5000 width=...)
  Filter: (user_id = 1)
  Planning Time: 0.1 ms
  Execution Time: 2.3 ms
```

`Seq Scan` — we walked the whole table. On ten thousand, 2 ms. On ten million it stops being a joke, and that's exactly what the interview asks about.
Create the index, again:

```
CREATE INDEX idx_tasks_user ON tasks (user_id);
EXPLAIN ANALYZE SELECT * FROM tasks WHERE user_id = 1;
```

```
Index Scan using idx_tasks_user on tasks  (cost=0.29..140.00 rows=5000 width=...)
  Index Cond: (user_id = 1)
  Execution Time: 1.1 ms
```

Your numbers will differ. Don't look at "twice as fast," look at the **word**: Seq vs Index. If after the index it's still Seq Scan — either stats didn't update (`ANALYZE tasks;`), or the condition is different (`WHERE user_id IS NOT NULL` on almost every row — the index is useless), or there are few rows and the planner isn't stupid.

> **Slow down**
>
> `EXPLAIN` without `ANALYZE` is the plan, how Postgres **intends** to. With `ANALYZE` — also how long it actually took, with live numbers. For the README you need both runs: before and after, on the same volume. Three rows in the table — the comparison lies like a polite sensor: "all nominal," and oxygen is already whistling.

Don't index `title` "because we might search." Might — when a `WHERE title = ...` or `LIKE 'foo%'` (prefix) shows up. `LIKE '%foo%'` a normal index often won't save. Say honestly in the interview "depends on the query," that's an adult answer, not "an index is always faster."

8.2.2 Pages of twenty, not a hundred thousand in one JSON

```
GET /tasks?page=0&size=20
```

Spring Data knows `Pageable`. Let it.

```
@GetMapping("/tasks")
public Page<TaskView> mine(Authentication auth, Pageable pageable) {
    Long me = currentUserId(auth);
    return tasks.findByUserId(me, pageable).map(this::toView);
}
```

`Page` in JSON will bring `content`, `totalElements`, `totalPages`. The browser and you deserve a better death than a hundred thousand tasks in one response. Server memory does too.

> **Mac, Windows, WSL**
>
> curl with pagination is the same on a Mac and in WSL:
>
> `curl -s -H "Authorization: Bearer ..." "localhost:8080/tasks?page=0&size=20"`
>
> Quotes around the URL in zsh/bash help because of `?` . In cmd.exe — different quotes, WSL is easier. PowerShell sometimes swallows `?` — then quotes too, or `curl.exe` .

Page zero is the first page, Spring-style. If you want "page 1 for humans" — translate yourself, don't get mad at the framework.
Index and pagination are friends: `WHERE user_id = ? ORDER BY id LIMIT 20 OFFSET 0`. Without an index — Seq again, then sort, then cut 20. With an index on `user_id` it's already warmer. A composite `(user_id, id)` is warmer still, when you grow into it. Today a plain `user_id` is enough if the README is honest.
Turn on `show-sql` and hit `page=2&size=20`. In the log something like:

```
select t.id, t.title, t.done, t.user_id
from tasks t
where t.user_id=?
order by t.id
limit 20 offset 40
```

`offset 40` — "skip two pages." On huge tables OFFSET gets expensive: the database still walks the skipped rows. For a junior and ten thousand rows — fine. In an interview you can say "cursor/keyset later, page for now." Honest.

> **Bad idea**
> `size=100000` in a client's hands — a hundred thousand JSON again. Put a ceiling: `@PageableDefault(size = 20)` and `MaxPageSize`. The station is not obliged to haul the whole hold because someone typed nonsense in the address bar.

Quest 14.L1 · Lisp

10000 pairs (person . task). Search by walking. Second version: a hash-table in advance. Feel the difference in your body, not in an article. `(get-internal-real-time)` before and after.

Quest 14.J1 · Java

Pagination + an index in Flyway `V2`. If V2 is already taken by projects — the next free number. The file matters, not a magic two.

Quest 14.J2 · Java

README: `EXPLAIN` before and after. On three rows Postgres may skip the index — pour in thousands, otherwise the comparison lies like a polite sensor.

Quest 14.J3 · Java

`GET /tasks?page=0&size=5` twice: second page `page=1`. In the README two JSONs — at least the ids. If the pages are identical, you forgot Pageable or always use page=0.

8.3 Lesson 15. Pockets, equality, and puzzles on your fingers

Lisp 40 min · Java 120 min · a little theory, then until it runs

Interviews love the same rakes. Not because they're villains. Because you've already stepped on these rakes in prod, you just didn't know the names yet.

8.3.1 Two crates with one number, and a map that betrayed you

```
class UserId {
    private final long value;
    UserId(long value) { this.value = value; }
}
```

You put it in a map:

```
Map<UserId, String> cabin = new HashMap<>();
cabin.put(new UserId(1), "Alex");
System.out.println(cabin.get(new UserId(1)));
```

It prints `null`. Alex is in the cabin. The key "same number" is, to the map, **a different crate**. By default `equals` is `==`: the same object in memory, not "similar inside." `hashCode` is from the address too. Two `new UserId(1)` — two planets.

> **Slow down**
>
> HashMap first computes `hashCode`, runs to a bucket, then in the bucket compares `equals`. If `equals` says "we're the same" and `hashCode` is different — the key sits in one bucket, they look in another, forever a miss. The contract: equal objects must have the same hashCode. The reverse is false: one hashCode is not yet equality (a collision). Break one without fixing the other — the map lies quieter than Hibernate.

A hand fix:

```
@Override
public boolean equals(Object o) {
    if (this == o) return true;
    if (!(o instanceof UserId other)) return false;
    return value == other.value;
}

@Override
public int hashCode() {
    return Long.hashCode(value);
}
```

Or, Java 21, without shame:

```
record UserId(long value) {}
```

`record` already gives `equals` / `hashCode`. That's not cheating, that's the 21st century. In an interview say "record"; if they ask "write equals" — write it, so they see you understand, not only the syntax.

> **Bad idea**

> You cannot mutate a map key later: you put a `UserId`, then a setter changed `value` — the object moved to another universe, the map looks up by the old hash. `final` fields. A mutable key is pain. Like a tag on a crate they restick while the crate is in transit.

`==` for objects — "same crate?" `equals` — "similar inside?" For strings almost always `equals`. `==` on strings sometimes accidentally works from the intern pool, and you'll believe in success. Don't. For `int`, `==` is fine. For `Integer` with a big number `==` is a trap again: cache from −128 to 127, after that new crates. Interviews love this to trembling.
Type it, don't take it on faith:

```
Integer a = 127;
Integer b = 127;
System.out.println(a == b);        // often true, cache
Integer c = 128;
Integer d = 128;
System.out.println(c == d);        // often false, new crates
System.out.println(c.equals(d));   // true, same number inside
```

`int` is just a number in a register, `null` cannot happen. `Integer` is a crate, can be empty, rides in collections. One sentence for the cat: "int is a value, Integer is a wrapper object."

8.3.2 Buckets: how a map loses a key, step by step

Imagine a cabinet with 8 shelves. `hashCode() % 8` is the shelf number.

1. `put(new UserId(1), "Alex")`. Hash from the address, say shelf 3. Alex lies on 3.
2. `get(new UserId(1))`. New object, different address, different hash, shelf 6. Shelf 6 is empty. `null`.
3. You fixed only `equals`, hash still from the address. Lookup arrived on shelf 6, nobody there to equals. `null` again.
4. You fixed both: both `UserId(1)` give hash, say, `1`, one shelf, equals says "yes." Alex is home.

> **What just happened**
>
> Break it the other way: `hashCode` always `42`, `equals` is normal. The map works, but every key is on one shelf. At tens of thousands it gets slow, like a LinkedList pretending to be a HashMap. In an interview that's "degrades to a list with a bad hash."

8.3.3 The list you almost always want

`ArrayList` — an array under the hood, it grows. Get by index is fast. Insert in the middle — shift the tail. `LinkedList` — little train cars. Cheap into the middle **if you're already standing on the car**. From an index — walk first. In practice a junior hauls `LinkedList` "for beauty" and gets a slow `get(i)` in a loop. A rare guest. Don't haul it for beauty.
`HashMap`: key, bucket, average get is fast. Worse when everyone has the same hash — one bucket, like a queue at the only airlock.

Complexity out loud, roughly, no exam theater:

- list, find an element: walk with your eyes, the longer the slower;
- map by key: fast on average;
- put at the end of an ArrayList: usually cheap.

Streams `.filter().map()` — in moderation. A stream for the sake of a stream reads like poetry you'd be ashamed of. A `for` loop is not shameful. Shameful is not understanding what the stream does.

```
List<String> titles = tasks.stream()
    .filter(t -> !t.isDone())
    .map(Task::getTitle)
    .toList();
```

Three lines — ok. Twelve operations with `collectingAndThen` — no, not this week and not in an interview "write it on paper."

8.3.4 Lisp: the same word frequency, the same pocket

```
(defun freq (lst)
  (let ((h (make-hash-table :test 'equal)))
    (dolist (x lst)
      (incf (gethash x h 0)))
    h))
```

`:test 'equal` — otherwise strings with the same letters will be different keys (`eq` looks at "same crate"). Java `HashMap` for strings already calls `equals`. Lisp makes you choose. That's honest.

```
(freq '("airlock" "antenna" "airlock"))
; table: "airlock" → 2, "antenna" → 1
```

A stack for brackets is an ordinary list: `push` onto the head, `pop` from the head. Opened `(` — put the expected `)`. A `)` arrived — take it off, match? No — the station listed. At the end the stack must be empty, otherwise the hatch never closed.

8.3.5 Algorithms: pictures, not a medium marathon

Grokking Algorithms (there are pictures, Barski would approve) and 3–4 easy problems a week. Not mediums in batches. Mediums in batches is how you burn out and start hating parentheses already on Java.

Three things they actually ask with fingers:

1. Reverse an array in place — two indexes from the ends, swap until they meet.
2. Character frequency — `HashMap<Character, Integer>` or an array of 26 if it's only Latin.
3. Brackets — a stack, see above.

Write it on paper. Then in the IDE. Then break the input `"]"` and `""`. Empty — brackets are balanced, by the way. A station with no hatches is airtight too. Philosophically debatable, convenient in a test.

Quest 15.L1 · Lisp

Balance of `()` `[]`. The stack is an ordinary list, `push` / `pop`. If extras remain — the station listed.

Quest 15.J1 · Java

Map key `UserId`. Two objects with one number inside — one key. `record` already gives `equals` / `hashCode`, that's not cheating, that's the 21st century.

Quest 15.J2 · Java

Three things: reverse an array in place; character frequency; brackets. No "ten more from LeetCode while it's hot."

Quest 15.J3 · Java

Break it on purpose: a key class **without** equals/hashCode, `put` and `get` with different `new`. README: which `null` you saw. Then a record. The same `get` — not null. Two lines of output. That's a story you'll later tell at the board.

8.4 Lesson 16. Two requests at once, and why the counter lies

Lisp 40 min · Java 120 min · a little theory, then until it runs

Tomcat is multithreaded anyway: two HTTPs can poke the service at the same time. You didn't order this. It's already like that. One user — one thread per request, two users — two threads, one `Counter` for both, if you hung it that way.

8.4.1 A race with numbers, not slogans

```
class Counter {
    int n;
    void inc() { n++; }
}
```

`n++` looks like one step. For the hardware it's roughly:

1. read `n` into a register;
2. add 1;
3. write it back.

Two threads, `n` is currently 42:

```
thread A read 42
thread B read 42
thread A wrote 43
thread B wrote 43
```

Two additions. The counter grew by **one**. The second addition got eaten. That's a race. Not "Java glitches sometimes." That's you letting two mechanics turn one valve without looking at each other.
Again, slower, with step numbers. Goal is 2, two threads one `inc` each, start `n = 0`:

```
time   | thread A             | thread B             | n in memory
-------+----------------------+----------------------+-----------
 t1    | read 0               |                      | 0
 t2    |                      | read 0               | 0
 t3    | 0+1, writes 1        |                      | 1
 t4    |                      | 0+1, writes 1        | 1
```

Expected 2. Got 1. Scale that to 100000 such crossings — a hole of tens of thousands. Not every `inc` has to cross. A fraction is enough.
Run it:

```
Counter c = new Counter();
Thread a = new Thread(() -> { for (int i = 0; i < 100_000; i++) c.inc(); });
Thread b = new Thread(() -> { for (int i = 0; i < 100_000; i++) c.inc(); });
a.start();
b.start();
a.join();
b.join();
System.out.println(c.n);
```

You waited for 200000. You saw, say, 143882. Or 178001. Or a rare jackpot 200000 — and decided there is no race. No: you got lucky with timing. On Windows the threads are the same, surprise. On a Mac too. The number will differ from run to run. Write three runs in the README. The spread is evidence, not shame.

> **Slow down**
>
> Why "sometimes 200000": chunks of the loops don't always cross on the same `n`. The scheduler sometimes gives A a hundred steps in a row, sometimes shuffles. A race is not "always broken." It's "not promised to be whole." The station cannot live on "well yesterday it added up." An elevator that sometimes falls is a broken elevator.

A textbook fix:

```
synchronized void inc() { n++; }
```

At one moment one thread turns the valve. The other waits. 200000 again. Boring. Correct.
In real life for a counter — `AtomicInteger` (`incrementAndGet`), for money and tasks — the database and a transaction. A handmade `synchronized` on the whole service is a queue at one airlock: safe and slow. Handmade `new Thread` in prod is like welding in a spacesuit: you can, but why.
A thread pool: `ExecutorService`. Don't breed an infinite army.

```
ExecutorService pool = Executors.newFixedThreadPool(4);
pool.submit(() -> System.out.println("watch"));
pool.shutdown();
```

Four workers. As many tasks as you like in the queue. A thousand `new Thread` for a thousand tasks — a thousand spacesuits, the oxygen runs out.

8.4.2 Where the race is in the web if I "just Spring"

Two `POST /tasks` at once — usually fine: two different rows, `BIGSERIAL` hands out ids. The race starts when two people change **the same thing**:

- a counter "how many tasks the project has" in a Java field, not in `COUNT`;
- "transfer money" with two UPDATEs and no transaction;
- `if (tasks.size() < 10) tasks.add(...)` — both see 9, both add, it became 11.

A database with `@Transactional` and a proper `UPDATE accounts SET balance = balance - 100 WHERE id = ? AND balance >= 100` is an adult valve. A check "is there enough money" in Java, then UPDATE — childish: between the check and the write a neighbor already debited.

In numbers. Alex has 100. Two requests "debit 100" at once:

```
A read balance=100, 100 >= 100? yes
B read balance=100, 100 >= 100? yes
A writes 0
B writes 0   // both "succeeded," money gone twice, lucky it was zero
```

Or worse: both write `100-100`, but if the logic is "add to the other account," Borya gets 200 from nowhere, Alex zero. A transaction + a condition in SQL: the second UPDATE hits 0 rows, the service says "not enough." Not `Thread.sleep(50)`.

Don't fix a race with "we'll just wait Thread.sleep." Sleep in an interview is a red light. Sleep in prod is duct tape on a sensor.

8.4.3 Eight questions for the cat (yes, the cat again)

By the end of the week answer **out loud**:

- how `int` differs from `Integer` in one sentence;
- why you cannot change a string "from the inside";
- collections and "this is fast / this isn't";
- checked versus unchecked;
- `try-with-resources`;
- inheritance vs "embed an object" on your own `Task`;
- what `@Transactional` rolls back;
- why `equals` together with `hashCode`.

Can't do it without peeking — not "another chapter." Retell your own code. Field names are a cheat sheet. If you're ashamed to say `mgr` out loud, rename it to `manager`, the story will go.

Lisp this week is purity: no global `defparameter`, only arguments. There's no race in single-threaded SBCL, but the habit "state flows through arguments" later helps you not breed a shared field `int n` for the whole server.

You may start a tiny language of your own: numbers and `+`. If it's a number — return it. If it's a list with `+` — add up `ev` of the tail. That's legal. That's even desirable. Java Bro does not cancel.

```
(defun ev (form)
  (if (numberp form)
      form
      (let ((op (first form))
            (args (mapcar #'ev (rest form))))
        (cond
          ((eq op '+) (apply #'+ args))
          ((eq op '-) (apply #'- args))
          (t (error "unknown ~a" op))))))

(ev 3)            ; 3
(ev '(+ 1 2 3))   ; 6
(ev '(+ 1 (- 5 2))) ; 4
```

What just happened

`mapcar #'ev` — compute the children first, then add. Like Lisp in general: inner parentheses earlier. You just wrote a tiny Lisp inside Lisp. The station loves that kind of recursion. Don't drag it into Spring. Drag it into `lisp-experiments`.

Quest 16.L1 · Lisp

Rewrite one station étude with no global `defparameter`, only arguments. Purity. Then thinking about races is easier.

Quest 16.J1 · Java

A race and `synchronized` — a separate class in `java-basics`, not in the live server. README: which numbers you saw. On Windows the threads are the same, surprise.

Quest 16.J2 · Java

Forty-five minutes, a voice recorder, eight questions from the list. Listening to yourself is embarrassing. On the interview there are fewer surprises.

Quest 16.J3 · Java

The same counter via `AtomicInteger`. Three numbers in the README: race / synchronized / atomic. The last two should match 200000. If atomic lied — you called `get` + `set`, not `incrementAndGet`.

SICP · open only if it itches

An object as a bundle of functions with a secret inside. If it suddenly got interesting why `private`.

Sunday sabotage

Draw on paper two threads and `n++` in cells: reads / plus / writes. Label 42 and 43. That's the best cheat sheet for lesson 16, and the whiteboard won't take it away — the whiteboard is waiting for it.

Hide it on GitHub

Commit `week16: auth indexes races`. In the README — EXPLAIN, three race numbers, two users. That's a portfolio, not "a little more Kafka."

Chapter

9 Month 5. Vacancy words, on your own hardware

This month's job posts sound like a parts list from someone else's station: Redis, a queue, Kafka, Docker, CI. Pretty words. In interviews people say them with a smart face. Your job is not to memorize the words. It's to touch each thing **on your own** monolith, while it lies, screams, and refuses to start.

If the cache lies — you'll see it with curl. If the queue doesn't drain — you'll see it in the log. If in Docker the database "can't be found" — you almost certainly wrote `localhost` inside the container. This isn't theory. This is an evening with a kettle.

> **Station law**
>
> Entry here is with a living monolith: a database, password login, tests, a README. Otherwise finish month 4. Better a delay than Kafka on an empty CRUD. Empty CRUD plus Kafka is a station with no walls and a loudspeaker.

> **Mac, Windows, WSL**
>
> Redis, Postgres, and the app this month live in Docker. On a Mac — Docker Desktop or Colima, whichever you already have. On Windows — Docker Desktop + WSL2. One YAML. The same `docker compose up` commands. Don't install Redis "also brew, and MSI, and apt in WSL" all at once: three Redises on one machine is a quest nobody ordered.

9.1 Lesson 17. Cache: fast, cheeky, sometimes a liar

Lisp 40 min · Java 120 min · a little theory, then until it runs

A cache is a copy closer by. Faster than the database. It can also tell a fairy tale about a task you already deleted. On station MODULE that's a plaque on the airlock wall: "oxygen: nominal." They hang the plaque so you don't run to the hold every time. If the tank is already empty and the plaque is old — you'll open the hatch anyway. That's a cache.

The database is the hold. The cache is the plaque. HTTP without a cache goes down to the hold every time. With a cache — it looks at the wall. While the plaque is fresh, life is beautiful. Then someone changed the tank, forgot the plaque, and the interview asks: "you updated, the cache is stale — what happens?"

If you answer "well it does it itself" — it doesn't. Itself it only expires on TTL, if you set a TTL. Or it lies forever, if you set "forever" and forgot the eviction.

9.1.1 Why bother, if the database is "fast anyway"

On three tasks Postgres answers instantly, and a cache looks like theater. On three thousand tasks that get read a hundred times a second, the hold starts puffing. A cache makes sense when:

- the same data gets read often;
- you can write less often than you read;
- you **agree** to sometimes show something slightly stale, or you know how to drop the key on write.

A user's task list for 30 seconds is a textbook ceiling. Don't cache "every task of every person forever." That's not speed. That's a museum of lies.

9.1.2 Redis in a box, not "I'll install it on the system"

Image `redis:7`. Not latest. Latest is "whatever environment," and you're already old enough to get angry at sudden surprises.

In `docker-compose.yml` for now you can have **only** Redis, if Postgres is still local. Later, in lesson 20, you'll put everyone in one play. Today this is enough:

```yaml
services:
  redis:
    image: redis:7
    ports:
      - "6379:6379"
```

`docker compose up -d redis`. Then:

```
docker compose exec redis redis-cli ping
```

It should answer `PONG`. That's not a joke and not a password. That's Redis saying "alive."

> **Mac, Windows, WSL**
>
> Port 6379 on a Mac and in Docker Desktop on Windows is the same: from the host, `localhost:6379`. Java on the host (IDEA) talks to `localhost`. Java **inside** the app container in lesson 20 will go to host `redis`, not `localhost`. Write that in pencil on your forehead. In two weeks the forehead will come in handy.

In Spring Boot 3 / Java 21 usually a cache starter plus Redis. Check property names against **your** version, not a 2019 guide. Roughly:

```yaml
spring:
  cache:
    type: redis
  data:
    redis:
      host: localhost
      port: 6379
```

Cache the task list for 30 seconds. On write — throw the key away. Don't "update the cache by hand with the same JSON": throw it away. Let the next GET go to the database and put something fresh. Two actions, not twelve.

9.1.3 Story one: the plaque lies (stale cache)

Someone created a task "fix the antenna." GET `/tasks` — they see it. Then they renamed it to "fix the antenna now." GET — still "fix the antenna," no "now." Ten seconds passed. The person screams that the server is broken. The server isn't broken. **You** are: you cached the list and forgot to drop the key on update.

That's the main interview question about cache. Not "what is Redis." It's: you updated, the cache is stale — what happens, and how do you fix it.

Fix it like this:

1. On any write (create / update / delete) delete the key. Not "later." In the same request, after a successful write to the database.
2. A TTL, so even a forgotten key dies on its own. 30 seconds for study. In battle you pick a TTL, it isn't magic of the number 30.
3. Don't cache things that are different every time (a random quote of the day — fine; "the current user just saved" — no).

> **Slow down**
>
> Reproduce the lie on purpose. Comment out the key drop. Create a task. GET. Update the title. GET immediately — the old name. Wait 31 seconds. GET — the new one. There, you **saw** TTL in your body, not in an article. Put the drop back. Now the second GET is immediately fresh. README: how to repeat both variants. It's useful to know how to break your own station on purpose. Otherwise in the interview you'll say "well in theory," and theory will smile at you with a yellow card.

9.1.4 Story two: a hundred elephants on a footbridge (cache stampede)

The cache expired. No key. In that millisecond a hundred GET `/tasks` arrived. A hundred requests didn't find the plaque, a hundred went down to the hold. Postgres got a herd. That's called cache stampede or thundering herd — a loud herd. On the station: the sensor went dark, and the whole crew ran for one hatch.

On three users you won't see this. In a demo you can draw it in the log: turn Redis off, or set TTL to 1 second, and throw a bunch of curls in a loop. In the log — a bunch of identical `SELECT`s. The database is alive, but that's already a smell.

You don't fix it with "install Redis again." Redis just expired, that's the joke. You fix it like this:

- **drop on write** matters more than a short TTL "so it's always fresh." Fresh and stampede are neighbors;
- one recompute: whoever first didn't find the key goes to the database, the others wait for them (in Java that's already a lock / `synchronized` on the key, or libraries like Caffeine with refresh;

in prod also singleflight — remember the word, don't write the implementation from scratch today);

- sometimes honestly serve something slightly stale while the new key computes. That's adult games. For the textbook it's enough to know the **name of the trouble**.

In an interview this is enough: "if a lot of requests hit an empty key at once — they all go to the database; I'd keep a short lock on recompute or not set a one-second TTL on a hot key." That's better than "Redis is very fast."

9.1.5 Lisp: a stick-and-string Redis

We don't need a socket. We need a hash-table and time. Time in Common Lisp is `(get-universal-time)`, seconds since 1900, like a ship's clock that doesn't care about your timezone.

```
(defstruct centry value expire)

(defun cset (h key value ttl)
  (setf (gethash key h)
        (make-centry :value value
                     :expire (+ (get-universal-time) ttl))))

(defun cget (h key)
  (let ((e (gethash key h)))
    (when (and e (< (get-universal-time) (centry-expire e)))
      (centry-value e))))
```

`cset` puts a value and writes on the crate "good until." `cget` looks at the clock. Expired — `nil`, as if the key was never there. Doesn't have to delete: you can delete, you can leave a corpse. For the étude `nil` is enough.

> **What just happened**
>
> Put `'tasks` for 2 seconds. Immediate `cget` — the list. Wait three seconds in the REPL: `(sleep 3)`. `cget` again — `nil`. You just touched TTL without Docker. Redis in prod does roughly this gesture, only with a network, persistence, and a pile of buttons you don't need yet.

Drop on write is just `(remhash key h)`. Forgot `remhash` — you get a fairy tale until the deadline. Same lie as in Java, only without YAML.

Quest 17.L1 · Lisp

A hash-table that expires by time. `get` after the deadline — `nil`. A stick-and-string Redis.

Quest 17.J1 · Java

Cache the list, 30 seconds, drop on write. README: how to see the cache lie **if you forget** the drop. It's useful to know how to break your own station on purpose.

Quest 17.J2 · Java

A short TTL (a second or two) and a loop of twenty `curl` s in a row while the key is dead. In the log count how many times the service went to the database. Write the number in the README. This isn't a benchmark, it's touching a stampede with a finger. If it's always one SELECT — either the cache didn't turn off, or you got lucky with one thread; then add parallelism (`xargs -P` or two windows).

Bad idea

Don't cache responses with personal data "for everyone" under one key `tasks` . The key must know **whose** this is: `tasks:user:42` . Otherwise a neighbor on the station reads your list titled "don't tell anyone." A cache is not where permissions suddenly vanish.

Hide it on GitHub

A commit like `week17: redis cache ttl 30s` . In the README — two scenarios: the lie without a drop and the truth with a drop. A log screenshot is optional. The words "I saw stale" are required.

9.2 Lesson 18. "Do it later": a queue, not a second server

Lisp 40 min · Java 120 min · a little theory, then until it runs

An email "a task was created" must not knock HTTP over if mail is down. Imagine: you're on watch, someone asks "write down the task and tell ground." If you call ground first and the antenna is silent for three minutes, the person at the airlock stands there and hates you. Right: write the task, answer "got it," and put the note "tell the human" in a tray. The tray is a queue.

First a Spring event or a `BlockingQueue` in the same process. Then RabbitMQ — **one** scenario. Kafka isn't needed today, however many times it shows up in the dream vacancy. Kafka is a log in the next lesson. A queue is a tray. Those are different objects, even if résumés write them through a comma like siblings.

9.2.1 First a tray in the cabin: `BlockingQueue`

Java has a queue that can **wait**. Not spin in `while (true)` and burn CPU, but sleep until someone puts a note.

```
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

BlockingQueue<String> q = new ArrayBlockingQueue<>(16);
```

Capacity 16 — on the station the tray isn't rubber. If you put a 17th while nobody took one, `put` **stops and waits**. That isn't a bug. That's "don't lose notes and don't inflate memory."

Slow down

Two characters. The producer is the watch officer. The consumer is the runner.

```
Thread producer = new Thread(() -> {
    try {
        q.put("tell ground: task 7");
        System.out.println("put");
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "producer");

Thread consumer = new Thread(() -> {
    try {
        String note = q.take();
        System.out.println("took: " + note);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
}, "consumer");

consumer.start();
producer.start();
```

`take` with no note — waits. `put` with no room — waits. `InterruptedException` is "they asked me to stop"; we put the flag back and don't pretend nothing happened. Run it in `java-basics`, not in the live server. Watch the order of the lines: sometimes "put" before "took," sometimes the other way — that's two threads, not a detective story.

`offer` doesn't wait: didn't fit — `false`. `poll` doesn't wait: empty — `null`. For "do it later" in the same process you usually want `put` / `take`: better to stand than to lose "tell the human."
Why not an `ArrayList` plus your own thread with `sleep(100)`? Because `sleep` is fortune-telling. `take` is a contract. In an interview "I spun sleep in a loop" sounds like "I fixed the airlock with a hammer." A hammer sometimes works. Then the hatch falls off.

9.2.2 Spring: an event, not a second microservice

In a monolith it's even simpler than handmade threads. Created a task — published an event. A listener writes to the log. HTTP already answered 201, or is about to, depending on **when** you listen. A rough skeleton:

```
public record TaskCreatedEvent(long id, String title) {}
```

In the service after `save`:

```
publisher.publishEvent(new TaskCreatedEvent(saved.getId(), saved.getTitle()));
```

Listener:

```
@Component
public class NotifyListener {
    private static final Logger log = LoggerFactory.getLogger(NotifyListener.class);

    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void on(TaskCreatedEvent e) {
        log.info("tell ground: task {} \"{}\"", e.id(), e.title());
    }
}
```

`AFTER_COMMIT` — don't shout "created" if the transaction later rolled back. Otherwise the runner sprints down the corridor with news of a task that doesn't exist. This is a tray-queue tied to the fact "it's already in the database."

Don't hook up real mail. A log is enough. SMTP on a textbook station is an invitation to spend Friday on a Gmail password.

9.2.3 When RabbitMQ already, and when not yet

RabbitMQ is a separate broker: you put a message, another process takes it, even if your server died and came back. That's already "a tray in the next room." It makes sense when:

- "tell the human" must survive a JVM restart;
- or another service will do it (you are **not** carving that out today).

One scenario: after create you put JSON on queue `task-created`. A separate `@RabbitListener` writes the same log. Docker image `rabbitmq:3-management`. If the broker doesn't stand up in an evening — fall back to `BlockingQueue` / a Spring event and write that honestly in the README. Honesty is prettier than a checkbox again.

> **Bad idea**
>
> Don't stand up a "notifications microservice" as a separate repository while the monolith's compose doesn't stand up. Carving out a service is easy. Then fixing "the task is in the database but the email went out about a rollback" is not. Pain of transactions between two databases we did not order.

9.2.4 Lisp: a queue as a list with a bad temper

```
(defparameter *q* nil)

(defun enqueue (x)
  (setf *q* (append *q* (list x))))

(defun dequeue ()
  (let ((x (first *q*)))
    (setf *q* (rest *q*))
    x))
```

`append` walks the whole list every time. For an étude that's fine. For a station in battle — like carrying bolts one at a time from the hold to the nose, starting from the hatch every time. In Java for a queue — `ArrayDeque` or `ArrayBlockingQueue`, don't invent `append`.
`drain` — until empty, print:

```
(defun drain ()
  (loop while *q*
        do (format t "~a~%" (dequeue))))
```

What just happened

Three `(enqueue ...)`, then `(drain)`. The queue is empty. Another `(drain)` — silence. FIFO: whoever lay in the tray first gets read first. This is not priority "the antenna beats washing a porthole." Priority is another story, not today's.

Quest 18.L1 · Lisp

A notification queue. `drain` prints them all. Like a watch officer who finally reached the stack of notes.

Quest 18.J1 · Java

After create — an event, the listener writes to the log "tell ground …". A listener test is optional. Working code is required.

Quest 18.J2 · Java

A separate class in `java-basics`: a `BlockingQueue` of 4 slots, the producer puts 8 notes, the consumer takes them. Log: when `put` waited, when `take` waited. README — five lines "what a blocking queue is." No Spring. By hand.

Sunday sabotage

Draw on paper: HTTP request → service → database → event → tray → "email." Where did the human already get 201? If after the database — good. If after the email — the antenna is holding the airlock again. Paper is cheaper than Kafka.

9.3 Lesson 19. Kafka: a log, not a telephone

Lisp 40 min · Java 120 min · a little theory, then until it runs

Kafka is not "call the neighboring service." It's a log: a writer appended to a topic, a reader crawls from an offset. Like a ship's log you cannot erase with a rubber. On MODULE they keep a log like that at the reactor: the watch wrote "pressure 1.2," the next watch reads **from a bookmark**, they don't call the previous one and ask "repeat, I didn't write it down."

If Docker is already a friend: one broker, one topic `task-events`, a producer from the service, a consumer that logs. If it's hard — an honest outline in `docs/kafka.md` and **don't break** the monolith. Kafka on a résumé only if **this** code opens. A word with no repository is a whistle with no air.

9.3.1 How a log is not a telephone (Kafka vs HTTP)

HTTP is a telephone. You call, wait for a tone, hear "201" or "500," hang up. If the other party is asleep, you hang or get an error **now**. Two services over HTTP know each other by name and by the contract "I wait for an answer."
Kafka is a log on the watch desk.

- The writer doesn't know whether anyone is reading. Appended a line — went to fix the antenna.
- The reader comes when they want. Slept an hour — they'll catch up. The bookmark (offset) remembers how far they read.
- Lines are not "rewrite." Usually you append. The old stays. That infuriates until you understand why: you can replay the day, you can debug "who put what at 03:12."
- If two readers have different bookmarks — both read the same thing, each at their own pace. A telephone can't do that unless you're a conference call and an admin.

So the vacancy phrase "integration via Kafka" does not mean "instead of REST." Often REST stays for the human and for "do it now." The log is for "it happened, you'll figure it out." Created a task — HTTP 201 to the human. Into the log — event `TaskCreated`, so later email, search, analytics, whoever, **without holding the receiver**.

> **Slow down**
>
> Compare out loud, even to the cat:
>
> 1. A human POSTs `/tasks`. The service writes to Postgres. Response 201. That's HTTP.
> 2. The same service additionally puts JSON `{ "id": 7, "title": "airlock" }` on topic `task-events`. That's a producer. It doesn't wait for the email to leave.
> 3. Another process (or thread) reads the topic from offset 0, then 1, then 2. That's a consumer. It crashed — stood up, asked the broker "I'm at 2," reads on.
> 4. If instead of a log there was HTTP to a "mail service," and mail is down, the task POST would get 502. A log survives that: the event is already in the log, mail will catch up.
>
> If you mix them up — go back to the paper. Telephone vs ship's log. Not "another queue, only fashionable."

A queue (lesson 18) **takes** the note from the tray. A log **doesn't take**: the bookmark moves forward, the paper stays. So "Kafka = a queue" in an interview is half a point and a polite smile. Finish: "sometimes they use it as a queue, but the model is a log with an offset."

9.3.2 One broker, one topic, no clusters in the kitchen

In Docker for study people often take an image where Kafka no longer needs a separate ZooKeeper (versions change, read the image README today, not "like the 2020 article"). One broker. One topic `task-events`. Replication "three brokers, rack awareness" is not your watch.

A producer from Spring (class names again, check the version):

```
kafkaTemplate.send("task-events", String.valueOf(id), json);
```

Consumer:

```
@KafkaListener(topics = "task-events")
public void on(String payload) {
    log.info("from the log: {}", payload);
}
```

If this stands up — in the README: how to `compose up`, how to see a line in the log after POST. If it doesn't stand up in an evening — **stop**. The monolith isn't guilty. Write `docs/kafka.md`.

9.3.3 An honest outline if the broker ate you

In `docs/kafka.md` in your own words, not a Wikipedia paste:

- why a log if you already have HTTP;
- what a topic is, a partition (at least: "a folder of the log" and "several notebooks so you can write in parallel");
- what an offset is;
- how a consumer group differs from "just two readers";
- what you **didn't** stand up (a cluster, exactly-once, Schema Registry).

That's stronger in an interview than "I took a course." A course doesn't open on GitHub. Your file does.

9.3.4 Lisp: a tiny Kafka in the REPL

A vector, append only, read from an index. Don't delete the old.

```
(defparameter *log* (make-array 0 :adjustable t :fill-pointer 0))

(defun produce (x)
  (vector-push-extend x *log*))

(defun consume (offset)
  (when (< offset (length *log*))
    (aref *log* offset)))
```

`produce` always at the end. `consume` from zero, from one, from seven — like a bookmark. No key "delete event 3." Want "don't read" — move the offset.

> **What just happened**
>
> `(produce 'airlock)` `(produce 'alarm)` `(consume 0)` → `AIRLOCK`. `(consume 1)` → `ALARM`. `(consume 2)` → `nil`. The log is still length 2. You didn't erase the air. You only looked.

Barski would probably have made a game of this. We'll make a log. The game can wait until Sunday.

Quest 19.L1 · Lisp

A vector-log: append only, read from an index. Don't delete the old. A tiny Kafka in the REPL. Barski would probably have made a game of this. We'll make a log.

Quest 19.J1 · Java

Either a living producer/consumer, or a page in your own words: why a log, how it isn't HTTP. Honesty matters more than a checkbox. Truth looks better in an interview than "well I took a course."

Quest 19.J2 · text

In `docs/kafka-vs-http.md` a five-row table: the human's action, HTTP, the log. Example: "the mail service is down." What POST `/tasks` sees in each world. No English bedsheet. Your own words, like a ship's mechanic, not like a landing page.

> **SICP · open only if it itches**
>
> A log as a sequence is a relative of the "stream of events" from the chapters on streams. If it suddenly itches, not instead of Docker. After. The station first has to leave in a box.

9.4 Lesson 20. The box the station leaves in

Lisp 40 min · Java 120 min · a little theory, then until it runs

Until now the station lived in the cabin: IDEA, local Postgres, "works on my machine." Today it has to leave for another person (or for you on another OS) with a command from the README. The box is Docker. The schedule of boxes is Compose. On Windows Docker Desktop is the same compose, the same files. WSL and a Mac don't fight over YAML. They fight over CRLF and over someone forgetting `.dockerignore`.

The check: another person does `docker compose up` and walks the curls from the README. If it's only "works in my IDEA" — that isn't a station yet, that's a mockup in the cabin.

9.4.1 Dockerfile, line by line

A file with no extension, name `Dockerfile`, at the root of `task-manager`. Each line is a layer. Layers are cached: you changed Java code — you don't have to download the JDK again if `FROM` and the dependencies above didn't move. So **manifest first**, then sources. Not the other way around.

Textbook, honest, Java 21:

```
FROM eclipse-temurin:21-jdk AS build
WORKDIR /src
COPY pom.xml .
COPY src ./src
RUN ./mvnw -q -DskipTests package || mvn -q -DskipTests package

FROM eclipse-temurin:21-jre
```

```
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

> **Slow down**
>
> Read it like a watch log, not like a spell.
>
> `FROM eclipse-temurin:21-jdk AS build` — take an image with JDK 21 (you need a compiler). Stage name `build`. Not `latest`. Temurin is a normal distro, the same one they often put in CI.
>
> `WORKDIR /src` — "from here on every command is from this folder." Like `cd`, only in a layer.
>
> `COPY pom.xml .` — dependencies first. If you haven't copied sources yet, Docker can cache the Maven layer when you only change `.java`. For simplicity we `COPY src` right after — a textbook compromise. The adult version: copy `.mvn` and `mvnw`, run `dependency:go-offline`, then `src`. If it didn't fit in an evening — don't die. The main thing: the jar is built in Linux, not "I brought a jar from a Mac."
>
> `RUN ... package` — build the jar inside Linux. `skipTests` in the image is arguable: tests should live in CI, not necessarily inside an image build. If tests poke localhost-Postgres, the image just won't build. So tests — outside, in GitHub Actions.
>
> The second `FROM eclipse-temurin:21-jre` is a **different** image, no compiler. In prod (and in textbook prod) a JRE is lighter. That's multi-stage: from the first stage we take only the jar.
>
> `COPY --from=build ... app.jar` — move the artifact. Not the sources. Not `.git`. Not the host `target/` wholesale.
>
> `EXPOSE 8080` — a plaque "I listen on 8080." By itself it doesn't punch a hole to the host. `ports:` in compose punches the hole.
>
> `ENTRYPOINT ["java", "-jar", ...]` — **exec form** (a JSON array). Not `java -jar` as a string through a shell, unless you need it. That way "stop" signals reach the JVM, not `/bin/sh`.

Next to it, `.dockerignore`:

```
.git
target
.idea
*.md
```

Otherwise half a disk and your secrets from `.env` ride into the context, if they happen to sit nearby. Don't copy `.env` into the image. Never. Even a textbook password `postgres/postgres` is better fed as a variable than baked in.

> **Mac, Windows, WSL**
>
> Building the image: `docker build -t task-manager:dev .` in the project folder. Same on a Mac and in WSL. In PowerShell too, if Docker Desktop is running (the whale in the tray is alive, not dead). If the build screams at `mvnw` — no execute bit: in git, `mvnw` needs the executable

> bit, on Windows it sometimes gets lost. Then in the Dockerfile call `mvn`, and put Maven in the stage, or copy the wrapper carefully. Don't surrender to "well then only IDEA."

9.4.2 Compose: three tenants, one network, and the `localhost` trap

```yaml
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: taskdb
      POSTGRES_USER: task
      POSTGRES_PASSWORD: task
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U task -d taskdb"]
      interval: 5s
      timeout: 3s
      retries: 10
    ports:
      - "5432:5432"

  redis:
    image: redis:7
    ports:
      - "6379:6379"

  app:
    build: .
    depends_on:
      db:
        condition: service_healthy
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/taskdb
      SPRING_DATASOURCE_USERNAME: task
      SPRING_DATASOURCE_PASSWORD: task
      SPRING_DATA_REDIS_HOST: redis
    ports:
      - "8080:8080"

volumes:
  pgdata:
```

Service names `db`, `redis`, `app` are DNS inside the Compose network. Container `app` knocks on host **`db`**, not `localhost`.

> **Slow down**
>
> Why everyone falls into one pit.

> Inside a container `localhost` is **itself**. The app asks Postgres at `localhost:5432` — it knocks on its own empty pocket. There's no Postgres there. Postgres is in the neighboring container named `db`.
> From the host (your curl, your IDEA) `localhost:5432` is the `ports:` forward. So **from a Mac** `psql localhost` works, and **from the app in Docker** it doesn't.
> Two worlds, two names:
> - Java in IDEA on the host → `localhost`
> - Java in container `app` → `db` and `redis`
>
> The database URL is from the environment, not baked in. One `application.yml` with a default for the cabin, in compose — variables that override. Inside an image with an unmarked `localhost` — a trap. Almost everyone falls in. You now know the "wet floor" sign.

`depends_on` without a healthcheck is "the container **started**," not "Postgres **accepted** the password." That's a race: the app died five times while the database yawned. `pg_isready` is a sensor. Let it be there.

Password `task/task` is fine for study. The same password on public GitHub is fine only because this is a textbook sandbox. A real password — no. Not even as a joke. Not even in `docker-compose.override.yml` that you "accidentally" committed.

9.4.3 CI: a robot that doesn't care which IDEA you have

File `.github/workflows/ci.yml`. Not "someday." On every push. A green check on GitHub is nicer than self-esteem.

```yaml
name: ci

on:
  push:
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "21"
          cache: maven
      - name: tests
        run: mvn -B test
```

> **Slow down**
>
> `name: ci` — like signing the watch. The Actions tab will show this name.

> `on: push` — any shove into the repository. `pull_request` — also on a PR, so you see red **before** the merge. You can narrow branches later. Right now let it scream always.
> `jobs.build` — one job. You can have several, you don't need to.
> `runs-on: ubuntu-latest` — someone else's machine, Linux. So "on my Mac" does not interest the robot. Tests must be green without mouse clicks.
> `actions/checkout@v4` — fetch the code. Without this the robot sits in an empty cabin.
> `actions/setup-java@v4` + `temurin` + `21` — the same JDK as in the book. `cache: maven` — don't download half the internet every time.
> `mvn -B test` — `-B` is batch: less interactive, less "press Enter." Tests. Not `package` skipping tests. If a test needs Postgres, either Testcontainers, or H2 **only** in `src/test`, and the README says so. Lying to the robot "green, but I didn't bring up the database" works exactly until the first person who believes the check.

Don't put secrets in the YAML. The robot doesn't need your password to Earth. If a token is ever needed — GitHub Secrets, not a string in a file.

> **Mac, Windows, WSL**
> First push from Windows: watch that `ci.yml` is LF, not CRLF with a surprise. Git usually handles it. If Actions screams at a weird character at the start of the file — BOM. Save UTF-8 without BOM. On a Mac this trouble is rarer. That doesn't mean "Mac is better." It means "Windows Notepad sometimes helps too hard."

9.4.4 A check that the station left

A fresh folder. Someone else's laptop, or your second OS, or just `docker compose down -v` and again.

```
docker compose up --build
```

Then from the host:

```
curl -s localhost:8080/health
```

Login, task list — as in the README. If health is alive and the database isn't — `localhost` inside again. If the port is taken — an old Postgres from month 3 is already sitting on the host. Turn the local one off or change the forward to `5433:5432` and **write that in the README**. A silent port is an enemy.

> **Quest 20.J1** · Java
> Compose brings up the database and the app. The database URL is from the environment, not a baked `localhost` inside the image with no note. Inside the Docker network the database host is often called `db`, not `localhost`. Yes, that's a trap. Almost everyone falls in.

Quest 20.J2 · Java

`.github/workflows/ci.yml` — build and tests. A green check on GitHub is nicer than self-esteem.

Quest 20.J3 · text

In the README a section "Why not localhost": five to seven sentences. Host vs container. Names `db` and `redis`. Where curl from a Mac goes. Where the JVM inside `app` goes. If a classmate still writes `localhost` in the image after that — that's their watch, not yours.

Bad idea

Don't carve out a "notifications microservice" while the monolith's compose doesn't stand up. If you carved it — write in the README what got more painful: data, transactions, debugging. Pain is study material, not shame.

Hide it on GitHub

Commit `week20: compose postgres redis app`. A tag if you want. The README starts with `docker compose up`, not "open IDEA." Otherwise the tag lies.

Sunday sabotage

Break one Dockerfile line on purpose (`FROM` a nonexistent tag, or a typo in `ENTRYPOINT`). Read the error out loud. Fix it. The fear "Docker is too big" passes when the red became specific, not cosmic.

Station law

By the end of the month you can honestly underline in a vacancy: cache, a queue **or** a log, a box. Underline means open a file. Don't underline a word that isn't in `git log`.

Chapter

10 Month 6. Out into open space, meaning the market

Almost no new sacred technologies. There is a product, and there are emails to strangers. Forty minutes of Lisp stay: when they reject you (they will), the parentheses still listen, unlike HR.
This is a strange month. You already know more than it feels like at 2 a.m. The market doesn't know that until you write a README so a stranger gets, in a minute and a half, **what to open**, and until you send a letter that has truth in it, not "stress-resistant." Station MODULE has nothing to do with this. The calendar and your voice do.

> **Station law**
> The measure of the month is slots on the calendar and letters sent, not a new chapter on Kubernetes. Kubernetes can wait. HR will not wait for "just a little more Kafka": they'll just take someone else whose README opens.

10.1 Lesson 21. README as a porthole, not as an attic

Lisp 40 min · Java 120 min · a little theory, then until it runs

The person on the other side looks for a minute and a half. Sometimes forty seconds. They don't read your inner monologue about "I'll finish it later." They look at the top of the screen. If it's empty, or "task manager on spring boot" with no verb, or a wall of emoji — they close the tab. Like a porthole smeared from the inside: from outside it looks like there is no station.
The order that works:

1. README `task-manager`: what it does, a curl or a screenshot, how to run it on a Mac **and** on Windows/WSL, what it's made of.
2. The repo has no `.env` with secrets and no `target/` folder.
3. Commits are not `asdf`.
4. Let `java-basics` live — that's a path, not a shame. Shame is pretending Hello never happened.

Résumé: one page. Stack, two projects with links, how to reach you. Courses at the bottom or not at all. Lisp — under "also."

10.1.1 The black box you have to drag into the sun

Open your README now, before edits. Time one minute. Pretend you're a classmate on Windows, Docker Desktop installed yesterday, a different IDEA. What breaks first?
Usually:

- no run command, just "open the project";
- Postgres is "you know, the usual";

- curl only for GET, and login is silence;
- Java 17 in the text, 21 in the `pom`, or the other way around;
- a screenshot from 2024, the endpoints have moved since.

That isn't shame. That's an attic. Everybody has an attic. You wipe a porthole with a rag, not with self-esteem.

10.1.2 Sample README (steal the structure, not the biography)

Below is a **teaching** text. Put in your URLs, your endpoints, your study-database password. If you have JWT — show two curls: registration and "the list with your token." If you have a session — show the cookie. Don't write "etc." "Etc." in an interview means "I didn't check."

```
# task-manager

Personal tasks over HTTP. Not a tracker for a million people — a study
monolith you can poke CRUD, login, and tests on.

## What it does

- registration and login (JWT)
- your tasks: create / list / update / delete
- list with pagination
- list cache in Redis for 30 seconds, drop on write

## Run (Docker)

You need Docker Desktop (Mac or Windows+WSL2) and free
ports 8080, 5432, 6379.

    docker compose up --build

Check:

    curl -s localhost:8080/health

Then registration, login, POST /tasks. Examples below.

## Run without Docker (cabin)

JDK 21, PostgreSQL 16, Redis 7. Create a database `taskdb`.
Copy `application-example.yml` → `application.yml`
(local password, don't put the file in git).

    ./mvnw spring-boot:run          # Mac, WSL
    mvnw.cmd spring-boot:run        # Windows

## Stack

Java 21, Spring Boot, PostgreSQL, Flyway, Redis, JUnit.
```

```
## What isn't here (so we don't lie)

Kafka is not running in production. The notification queue is an event
inside the monolith and a log. There is no cluster. That's fine.
```

A Postman screenshot or a curl screenshot is a plus, not a requirement. The requirement is that a classmate **without you in chat** sees health `UP`.

> **Mac, Windows, WSL**
>
> Two columns of commands in the README aren't aesthetics, they're survival. Mac and WSL: `./mvnw`, backslash in a multiline curl. Windows cmd: `mvnw.cmd`, `^`. PowerShell is often easier as one line. Write "if curl screams — go into Ubuntu (WSL) and hit it from there, the server on localhost is shared." That one sentence saves an hour of someone else's life and your evening of "well it works on my machine."

10.1.3 A repository that doesn't smell like a dorm

`.gitignore` has to exist: `target/`, `.idea/`, `.env`, `*.iml`, `.DS_Store`. If `target/classes` is on GitHub — a recruiter may not get it, but a developer in the interview will get it and sigh. A sigh in an interview is bad currency.

Secrets: a study `task/task` in compose — fine, that's a sandbox. An email token, a password "like production," an AWS key you dropped "for a second" — not fine. Even in git history. If you already pushed — rotate it, don't pretend "nobody saw." Bots saw.

Commits: `week12: flyway tasks table` reads. `fix`, `asdf`, `doesn't work` — the log of a drunk watch. Don't rewrite the whole history for beauty. Starting next Monday, write them properly.

Let `java-basics` and `lisp-experiments` stay public. That's the path from Hello. A person who hides the first program often hides the second. You don't hide.

10.1.4 A résumé on one page, not a novel

Name, city (or "remote"), email, GitHub, phone if you're ready to hear unknown numbers.

Then **not** "goal: grow in a friendly team." The goal is already clear: a job. Then the stack in one line: Java 21, Spring, PostgreSQL, Docker, Git.

Two projects:

- task-manager — what it does, a link, one detail ("JWT, Redis cache, compose").
- a second, if you have one: shop, android-calc, even a console list. Better a live small one than "microservices planned."

Experience: school, a side job, "I wrote such-and-such." Don't lie about dates. Courses — at the bottom, one line, or not at all. Lisp: "for myself, Common Lisp, study études" — under "also," not in the header instead of Java.

Quest 21.J1 · Java

Rewrite the README so a classmate can bring the project up without you in chat. Mac and Windows. If "well that's obvious" pops up — it isn't obvious.

Quest 21.J2 · text

Eight to twelve lines about task-manager: why, what you built, what you used, a link. No "communicative, stress-resistant."

Quest 21.J3 · text

A résumé on one page. Photograph it or PDF. Read it out loud in forty seconds. If in forty seconds it isn't clear **how you're useful** — cut adjectives, not the stack.

Hide it on GitHub

Check with your eyes, not "git status is green": opened GitHub in incognito, README, no `target/`, there is compose, there is a CI checkmark. Incognito — so you don't confuse "I'm logged in and I know everything" with "a stranger."

10.2 Lesson 22. Rehearsal in the cabin

Lisp 40 min · Java 120 min · a little theory, then until it runs

Three chunks of fifteen–twenty minutes. Not "when I feel like it." The feeling will not arrive at the interview. A person with a laptop will arrive, and the question "tell me about the project."

1. Java and SQL (joining tables, an index, a transaction, how 401 is not 403).
2. "Tell me about task-manager" without a screen, three minutes, then "what if the database dies."
3. A twenty-minute problem out loud. Silence in an interview reads as "I died."

Record your voice. Listening to yourself is embarrassing. Embarrassment is cheaper than a rejection where you never figured out **where** you drifted.

10.2.1 A bad pitch, three minutes (don't do this)

"Well it's like a task manager on Spring. Well Boot kinda brings it up itself, and Java, and a Postgres database. I added endpoints. And Redis, because Redis is in the job posts. I also wanted Kafka but didn't have time. Anyway, regular CRUD. And there are tests, kind of."
What the other side hears:

- you aren't the author, you're a witness to Spring;
- Redis "because job posts" — a red flag;
- Kafka, which isn't there, took story-time from what **is** there;
- "regular CRUD" — you shrunk yourself, even though you have JWT and other people's tasks don't leak.

Spring will **not** tell the interview anything by itself. You will.

10.2.2 A good pitch, the same three minutes (steal the frame)

"I built a personal-task service — REST, Java 21, Spring Boot, PostgreSQL.
The problem is a study problem, but a real one: a to-do list that survives a restart, and you can't read other people's items. So a task has an owner: the user id from login, not from a URL parameter.
The moving parts: registration and login, JWT, CRUD, pagination. Flyway migrations — the schema is in git, not 'in my head.' On list reads there's Redis for 30 seconds, on write I drop the key; without the drop the cache lies, I broke that on purpose and wrote it in the README.
The email queue isn't a separate service, it's an event after the transaction commits and a log line 'tell the human.' I didn't put Kafka in production: I understand a log versus HTTP, and in the repo there's either a tiny broker or notes — whichever is honest.
If the database dies — the API returns 5xx, the cache may show stale for a while; that isn't a rescue, that's a delay. Compose brings up the app, Postgres, and Redis; the README has curl from a Mac and from Windows.
If I rewrote it — I'd pull notifications out more cleanly and add Testcontainers so CI doesn't depend on 'the database on my desk.'"
Time it. If it's over four minutes — cut. If it's under a minute and a half and there's not a single **your** decision — add not technologies, but "why."

> **Slow down**
>
> A frame on paper, four blocks, one sentence each, then grow them:
>
> 1. What it is, for whom.
> 2. One rule you're proud of (your own tasks, a transaction, dropping the cache).
> 3. What it **isn't** (not microservices, not Kafka in production).
> 4. How to bring it up and what breaks if Postgres goes down.
>
> No screen. Hands are allowed. A laptop is not. In an interview they may not give you a screen to poke — they may give you "tell me." The story has to stand on its own.

10.2.3 A Java and SQL chunk, so you don't mumble

Speak short truths. Not a lecture.

- **Joining tables:** `tasks.user_id` points at `users.id`. JOIN is "put them next to each other." Without JOIN you get either a raw id or N+1, which you already smelled in month 3.
- **Index:** a cheat sheet for "where are this user's tasks." Writes a bit slower, finds faster. On three rows Postgres may skip the index — you already wrote that in the README too.
- **Transaction:** several writes as one gesture. It fell over — nothing happened. `@Transactional` on a public service method, not on `private`, not "on every controller just in case."
- **401 and 403:** 401 is "I don't know who you are" (no token / expired). 403 is "I know who, you can't come in here." Mixing them up is like mixing "the airlock is locked because you didn't give a name" and "you gave a name, but you can't go into the reactor."
- **equals and hashCode:** `==` is the same box. For a map key — the contents. A `record` already gives you both.

If you don't remember — say "I can't phrase it exactly right now, here's how I did it in code" and describe your file. "I don't know" plus a map beats a fairy tale about MVCC from the middle.

10.2.4 Lisp: a tiny `eval`, the reason the parentheses are on the schedule at all

Not the whole language. One evening of joy. Being able to evaluate `(+ 1 2)` and `(- 5 3)` and nesting. That isn't "write SBCL." That's understanding that parentheses are a tree, and a tree can be walked.

Slow down

A form is either a number, or a list whose head is an operation and whose tail is arguments. The arguments are forms too: that's why nesting works.

```
(defun ev (form)
  (if (numberp form)
      form
      (let ((op (first form))
            (args (mapcar #'ev (rest form))))
        (cond
          ((eq op '+) (apply #'+ args))
          ((eq op '-) (apply #'- args))
          (t (error "unknown ~a" op))))))
```

`(ev 3)` — a number, return it as is. `(ev '(+ 1 2))` — not a number, head `+`, tail `(1 2)`, run each tail through `ev` (they're numbers), add. `(ev '(+ 1 (- 5 3)))` — the inner `(- 5 3)` becomes `2`, then `(+ 1 2)`.

Break it: `(ev '(+ 1 foo))`. Error. Fix it either by not accepting symbols, or by teaching `foo` to be a variable — that's already the next evening, don't get greedy.

Want a little more language — add `*` with the same `cond`. Want unary minus `(- 3)` — remember that `apply #'-` in Common Lisp already can. Don't drag `lambda` and `defun` into `ev` the same evening: you'll get a compilers textbook, and tomorrow you have a letter. A thimble. Not a station inside a station.

Draw the tree on paper. Without paper, half the people decide `eval` is REPL magic. It isn't magic. It's `first` / `rest` and the faith that a smaller piece evaluates.

What just happened

`(ev '(+ 1 2))` → `3`. `(ev '(- 5 3))` → `2`. `(ev '(+ 1 (- 5 3)))` → `3`. You just wrote a language the size of a thimble. The thimble matters more than "I read about compilers." In an interview you can smile: "I evaluated parentheses for myself." Someone will smile back. That's enough.

Quest 22.J1 · practice

Record a talk about the project. Listen. If every five seconds it's "well Spring does that itself" — retell until **you** show up.

Quest 22.L1 · Lisp

A tiny `eval`: `(+ 1 2)`, `(- 5 3)`, nesting. Not the whole language. One evening of joy. That's the game Lisp is on the schedule for.

Quest 22.J2 · text

On one page: a bad pitch (yours, honestly, how you talk now) and a good one (from the frame in this chapter). Don't invent technologies that aren't in the repo. If there's no Kafka — it isn't in the good pitch either; there's an event-queue and the words "I didn't put it in."

> **Sunday sabotage**
>
> Call a live human for fifteen minutes. Let them ask "what if the database dies" and "how is 401 not 403." A live human is worse than a cat: a cat doesn't wince. Wincing is useful.

10.3 Lesson 23. Letters into the unknown

Lisp 40 min · Java 120 min · a little theory, then until it runs

Five to ten applications a week. Not fifty by copy-paste. Not one "when the letter is perfect." Cover letters are about **your** server, not "I learn fast." A posting with Kafka and kube: you can write if you're ready to say the truth: "I didn't put it in production, here's a monolith and here's a queue." A lie burns on the second question. Sometimes on the first.
HR does not read a soul. HR reads whether there's Java, whether there's a link, whether the letter screams "READY FOR CHALLENGES!!!" Then the letter reaches a person who **can** open GitHub. They need a hook, not a novel.

10.3.1 Sample letter (short, with one truth)

Subject: Java / junior, task-manager — applying for "Backend Java, team X"

```
Hello.

I saw the "Java developer" posting at N: you have PostgreSQL,
REST, and a message queue.

I've spent six months building a study monolith: Java 21, Spring Boot,
PostgreSQL, JWT, tests, Docker Compose.
Tasks are yours only, Redis cache dropped on write.
The notification queue is an in-process event, not a separate service.

I haven't put Kafka in production. I understand a log vs HTTP; the repo
has honest notes / a study broker (as in the README).
```

```
Happy to talk about what's missing for your posting.

GitHub: https://github.com/<nick>/task-manager
Email / telegram: ...

Name
```

What works here: the company name, **one** detail from the posting that you actually have (Postgres, REST), one detail you don't have — honest right away. A link. How to reach you.
What doesn't work: "I am a goal-oriented individual." "I'll consider an internship and junior and middle." "Kafka, Kubernetes, AWS, Grafana" as a list, if of those you only have the word Grafana in a course title.

10.3.2 A bad letter (so you have something to compare)

```
Hello!!! My name is Alex.
I really want to work at your company, I learn fast,
I'm stress-resistant, communicative, ready for challenges.
I've studied Java, Spring, Kafka, Docker, k8s, React.
Please find my résumé attached.
```

Three exclamation marks — a panic sensor. A wall of stack — a lying sensor: React next to k8s with no link means "I read words." No company name. No GitHub. Not **one** sentence about what you **built**. You can not send a letter like that: it saves someone else's time and your own embarrassment.
The second bad genre — a sheet three screens long: childhood, courses, "I loved math in school." The person on the other side is not hiring a school. They're hiring whether the link opens and whether you're lying about Kafka.

10.3.3 Honest answers you can say out loud

In an interview (and in a letter, if they ask) a short truth beats a long fairy tale. Here are drafts. Say them in your own words, don't memorize them as an oath.
"What's your Kafka experience?"
"I haven't put it in production. I stood up a study broker / I read the log model: topic, offset, how it isn't HTTP. In the project, events go through a Spring event after commit. If you have Kafka — I'll need time to learn **your** cluster and conventions, not Wikipedia."
If the broker is actually in your compose and messages show up in the log — say that. Don't undersell. Don't undersell **and** don't say "well basically production."
"Why Redis?"
"The task list is read often, written less. I cached it for 30 seconds, on write I delete the key. I broke the drop on purpose — saw stale. I didn't catch a stampede with three users, but I know a short TTL plus a crowd of simultaneous GETs will hit the database."
"Microservices?"
"One process, one database. I didn't carve notifications out: otherwise the email leaves on a rollback. When a compound gesture survives one transaction — then we talk."

"Why is Lisp on the résumé?"
"Forty minutes a day so I don't get bored and so I see lists. I'm not applying as a Lisper. I can show a tiny eval. If that's not interesting — fine, let's do Java."
"Where do you see yourself in five years?"
Not "CTO of your company." "I write a backend I'm not ashamed to fix at night, and I understand what I'm doing." Boring. Boring beats science fiction.
"Any questions for us?"
Yes. "What does a junior's first month look like?" "Who reviews?" "What's prod on, and will I get to the logs?" The question "do you have Kafka?" makes sense if you're ready for the answer. The question "how much do you pay" — better later, when they bring it up, or very calmly if this is already the second conversation.
"Tell me about HashMap"
"A key, a bucket, on average you get it fast. Two objects with the same meaning must be equal via equals and have the same hashCode, or the map will put them on different shelves. A mutable key is pain: you put it in, you change a field, you can't find it. For ids I use a record."
"What's a transaction?"
"Several writes to the database as one gesture. Either all of them, or none. In Spring — `@Transactional` on a service method. If I create a project and a task, and the title is empty — I want the project gone too. That's in my month-3 code, I can open it."
If you can't open it — don't cite it. Say the idea. A link to a file that isn't there burns faster than a Kafka lie.

10.3.4 Where to send so you aren't yelling into a vacuum

Job sites, internship telegram chats, people from the course, **warm** letters ("you said they were hiring"). Five to ten cold ones a week. If you sent fifty copies overnight — you aren't a hero, you're spam. Spam gets marked.
A posting "three years experience, Kafka, k8s, highload" — you can send one honest letter if the rest (Java, SQL) matches. You can't spend ten hours bolting on Kafka for that one posting. The calendar matters more than their dream stack.
An unpaid internship: decide yourself, don't hero it out of shame. Three months "for experience" with no mentor and no code in git is often more expensive than ten more letters. If there's a mentor, there's code, Java is alive — maybe. If it's "make coffee and look at Jira" — no, even if it says Kafka on the door.

Quest 23.J1 · text

A letter template: company name + one detail from the posting that you **actually** have. Not "ready for challenges."

Quest 23.J2 · practice

Out loud, on a recorder: the Kafka answer (didn't put it in / here's what I did put in), Redis, microservices. Three minutes for all three. Listen. If it sounds like an apology for being alive — rewrite the tone. Truth without a hedgehog in the voice.

> **Bad idea**
>
> Don't paste someone else's README and someone else's curls into the letter. In the second interview they'll ask you to "open your repo and change this." Someone else's repo will not open with your hands.

10.4 Lesson 24. After a miss — the same day, not a month of shame later

Lisp 40 min · Java 120 min · a little theory, then until it runs

They will reject you. Sometimes with silence, which is worse than words. Sometimes "we've decided to move forward with another candidate." Sometimes on a HashMap question where you started a fairy tale. That isn't a sentence on the station. That's a sensor.

A list of questions where you drifted. Usually SQL, `HashMap`, transactions, HTTP codes. Close the hole locally. Not "reread the whole textbook from lesson 0." They repair the station by the sensor too, they don't rebuild from the keel every watch.

The same day, while memory is hot:

1. What they asked (literally, even crooked).
2. What you answered (honestly, "I mumbled").
3. What answer wouldn't have been embarrassing — five to eight sentences.
4. A link to your file if the topic is in your code. If not — a small étude in `java-basics`, not a new framework.

The card lives in a folder `interview-log/` or in a paper notebook. Paper isn't ashamed. You don't have to push to GitHub if it says "I was dumb" inside. You can push without the dumb: just the question and a careful answer. Like a station log.

10.4.1 Typical holes and how not to treat them

Drifted on JOIN — don't buy a "SQL in 48 hours" course. Write three queries on **your** tables: a user's tasks, tasks with the owner's name, task count per person. Out loud.

Drifted on "what if two requests at once" — go back to the counter and `@Transactional`, not to a whole book on the JMM.

Drifted on "tell me about the project" — that's lesson 22, not Kafka. Kafka has nothing to do with it, however much you'd like to hide in a new topic.

Silence after ten letters is a sensor too. Check: is there a link, does compose come up, does the README scream on Windows. Sometimes the market is just slow. Sometimes the porthole is dirty. Dirt you fix. A slow market — more letters, not "I'll wait it out in shame."

10.4.2 Sample card (you can almost copy the form)

```
Date: 2026-08-13
Where: company N, screen
Question: how is 401 different from 403?
What I said: "well both are errors"
How it should go:
  401 — didn't identify (no token, expired).
  403 — identified, can't go here (someone else's task).
  In task-manager a foreign id gives 403 or 404 —
  see TaskService.java, method getOwned.
Étude: no, the code is already there.
```

Three of those sheets in a month — no longer "I'm always dumb," it's a map of holes. A map can be fixed. Eternal dumbness cannot.

Quest 24.J1 · Java

A card for a hole: the question, an answer of five to eight sentences, a link to your file if there is one. That's your new station log.

Quest 24.J2 · text

Three cards in advance, before the first miss: 401 vs 403; why an index; what a transaction does on creating a project with a task. Write how you talk, not like Wikipedia. Then, when it comes up in an interview, it won't be "I should have prepared," it'll be "I've already said this out loud."

> **Station law**
>
> Forbidden after a rejection: deleting the repo, going silent for a month, starting a "full rewrite on microservices." Allowed: a card, one étude, the next letter this week.

10.5 Lesson 25. Code by hand, no magic button

Lisp 40 min · Java 120 min · a little theory, then until it runs

Forty-five minutes, an easy problem, the standard library, even in Notepad. On Windows — Notepad++ or IDEA with no autocomplete, if you have the nerve. Out loud: what's the invariant, what's an example, how fast is this.

A whiteboard (or Miro, or an A4 sheet) is not a place to show you memorized streams. It's a place to show you **think**: an example, an edge, complexity "I'll walk the list / I'll put it in a map."

Three study ones, mean on a timer:

- anagrams: the same letters, different order;
- merge two already-sorted arrays into one sorted array;
- unique emails (as the problem says: dots, plus — read the statement, don't guess Gmail).

> **Slow down**

> Before the timer ticks, rehearse the ritual:
>
> 1. Restate the problem in your own words. If you didn't restate — already a miss, you solved the wrong problem.
> 2. Example: `"ab"`, `"ba"` — yes; `"ab"`, `"abc"` — no.
> 3. Edge: empty string, one character, different case — ask, or pick and say it out loud.
> 4. Idea: sort the characters or a frequency array. Complexity.
> 5. Code. Don't go silent. "I'll put frequencies in a HashMap" — they can hear you're alive.
> 6. Walk the example with a finger. Find the hole yourself, before the interviewer.
>
> If you're stuck for five minutes — say "I see two paths, I'm taking the simpler one, this one." Silence reads as "I died." A bug in the code reads as "a person you can fix things with."

On Java 21 you can use a `record`, you can use `char[]` arrays. You don't need `Stream` for Stream's sake. On a whiteboard a stream is often crooked and you can't run it.
A Lisp version of anagrams is the same gesture, different parentheses. In an interview that's a joke at the end, not the main act. The main act is Java, which they're hiring for.

10.5.1 Anagrams with a finger, while the timer isn't mean yet

The problem: two strings are made of the same letters.

```
static boolean anagram(String a, String b) {
    if (a.length() != b.length()) {
        return false;
    }
    char[] x = a.toCharArray();
    char[] y = b.toCharArray();
    Arrays.sort(x);
    Arrays.sort(y);
    return Arrays.equals(x, y);
}
```

Out loud: "different length — no right away. Otherwise I sort the characters and compare the arrays." Example: `listen` / `silent` — yes. Complexity: sorting, not "I'll walk all permutations." Permutations are a path to heroics and dying on `abcd`.
Second path — frequencies in `HashMap<Character,Integer>`: plus on the first string, minus on the second, all zeros. On a whiteboard sorting is shorter. In the interview say both, take one, write one. Two unfinished paths are worse than one working one.
Merge arrays: two indices `i` and `j`, a third array, whoever is smaller — put that one, advance. Write the tail. Don't `sort` everything after `concat`: they're checking whether you can use the fact that it's **already** sorted.

> **Mac, Windows, WSL**

> Practice in the layout you'll walk into. If the interview is on Windows in their office and you've lived on a Mac — one evening in Notepad won't kill you. Parentheses and semicolons are the same. The buttons are different. Better to swear at home than in a room with a marker.

Quest 25.J1 · Java

Three on a timer: anagrams; merge two sorted arrays; unique emails. The timer is mean. That's the point.

Quest 25.L1 · Lisp

The same anagrams in Lisp. Then in an interview you can say: "I also did this with parentheses." Someone will smile. That's enough.

10.6 Lesson 26. Walk, don't "just a little more Kafka"

Lisp 40 min · Java 120 min · a little theory, then until it runs

The measure of the month is calendar slots, not a new chapter. Lisp forty minutes. Sunday — a walk. Seriously, a walk.
"Walk" means: an application went out, a slot is on the calendar, you slept, the README opens, the pitch was said out loud at least twice. It does not mean: you learned one more broker the night before the call. A night broker in an interview smells like fear. Fear is audible.

10.6.1 What a week looks like when you're already on the market

Monday: two applications from the template, different postings, in each letter one truth from **their** text.
Tuesday: a card from the last rejection or a practice question. Forty-five minutes of whiteboard.
Wednesday: a slot, if they gave you one. If they didn't — two more letters. Not "waiting for fate."
Thursday: Lisp, a small bug in task-manager you promised yourself after the pitch ("Testcontainers," "a clearer 401 error"). One bug, not a universe refactor.
Friday: finish what's broken. Don't open new textbooks. Especially about Kubernetes.
Saturday: the project as a project, not as a sacrifice to the interview.
Sunday: legs, air, station MODULE with no laptop. A brain with no air lies that "one more chapter will save you." It will not.

10.6.2 On the interview day itself

Laptop charged. IDE open on the project, if they ask you to "show." Zoom/Meet checked yesterday, not five minutes before. Headphones. A glass of water. You know the README by heart — not because you memorized it, because you wrote it.
Three minutes late — write. Silence is worse than three minutes.

They asked something you don't know: "I don't know it in production, here's how I understand it, here's where my model lies." Then you can ask "how is it for you." A conversation. Not an expulsion exam, even if it feels like one.
After the call — a card the same day. Even if it feels like "everything went well." Well also has to be repeatable.

10.6.3 A call, an office, a whiteboard in someone else's room

Zoom: camera at eye level, not up the nostrils. A calm background. Phone in airplane mode, a **second** internet channel (hotspot) just in case. The Meet link open ten minutes early, not ten seconds: a browser update loves to coincide with your entrance.
Office: passport, if there's a badge. A notebook. Your own pen. Water. Headphones not needed if they take you to a meeting room. Laptop — if they said "bring it," and a charger for **their** outlets. In Russia the outlets are usually the same; in a coworking space sometimes not.
On the whiteboard write large. Tiny handwriting in a corner is "I'm ashamed of my code." At the top, the problem in your own words. On the left, an example. On the right, code. Erase not from panic, from "this chunk lies, here's a new one." Ask for a minute to think — normal. Ask for a hint after seven minutes of honest struggle — also normal. Ask for a hint after thirty seconds of silence — too soon.
If they share a screen and ask you to open IDEA: turn off autocomplete if you have the nerve, or leave it and **don't** accept the first nonsense Guava whispers. The interviewer can see when you hit Tab without looking.

10.6.4 When the book is over

You've finished the book when:

- on GitHub you can see the path from Hello to a server with a database;
- you fix your monolith without theater;
- you honestly say what you haven't done.

An offer is not a button at the end of a chapter. It's a probability. A plan raises it. It doesn't promise a date. Nobody promised station MODULE wouldn't fall apart either — and it's still there. Somehow.
The evening before a slot: not Kafka. README out loud. One curl. The pitch once on a recorder. Sleep. If you can't sleep — a scrap of paper with the four pitch blocks under the pillow, not a Habr article "how Raft works." Raft can wait for Tuesday after a rejection. Or after an offer. It doesn't care.
If the letters are silent — it isn't necessarily another six months of Kafka. Further in the book there's a short spare hatch: Android, a calculator, Studio from developer.android.com. Same Java. Different screen.

Quest 26.J1 · practice

A calendar for two weeks: application slots (dates), one slot "rehearse the pitch," one slot "three problems on a timer." Hang it on the wall or put it in a calendar you **see**. An invisible plan is a dream, not a watch.

Quest 26.L1 · Lisp

Forty minutes on the day they rejected you. Any station étude, even `ev`, even an oxygen sensor. Parentheses don't know about HR. That's the point of the forty minutes.

> **Hide it on GitHub**
>
> On the GitHub profile: pin `task-manager`. Profile bio — one line, not a quote about passion for code. The link in the résumé opens. Check from a phone.

A profile with no pinned repo is an attic. The person on the other side will not hunt `task-manager` in a list of twenty forks of other people's tutorials. Pin it. Unpin the `spring-petclinic` fork if you didn't fix it. A fork with no commits is not a project.

Check the links from a phone: GitHub, résumé PDF, email in the profile. A recruiter often sits on the subway, not at a big monitor. If the README on a narrow screen is a wall — cut. If the email has a typo — the letters "went into space," and the station has nothing to do with it.

And with that the book as a textbook ends. Next — other people's rooms, other people's questions, your files. Station MODULE isn't going anywhere. Neither are the parentheses. Go.

If you're scared — that's normal. It's scary to fix a reactor in month one, it's scary to hit Send. Hit it anyway. A letter that didn't go out doesn't open. A station that wasn't turned on doesn't scream with sensors — and that isn't calm, that's the silence of an empty corridor.

Tea. Send. A card. Parentheses.

> **Bad idea**
>
> Don't vanish into "I'll rewrite everything in Kotlin + Kafka + k8s in August." August will end, there will be no letters, the monolith will be dead on a `rewrite` branch. The market takes what's alive. Alive is what comes up today.

Chapter

11 If they won't take you for backend

Sometimes the backend market looks at a junior the way an airlock looks at a meteor: politely, but it doesn't open. Kafka in the posting, "a year of experience," silence after ten letters. That doesn't mean you threw six months away. Java didn't go anywhere. The door is just different: a **pocket**. A phone in your hand is the same computer, only without a server room and with a Back button.

Android apps have been written in Kotlin for a while, but Java is alive there, and Studio eats it. You already know classes, `if`, and "why is it underlined in red." That's enough to build a calculator and not die. If it goes well — you'll catch Kotlin in a couple of weeks: it's from the same family as Java, just less water.

You don't have to put Lisp on a phone. The forty minutes of parentheses can stay. Station MODULE doesn't care which screen you fix sensors from.

This isn't "drop backend." This is a spare hatch. A calculator in an evening is cheaper than another three months of "I'll tweak Kafka and then they'll definitely take me." People need phones every day. They need servers too, but server HR is fussier.

11.1 Where to get Studio (and why the first evening is pain)

The site: developer.android.com/studio — official, for Mac, Windows, and Linux. Don't download "android studio cracked" from page fourteen of search. That isn't savings, that's an adventure with a virus. The station is already on duct tape; it doesn't need a virus.

It takes a long time to install: Studio itself, the Android SDK, an emulator, system images. Disk space is gigabytes, not "one more VS Code." Twenty gigabytes is a normal bill, not a bug. If the disk is a hundred and ninety-eight are used — either clean, or a USB phone without an emulator. An emulator on a packed disk dies bashfully, with android-30 errors that aren't about android-30.

> **Mac, Windows, WSL**
>
> On a Mac, Studio is native; Apple Silicon (M1/M2/M3) wants an **arm** emulator image, not an old x86 "because the 2019 guide said so." Studio usually hints itself. If you downloaded x86 onto an M-chip — it'll either be slow through translation, or not at all. Don't hero it.
>
> On Windows this is a **native** window, not WSL: the emulator needs Hyper-V or its own engine, and inside Ubuntu-in-Windows it often sulks. WSL runs your Spring beautifully. A phone emulator — no, don't mix up the doors.
>
> Linux: possible, but GPU drivers and KVM are a separate watch. If Linux is "so I have it" and the main laptop is Windows — put Studio on Windows. One pain is better than two.

First launch: a wizard, a Google login is optional for a calculator (you can skip it). Then SDK Manager will download another chunk of the internet. Tea. Not two teas and "I probably broke it" — that's how an Android progress bar breathes.

New Project → **Empty Views Activity** (not Compose, don't panic at the word Jetpack yet). Language: **Java**. Minimum SDK — whatever Studio suggests, not the most ancient and not the newest "pixels from the future only." A package name like `com.example.calc` — you'll replace it later if `example` is embarrassing. For study, `example` isn't embarrassing.

The project indexes for minutes the first time. The Gradle strip at the bottom wriggles. A red `R` in the first two minutes is often "hasn't woken up yet," not "you're an idiot." Build → Make Project. Wait. If after Make `R.id` is still red — Build → Clean Project, then Make again. Sometimes Studio just hasn't woken up. Like a sensor on MODULE.

11.1.1 Emulator versus a cable: two ways to see a button

Emulator: Device Manager → Create → some Pixel without a 16K screen and without folding wings. System image — a fresh stable one, with Google APIs, not "Android Canary from Mars." First start — tea. Sometimes two. A cold emulator start on a weak laptop is a small industry sin, not yours.

If the laptop is weak, the fan screams, the emulator draws a slideshow — **don't hero it**. Turn on USB debugging on your own phone and run on hardware:

1. Settings → About phone → seven taps on Build number. "You are now a developer" appears. Yes, seven. That isn't a 2012 joke, it's still like that.
2. Developer options → USB debugging.
3. A cable that **moves a file**, not "charge only" from a drawer.
4. Allow debugging on the phone screen when it asks. The "always from this computer" checkbox — to taste.
5. At the top of Studio the device should change name from the emulator to the phone model.

Not embarrassing. On a real station they also look at the sensor in the corridor first, they don't build a corridor simulator.

> **Bad idea**
>
> A work phone is a separate world: corporate profiles, "nope." For study, your own device is fine. Don't put debug on grandma's phone "for a minute." Grandma didn't sign up for this quest.

If Studio doesn't see the phone: another cable, another port, on Windows — the manufacturer's driver, File transfer / PTP mode, not Charge only. That's the dullest pain in the chapter. Dull pain is treated by trying things, not by a new textbook.

11.1.2 Licenses, a hypervisor, and Gradle that is "still downloading"

The first build can spend half an hour downloading the internet into `~/.gradle`. It isn't stuck if the bar is moving. If it hasn't moved in ten minutes — look at the network, a company proxy, a VPN that cuts dl.google.com. On Windows antivirus sometimes pets every jar in the Gradle cache —

then the build is "eternal." An exclusion on the `.gradle` folder saves nerves. Don't turn antivirus off entirely "per a forum guide."

SDK licenses: Studio usually asks Accept. If the log says `You have not accepted the license agreements` — SDK Manager, checkboxes, Accept. On the command line you'll someday meet `sdkmanager --licenses`. Today a GUI button is enough.

An emulator on Windows without virtualization is a slideshow or a refusal. In BIOS/UEFI you turn on VT-x / AMD-V. On a home laptop that's sometimes "I don't know what BIOS is." Microsoft's docs on Hyper-V and WSL2 are already on the machine if you installed Docker Desktop: virtualization is often already on. If Docker is alive and the emulator isn't — it isn't necessarily BIOS; check whether Hyper-V ate everything, and which image you downloaded.

On a Mac "the emulator won't start" is often fixed by: download the system image, Cold Boot Now in Device Manager, don't panic at the first `Waiting for target device`. Cold boot is like restarting the station, not like "delete everything."

11.1.3 Red that doesn't mean "you're an idiot"

`SDK location not found` — Studio doesn't know where the SDK is. File → Settings → Appearance & Behavior → System Settings → Android SDK (on a Mac that's Android Studio → Settings). The path has to exist. If you moved to a new disk — the path is old, Studio screams. Point it at the new one, don't reinstall everything from scratch.

`Failed to install the following Android SDK packages` — network, license, or the disk ran out. Download through SDK Manager. Don't download a "sdk-tools" zip from page fourteen.

`INSTALL_FAILED_INSUFFICIENT_STORAGE` on the emulator — Wipe Data in Device Manager or a new, bigger AVD. Phone: clear the cache, that's daily life, not Java.

`Duplicate class` / Gradle conflict — you added a library Studio already pulls. For a calculator you barely need libraries. If you wandered into Firebase "for five minutes" — there it is. Throw it out until the calculator counts.

A red `R` after renaming an id in XML — Make Project. The name in XML and `R.id.name` have to match letter for letter. `plus` and `Plus` are different. Android doesn't forgive case, like Java classes, only meaner, because the error doesn't surface right away.

Invalidate Caches is the last button, not the first. First Make, Clean, "are you running the right module." Invalidate is like restarting the whole station: it helps when nothing makes sense anymore, and sometimes it doesn't help.

Also: you're running the wrong Run Configuration — the green arrow is on a library, not on `app`. On the left pick `app`. Or the device is "No devices": the emulator didn't wait, the phone fell off. That isn't Java. That's a plug in the wrong outlet.

11.2 A calculator on the knee

Two numbers, plus and minus, then multiply and divide, the answer on the screen. Not a bank and not Telegram. But you'll see: a button → your Java code → digits. Like `Scanner` in the console, only with a finger.

In `activity_main.xml` (the Code tab, not just Preview):

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/a"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="first number"
        android:inputType="numberDecimal|numberSigned" />

    <EditText
        android:id="@+id/b"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="second number"
        android:inputType="numberDecimal|numberSigned" />

    <Button
        android:id="@+id/plus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="+" />

    <Button
        android:id="@+id/minus"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="−" />

    <Button
        android:id="@+id/mul"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="×" />

    <Button
        android:id="@+id/div"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="÷" />

    <TextView
        android:id="@+id/out"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```
        android:textSize="24sp"
        android:text="?" />
</LinearLayout>
```

`LinearLayout` — boxes in a column. `orientation="vertical"` — like a list in a corridor. `EditText` — a field you poke. `inputType` hints to the keyboard "numbers, minus and a dot are ok," not a whole novel. `TextView` — only shows, you don't type there with a finger.

The Preview tab lies less often than it used to, but Code is the source of truth. If Preview is pretty and the phone has no button — you were looking at the wrong XML.

`dp` is "density," so a button isn't microscopic on one phone and huge on another. `sp` is the same for text, and it also respects "large font" in settings. Don't set text height in `dp` without a reason: a person with bad eyesight has a right to large letters. `match_parent` — take the parent's width/height. `wrap_content` — by contents. If everything is `wrap_content` in a column — the fields are fine. If a `ListView` is also `wrap`, it may shrink and you'll decide the list "doesn't work."

In `MainActivity.java` — the same Java, only the entrance isn't `Scanner`, it's fields on the screen:

```
package com.example.calc;

import android.os.Bundle;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        EditText a = findViewById(R.id.a);
        EditText b = findViewById(R.id.b);
        TextView out = findViewById(R.id.out);
        Button plus = findViewById(R.id.plus);
        Button minus = findViewById(R.id.minus);
        Button mul = findViewById(R.id.mul);
        Button div = findViewById(R.id.div);

        plus.setOnClickListener(v -> show(out, num(a) + num(b)));
        minus.setOnClickListener(v -> show(out, num(a) - num(b)));
        mul.setOnClickListener(v -> show(out, num(a) * num(b)));
        div.setOnClickListener(v -> {
            double y = num(b);
            if (y == 0) {
                out.setText("can't");
                return;
            }
            show(out, num(a) / y);
```

```
        });
    }

    private void show(TextView out, double value) {
        out.setText(String.valueOf(value));
    }

    private double num(EditText field) {
        String s = field.getText().toString().trim();
        if (s.isEmpty()) {
            return 0;
        }
        try {
            return Double.parseDouble(s);
        } catch (NumberFormatException e) {
            return Double.NaN;
        }
    }
}
```

Replace the package `com.example.calc` with the one Studio picked itself. Green arrow. If `R.id` is red — Clean; sometimes Studio just hasn't woken up.

11.2.1 Empty field, letters, and divide by zero — this isn't "later"

Empty input: a person opened the app and immediately poked `+`. Without a check, `parseDouble("")` screams `NumberFormatException`, the activity dies, Android draws "the app has stopped." That isn't how you fix an oxygen sensor on the station: you don't blow the compartment because someone forgot a digit.

In the code above, empty → `0`. That's a teaching gesture, like in the console "zero if they stay quiet." You can do otherwise: write "enter two numbers" in `out` and don't compute. Pick one and stick to it. The main thing is **don't crash**.

`NumberFormatException`: `inputType` is not an iron curtain. Weird stuff still leaks from the keyboard sometimes, and from paste even more. `parseDouble` has a right to be offended. Catch it. `NaN` in the answer beats a crash; even better — the word "nope" on the screen. `Double.NaN` in `+` infects the sum, the user will see `NaN`. You can check `Double.isNaN` and write "not a number." Do that when the calculator already adds.

Divide by zero on a phone doesn't blow the station: `double` will be `Infinity`. The user is better off with "can't" than `Infinity` like in a math textbook they didn't ask for. We don't use integer division: the fields are decimal.

> **Slow down**
>
> With a finger, not with eyes.
>
> 1. Enter `2` and `3`, tap `+`. Should be `5.0` or `5`.
> 2. Clear both fields, tap `+`. Must not crash. Either `0.0`, or your message.

3. Enter `8` and `0`, tap `÷`. "can't", not a crash, not `Infinity`.
4. Multiply `1.5` and `2`. If you got `2` — you cut the fraction to `int` somewhere. Go back to `double`.

Quest A.J1 · Java

The `×` and `÷` buttons are alive. Divide by zero — the text "can't", the app is alive. A screenshot in the README of the `android-calc` repo.

Quest A.J2 · Java

An empty field doesn't kill the activity. Catch `NumberFormatException`. Write in `out` that the input is bad, or carefully treat empty as zero — but then **write in the README** which rule. A hidden "empty is zero" with no words is a trap for future you.

You can be explicit, without NaN:

```
private Double parseOrNull(EditText field) {
    String s = field.getText().toString().trim();
    if (s.isEmpty()) {
        return null;
    }
    try {
        return Double.parseDouble(s);
    } catch (NumberFormatException e) {
        return null;
    }
}
```

In the listener: if `null` — `out.setText("enter numbers")` and `return`. Empty and "asdf" behave equally honestly. Zero as "the human is silent" sometimes lies: `2+` empty became `2+0`. For a station calculator it's better to yell than to invent.

11.3 Lifecycle: why an activity isn't "just main"

In the console there was `main`. You called it — it lives until it ends. On a phone the screen is longer-lived and fussier: the person minimized, someone called, they rotated the phone, the system is low on memory. An activity isn't an eternal process. It's a watch with a log "I'm on screen now / I'm not."

Roughly, no dissertation:

- `onCreate` — born. Here `setContentView`, here you find buttons. The first time, or after the system killed it.
- `onStart` — they **can** see me now.
- `onResume` — I'm in the foreground, they're poking me.

- `onPause` — something covered you: the shade, a call, another activity on top. Don't compute big sums here, here is "pause."
- `onStop` — I'm not visible (they went to another screen, they pressed Home).
- `onDestroy` — that's it, I'm off the watch. Not a fact that "the user pressed back": the system also kills when it's stingy with memory.

Screen rotation by default often **kills** the activity and creates it again. So a field you kept in an ordinary variable may forget the digit. For a five-minute calculator that's tolerable. For a task list — already insulting. Later there will be `ViewModel` and "state survives rotation." Today it's enough to know the name of the trouble: "I rotated the phone and the counter reset — that isn't magic, they recreated `onCreate`."

> **Slow down**
>
> Put a log in every method:
>
> ```
> Log.d("CALC", "onCreate");
> ```
>
> The same in `onStart`, `onResume`, `onPause`, `onStop`. Logcat at the bottom of Studio, filter `CALC`. Press Home. Come back. Rotate the phone. Read the tape. This isn't ritual for ritual's sake: you'll see that Home is not equal to "the process died," and rotation often is equal to "the activity died, a new one was born."
>
> If Logcat is empty: Logcat tab on the right, the device is the same one as the green arrow, level Debug, not Error. The app package in the filter if there are too many logs: the emulator is chatty, like MODULE's sensors at a full moon. `Log.d` isn't visible at Error level — you're looking at the wrong shelf.

Officially and without our jokes: the lifecycle guide in the Android docs. English. We warned you back in the preface.

Rotation and the death of an activity aren't the only trap. The system can kill the process when you've been in the background a long time, and then return you to `onCreate` with `savedInstanceState`. For the calculator: if you want the two fields not to wipe, put the strings in a `Bundle` in `onSaveInstanceState` and read them in `onCreate` if `savedInstanceState != null`. That's already "state survives the watch." Not required the first evening. Required to know that a class variable is not a safe.

> **Quest A.J3** · Java
>
> Logs on `onCreate` / `onStart` / `onResume` / `onPause` / `onStop`. README: what prints if you press Home and come back, and what — if you rotate the phone. In your own words, five sentences. Not a paste from the guide.

11.4 A task list on the phone — a sketch, not a third textbook

The calculator counts. You already wrote a to-do list in the console and over HTTP. On a phone it's the same `ArrayList<String>`, only the list is **drawn**. Don't drag in Firebase, maps, and "like Instagram" right away. First a list that **shows**.

A sketch for one or two evenings, not for Google architecture:

1. A new screen or the same one: an `EditText` "title", an "add" button, a `ListView` or `RecyclerView`. The first time, `ListView` + `ArrayAdapter` is simpler. `RecyclerView` is more correct and meaner. If `ListView` worked — you already won. Then `RecyclerView`.
2. In memory, `ArrayList<String> tasks`. You added — `adapter.notifyDataSetChanged()`. You minimized the app — the list died. Like month 1. Familiar.
3. Want immortality — `SharedPreferences` (a small key-value pile) or a file, like `tasks.txt`. SQLite / Room — when the list is alive and you're angry at the file. Not the first evening.
4. Later you can tap an item and mark it done. Later — talk to **your** `task-manager` over HTTP. That's already "the phone as a client to your server." Pretty for a portfolio. First let the list live without a network: on the subway, server MODULE doesn't answer.

XML for the sketch (don't copy ids blindly if they're already taken):

```
<EditText
    android:id="@+id/title"
    android:hint="task"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />

<Button
    android:id="@+id/add"
    android:text="add"
    android:layout_width="match_parent"
    android:layout_height="wrap_content" />

<ListView
    android:id="@+id/list"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1" />
```

`layout_weight` in a vertical `LinearLayout` is "take the rest of the screen." Without it the list sometimes shrinks to a strip and you think Android is broken. The layout is squashed, not Android.

In Java — a sketch, not sacred code:

```
List<String> tasks = new ArrayList<>();
ArrayAdapter<String> adapter = new ArrayAdapter<>(
        this, android.R.layout.simple_list_item_1, tasks);
list.setAdapter(adapter);
add.setOnClickListener(v -> {
    String t = title.getText().toString().strip();
    if (t.isEmpty()) {
        return;
```

```
    }
    tasks.add(t);
    adapter.notifyDataSetChanged();
    title.setText("");
});
```

Don't put an empty task. You already set that rule in the service on the server. Same here: the watchstander doesn't take an empty note.
Want the list to survive the Back button — a sketch on `SharedPreferences`:

```
prefs.edit().putString("tasks", String.join("\n", tasks)).apply();
```

Read on start: `getString`, `split`, into the list. That isn't a database. That's `tasks.txt` in a pocket. For ten items it's enough. For a thousand — Room, when you live that long. Don't write your own SQL "because on the backend it's Postgres": on a phone the daily life is different, the disk is different, "the process got killed" is different.
Network to your `task-manager`: on the emulator `localhost` is **the emulator itself**, not your Mac and not Docker. People often write `10.0.2.2` as "the host machine of the emulator." On a real phone on the same Wi-Fi — the computer's LAN IP, and the server has to listen on more than `localhost`. That's the same trap as `db` versus `localhost` in Compose, only in a pocket. While the list has no network — you don't owe yourself that trap yet. When you go there — a README, not "well it's local."
Cleartext HTTP: Android by default doesn't like `http://` without TLS. A study server at `http://10.0.2.2:8080` may get `Cleartext HTTP traffic not permitted`. For study, `usesCleartextTraffic` or a network-security-config — yes, with a note in the README "local only." You don't take that to prod. Like the study password `task/task`: fine at home, not onto Earth.

Quest A.J4 · Java

A list sketch: a field, a button, a `ListView`, three tasks with a finger. A screenshot. Doesn't have to survive rotation and disk. In the README, honestly: "memory only for now." Honesty is again prettier than a checkbox "almost like Todoist."

Hide it on GitHub

Repo `android-calc`. README: how to open it in Android Studio, which API, a screenshot. That's a portfolio too — "I didn't only poke JSON in Postman." If you got to the list — either a second repo `android-tasks`, or a folder in the same one. Don't mix calculator and list in one chaos without words unless you have to.

Bad idea

Don't drag in Firebase, maps, and "like Instagram" right away. First a calculator that **counts**. Then a task list on the phone — you already wrote it in the console.

11.5 Manifest, strings, and a second door

`AndroidManifest.xml` is the app's passport. Without it the system doesn't know what the activity is called and which one to start. Studio writes it itself. You go in there when:

- you add a **second** activity — you have to declare it, or "activity not found";
- you need the internet: `<uses-permission android:name="android.permission.INTERNET" />`;
- you want the icon and the name on the home screen to not be `com.example.calc`.

Don't hardcode the home-screen name as a string in the manifest. Put it in `res/values/strings.xml`:

```
<resources>
    <string name="app_name">MODULE calculator</string>
    <string name="hint_a">first number</string>
</resources>
```

On the screen XML: `android:hint="@string/hint_a"`. Why: tomorrow a translation, the day after a large font, and you aren't crawling through every layout. For study, two strings are already an adult gesture. For Instagram — required. We aren't Instagram. Two strings are still worth it.

A second activity is a second watch. New → Activity → Empty Views Activity, name `AboutActivity`. Studio usually writes it into the manifest itself. The jump:

```
startActivity(new Intent(this, AboutActivity.class));
```

`Intent` is a note: "open this class." You can put extra data:

```
Intent i = new Intent(this, AboutActivity.class);
i.putExtra("last", out.getText().toString());
startActivity(i);
```

On the other side: `getIntent().getStringExtra("last")`. That isn't HTTP. That's a note in a pocket while both screens are in one process. The process got killed — the note died. For "about the app" it's enough. For a task list that has to survive rotation — `onSaveInstanceState` or ViewModel, not Intent.

> **Slow down**
>
> An "about the station" button on the calculator. Second screen: two sentences and the last computed number. Back — the system arrow, you don't program it the first evening. If the number on the second screen is empty — you forgot `putExtra` or typo'd the key. Keys are strings, Java won't check them. A `last` / `Last` typo is a classic, like `R.id.plus` and `Plus`.

> **Quest A.J5** · Java
>
> A second activity, Intent, `putExtra` with the calculator's last answer. A screenshot of both screens. If the system Back doesn't return to the calculator — you probably wandered into some weird launchMode in the manifest. Put it back the way Studio set it.

11.6 Gradle: why the green arrow sometimes thinks for half an hour

An Android project isn't one `javac Hello.java`. It's Gradle: a script that downloads the SDK, dependencies, resources, glues an APK. Two files you touch:

- `build.gradle.kts` (or `.gradle`) of the `app` **module** — `minSdk`, `targetSdk`, dependencies;
- sometimes the root one — repositories to download from.

`minSdk` is the oldest Android you'll agree to. The lower it is, the more grandma phones, the more "this API isn't there." For study, leave what Studio suggested. Don't set 14 "to cover everyone": you won't cover them, but `HttpURLConnection` will suddenly be from 2009.

Dependencies for a calculator are almost unneeded. `implementation(...)` shows up when you go into the network or into Room. Each line is a promise to download the internet and someday catch a version conflict. An empty calculator with twenty libraries is a station with no walls but a spare-parts catalog.

The first build downloads the world. The second is faster. `Build → Clean` — when Studio is losing its mind, not every time "just in case." Clean every time is like rebuilding the reactor before every sensor check.

> **Mac, Windows, WSL**
>
> The Gradle cache lives in the home folder: `~/.gradle` on a Mac and in WSL, `C:\Users\<you>\.gradle` on native Windows. Android Studio is a native Windows window, not WSL: don't look for the cache inside Ubuntu, it isn't there. The disk ran out — most often it's `.gradle` that bloated, plus emulators in `Android/Sdk`.

11.7 Rotation memory and SharedPreferences without mysticism

Rotation kills the activity. Class variables die. An `ArrayList` in a `MainActivity` field — too. So a "three tasks with a finger" list empty after rotation isn't an Android bug, it's you storing daily life in a creature the system has a right to kill.

A minimal safe for one screen:

```
@Override
protected void onSaveInstanceState(Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putString("a", a.getText().toString());
    outState.putString("b", b.getText().toString());
}
```

In `onCreate`:

```
if (savedInstanceState != null) {
    a.setText(savedInstanceState.getString("a", ""));
    b.setText(savedInstanceState.getString("b", ""));
}
```

A `Bundle` is a key-value bag the system **may** return on recreation. Not a database. Not a file. Rotation and "not enough memory in the background" — yes. You reinstalled the app — no.

To survive being closed — `SharedPreferences`:

```
SharedPreferences prefs = getSharedPreferences("station", MODE_PRIVATE);
prefs.edit().putString("tasks", String.join("\n", tasks)).apply();
```

Read: `getString("tasks", "")`, `split`, into the list. `apply()` is async, doesn't block the finger. `commit()` waits for disk, rarely needed for study. Don't put passwords in there "like in Spring Security." That's a note on the fridge, not a safe.
ViewModel is a pocket that survives rotation but doesn't survive process death. For an evening list you can skip it. For an Android-junior interview the word is useful to know: "screen state is in the ViewModel, disk is in the repository, the activity only draws." You've already heard this as controller / service. Same gesture, different hatch.

Quest A.J6 · Java

Calculator: two fields survive rotation through a `Bundle`. README: what happens if you kill the app from Recents and open it again (the fields are **not** required to survive — that's honest). Then, if you want — a task list in SharedPreferences, so Recents no longer wipes it.

11.8 The phone as a client to your server

When a list in memory gets boring, you can visit **your** `task-manager`. That's pretty for a portfolio: "here's the backend, here's the pocket." Read the traps first, then write `HttpURLConnection`.

- Emulator: `localhost` is the emulator itself, not the Mac and not Docker. The host machine is often `10.0.2.2`. A real phone on Wi-Fi — the computer's LAN IP, the server listens on `0.0.0.0`, not only `127.0.0.1`.
- HTTP without TLS Android cuts. A study cleartext flag — in the README "local only."
- You can't do network from the UI thread: `NetworkOnMainThreadException`. Either `new Thread`, or (better) an executor. Otherwise the calculator "froze" on slow Wi-Fi, and that's you going to the network from `onClick`.
- JSON: the same text curl printed in month 2. `org.json.JSONObject` is built in. Dragging Jackson onto a phone for two fields is greed.

A sketch, not a sacred client:

```
new Thread(() -> {
    try {
        URL url = new URL("http://10.0.2.2:8080/health");
        HttpURLConnection c = (HttpURLConnection) url.openConnection();
        c.setConnectTimeout(3000);
        int code = c.getResponseCode();
        runOnUiThread(() -> out.setText("health " + code));
    } catch (Exception e) {
        runOnUiThread(() -> out.setText(e.getMessage()));
    }
}).start();
```

`runOnUiThread` — back to the screen: you can touch a `TextView` only from the UI watch. Otherwise a rare crash that on the emulator is "sometimes." The station doesn't like those "sometimes."

> **Quest A.J7** · Java
>
> A "ping" button: GET `/health` of your server from the emulator **or** from the phone. In the README — which URL and why not `localhost`. If the server isn't running — an error on the screen, not a crash. That's a client.

11.9 Kotlin from the same button, not from a new textbook

Studio can mix Java and Kotlin in one module. Don't. But seeing once what a listener looks like is useful — so "Kotlin required" in a posting doesn't sound like another language from Mars.

```
plus.setOnClickListener {
    out.text = (num(a) + num(b)).toString()
}
```

The same `setOnClickListener`, fewer letters, `text` instead of `setText`. Old ideas. New syntax. Learn it when the Java calculator **counts**, the list **shows**, rotation **doesn't wipe** the fields. Earlier — three magics, zero fixes.

Official courses: developer.android.com/courses — often Kotlin right away. You can watch them like subtitles: "aha, that's my `onCreate`." Not as an order to throw Java out.

11.10 An interview from a phone, if the hatch became a door

They won't ask Kafka. They'll ask:

- how an activity isn't `main`, what happens on rotation;
- why `strings.xml`, why the manifest;
- why network isn't from the UI thread;
- how a study `ListView` differs from a "proper" `RecyclerView` (`RecyclerView` reuses cards, a list of a thousand doesn't draw a thousand views at once);
- where to put state: screen / ViewModel / disk.

The answers are already in this chapter plus month 1 (the list) and month 2 (HTTP). Don't learn "Android Architecture Components" as a pack before the first screen. A screen that opens, plus an honest README — again stronger than a certificate.

An APK onto your phone without the store: Build → Build APK(s). The file is in `app/build/outputs/apk/`. On the phone, allow installs from this source. That isn't publishing to Google Play. Play is a console, a signature, fees, moderation. For a junior, "here's an APK and here's a screenshot" is enough. The store can wait for an offer or for stubbornness.

11.10.1 `RecyclerView` in two words, when ListView is no longer embarrassing

`ListView` for study is honest. In an interview they'll say "why not RecyclerView?" Short answer: RecyclerView doesn't draw a thousand rows at once — it keeps about ten cards on screen and **reuses** them when you scroll. For three tasks that's theater. For a feed like the grown-ups — a necessity.
It's meaner to write: your own `Adapter`, your own `ViewHolder`, `onCreateViewHolder` / `onBindViewHolder`. Studio can scaffold it. Don't copy the first 2016 guide with `android.support` — that's a dead package. Now it's `androidx.recyclerview`.
Hatch rule: ListView alive → then RecyclerView. Not the other way around. Otherwise you'll spend a week befriending a holder, and the plus button still won't add.

11.10.2 When the app "has stopped"

A red dialog on the phone isn't mysticism. In Logcat look for `FATAL EXCEPTION` and the first line of **your** package, not three screens of `android.view`. Then ordinary Java: NPE, the wrong id, network from the UI thread, `findViewById` returned null because the layout is different.
`NullPointerException` on `plus.setOnClickListener` — the button isn't in **this** XML. You're looking at `activity_main`, and `setContentView` pulls another layout. Or the id `plus` is in Preview, and in Code there's a typo. Match `R.id` and XML letter for letter.
`CalledFromWrongThreadException` — you touched a `TextView` from `new Thread`. Bring it back through `runOnUiThread`. That isn't "Android is hard." That's two watches: network and screen, and they aren't the same.
Read the stack top-down, like in Spring: the first **your** line matters more than `Handler.dispatchMessage`. Paste it into search without the package name `com.example.calc` if that's embarrassing. The error doesn't change from that.

11.11 Next — the docs, not a third textbook

Official and to the point:

- developer.android.com/docs — how an activity, a layout, a lifecycle are built. English. We warned you.
- developer.android.com/courses — Google's courses, often with Kotlin. Don't panic: new syntax, old ideas.
- Search in Studio: double Shift, the name `findViewById` — you'll jump into the sources. That isn't magic, that's Java.

If backend took you after all — you can leave Android alone. If it didn't — a calculator in an evening is cheaper than another three months of "I'll tweak Kafka and then they'll definitely take me." People need phones every day. They need servers too, but server HR is fussier.
Kotlin, when the calculator gets boring: the same `onCreate`, the same ids, fewer words around them. Don't learn Kotlin **instead of** the first screen. The screen matters more than the dialect.
View Binding and Jetpack Compose are words Studio whispers from the doorstep. Binding — so you don't write `findViewById` by hand. Compose — another way to draw a screen, without XML.

Both can wait until `findViewById` and XML make sense to you. Otherwise you'll learn three magics at once and fix none. The station likes one sensor per evening.

Dark theme: a `Theme` in `themes.xml`, or just a dark background on the `LinearLayout` and light text. For a portfolio, a "light / dark" screenshot looks more grown-up than one more dependency. Don't install a theme library for two colors.

> **Sunday sabotage**
>
> Add a `×` and `÷` button if somehow they still aren't there. Then a darker theme. Then "hey, this could be a station calculator: watts and oxygen." If you slip into a game — that's normal. That's how Barski meant it, only he had orcs and we have orbit.

> **Station law**
>
> A spare hatch isn't shame. Shame is a year of pretending "just a little more backend" and not having a single screen that opens without you. A green arrow on a phone counts. Even if it only computes `2+2`.

If in a week the calculator is boring and backend is still silent — it isn't embarrassing to stay on the phone longer. Same Java. An Android-junior interview also asks about lists, lifecycle, and "why it crashed." You already know what to say about empty input. That's more than "I downloaded Studio."

Forty minutes of Lisp fit this hatch too. Parentheses don't compete with XML. Evening: XML and buttons. Morning: `ev` or an oxygen sensor. Station MODULE isn't jealous of a Pixel.

A screenshot in the README isn't vanity. It's proof that you **had** a green arrow, not "well it kind of launched." A phone on the desk, a calculator with `2+2`, a date. Like curl in month 2. Same gesture, different porthole.

If Studio is still downloading the SDK while you read this — let it download. Tea. Then Empty Views Activity, Java, two buttons. Not a third textbook. Not Firebase. Two buttons. Station MODULE also once started with one sensor that lied, and a person who noticed.

Hatch checklist, if you forgot:

- Studio from developer.android.com, not "cracked";
- Empty Views Activity, Java;
- emulator or USB debugging;
- plus, minus, multiply, divide, empty doesn't crash;
- a screenshot in `android-calc`.
- a second door (`AboutActivity`) — if it isn't scary anymore;
- fields survive rotation — if it really isn't scary anymore.

Next, Google's docs, not a third textbook and not Firebase "for five minutes." Station MODULE also once started with one sensor.

Chapter

12 Answers

Try it yourself first. Really. If you jump straight here — the brain gets nothing, only the warm feeling of "I kind of know this." Below is one working version, not holy scripture.

12.0.1 Lesson 0

Answer 0.L1

```
(+ 1 2 3 4 5)        ; 15
(* 2 3 4)            ; 24
(* (- 10 3) 2)       ; 14
```

Answer 0.J1

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Name: Alex");
        System.out.println("Date: 2026-08-13");
    }
}
```

In the terminal: `javac Hello.java && java Hello`.

Answer 0.G1

`git add -A && git commit -m "week0: hello name" && git push`. In the browser on GitHub you can see the files on branch `main`.

Answer 0.L2

`(+ 2 "station")` — type error: `+` eats numbers. `(concatenate 'string "MODULE-" "1")` → `"MODULE-1"`. Log: two different errors side by side, so in a month you aren't hunting through memory.

Answer 0.J2

A second `println` with the number of compartments. Without running `javac` again the screen still shows the old number: the JVM runs the `.class`, not the `.java`. After `javac` — the new one. That's why there are two steps.

12.0.2 Lesson 1

Answer 1.L1

```
(defun dock-ok (speed)
  (if (< speed 5) "soft" "too fast"))
```

Answer 1.L2

```
(defun airlock (inside outside)
  (if (= inside outside) 'open 'sealed))
```

Answer 1.J1

```
Scanner in = new Scanner(System.in);
int a = in.nextInt(), b = in.nextInt(), c = in.nextInt();
double avg = (a + b + c) / 3.0;
System.out.println(avg);
if (a < 0 || b < 0 || c < 0) System.out.println("ALARM");
```

Integer division `(a+b+c)/3` throws away the fraction — for an average use `3.0`.

Answer 1.J2

```
int secret = 1 + (int) (Math.random() * 10);
int g = new Scanner(System.in).nextInt();
if (g == secret) System.out.println("right");
else if (g < secret) System.out.println("too low");
else System.out.println("too high");
```

Answer 1.L3

```
(defun lamp (energy)
  (cond
    ((>= energy 80) 'green)
    ((>= energy 40) 'yellow)
    (t 'red)))
; (lamp 100) GREEN, (lamp 80) GREEN, (lamp 79) YELLOW,
; (lamp 40) YELLOW, (lamp 0) RED
```

Answer 1.J3

```
while (true) {
    System.out.print("Energy 0..100: ");
    String raw = in.nextLine().trim();
```

```
    try {
        int n = Integer.parseInt(raw);
        if (n >= 0 && n <= 100) {
            System.out.println(n);
            break;
        }
    } catch (NumberFormatException e) {
        // prompt again
    }
}
```

12.0.3 Lesson 2

Answer 2.L1

```
(defun rooms-report (lst)
  (loop for room in lst
        for i from 1
        do (format t "~a. ~a~%" i room)))
```

Answer 2.L2

```
(defun has-reactor-p (lst)
  (not (null (member 'reactor lst))))
```

Answer 2.J1

Loop `while (true)`, `String line = in.nextLine()`, `split(" ", 2)`. `add` — `tasks.add(parts[1])`. `list` — numbered `for`. `del` — `remove(Integer.parseInt(n) - 1)` with a bounds check. `quit` — `break`.

Answer 2.J2

```
Map<String, Integer> ages = Map.of("Ann", 20, "Bob", 15, "Cyd", 33);
ages.forEach((name, age) -> {
    if (age > 18) System.out.println(name);
});
```

`Map.of` — an immutable map; enough for a teaching printout.

Answer 2.L3

```lisp
(defun prepend-airlock (rooms)
  (cons 'airlock rooms))
; (defparameter *r* '(corridor))
; (prepend-airlock *r*) => (AIRLOCK CORRIDOR), *r* is still (CORRIDOR)
```

Answer 2.J3

```java
Map<String, Integer> energy = new HashMap<>();
energy.put("reactor", 80);
energy.put("antenna", 20);
energy.put("garden", 0);
energy.forEach((room, n) -> {
    if (n < 30) System.out.println(room);
});
```

12.0.4 Lesson 3

Answer 3.L1

```lisp
(defun clamp (n lo hi)
  (min hi (max lo n)))
(defun spend (energy cost) (clamp (- energy cost) 0 100))
(defun recharge (energy delta) (clamp (+ energy delta) 0 100))
(defun simulate (costs)
  (let ((e 100))
    (dolist (c costs) (setf e (spend e c)))
    e))
```

Answer 3.L2

```lisp
(defun render-room (room) (format nil "[~a]" room))
(defun print-rooms (lst)
  (dolist (r lst) (format t "~a~%" (render-room r))))
```

Answer 3.J1

`done 2` → `store.complete(2)` finds the `Task` with `id == 2` and calls `complete()`. Print: id, title, the done flag.

Answer 3.J2

Field `private int priority` defaults to `0` in the constructor. A setter or an `add(title, priority)` parameter. `list` prints `priority + " " + id + " " + title`.

Answer 3.L3

```
(defun docking-score (speed fuel angle)
  (let ((speed-part (if (< speed 5) 10 0))
        (fuel-part (if (> fuel 20) 5 0))
        (angle-part (if (< (abs angle) 10) 3 0)))
    (+ speed-part fuel-part angle-part)))
```

Answer 3.J3

In `TaskStore`: loop over `tasks`, `t.getId() == id`, otherwise `null`. In `main`: `show 2` → `findById(2)`, on `null` print `missing`.

12.0.5 Lesson 4

Answer 4.L1

```
(defun max-list (lst)
  (if (null (rest lst))
      (first lst)
      (let ((m (max-list (rest lst))))
        (if (> (first lst) m) (first lst) m))))
```

Answer 4.L2

```
(defun filter-positive (lst)
  (cond
    ((null lst) nil)
    ((> (first lst) 0)
     (cons (first lst) (filter-positive (rest lst))))
    (t (filter-positive (rest lst)))))
```

Answer 4.J1

On start `titles = TaskFile.load(Path.of("tasks.txt"))`, rebuild `Task` with new ids in order. On `quit` — `titles` from `store.all()` via `getTitle()`, `save`. Empty file / no file → empty list.

Answer 4.J2

README: JDK 21, `javac` / `java` or IDEA, a command table, a sample dialogue. Without that, week 4 is not closed.

Answer 4.L3

```
(defun rooms-length (lst)
  (if (null lst) 0 (+ 1 (rooms-length (rest lst)))))
(defun find-room (room lst)
  (cond
    ((null lst) nil)
    ((eq (first lst) room) lst)
    (t (find-room room (rest lst)))))
```

Answer 4.J3

Four disasters in the README: no `javac` → install JDK 21 and PATH; `cannot find symbol` → import or compile every `.java`; no `tasks.txt` → empty list, not a crash; letters instead of a number → prompt again.

12.0.6 Lesson 5

Answer 5.L1

```
(mapcar (lambda (n) (min 100 (* n 2))) '(10 60 40))
; (20 100 80)
```

Answer 5.L2

```
(defun offline-modules (alist)
  (mapcan (lambda (pair)
            (if (eq (cdr pair) 'down) (list (car pair)) nil))
          alist))
```

Answer 5.J1

`addIncreasesSize`; `completeMarksDone`; `completeMissing` — `assertThrows(NoSuchElementException.class, () -> store.complete(99))` or `assertFalse(store.complete(99))`, if you chose `boolean`.

Answer 5.J2

Temporarily comment out `task.setDone(true)`. Red test. Put it back. README: `mvn test` or IDEA's green arrow.

Answer 5.L3

```lisp
(defun energy-of (module alist)
  (cdr (assoc module alist)))
; (energy-of 'reactor *energy*) => 80, 'garden => NIL
```

Answer 5.J3

`git checkout -b feat/tests`, commit, `git switch main`, `git merge feat/tests`. In the README three lines of `git log --oneline`.

12.0.7 Lesson 6

Answer 6.L1

```lisp
(defun http-not-found ()
  (format nil "HTTP/1.1 404 Not Found~%Content-Type: text/plain~%~%missing"))
```

Answer 6.L2

```lisp
(defun json-task (id title)
  (format nil "{\"id\":~a,\"title\":\"~a\"}" id title))
```

Answer 6.J1

```java
@GetMapping("/tasks/{id}")
public ResponseEntity<Task> one(@PathVariable int id) {
    return tasks.stream()
        .filter(t -> t.getId() == id)
        .findFirst()
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}
```

Answer 6.J2

Five curls: health, list empty, POST, list one, GET by id. Expected JSON, short.

Answer 6.L3

```lisp
(defun http-created (body)
  (format nil "HTTP/1.1 201 Created~%Content-Type: application/json~%~%~a" body))
; (http-created (json-task 2 "antenna"))
```

Answer 6.J3

`curl -i -X POST ... -d '{}'` — copy the `HTTP/1.1 ...` line into the README. Before lesson 7 it's often 500; after you catch it — 400. Honestly record what you see.

12.0.8 Lesson 7

Answer 7.L1

```
(defun create-task (title)
  (if (or (null title) (string= title ""))
      (error "title required")
      (list :title title :done nil)))

(defun try-create (title)
  (handler-case (create-task title)
    (error () 'bad-request)))
```

Answer 7.J1

A controller with `private final TaskService service` and a constructor. Annotations `@RestController`, `@Service`. No list in the controller.

Answer 7.J2

Service `boolean delete(int id)` / `void` + an exception. Controller 204 `noContent` or 404. A unit test of the service without `@SpringBootTest`.

Answer 7.L2

```
(defun blank-p (s)
  (or (null s) (string= (string-trim '(#\Space) s) "")))
(defun create-task (title)
  (if (blank-p title)
      (error "title required")
      (list :title (string-trim '(#\Space) title) :done nil)))
```

Answer 7.J3

`create` catches `IllegalArgumentException` → 400 and `{"error":"title required"}`. Two `curl -i` in the README: `""` and `"  "`.

12.0.9 Lesson 8

Answer 8.L1

```
(defun log-info (fmt &rest args)
  (format t "INFO ")
  (apply #'format t fmt args)
  (terpri))
```

Answer 8.J1

`LoggerFactory.getLogger . log.info("created id={}", id) .`

Answer 8.J2

```
@GetMapping("/tasks")
public List<Task> all(@RequestParam(required = false) Boolean done) {
    if (done == null) return service.findAll();
    return service.findAll().stream()
        .filter(t -> t.isDone() == done)
        .toList();
}
```

Answer 8.L2

```
(defun log-warn (fmt &rest args)
  (format t "WARN ")
  (apply #'format t fmt args)
  (terpri))
; in try-create: success — log-info, error — log-warn and 'bad-request
```

Answer 8.J3

`log.warn("task not found id={}", id)` in the service or the controller on 404. In the README a line from the Run console / `mvnw` .

12.0.10 Lesson 9

Answer 9.L1

See `save-tasks` / `load-tasks` in the chapter text. In a new session `(load-tasks "module-tasks.lisp-data")` .

Answer 9.J1

Three `INSERT` , `SELECT id, title FROM tasks;` , copy the output into the README in a code block.

Answer 9.J2

`spring.datasource.url=jdbc:postgresql://localhost:5432/taskdb`. `JdbcTemplate.query` with a `RowMapper`. `GET /tasks` reads the DB.

Answer 9.J3

Wrong password: `password authentication failed`. No database: `database "taskdb" does not exist`. No table: `relation "tasks" does not exist`. Three quotes in the README, then a working yml.

12.0.11 Lesson 10

Answer 10.L1

```
(defun tasks-of (pid projects)
  (cdr (assoc pid projects)))
; example: '((1 . (a b)) (2 . (c)))
```

Answer 10.J1

`JpaRepository<Task, Long>`, `POST` → `save`, Flyway `V1__tasks.sql` matches `@Table`. `ddl-auto: validate`.

Answer 10.J2

`GET` builds a DTO: project id, name, a list of `{id,title}` without a nested project on every task.

Answer 10.J3

A field on the entity with no column → `Schema-validation: missing column [...]`. Quote the first `Caused by` in the README. Then `V2__...sql` with `ALTER TABLE ... ADD COLUMN`.

12.0.12 Lesson 11

Answer 11.L1

```
(defun apply-ops (state ops)
  (if (find 'fail ops)
      state
      (append state ops)))
```

Teaching model: "all or nothing."

Answer 11.J1

One `@Transactional` method: `projectRepository.save` then `taskRepository.save`. Empty title — `IllegalArgumentException` before the second save or after a check. Test: the project count did not grow.

Answer 11.J2

In the log, count the `select`s. Put the number in the README. If it's one query per item — you saw N+1.

Answer 11.J3

`JOIN FETCH` or a DTO query. In the log one (or two: count + data) `select` with a `join`, not a spray of identical `where project_id=?`. Two numbers: before / after.

12.0.13 Lesson 12

Answer 12.L1

```
(save-tasks "t.dat" '((1 . "airlock")))
; new SBCL session:
(equal (load-tasks "t.dat") '((1 . "airlock")))
; T, otherwise (error "station lost the log")
```

Answer 12.J1

Four tests: create returns an id; get of unknown — empty/404; update changes the title; delete then get is empty.

Answer 12.J2

README: Postgres version, `flyway`, `mvn test`, curl CRUD, the H2 limitation if there is one.

12.0.14 Lesson 13

Answer 13.L1

```
(defparameter *users* (make-hash-table :test 'equal))
(defun register (email password)
  (when (gethash email *users*) (error "exists"))
  (setf (gethash email *users*) (sxhash password))
  t)
(defun login (email password)
  (eql (gethash email *users*) (sxhash password)))
```

Answer 13.J1

`Task.userId` from `Authentication`. Repository `findByUserId`. Security: authenticated for `/tasks/**`.

Answer 13.J2

Service: if `task.getUserId() != current` → 404 (hiding the fact) or 403. A test with two users.

Answer 13.J3

Without `Authorization` — 401. Someone else's id — 404 or 403 (as you chose in 13.J2). Your own — 200 and a body. Three curls in the README.

12.0.15 Lesson 14

Answer 14.L1

Scan: `(loop for (u . tid) in pairs if (eql u user) collect tid)`. Index: `gethash user table`. For 10000 that's already noticeable in the REPL if you spin it thousands of times.

Answer 14.J1

`V2__idx_tasks_user.sql`: `CREATE INDEX ...`. Controller `Page<Task>` + `Pageable`.

Answer 14.J2

Two `EXPLAIN ANALYZE` blocks in the README: Seq Scan vs Index Scan (depends on volume; on three rows the optimizer may skip the index — pour in thousands of rows).

Answer 14.J3

`page=0&size=5` and `page=1&size=5`. Ids in `content` don't overlap. If they overlap — Pageable never reached the repository.

12.0.16 Lesson 15

Answer 15.L1

```
(defun balanced-p (s)
  (let ((st nil))
    (loop for ch across s do
      (cond
        ((char= ch #\() (push #\) st))
```

```
        ((char= ch #\[) (push #\] st))
        ((member ch '(#\) #\]) :test #'char=)
         (unless (and st (char= ch (pop st)))
           (return-from balanced-p nil)))))
    (null st)))
```

Answer 15.J1

```
record UserId(long value) {}
// a record already gives equals/hashCode.
// If you write your own class — Objects.equals / Objects.hash(value).
```

Answer 15.J2

Reverse: two indices `i,j`. Frequency: `HashMap<Character,Integer>`. Brackets: `ArrayDeque<Character>` as a stack.

Answer 15.J3

A class without equals: `get(new UserId(1))` → `null` after `put(new UserId(1), ...)`. `record UserId(long v)` — finds it. Two `println`s in the README.

12.0.17 Lesson 16

Answer 16.L1

Instead of `*energy*` — `(defun tick (energy cost) ...)`. The call chain passes the new value. The global is not read.

Answer 16.J1

Two `Thread`s, 100000 `inc` each. Print `n`. Then `synchronized void inc()`. Compare.

Answer 16.J2

Not code: a file `core-answers.md` or your voice. The checklist from the lesson 16 text.

Answer 16.J3

`AtomicInteger n = new AtomicInteger();` in `inc` — `n.incrementAndGet()`. Three runs: the hole / 200000 / 200000.

12.0.18 Lessons 17–20

Answer 17.L1

```
(defstruct centry value expire)
(defun cget (h key)
  (let ((e (gethash key h)))
    (when (and e (< (get-universal-time) (centry-expire e)))
      (centry-value e))))
(defun cset (h key value ttl)
  (setf (gethash key h)
        (make-centry :value value
                     :expire (+ (get-universal-time) ttl))))
```

Answer 17.J1

`@Cacheable` / a hand-rolled `RedisTemplate`. On update `convertAndSend` is not needed — `delete(key)`. README: reproducing stale without delete.

Answer 17.J2

TTL 1–2 s, the key goes stale, a burst of GET. In the log several identical SELECT — stampede. README: the number and how you launched it (`xargs -P` or two windows).

Answer 18.L1

`enqueue` via `append` or a tail. `drain`: `dolist` while the queue isn't empty, `format t`.

Answer 18.J1

`@ApplicationEvent` `TaskCreatedEvent` + `@TransactionalEventListener`. A log in the listener.

Answer 18.J2

`ArrayBlockingQueue<>(4)`, producer `put` 8 times, consumer `take`. Log "waiting on put" / "took". No Spring.

Answer 19.L1

```
(defparameter *log* (make-array 0 :adjustable t :fill-pointer 0))
(defun produce (x) (vector-push-extend x *log*))
(defun consume (offset)
  (when (< offset (length *log*)) (aref *log* offset)))
```

Answer 19.J1

Either `KafkaTemplate.send("task-events", json)`, or an honest `docs/kafka.md`.

Answer 19.J2

A table: POST of tasks while the mail is dead — HTTP 201 vs the log catching up. Five lines in your own words, not Wikipedia.

Answer 20.J1

Compose: services `db`, `app`, `depends_on`, a Postgres healthcheck. App: `SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/taskdb`.

Answer 20.J2

```yaml
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 21 }
      - run: mvn -B test
```

Answer 20.J3

From the host, curl → `localhost:8080`. The JVM in the `app` container → host `db`, not `localhost`. `localhost` inside a container is the container itself.

12.0.19 Lessons 21–26

Answer 21.J1

Check: a new directory, README only. Stopwatch. If you get stuck — a hole in the README, not in "obviousness."

Answer 21.J2

Template: "REST for personal tasks. Java 21, Spring Boot, PostgreSQL, JWT. Registration, CRUD, pagination. Link: github.com/…/task-manager".

Answer 21.J3

One page: name, contact, stack in a line, two projects with links. Courses at the bottom. Lisp under "also this." Forty seconds out loud.

Answer 22.J1

3 minutes: problem → demons (API, DB, auth) → one technical choice you're proud of → what you'd rewrite.

Answer 22.L1

```
(defun ev (form)
  (if (numberp form)
      form
      (let ((op (first form))
            (args (mapcar #'ev (rest form))))
        (cond
          ((eq op '+) (apply #'+ args))
          ((eq op '-) (apply #'- args))
          (t (error "unknown ~a" op))))))
```

Answer 22.J2

Bad: "Spring itself, CRUD, I wanted Kafka." Good: problem → my own tasks/JWT → a cache with invalidation → what's missing (Kafka in prod) → compose. Only what's in git.

Answer 23.J1

"Saw vacancy N. Built CRUD with PostgreSQL and tests (link). I haven't shipped Kafka in prod; queue/events live in the monolith. Happy to talk."

Answer 23.J2

Out loud: "haven't shipped it in prod, here's log vs HTTP; Redis — TTL and invalidation, I broke stale; I didn't carve out microservices because of the transaction." No apology for existing.

Answer 24.J1

One card = question + answer + a link to a project file (`TaskService.java:40`).

Answer 24.J2

Three cards before the interview: 401 vs 403; an index; the transaction that creates a project+task. In your own words, not Wikipedia.

Answer 25.J1

Anagram: sort the char array or frequencies. Merge: two indices. Emails: normalize `+` and dots in the local part per the problem statement.

Answer 25.L1

```
(defun anagram-p (a b)
  (string= (sort (copy-seq a) #'char<)
           (sort (copy-seq b) #'char<)))
```

Live coding matters more than the ideal: it compiles, there are examples, you can say the complexity.

12.0.20 Chapter "how a computer is built"

Answer 0b.1

Mac / WSL:

```
mkdir module-cabin
cd module-cabin
echo "station MODULE" > note.txt
pwd
ls
cat note.txt
```

The second line in `note.txt` is exactly what `pwd` printed (for example `/Users/you/dev/module-cabin` or `/home/you/module-cabin`). PowerShell: `New-Item`, `Get-Location`, `Get-Content`. If the path is "wrong" — you created the folder from somewhere other than where you were standing. That is the answer.

Answer 0b.2

Pantry: the file `Hello.java` (and after `javac` — `Hello.class`). Counter: bytecode and variables of the running `java` process. Cook: the CPU executing JVM instructions. The Run icon only asks the chef (the OS) to start that recipe.

Answer 0b.3

The bytes are the same. What broke is the contract for how to read them. UTF-8 puts a Russian letter in two bytes, Windows-1251 expects one byte per letter — so the eye sees mojibake. Restore UTF-8 — the word is `модуль` again. Don't retype the mojibake "as is" into a new file: then you're corrupting the numbers themselves.

12.0.21 Station log MODULE

Answer S.L1

```
(defun ticks (n)
  (loop repeat n do (tick))
  *energy*)
```

Recursion: `(defun ticks (n) (if (<= n 0) *energy* (progn (tick) (ticks (- n 1)))))`. `clamp-energy` is already in `tick` — you won't go below zero.

Answer S.L2

```
(defun exits-report ()
  (dolist (pair *doors*)
    (format t "~a -> ~a~%" (car pair) (cdr pair))))
```

Answer S.L3

```
(defun drop (item)
  (if (member item *pocket*)
      (progn
        (setf *pocket* (remove item *pocket*))
        (setf (cdr (assoc *here* *at*))
              (cons item (stuff-here)))
        (format t "dropped ~a~%" item))
      (format t "not in pocket: ~a~%" item)))
```

Check: pick it up in the reactor, `walk` to the corridor, `drop`, go back — the reactor is empty.

Answer S.L4

```
(defparameter *lamp* 'dead)
(defun whack ()
  (cond
    ((not (eq *here* 'galley)) 'wrong-room)
    ((not (member 'wrench *pocket*)) 'need-wrench)
    (t (setf *lamp* 'ok) 'fixed)))
```

In the corridor `look`: if `*lamp*` is `'ok` — a different line about the ballast. Otherwise an honest comment: it's a prop.

Answer S.L5

```
(defun save ()
  (save-world "module-save.lisp-data"))
(defun load ()
  (if (probe-file "module-save.lisp-data")
      (load-world "module-save.lisp-data")
      'no-save))
```

`probe-file` — "is the jar on the shelf." After you tape the leak, `*fixed*` must be in the plist, or the load will forget the duct tape.

Answer S.L6

For five rooms an honest table from `airlock`: corridor 1, galley 2, reactor 2, cupola 3. Otherwise BFS/recursion with a queue. `airlock` → `cupola` is not 1: there's no hatch, the path goes through the corridor and the galley.

Answer S.L7

```
(defun coffee ()
  (cond
    ((not (eq *here* 'galley)) 'wrong-room)
    ((< *energy* 4)
     (format t "steam. energy ~a~%" *energy*)
     'steam)
    (t
     (setf *energy* (- *energy* 4))
     (if (member 'mug *pocket*)
         (format t "into Ш's mug. energy ~a~%" *energy*)
         (format t "on the floor. energy ~a~%" *energy*))
     'sludge)))
```

12.0.22 Java lab

Answer J.L1

First the pit: `nextInt` + `nextLine` → empty brackets. Then:

```
int energy = Integer.parseInt(in.nextLine().trim());
String cmd = in.nextLine();
```

Comment: `nextInt` leaves the `end of line` in stdin, and the next `nextLine` will eat it.

Answer J.L2

`split(" ")`, length 3, `parseInt` in a `try`. `energy` / `oxygen` 0..100, `temp` −20..40. Not a number — a message, `continue`. The process does not die.

Answer J.L3

`Files.exists` — no file → defaults and the line `no station.txt, using defaults`. Each line gets its own `try` around `parseInt`, a broken one — `System.err`, the other keys live. Not one `catch (Exception e)` around the whole load.

Answer J.L4

```
return "{\"energy\":" + e + ",\"oxygen\":" + o + ",\"temp\":" + t + "}";
```

Validator green. A room name with a `quote` and no escape — red. README: Jackson escapes strings and nesting so you don't assemble JSON by concatenation.

Answer J.L5

`HttpClient.send` GET `https://httpbin.org/get` — status and `body.substring(0, Math.min(200, body.length()))`. Second URL `/status/404` — print 404, that is not a broken client. Third — `catch`, the exception class name + `getMessage()`.

Answer J.L6

Three files, state only in `Station`.

```
javac Station.java StationFile.java StationDash.java
java StationDash
```

README: those commands + a dialogue of `status` / `set energy 55` / `save` / `quit`.

Answer 26.J1

Two weeks on a calendar: dates of emails, a pitch slot, a slot for three problems. Visible to the eye, not "in your head."

Answer 26.L1

On the day of the rejection — any station étude, even `(ev '(+ 1 2))`. Parentheses don't know HR.

12.0.23 Android

Answer A.J1

Listeners on `mul` and `div`. If the divisor is 0 — `out.setText("can't")`, not `Infinity` and not a crash.

Answer A.J2

Empty: `trim().isEmpty()` before parse. Catch `NumberFormatException`. README: empty means zero **or** the label "enter numbers."

Answer A.J3

`Log.d` in `onCreate` / `onStart` / `onResume` / `onPause` / `onStop`. Home: pause+stop, return start+resume. Rotate: often destroy+a new onCreate.

Answer A.J4

`ArrayList` + `ArrayAdapter` + `ListView`. Three finger-sized tasks. README: "while it's in memory." A screenshot.

Answer A.J5

`AboutActivity`, `Intent` + `putExtra("last", ...)`. On the other side `getStringExtra`. The system Back with no magic of your own.

Answer A.J6

`onSaveInstanceState` / read it in `onCreate` for two fields. Recents killing the process — fields may vanish, that's in the README. The list — optional `SharedPreferences`.

Answer A.J7

Not the UI thread, GET health, URL `10.0.2.2` from the emulator. An error on screen, not a crash. README: why not localhost.

12.0.24 Macros, CLOS, the live image

Answer M.L1

`macroexpand-1` on `(module-when test a b)` gives `if` with `(progn a b)`. Without `progn`, `if` has one form per branch: `b` is lost or becomes the "else."

Answer M.L2

`with-energy`: `gensym` for the cost, compare with `*energy*`, if too low `'too-low` without `decf`. If enough — `decf`, then the body. Two runs: 10 against 15 and 20 against 15.

Answer M.L3

`twice-wrong` substitutes the argument twice: two `tick`s. `twice-right` — `let` + `gensym`, one side effect. Look at `macroexpand-1` of both, not "yeah they look the same."

Answer C.L1

`defclass module`, `make-instance`, `setf` via the accessor, `module-status` — one line with the name and the energy.

Answer C.L2

`antenna` inherits from `module`. Its own `defmethod module-status`. `(mapcar #'module-status (list reactor dish))` — two different lines.

Answer C.L3

`:around` + `call-next-method`, when energy is 0 append `ALARM`. One wrapper for everyone, not two identical `if`s.

Answer C.L4

Class `station`, a slot that is a list of modules, `station-report` = `mapcar #'module-status`. Two different classes in the list.

Answer C.L5

`(defclass comm-antenna (module powered radio) ())`, `typep` on all three — `t`. Slots from both sides are readable. In Java there are no two parent classes with slots, only one `extends`.

Answer C.L6

`class-precedence-list` before and after swapping the order in `defclass`. Names in the CPL move. That isn't a bug, that's a lever.

Answer I.L1

Without `(quit)`: old `defmethod`, a call, new `defmethod`, a call on the same instance. Data the same, status string different. If only a new `make-instance` helped — you killed the image.

Chapter

13 Appendices

13.1 A. A week on one napkin

- **Mon–Thu:** 40 min Lisp (Barski and/or station MODULE) + 120 min Java (look a little, then until it runs).
- **Fri:** Lisp + finish whatever fell apart.
- **Sat:** two or three hours on the project only.
- **Sun:** a walk, or a small sabotage. Not a third chapter.

SICP — when you asked yourself. Microservices — after one normal program. GitHub — from lesson 0. English — every day, but not instead of Java: it isn't on the book's schedule.
If the evening got eaten — don't "catch up three chapters." One chunk of Lisp **or** one chunk of Java, and a commit. The napkin is not a judge.

13.2 B. A cheat sheet for the corridor before the interview

Read it out loud, not with your eyes. Eyes lie: "yeah I know this." The mouth checks.
Java: `==` and `equals`, strings, lists and "is this fast / not," `ArrayList` versus `LinkedList`, `HashMap`, generics without fanaticism, checked/unchecked, `try-with-resources`, objects on your own `Task`, `final`. `int` versus `Integer` in one sentence. Why you can't change a string from the inside.
Spring: a box of dependencies, a bean, a thin controller, a service, what `@Transactional` rolls back, the controller does not walk into the repository itself. Where the object in the constructor comes from if you never wrote `new`.
HTTP and SQL: GET you can repeat, POST twice — careful. 401 is not 403, 400 is not 500. Keys, an index, a transaction, JOIN versus two queries. `EXPLAIN`, at least the idea.
Hardware: how the app starts, where logs go, why Docker, how an image isn't a container, why inside compose the database host is `db`, not `localhost`.
If the question "what is a transaction" comes out as "well it's when" and then silence — go back to lesson 11, don't read about Kafka.

13.3 C. What else to read, if you aren't bored yet

- Conrad Barski, **Land of Lisp** — a neighbor in spirit, not the text of this book. The orcs are his, the station is ours.
- One Java reference, not three courses at once. Horstmann or anything, as long as it's **one**.
- Spring Guides — one per week's topic, not "all the guides over the weekend."
- Postgres on your own `tasks` table. The postgresql.org docs, the SQL section. Boring, useful.

- Aditya Bhargava, **Grokking Algorithms** — pictures, conscience clean.
- SICP — in spots, when you yourself asked about recursion or "data as code."
- Official Java tutorials: https://docs.oracle.com/javase/tutorial/ — dry, but they don't lie for clicks.

Don't read five books in parallel. Read one, write code. A book without code is a TV series.

13.4 D. The AI roommate

Allowed: "why won't it compile," "what's this wall of text in the error," "compare my two versions," "explain this line like I'm a person who just learned about parentheses."
Not allowed: "write a task-manager from scratch" and pass someone else's off as a portfolio. They'll ask about a line you never touched. It will be insulting and fair.
Grey zone: "sketch a test skeleton, I'll fill in the asserts." Allowed if you fill them in. Not allowed if the skeleton already has all the logic and you only renamed variables.

> **Station law**
>
> The roommate rule: after a hint you should be able to **close the chat** and repeat the step with your hands. If you can't — that wasn't a roommate, that was a genie, and you owe a debt that will surface in a room with a whiteboard.

13.5 E. About copying

Code from the textbook — into your study repos, please. Don't put Barski's and Abelson's books into your own PDF. Quest answers are not a commercial product without your own work. Station MODULE is already on duct tape; let's skip the lawsuits.
Other people's code from GitHub in a portfolio with no note — a lie. A fork signed "I changed this here" — fine. The difference is one sentence in the README.

13.6 F. Mac, Windows, WSL — a command cheat sheet

From here on, "like the book" = Mac or Ubuntu inside WSL2. Native Windows is the right-hand column, when you really must.

What for	Mac / WSL (Ubuntu)	Windows without WSL
Packages	`brew` / `sudo apt`	Installers, `winget`, Chocolatey
SBCL	`brew/apt install sbcl`	sbcl.org → Windows
JDK 21	`openjdk@21` / `openjdk-21-jdk`	Adoptium MSI, PATH
Git	`brew/apt install git`	git-scm.com
Postgres	`brew/apt` + service	Installer from postgresql.org
Maven wrapper	`./mvnw …`	`mvnw.cmd …`
Line break in curl	backslash `\`	cmd: `^`; better PowerShell in one line
Leave SBCL	`(quit)` or Ctrl+D	`(quit)`

Docker	Docker Desktop / engine	Docker Desktop + WSL2
Project in IDEA	open the folder	or `\\wsl$\Ubuntu\home\…`
Where am I	`pwd`	`cd` with no args in cmd; `pwd` in Git Bash
List files	`ls -la`	`dir` / `Get-ChildItem`
Copy a file	`cp a b`	`copy a b`
mvnw permissions	`chmod +x mvnw`	not needed: `mvnw.cmd`

A rule that saves a month of life: on Windows **WSL2 immediately**, copy Java commands from the textbook into Ubuntu. IntelliJ can stay "windowed" and open a folder inside WSL. Postgres and Docker — better from one side of the wall too: either everything in WSL, or everything native. Two captains, remember.

If `localhost` from WSL can't see a database installed by MSI on Windows — that's not you being dumb. That's two worlds. Install Postgres in Ubuntu (`apt`) and live in peace.

PATH broke: close every terminal window. Not one. All of them. Open a new one. If that didn't help — reboot. This isn't shamanism, it's an environment cache that clutches a dead path like a treasure.

13.7 G. If you're really stuck

Order, not panic:

1. Read the **first** error whole. Not a screenshot of the top half.
2. Check the folder: `pwd` / `ls`. Is the file there?
3. Check you called the right program: `which java`, `java -version`.
4. A minimal example: one function, not the whole server.
5. Then the AI roommate or a search. Put the error in the query, not "my spring doesn't work."

Stuck three days on Hibernate — that's the profession, not a sign to leave. Stuck three days installing a JDK — go back to the workshop, don't jump to month 3.

13.8 H. English on a napkin

Words it hurts to be without:

`build`, `run`, `debug`, `commit`, `push`, `pull`, `merge`, `branch`, `issue`, `stack trace`, `null`, `exception`, `timeout`, `port already in use`, `permission denied`, `not found`, `bad request`, `unauthorized`, `forbidden`.

Don't learn them like a school vocab list. Hit one in an error — translate **that** phrase, write it in the station notebook. In a month the notebook becomes a dictionary by itself.

13.9 I. What counts as "I finished the book"

Not the last page. This:

- on GitHub you can see the path from Hello to a server with a database;
- the README brings the project up on someone else's machine (or on your second OS);
- you honestly say what you didn't do;

- you go to interviews instead of stockpiling chapters.

An offer is not a button. The pages after month 6 are about letters and rejection, not one more framework. If you want one more framework instead of a letter — that's fear, not study. Understandable fear. The letter still has to go.

Chapter

14 Station glossary

Not Wikipedia. Wikipedia explains as if you already know the neighboring thirty words. Here — like a mechanic in the corridor: one joke so you don't fall asleep, and one explanation you can **use**. The words are English because they are English. Translating `null` as "nothing" is allowed once, in your soul. In code and in interviews it's still `null`.
Flip here when you need it. Memorizing from Monday is a bad quest.

14.1 REPL

Read-Eval-Print Loop: read, compute, print, wait again. A parrot with a calculator.
In practice: SBCL's black window with a star `*`. You type a form, hit Enter, see the answer. Not "run the project." A conversation. If you didn't touch it with your hands — you read about Lisp, you didn't do Lisp.

14.2 JDK

Java Development Kit. The box that holds the stove **and** the translator.
`java` runs what's already cooked. `javac` compiles the source. Without a JDK you sometimes only have `java` (JRE / some runtime) — you can run other people's stuff, you can't build your own. This book wants 21. Not "whatever the app store had."

14.3 JVM

Java Virtual Machine. The stove the bytecode runs on. Not your OS, another layer: "we'll pretend to be the same machine on a Mac, Windows, and a server."
That's why a `.class` is not "a Windows program." That's why OutOfMemory is about **this** stove's memory, not always the disk. That's why "it's one thing in IDEA, another in the terminal" often means: two JVMs, different ones.

14.4 class

A blueprint of a thing. Not the thing itself.
`class Task` says: a task will have an id, a title, a flag. `new Task(...)` — already a gadget in memory. The file is usually named after the blueprint: `Task.java`. Java loves that. The station also likes labels on the crates.

14.5 method

A verb on a thing. "What it can do."

`complete()` on a task, `add(...)` on the store. `main` is a special method: the JVM enters here, like the main hatch. A function in Lisp is a relative in spirit, just without the `public static` paperwork.

14.6 HTTP

The language of letters between a browser (or your `curl`) and a server. Not "the whole internet." The internet is a corridor. HTTP is the form: method, path, headers, body.
GET — "show me." POST — "accept this new thing." 200 — ok. 404 — no such door. 500 — there's a fire in their kitchen. Until you learn the form, Spring will look like button magic.

14.7 JSON

Text pretending to be data: `{"energy": 80, "room": "reactor"}`. Curly — object, square — list. Quotes on keys are mandatory, unlike your mood.
This is not a database. This is an envelope. The database lives at their place, JSON rides over HTTP. Break a comma — "their machine" will say "couldn't chew that," and it will be right.

14.8 Git

Not GitHub. Git is a diary of folders on your disk: what changed, when, why. GitHub is a locker on the internet where that diary also gets put.
Without Git you have "final version" and "final2_for_real." Stations don't fly like that. `git status` — look at your feet. Do that more often than `git commit`.

14.9 commit

A snapshot: "the folder looked like this, here's a message about why." Not saving a file. The editor saves the file. A commit is a mark you can come back to.
Write the message for a human in six months. `week4: save tasks` beats `asdf`. A commit doesn't have to be pretty. It has to exist.

14.10 SQL

The language for talking to tables. `SELECT` — give me rows. `INSERT` — put this in. `UPDATE` / `DELETE` — as they sound, just without "well, almost all of them."
This is not Java. This is a separate warehouse dialect. Spring hides it until it hides it badly — then you open a Postgres console anyway and write `SELECT` by hand. Better to know how before that day.

14.11 JOIN

Glue rows of two tables by a key. Tasks plus people's names, compartments plus sensors.
Without JOIN a beginner makes two queries and glues them in their head. With JOIN the database glues them, that's what it's for. Forget the join condition — you get a Cartesian product: every task with every person, a party, then disk space weeps.

14.12 index

A table of contents for a column. "Where are all tasks of user 42" without reading the whole table with a flashlight.

An index takes space and slows writes a little: you have to update the table **and** the table of contents. On three rows you don't need it. On three million — you very much do. `EXPLAIN` will show whether they use it. Guessing by vibes — no.

14.13 transaction

"All or nothing." Drain energy from the reactor **and** turn on the antenna. If the second falls over — the first must not stay half-done.

`@Transactional` on a service — "this work is one mash." Not speed magic. Honesty magic. Two Save buttons without a transaction — a way to get a station that doesn't exist in nature: money taken, ticket not issued.

14.14 bean

An object Spring manages: created it, handed it to others in a constructor, often one for everybody. Not a coffee bean. Not the coffee joke, though everyone makes it, including me. If you wrote `new` on a service yourself — that isn't a bean, that's you. The container then shrugs.

14.15 controller

A thin door from the street. HTTP arrives here. Further — not here.

The controller translates a letter into a service call and back into JSON. If there's SQL in the controller — the station is looking at you. If the business rule "no empty name" lives only in the controller — someone will call the service around it, and the name will be empty.

14.16 service

The station's rules. May we dock, is there enough energy, is the task a duplicate.

Meaning goes here. The transaction goes here. Tests without the web live here. If there's no service and everything is in the controller — you don't have architecture, you have one big `main` with annotations.

14.17 repository

The door into the database. "Put a task," "give me a task by id." Not "decide whether the user is allowed."

A JPA repository often looks like an interface with no body — Spring writes the implementation. That's convenient until it's too convenient: then you write the query yourself and remember SQL without shame.

14.18 Docker

A way to say: "run this same kitchen at your place, not only at mine." Not a virtual machine with Windows inside (though that lives nearby). A box with a process and its jars.
"Works on my machine" is cured not by a meme but by a Dockerfile and `compose`. The first time will hurt. The second — less. On Windows go straight to WSL2, or the pain is eternal.

14.19 image

A snapshot of the kitchen: which jars to put in, which recipe to run. A file (almost) you can download.
An image doesn't run. It sits. `postgres:16` — someone else's database image. Your app image — "JDK + my jar + how to start." Pin image versions. `latest` is a 3 a.m. surprise.

14.20 container

A running image. Mash on the stove from that snapshot.
You can kill a container — the mash is gone. Data you care about goes in a volume, or Postgres inside the container will forget the tasks along with you. Two containers from one image — two mashes, not one.

14.21 cache

A cheat sheet next to the counter: "we already computed this recently." Fast. Lies if you forget to throw it out.
A cache without invalidation is a sensor showing yesterday's air. `@Cacheable` is pretty. `delete` on update is mandatory. Otherwise the demo: "I changed it, the site didn't." The site is honest. The cheat sheet is old.

14.22 queue

A queue of work: you put a message, someone takes it later. Not "do it now in this same request." The letter "task created" can wait. The user already got 200. Another cook will handle the letter. If that cook died — the letter must **stay**, not dissolve. Otherwise the queue is theater.

14.23 Kafka

A queue that is also a log: messages are remembered, you can reread them from a place.
Not a junior's first tool. Not proof of adulthood. On a résumé with no code of your own — a red rag. In this book it shows up late and for a reason: "lots of events, we don't want to lose them." If the work fits one monolith — live without Kafka, the station will survive.

14.24 stack trace

A bedsheet of "who called whom when it fell over." Read **bottom up** or top down — whichever helps, but **your first understandable line** matters more than Spring's guts.

This is not a cipher. This is a map of corridors at the moment of a fire. The line number is a gift. If there isn't one, you're looking at the wrong build. Don't fear the length. Fear an empty `catch`.

14.25 null

"There is no box." Not zero. Not an empty string. No crate.
In Java `null` bites: `NullPointerException` — you asked a method of an absence. In Lisp the relative in spirit is `nil`, but with a different temperament. In an interview "what is null" is not a joke. Say: a reference to nowhere. Then say how you keep from bringing it into the service without need.

14.26 exception

A way to shout "you can't do that" instead of lying quietly.
`throw` — the shout. `try/catch` — you heard it. Catch the specific one. An empty `catch` — duct-taping the siren. Checked in Java (`IOException`) — a snitch compiler: admit that the disk is sometimes against you. Annoying. Useful.

14.27 recursive

A function calls itself on a **smaller** piece. You clean the corridor: this compartment, then the rest. Without a stop (`null` list, `n == 0`) — `Control stack exhausted` / `StackOverflowError`. Not mysticism. Nested dolls with no last doll. Recursion isn't "for smart people." Recursion is for lists and trees until it clicks. Then you can `loop`.

14.28 cons

Glue a head and a tail. The main list constructor in Lisp. `(cons 'airlock '(corridor))` → `(AIRLOCK CORRIDOR)`.
Looks like a typo in granddad textbooks. It isn't a typo. A list is a chain of cons. Understand cons — understand why `rest` is cheap and "append at the end" in the obvious way isn't.

14.29 symbol

A tag. A name. In Lisp `'reactor` is not the string `"reactor"`. Strings compare one way, tags another. A symbol is a word of the language. A variable, a function, just a label in a list of compartments. The apostrophe: "don't evaluate, put it as is." Without the apostrophe Lisp will try to **call** `reactor` and get offended if it isn't a function.

14.30 t

Yes. Truth. "In all other cases" in `cond`.
Not the number 1. Not the string `"true"`. In Lisp just `t`. You don't write `truth` in the code. You don't have to write `true` either — that's from the neighboring language. Here it's `t`.

14.31 nil

No. The empty list. "Nothing." One actor, two roles, and everyone's fine with it.
`(if nil ...)` won't enter. `(rest '())` gives `nil`. Falsehood and emptiness met in the airlock and didn't argue. Java can't do that: there `false`, `null`, and an empty `List` are three different citizens.

14.32 Maven

The Java project builder: dependencies, tests, packaging. The ritual file `pom.xml`.
`./mvnw test` — "install what's written, run the tests." The wrapper (`mvnw`) goes in the repo so a neighbor has the same Maven version. No need to "download Maven separately and configure it like a priest." On Windows without WSL — `mvnw.cmd`.

14.33 Flyway

Database migrations: files `V1__tasks.sql`, `V2__idx.sql`. A history of the schema, not "tweaked by hand in prod."
Order matters. Once applied — don't edit the old file as if nobody saw. They saw. The database saw. A new file — a new number. It's boring and it saves stations.

14.34 JPA

The contract "class ↔ table." Fields — columns. `save` — almost INSERT/UPDATE.
Hibernate is often under the hood. `@Entity` — "this is a table row, not just a POJO in a vacuum." Convenient until N+1 arrives. Then you remember there's SQL under the blanket, and that's fine, not cheating.

14.35 equals

"Is this the same task in meaning?", not "is this the same crate in memory?"
`==` for objects in Java — "the same address on the counter." `equals` — "we count them equal." For strings almost always `equals`. Forget — a bug in the interview and in prod. A record often gives `equals` itself. Your own class — write it, or live with the pain.

14.36 thread

A thread of execution. Several cooks in one program.
By default your `main` is one thread. A web server — many: per request. Two threads wreck one counter without `synchronized` / a proper queue — a race. Don't start with "parallelism for the résumé." Start when one cook can't keep up, and measure.

14.37 port

A door on the machine. A number. 443 — https, 8080 — the teaching server, 5432 — Postgres.
Two processes don't share one door: "port already in use." Means the old server is alive or Slack took it. `localhost:8080` — this machine, this door. Not "a site in the cloud." A door.

14.38 localhost

Yourself. `127.0.0.1`. The kitchen, not the street.
A browser on `localhost` doesn't go "to the internet," it knocks on a process in this same box. In WSL sometimes "Windows localhost" and "Ubuntu localhost" are two kitchens with a hole between them. If the database "isn't visible," check **whose** door it is.

14.39 API

A contract: how to call someone else's program. Often HTTP+JSON, not always.
"There's an API" means: you can do it by letter, not only by mouse. Your Spring controller **is** an API. Someone else's httpbin — also. API docs — which doors, which letters, which answers. Without them you're an archaeologist.

14.40 REST

A style of HTTP API: resources, verbs, statuses. `/tasks/3` — task 3. GET — read. PUT/PATCH — change.
Not a religion. Not "microservices required." On job posts the word means: "you can do CRUD without inventing your own Skype." If the argument "is this real REST" lasts an hour — go write a working GET.

14.41 bytecode

What `javac` chews `.java` into. A `.class` file. Not machine code of your CPU, a JVM dialect.
That's why one Java program runs on different OSes — the stove looks different outside, inside the bytecode is the same. You don't need to look at bytecode every day. You do need to know the green button produces it.

14.42 annotation

A label over code: `@Override`, `@RestController`, `@Test`. The compiler or the framework reads the label and behaves differently.
This is not a comment. People lie in comments. A tool eats the annotation. An extra `@Transactional` on a `private` method is a label that **won't work**, like a "wet floor" sign on a dry floor. Read where you stick it.

14.43 DTO

Data Transfer Object. A box of "here's what we give to the street," not necessarily the whole entity from the database.
So the JSON doesn't leak a `passwordHash` field. So you don't drag a lazy association and catch a lazy fire. A boring word. An adult habit. Early on you can return the entity — then you burn yourself, and the glossary suddenly feels like home.

14.44 IDE

An editor that also compiles, debugs, suggests. IntelliJ is ours. VS Code too, just install the Java plugins.
Not a compiler. Not Git. Not a substitute for a head. The Run button inside calls the same `javac` / `java`, only without your `pwd`. Once a week run from the terminal, so you remember.

14.45 PATH

The list of folders where the system looks for programs when you type `javac` without a full path. "Command not found" often means: the jar is there, it isn't in the corridor's PATH. Installed a JDK — set PATH or close and reopen the terminal. On WSL and on a Mac this is ordinary first-day pain, not a sentence.

14.46 stdin / stdout

Standard input and output. Keyboard and screen by default.
`Scanner(System.in)` reads stdin. `System.out.println` writes stdout. You can swap in a file: `java Energy < input.txt`. Errors often go separately: stderr. So "the output vanished" sometimes means: you're looking at the wrong stream.

14.47 hash

A fingerprint of data. Same input — same fingerprint. Change a little — a different one.
`HashMap` looks up by the key's fingerprint, so the key must equal properly (`equals` + `hashCode`). Passwords in a database aren't stored in the open — you store a hash (better: a special, slow one). `sxhash` in teaching Lisp is a toy, not a safe.

14.48 N+1

First one query "give me the list," then a query **per** item. A hundred tasks — a hundred and one trips to the database.
On three rows you don't see it. In prod you do. Cured by JOIN / `join fetch` / not poking associations in a loop. If the SQL log looks like a machine gun — you saw N+1. Congratulations. Now it has a name.

14.49 garbage collector

The garbage collector. The JVM (and SBCL too) throws away jars nobody points at anymore.
You don't `free` every object. That doesn't mean "memory is infinite." A leak in Java is holding a list of references to everything you ever saw. GC is honest: it doesn't touch what you can still reach.

14.50 endpoint

An API door: method + path. `GET /tasks`. `POST /tasks`.

Not "the whole server." One handle. In Postman / curl you pull an endpoint. In a controller one method often = one endpoint. If the handle does five different jobs depending on a parameter's mood — that isn't a door, that's an attic.

14.51 entity

An object that **lives in the database**. A table row as a class.
Not every class is an entity. `Task` with `@Entity` — yes. `EnergyReport`, which you assembled from three tables only to return JSON — more of a DTO. Mix them up — you'll get a weird `save` and weird JSON.

14.52 primary key

The main key of a row. Usually `id`. Unique. JOIN goes by it, "give me task 3" goes by it.
Not the task title: titles repeat. Not "the number in the list on screen": the list lies after a sort. A real id. The station hasn't argued with this in sixty years.

14.53 foreign key

A pointer to someone else's primary key. On a task — `project_id`. "This row is about that project." The database can watch that the project exists. That's a constraint, not a whim. Without it you get tasks of ghost projects. With it — an error on insert, and that's a gift.

14.54 log

A process diary: lines during its life. Not `print` for yourself in `main`, but proper levels: info, warn, error.
Logs are read when it already hurts. Write so future you understands **which id** and **what you wanted**. Don't write passwords. Don't write personal data "for debugging." The station records energy, not the crew's correspondence.

14.55 pom.xml

The Maven project's passport. Coordinates, Java version, dependencies, plugins.
XML is mean about spaces in your soul; in fact it's just wordy. Don't copy someone else's `pom` whole "just in case." Every dependency is someone else's code on your station. Sometimes needed. Sometimes a transitive circus.

14.56 Spring

A frame you hang web, beans, database access on. Boot — "the frame already has batteries, hit Run."
Not a language. Not a Java replacement. A layer of convenience and surprises. Until you can assemble a class, a list, and a file without Spring — too early. When you can — Spring saves weeks. When you can't — it saves understanding.

14.57 constructor

The method that builds the thing on `new`. Same name as the class. Without one Java still gives you an empty one if you didn't write another — and then fields will be zeros and `null`.
Station joke: the constructor is the only moment you can honestly say "you aren't born without an id." After that setters start lying.

14.58 static

"Belongs to the blueprint, not one gadget." `main` is static because the JVM hasn't done `new` of your class yet, and it still has to enter.
Abuse `static` on everything — you have globals with stars again, only without the stars. Sometimes you need it. Often it's laziness.

14.59 package

A corridor of names. `com.module.tasks`. So two `Task` classes don't fight. First line of the file. The folder on disk must match, or Java gets offended like a storekeeper.
Not a religion of "how big companies do it." Just don't put a hundred classes in the root with no sign.

14.60 import

"Take the blueprint from that corridor." Not copy-paste of code. Not "download from the internet." Without import — write the full name, like an address with a zip code.
A star `import java.util.*` works. Then in the IDE you can't see where `List` came from. For study, explicit imports are better.

14.61 boolean

Yes/no. `true` / `false`. Not `t` and not `nil`. Not 1 and 0, though other languages lie that it's the same. `if (energy)` in Java won't fly if `energy` is an `int`. The storekeeper wants an explicit question. Sometimes annoying. Less often lying.

14.62 String

Text. An object, not "an array of letters you solder yourself," though inside it's almost that.
Gluing with `+` is convenient and sometimes shameful for speed. For a dashboard it's enough. `equals` for comparison, we already yelled that. Quotes only double: `'a'` in Java is a character, not a string.

14.63 int

An integer in a fixed-size box. Division `7 / 2` gives `3`. The fraction went out the airlock with no siren.
For an average write `3.0` or `double`. For money at work — not `double`, but that's not the first month, thank goodness.

14.64 array / list

An array — a crate of N slots, N decided at birth. `ArrayList` — a crate that can add more.
Indices from zero. People from one. The translator is you. `IndexOutOfBounds` is a gift: there is no slot, not "the element is null."

14.65 404 / 500

404 — no door. 500 — there is a door, fire behind it.
Mixing them up is embarrassing in an interview and in logs. Sometimes you lie 404 to a client instead of 403, so you don't advertise that the secret exists. That's policy, not HTTP romance.

14.66 cookie / session / JWT

Ways to remember who you are between letters. HTTP itself is forgetful, like a CPU with no RAM. Cookie — a note in the browser. Session — a note on the server plus a number. JWT — a note the server signed and handed you to carry. For the glossary: **not a login on every click**. Details — when you get to Spring Security. Not sooner, please.

14.67 curl

A terminal browser with no pictures. `curl URL` — send GET, print the body.
Flags later: `-i` headers, `-X POST`, `-d` body. If you can curl, you can check an API without Postman. Postman is prettier. curl shows up on a server with no mouse.

14.68 WSL

Windows Subsystem for Linux. Ubuntu often lives **inside** Windows. Not a second Windows in a window and not "install Linux instead." A Linux terminal and files next door, with a hole to the outside.
This book on Windows lives here: `sbcl`, `javac`, `./mvnw` — in WSL, not in CMD. The Windows disk sticks out as `/mnt/c/`. You can. File permissions there are clowns. Put study code in the Ubuntu home directory, `~/dev/...`. If "I see it in Explorer, not in WSL" — you're looking at **two** kitchens. PATH is two as well: the one in PowerShell doesn't feed the station.

14.69 SDK

Software Development Kit. A box to **build** software for a platform: compiler, libraries, docs, sometimes an emulator.
The JDK is an SDK for Java. The Android SDK — for the phone. A runtime (`java` without `javac`) is a stove with no welder: you can run other people's, you can't cook your own. An IDE is not an SDK. An IDE is a workshop that **calls** the SDK. On a job post "experience with SDK" often means: you installed the box and built something, not just opened an editor.

14.70 JAR

Java ARchive. Almost a zip: `.class`, resources, a manifest. Extension `.jar`.
`java -jar app.jar` — run the cooked thing as a program. Maven `package` puts that file in `target/`. What goes to the server is a jar, not a folder of `.java`. You can unzip a jar like a zip to look. Patching `.class` with a hex editor is circus. You patch the source, then `package` again.

14.71 classpath

The list of places the JVM looks for classes and jars **when the program is already running**.
`ClassNotFoundException` often means: the file is on disk, it isn't in the classpath corridor. In the IDE the green button builds the classpath itself. In the terminal `java -cp lib/*:classes Main` — that's you. Maven stuffs dependencies into that list, so `./mvnw exec` and a "bare" `java` behave differently. Until that clicks, the phrase "it works in IDEA" will visit once a week, like a leak in the reactor.

14.72 new

The operator "make a gadget from the blueprint." `new Task()` — an object in memory, you're the owner.
Nearby: bean. Spring creates the service **itself** and hands it to the constructor. If in the controller you wrote `new TaskService()` — that isn't a bean, that's a second parallel station in your pocket. Tests lie, the transaction is the wrong one, the database is another. Rule: `new` for your own data (`Task`, `ArrayList`, DTO). For a service and a repository — the container. That's the whole bean vs `new` difference, not a philosophy of coffee beans.

14.73 migration

A script "the database schema became different." A file with a number. Flyway or Liquibase apply it **once** and remember it in a bookkeeping table.
This is not `UPDATE` by hand in prod. Prod already chewed V3, and you tweaked V1 — locally "it'll do," for the team it's hell. A new file — a new number. Rollback is often another migration that fixes, not a time machine. A replica reads the same history. Otherwise the nodes drift, and that's no longer a duct-tape joke.

14.74 replica

A copy of a database (or a service): so you can read from several counters and not die alone.
You usually write to the primary / leader. You sometimes read from a replica. Physics joke: the copy lags a second, and `GET` "doesn't see" the row you just saved. That's replication lag, not a bug in your `save`. On the teaching station one Postgres — you don't need a replica. In an interview the word means "a copy," not "a second Docker for the résumé."

14.75 offset

A bookmark in a Kafka log: "I've read up to here." A number. Not a task id and not a row number in SQL.

The consumer moves the offset when it has processed. Move it early — you lost the message for yourself. Don't move it — after a restart you'll eat it again. That's why "at least once" and idempotence come as a pair. Until there's Kafka — picture a bookmark in the station log: page 47, not "somewhere about duct tape."

14.76 consumer group

Several Kafka eaters that share **one** log so that one message inside the group is eaten by **one**.

Two groups — two independent readings: both the dashboard and billing can eat the same event. Two eaters **in one** group — they split partitions, not the work. The group name is part of the contract: change the string — different offsets, "as if you'd read nothing." Debugging starts with "which group?", not with restarting the broker in a panic.

14.77 CI

Continuous Integration. A robot runs tests on every push or PR, not "it's green on my laptop."

GitHub Actions, GitLab CI, Jenkins — different kitchens, one idea: a fresh clone, `./mvnw test`, a check or a cross. CI red, yours green — **someone's** kitchen is lying. Often: a file not in git, a different JDK version, a test that looks at the clock. Cured not by the "Run again" button. Cured by the robot's log. CD (delivery/deploy) is the neighbor: also **ship it**. First learn so the robot can say "tests failed."

14.78 linter

A program that nags about style and dumb errors **before** a run.

Not a compiler. The compiler says "this isn't Java." The linter says "this is Java, but `==` on strings, and a variable is hanging." Checkstyle, SpotBugs, ESLint in the neighboring world. Rules live in the repo, not in "it's yellow in my IDE." Turn the whole linter off — duct-tape on the siren. One rule is dumb — turn **that one** off. A formatter (spaces, line breaks) is a relative, not an enemy. "Tabs vs spaces" fights are cured by a file in the repo, not a chat until morning.

14.79 PR

Pull Request. In GitLab more often Merge Request. A request: "take my branch into the shared one." Not a commit. A PR can have a pile of commits. There's a diff, a description, a CI bot, people's comments. A small PR is easier to look at. A PR of three thousand lines "I did everything" — nobody reads it, they merge from fatigue, then hunt the fire. For study: a PR to yourself also teaches writing "what and why," not only code. Review — a human looks at the diff. Not a trial. Not "find ten nits."

14.80 rebase

Lay your commits on top of a fresh `main`, as if you started today.
History becomes a straight line. `merge` makes a knot: "two lines met here." Rebase lies elegantly. Don't rebase a branch other people are already pulling: you'll rewrite history under their feet. Your own study branch before a PR — you can. If you're scared — `merge`. The station won't fail you for missing zen. `git rebase -i` in week one is a way to break the watch and learn `reflog` sooner than you wanted.

14.81 conflict

Two changes in **one** place. Git doesn't know whose line is holy.
Markers `<<<<<<<`, `=======`, `>>>>>>>` are not a ruined file, they're a form. Remove the form, keep the meaning, `git add`. A conflict is not "Git broke." A conflict is two mechanics turning one valve. Cured by reading **both** chunks, not the "take all left" button. After — build and run tests. Otherwise you'll merge a station that doesn't compile, but the history is "clean."

14.82 8080

The teaching web-server port. Not 80: that's often taken and wants admin rights. Not 443: that's https in prod.
`localhost:8080` — the browser knocks on **this** machine, **this** door. "Connection refused" — no process behind the door. "Already in use" — there is a process, a second one won't fit. Kill the old one or change the port. 8081 in someone else's guide is not a sacred number, it's another door when 8080 is taken. localhost — see above: the kitchen, not orbit.

14.83 CORS

Cross-Origin Resource Sharing. The browser won't let a site from one address pull an API from another until the server says "you may."
This is protection of **the user in the browser**, not a cipher and not "the backend broke." `curl` does not obey CORS: curl is not a browser. So "200 in curl, red on the front" is often CORS. Until there's a separate front on another port, you don't have to fix it. When there is — a header on the **server**. An extension "turn off browser security" is not a profession, it's duct tape on the siren.

14.84 lazy

"Don't touch it yet." In JPA an association is loaded when you touch the field, not at the first `SELECT`. Convenient while the database session is alive. Session closed, you poked `task.getProject()` — `LazyInitializationException`. Station classic. Cured not by "put EAGER on everything" (hello, N+1 and a JOIN half a screen wide). Cured: fetch what you need while the repository is still in a transaction; or a DTO; or an explicit fetch. Laziness is not a lazy programmer. It's **when** to go to the database.

14.85 eager

"Bring it now." The association loads with the entity.

On one task with one project — fine. On a list of a hundred tasks, each with comments, comments with authors — a tree of SQL. Eager as duct tape on every pipe: it holds until you can't breathe. For many `@ManyToOne` the default is eager, and you're already paying without seeing the bill. Look at the SQL log. A machine gun — the phenomenon has names: N+1, lazy, eager. Primary key and foreign key have nothing to do with it: the association **exists**. The argument is only **when** to fetch it.

14.86 NULL

In SQL: "there is no value." Not zero. Not an empty string. A separate beast.

`WHERE energy = NULL` will **not** find rows with empty energy. You need `IS NULL`. Comparison with NULL gives UNKNOWN, not true — so `AND` / `OR` float differently than in Java. In Java `null` is "there is no reference." Similar letters, different ritual. `Integer energy` can be `null` (the box is empty). `int energy` — no, there's always a number; forgot to initialize — zero, and that's already a lie, as if "the sensor showed zero." JDBC: `wasNull()`, or zero and NULL glue into one mash. Turn the reactor off with a zero, thinking you turned it off with an absence — that's a Щ plot.

14.87 schema

A blueprint of tables: columns, types, primary key, foreign key, constraints.

Data — rows. Schema — the rules of the shelves. A migration changes the schema. `SELECT` without a schema is archaeology. In Postgres the word is also a namespace inside the database (`public`). In month one: "how the tables are built," not three meanings from the docs. Draw the tables on paper. Paper lies less than "there's an entity in there."

14.88 environment variable

An environment variable. A label on the **process's** door: `DATABASE_URL`, `JAVA_HOME`.

Not a class field. Not a global with stars. The OS (or Docker, or CI) puts a string, the program reads it. Passwords in `application.properties` in git — a bad joke. Passwords in env — ordinary adult. `echo $PATH` — you've already looked at env. "Works at the neighbor's" — compare env, not only code. The code is the same. The doors are different.

14.89 working directory

The folder **from which** relative paths are counted. `station.txt` with no path — "where the process stood up."

Not the source folder. Not the repo root, unless you chose it. `pwd` in the terminal. Working directory in IDEA. WSL vs Windows Explorer. Three different "here." A save "vanished" — almost always this. Print the absolute path once. Then argue with configuration, not with ghosts.

14.90 branch

A branch. A parallel line of commits. `main` — the shared one. `week4-save` — your sandbox. `git switch` doesn't copy the folder by magic: it rearranges files under that line. Uncommitted stuff may ride with you or get in the way. A branch is cheap. Fearing it is like fearing a second mug. Fearing uncommitted mash on two branches at once — healthy. `origin` is someone else's locker (often GitHub), not a holy synonym of a branch.

14.91 0.0.0.0

"Listen on all interfaces of this machine," not only on localhost.
A server on `127.0.0.1` is visible to itself. On `0.0.0.0:8080` — also from a phone on the same Wi-Fi, if the firewall lets it. A beginner opens "on all" and thinks that's the internet. That's a kitchen with a window onto the stairwell, not orbit. For study, localhost is enough. `0.0.0.0` — when you really need it from another box.

14.92 override

`@Override` — "I'm replacing an ancestor's method," not a new method with a similar name.
Without the annotation a typo in the name gives a **new** method, and the one you thought you were overriding stays old. The compiler is silent. The annotation yells. Stick it always. That's a siren on a typo in `equals` / `hashCode` / `toString`, not bureaucracy. Overload is the neighbor: same name, **different** arguments. Mixing overload and override in an interview is a classic, like `==` on strings.

14.93 interface

A contract: "you can do these methods," with no story of **how**.
`List` is an interface. `ArrayList` is one implementation, `LinkedList` another. In the variable's type write `List` unless you have a reason to stick. Spring loves repository interfaces: the container will slip in a body. In month one an interface is a socket, not "hexagonal architecture." Understood the socket — the hexagon can wait.

14.94 401 / 403

401 — you didn't introduce yourself (or the ticket wasn't accepted). 403 — you introduced yourself, you can't go here.
Mixing them with 404 is embarrassing in a different way: 404 no door, 403 there is a door, security is against you. Your API: don't smear everything 500. 400 — the client wrote it crooked. 500 — you. The client should understand **whose** problem it is, without reading your soul.

14.95 volume

A Docker volume: a folder that survives the container's death.
Postgres without a volume forgets tables when the container dies. A volume is a pantry **next door**, not inside a disposable mash. For study: one named volume for the database. `compose down -v` will

take the volume with the container — the `-v` flag here is not "verbose," it's "remove the pantry." Read twice, then press.

14.96 healthcheck

The door "am I alive?" Often `GET /health` or `/actuator/health`.
The orchestrator knocks here, not on the home page. Health green, responses 500 — you checked the wrong thing. If health goes to the database, a Postgres fall becomes "the whole service is dead" in the robot's eyes. Sometimes that's honest. For the station dashboard, health is a `status` command with energy. Same instinct, just without an orchestrator.

14.97 timeout

How long to wait before saying "the door is silent." Network, database, `HttpClient`.
Without a timeout the request hangs until the OS gets bored. Too short — a live database looks dead at rush hour. For study ten seconds on httpbin is fine. In prod the number comes from measurement, not from "a thousand, to be sure." Retry without a timeout — a way to hit someone who's already down, again.

14.98 idempotent

The same request twice gives the same **meaning**, not necessarily the same byte in the log.
GET is usually idempotent: "show task 3" ten times. POST "create a task" twice — two tasks, if you don't watch. Kafka at least once + a non-idempotent handler — duplicates in the database. On the station: `tape-leak` a second time must not spend a second roll of duct tape you no longer have. That's the word.

14.99 prepared statement

SQL with holes for values the driver will put in **as data**, not as query text.
`WHERE name = ?` plus a parameter — a name with a quote won't become a second query. Gluing `'... ' + userInput` is SQL injection, a classic, the station opening the airlock to a stranger. On JPA `save` this is hidden. Hand-rolled JDBC — not hidden. Even a teaching `SELECT` is better with `?` than with concatenation "it's my own number though."

14.100 record

A short class "just data": fields, constructor, `equals`, `hashCode`, `toString` — Java writes them itself.
Handy for a DTO. Not an entity: JPA likes an empty constructor and setters, a record argues with that. Not a replacement for everything. If you catch yourself on an eight-line class with three fields and a hand-written `equals` — record. If you catch yourself on "I'll add some behavior" — an ordinary class, don't be a hero.

14.101 CLOS

Common Lisp's object system: classes with slots, verbs on the outside (`defgeneric` / `defmethod`), several parents at once. Java still gives you one `extends`. Not "Java in parentheses." A different center: first the action, then the gadget's passport.

14.102 macro

Code that writes code. Gets forms, returns a form. `macroexpand-1` is an X-ray. A function you're "just too lazy to name" is not a macro. If an argument with a side effect got substituted twice — you wrote `twice-wrong`.

14.103 live image

A Lisp process where you can redefine a method without killing the data. That's how they patched Remote Agent on Deep Space 1. Gradle can't do that, and it isn't Gradle's fault: nobody invited it onto the probe.

If a word isn't in this glossary — no harm. Often it's either the name of someone else's library (google "what beast is this") or job-post marketing. First ask: **which sensor on the station does this fix?** If none — you don't have to learn it this week.

Chapter

15 Station log MODULE

This is not a schedule. The schedule is in months 1–6. This is an extra forty minutes for when the week's quest is already green and your fingers still want parentheses.
Station MODULE in the main course is always being repaired. In the log it also **gets played**. Its own plot: not orcs, not dungeons from someone else's book. Orbit, duct tape, notes from the previous mechanic. His name was… the watch log is smudged. A letter survived: ЩЦ. And a coffee stain.

> **Station law**
> These quests are not instead of Java. Not instead of a lesson. If the server is down — bring the server up. The log is for Sunday and for itchy parentheses.

We talk to SBCL. Put files in `lisp-experiments/station-log/`. Commit when it isn't embarrassing **or** especially when it is.

15.1 Watch 1. Energy that runs away

The previous mechanic left a scrap on the table:

> The reactor lies. The panel says 100. The corridor is cold. Don't trust the panel. Trust the function.

We will not make a game with pictures. We will make a world out of parentheses. A world more honest than the panel.
Start the REPL, `sbcl`, the star. First a global — yes, stars, yes, we yell:

```
(defparameter *energy* 100)
(defparameter *leak* 7)
```

`*leak*` — how much energy each tick eats just because. A crack in the hull. Or a kettle somebody forgot. For the model it doesn't matter.

```
(defun clamp-energy (n)
  (cond
    ((< n 0) 0)
    ((> n 100) 100)
    (t n)))

(defun tick ()
  (setf *energy* (clamp-energy (- *energy* *leak*)))
  *energy*)
```

> **What just happened**
>
> `tick` changes a global and **returns** the new value. In the REPL you'll see a number. This is not "a procedure that silently wrecks the world" — you got an answer too. Handy to look at, without wrapping in `print`.

```
(tick)  ; 93
(tick)  ; 86
```

Two ticks — and it isn't a hundred anymore. The panel would lie if we drew it once and forgot to call `tick`.
Recharge from the solar panel while the station is on the sunlit side:

```
(defun recharge (delta)
  (setf *energy* (clamp-energy (+ *energy* delta)))
  *energy*)

(recharge 20)  ; depends how much has already leaked
```

Emergency spend — open the antenna, blink at Earth:

```
(defun pulse-antenna ()
  (if (< *energy* 15)
      'too-dark
      (progn
        (setf *energy* (clamp-energy (- *energy* 15)))
        'ping)))
```

> **Slow down**
>
> `progn` — "do several forms, return the last." Without it `if` has only one "then." You want to subtract **and** return a symbol — pack them. Otherwise you get the number from `setf`, and `'ping` doesn't even ride.

Play in the REPL: five ticks, one `pulse-antenna`, another tick. Watch when `'too-dark` shows up. That isn't a bug. That's plot.

> **Quest S.L1** · Lisp
>
> `ticks`: a number n, repeat `tick` n times, return the final energy. Without a loop you can too — recursion. With a loop — `(loop repeat n do (tick))`. Don't go below zero: `clamp-energy` is already on watch, don't go around it.

15.2 Watch 2. Compartments are a graph, not a corridor from the brochure

On the chart at the entrance there are rectangles and arrows. Rectangles — rooms. Arrows — hatches. That's a **graph**: nodes and edges. Grown-up word. The thing is a list of neighbors.
Five compartments of the log. Remember them, they'll be useful in the adventure:

- `airlock` — the airlock. Smells of metal and other people's gloves.
- `corridor` — the corridor. A bulb blinks Morse, but it's just a bad ballast.
- `galley` — the galley. A mug with the letter Щ. The coffee ran out last quarter.
- `reactor` — the reactor. Warm. Hums. Lying to the panel here is indecent.
- `cupola` — the cupola. Earth in the window. For the plot: a strong receiver lives here.

Neighbors — an alist. Key — room, value — a list of where a hatch goes.

```
(defparameter *doors*
  '((airlock  . (corridor))
    (corridor . (airlock galley reactor))
    (galley   . (corridor cupola))
    (reactor  . (corridor))
    (cupola   . (galley))))
```

From the airlock only into the corridor. From the corridor — almost everywhere. From the reactor back to the corridor, no secret pipe. We could add a pipe. Then the graph gets merrier and more dangerous. No pipe for now: otherwise the first game will be about how you got stuck in the vents, not about parentheses.

```
(defun neighbors (room)
  (cdr (assoc room *doors*)))

(neighbors 'corridor)
; (AIRLOCK GALLEY REACTOR)
```

Can you step?

```
(defun connected-p (from to)
  (member to (neighbors from)))
```

`member` returns the tail from the find, or `nil`. For `if` that's enough: non-`nil` — yes.

> **What just happened**
>
> `(connected-p 'airlock 'reactor)` → `nil`. No hatch. Not "almost next to each other on the paper." Next to each other on the paper is geometry. In a graph there are only edges. The station is not required to be an honest office floor plan.

Where you stand:

```
(defparameter *here* 'airlock)

(defun look ()
  (format t "you're in ~a~%" *here*)
  (format t "hatches: ~a~%" (neighbors *here*))
  *here*)
```

```
(defun walk (to)
  (if (connected-p *here* to)
      (progn
```

```
        (setf *here* to)
        (look))
      (format t "no hatch to ~a~%" to)))
```

In the REPL:

```
(look)
(walk 'corridor)
(walk 'cupola)   ; no hatch, it yells
(walk 'galley)
(walk 'cupola)
```

You just wrote an adventure engine. No picture. A picture would have been a lie: as if the world were bigger than five symbols. The world is five symbols and hatches. Honest.

Quest S.L2 · Lisp

`can-reach-p` : from room A to B **in one step** already exists. Make `exits-report` : for **each** room print the name and the neighbors. Take the data from `*doors*` , don't copy five `format` s by hand. `dolist` over `*doors*` .

15.3 Watch 3. Щ's notes and a mug

Now items. Not an inventory of forty slots. A pocket: a list of symbols.

```
(defparameter *pocket* nil)

(defparameter *at*
  '((airlock  . nil)
    (corridor . (tape))
    (galley   . (mug note))
    (reactor  . (wrench))
    (cupola   . (photo))))
```

`tape` — duct tape. You'll patch the leak with it, otherwise what's the plot for. `mug` — Щ's mug. `note` — a note. `wrench` — a seventeen-mil wrench, the only one. `photo` — Earth, shot on film, because the digital module is "updating" again.

```
(defun stuff-here ()
  (cdr (assoc *here* *at*)))

(defun take (item)
  (let ((here-stuff (stuff-here)))
    (if (member item here-stuff)
        (progn
          (setf *pocket* (cons item *pocket*))
          (setf (cdr (assoc *here* *at*))
                (remove item here-stuff))
          (format t "took ~a~%" item))
        (format t "no ~a here~%" item))))
```

> **Slow down**
>
> `setf` on `(cdr (assoc …))` changes a cell in **the same** cons that lives in `*at*`. That's why the world changes. If we built a new list "outside" and forgot to assign it — the room would hold the mug forever. This isn't magic. This is a pointer to a pair.

```
(defun read-note ()
  (if (member 'note *pocket*)
      (format t "Ш.: leak in the reactor. duct tape in the corridor. don't drink from
the mug.~%")
      (format t "no note in the pocket. maybe it's still in the galley.~%")))
```

Play it as a rehearsal:

```
(defparameter *here* 'airlock)
(walk 'corridor)
(take 'tape)
(walk 'galley)
(take 'note)
(read-note)
(take 'mug)
```

You can take the mug. We won't drink: the previous mechanic warned us, and a textbook is not insurance.

> **Quest S.L3** · Lisp
>
> `drop`: put an item from the pocket into the current room. If the item isn't there — say so honestly. `remove` from `*pocket*`, `cons` onto the room's list. Check: took the wrench in the reactor, left for the corridor, dropped it, came back — no wrench in the reactor. Otherwise you changed the wrong list.

15.4 Watch 4. Five rooms, one leak

The whole game. An energy tick on every `walk`. You can patch the reactor only if you're **in** `reactor` and `tape` is in the pocket.

Put it in a file `module-adventure.lisp` (in the REPL later `(load "module-adventure.lisp")` from that folder, remember `pwd`).

The world resets by a function, not "close SBCL and pray":

```
(defun reset-world ()
  (setf *energy* 100)
  (setf *leak* 7)
  (setf *here* 'airlock)
  (setf *pocket* nil)
  (setf *at*
        '((airlock  . nil)
          (corridor . (tape))
```

```
          (galley   . (mug note))
          (reactor  . (wrench))
          (cupola   . (photo))))
  (setf *doors*
        '((airlock  . (corridor))
          (corridor . (airlock galley reactor))
          (galley   . (corridor cupola))
          (reactor  . (corridor))
          (cupola   . (galley))))
  (setf *fixed* nil)
  'ready)
```

`*fixed*` — whether the crack is patched. While `nil`, every move eats `*leak*`.
A move is not a separate button. A move is when you walked:

```
(defun walk (to)
  (cond
    ((not (connected-p *here* to))
     (format t "no hatch to ~a~%" to))
    ((<= *energy* 0)
     (format t "dark. energy 0. the station is not in the mood.~%"))
    (t
     (setf *here* to)
     (unless *fixed*
       (tick))
     (look)
     (when (stuff-here)
       (format t "on the floor: ~a~%" (stuff-here))))))
```

The patch:

```
(defun tape-leak ()
  (cond
    ((not (eq *here* 'reactor))
     (format t "the leak isn't here. the panel lies, but the pipes are in the
reactor.~%"))
    ((not (member 'tape *pocket*))
     (format t "no duct tape. Щ. wrote: corridor.~%"))
    (t
     (setf *fixed* t)
     (setf *leak* 0)
     (setf *pocket* (remove 'tape *pocket*))
     (format t "taped it. the hum got duller. that's a compliment.~%"))))
```

Victory is not a separate screen. Ask yourself:

```
(defun status ()
  (format t "energy ~a, crack ~a, pocket ~a~%"
          *energy*
          (if *fixed* 'taped 'hissing)
          *pocket*))
```

> **What just happened**
>
> A round: `(reset-world)`, `(walk 'corridor)`, `(take 'tape)`, `(walk 'reactor)`, `(tape-leak)`, `(status)`. Energy leaked a little on the way. Further ticks don't eat. You can go look at Earth from the cupola, like a person whose shift suddenly got boring.

Losing: wander without duct tape until `*energy*` hits 0. Then `walk` refuses. The station doesn't explode in ASCII art. It just doesn't open the hatch. That's meaner.
The "adventure" text lives in `look` and in the note. You can add room descriptions — an alist of strings.

```
(defparameter *flavor*
  '((airlock  . "cold floor. someone else's gloves, not your size.")
    (corridor . "the bulb lies in Morse. it's just the ballast.")
    (galley   . "Щ's mug and the smell of what used to be coffee.")
    (reactor  . "warm. if it hisses — not poetry, a crack.")
    (cupola   . "earth is big. you are small. duct tape beats lyrics.")))

(defun look ()
  (format t "~a~%" (cdr (assoc *here* *flavor*)))
  (format t "compartment ~a | hatches: ~a~%" *here* (neighbors *here*))
  *here*)
```

Don't turn this into a novel. Two lines per room. A REPL game lives on moves, not paragraphs.

> **Quest S.L4** · Lisp
>
> Item `wrench` in the reactor. Command `whack`: if the wrench is in the pocket and you're in `galley` — you "fixed" the corridor bulb, set `*lamp*` to `'ok`. If there's no wrench — `'need-wrench`. Decide yourself whether the lamp affects the game. Even one line in `look`. No effect — a prop, that also counts, but write honestly in a comment that it's a prop.

15.5 Watch 5. Save the world, or sleep will eat the mash

RAM is the counter. You quit SBCL — the mash is wiped. A jar on disk — `with-open-file`.
We'll save not "a pretty format for humans," but what Lisp can read back: `print` / `read`.

```
(defun world-plist ()
  (list :energy *energy*
        :leak *leak*
        :here *here*
        :pocket *pocket*
        :at *at*
        :doors *doors*
        :fixed *fixed*))

(defun save-world (path)
  (with-open-file (out path :direction :output :if-exists :supersede)
```

```
    (print (world-plist) out))
  path)
```

> **Slow down**
>
> `print` writes so `read` understands. `format` writes so you understand. For a save you need `print`. For "on the floor: mug" — `format`. Don't mix up the stoves.

```
(defun apply-world (plist)
  (setf *energy* (getf plist :energy)
        *leak*   (getf plist :leak)
        *here*   (getf plist :here)
        *pocket* (getf plist :pocket)
        *at*     (getf plist :at)
        *doors*  (getf plist :doors)
        *fixed*  (getf plist :fixed))
  'loaded)

(defun load-world (path)
  (with-open-file (in path :direction :input)
    (apply-world (read in))))
```

A round:

```
(reset-world)
(walk 'corridor)
(take 'tape)
(save-world "module-save.lisp-data")
```

Quit SBCL. Open it again. Load the file with the definitions, then:

```
(load-world "module-save.lisp-data")
(status)
(look)
```

The pocket should remember the duct tape. If it doesn't — either you saved in the wrong `pwd`, or you loaded the definitions **after** the save and `reset-world` overwrote it. Order: game code first, then `load-world`.

> **Bad idea**
>
> `read` eats lisp-data. Don't feed it a file from the internet "it's just text." This is not a JSON parser on a diet. This is the language with its mouth open. A teaching save is yours, from the disk next to you.

Quest S.L5 · Lisp

Command `save` with no argument writes `module-save.lisp-data` in the current folder. `load` reads. If there's no file — `'no-save`, not a crash. Check: patched the crack, saved, `reset-world`, loaded — the crack is taped again. If not, you forgot `*fixed*` in the plist.

15.6 Watch 6. A map on a napkin and a little "smart" move

The graph is already there. You can ask: is there a path **longer** than one hatch.
A teaching breadth-first walk, without queues from an algorithms textbook: recursion with a pocket of "already been."

```
(defun reachable (from)
  (labels ((walk-rooms (room seen)
             (if (member room seen)
                 seen
                 (let ((seen2 (cons room seen)))
                   (dolist (n (neighbors-from from-table room) seen2)
                     (setf seen2 (walk-rooms n seen2)))
                   seen2)))
           (neighbors-from (table room)
             (cdr (assoc room table)))
           (from-table () *doors*))
    (walk-rooms from nil)))
```

Stop. This already wants simplifying, because the nesting lies. Simpler like this:

```
(defun reachable (from)
  (let ((seen nil))
    (labels ((visit (room)
               (unless (member room seen)
                 (push room seen)
                 (dolist (n (neighbors room))
                   (visit n)))))
      (visit from)
      seen)))
```

> **What just happened**
>
> `(reachable 'airlock)` should return all five compartments: the graph is connected. Cut the corridor↔galley hatch in your head: then the cupola becomes an island if you start from the airlock. Try a temporary `(setf *doors* …)` without `galley` on the corridor — and look at `reachable`. Then `(reset-world)`.

This is not a term paper on graphs. This is the answer to "why is the room on the chart, but `walk` won't let me in": because a path is a chain of edges, not neighborhood on a picture.

Quest S.L6 · Lisp

`steps-to` : from `*here*` to room `to` , return the number of hatches on **one** known path (doesn't have to be shortest, but not infinite). No path — `nil` . You can do it by hand for five rooms as a table, or by a walk. A table on five nodes is not shameful. Shameful is returning 1 for `airlock` → `cupola` .

15.7 Watch 7. A mini-loop "the game asks itself"

Tired of typing `(walk 'x)` — make a loop. Still REPL spirit: you aren't compiling an engine. You're reading a line.

```
(defun play ()
  (reset-world)
  (look)
  (loop
    (when (<= *energy* 0)
      (format t "energy gone. watch closed.~%")
      (return))
    (format t "> ")
    (finish-output)
    (let* ((line (string-downcase (read-line)))
           (parts (split-line line))
           (cmd (first parts))
           (arg (second parts)))
      (cond
        ((string= cmd "quit") (return))
        ((string= cmd "look") (look))
        ((string= cmd "status") (status))
        ((string= cmd "take") (take (intern (string-upcase arg))))
        ((string= cmd "walk") (walk (intern (string-upcase arg))))
        ((string= cmd "tape") (tape-leak))
        ((string= cmd "save") (save-world "module-save.lisp-data"))
        ((string= cmd "load") (load-world "module-save.lisp-data"))
        (t (format t "commands: look walk take tape status save load quit~%"))))))
```

Write `split-line` yourself: cut on a space. Teaching:

```
(defun split-line (s)
  (let ((p (position #\Space s)))
    (if p
        (list (subseq s 0 p) (subseq s (1+ p)))
        (list s))))
```

Two words are enough: `walk corridor` . A third word the station will swallow with the second. Live with it or write a real split later.

`intern` + `string-upcase` — a bridge from "human text" to a Lisp symbol. Without it `walk` compares a symbol with a string and never walks. A classic. Щ. probably tripped on this, hence the mug.

> **Bad idea**
>
> `(play)` will take the REPL into itself. Exit is `quit`. Don't Ctrl+C right away, give it a chance. If you still Ctrl+C — you're back at `*`, the world in memory may be stuck mid-way. `(reset-world)` cures it.

15.8 Watch 8. A coffee machine that lies

There's a machine in the galley. The button says "coffee." Inside — a function. Previous mechanic Щ. wired it to the station's `*energy*`. Every cup — 4 units. If energy is low, the machine prints steam and pours nothing.

```
(defun coffee ()
  (cond
    ((< *energy* 4)
     (format t "steam. the button lies. energy ~a~%" *energy*)
     'steam)
    (t
     (setf *energy* (- *energy* 4))
     (format t "sludge. not coffee. but it's hot. energy ~a~%" *energy*)
     'sludge)))
```

> **What just happened**
>
> This is not a new game. This is **the same world**. `coffee` changes the same global as `tick`. If after patching the crack you drink twenty cups, energy still runs out. Honest plot: duct tape is not an infinite reactor.

Add a `coffee` command with no arguments to `play`. Nothing new in `world-plist`: energy is already there. The save itself remembers how much you "drank."

> **Quest S.L7** · Lisp
>
> The machine pours only if you're in `galley`. Otherwise `'wrong-room`. If the `mug` is in the pocket — different text: "at least into Щ's mug, not on the floor." Without the mug — on the floor, energy still gets charged. The station is not a nanny.

The napkin map doesn't change after the machine. What changes is the meaning of the galley: now it isn't a set with a note, it's a room with a side effect. That's how graphs come alive. Not with a picture. With a verb.

15.9 Watch 9. Antenna and cargo — the graph grows without a picture

Five circles on the napkin. The station lies: there are two more hatches Щ. didn't finish drawing, because the pencil broke. We'll finish them with parentheses.

- `cargo` — the hold. Smells of dust and boxes that say "this side up" in three languages, all lying.

- `antenna` — the antenna bay. Cold. A small window. From here they blinked at Earth until the connector froze.

Hatches. Cargo from the airlock: makes sense, you don't haul freight through the galley. Antenna from the cupola: also makes sense, the cable already goes there. In a graph, logic is an edge, not "well it's nearby."

```
(defparameter *doors*
  '((airlock  . (corridor cargo))
    (corridor . (airlock galley reactor))
    (galley   . (corridor cupola))
    (reactor  . (corridor))
    (cupola   . (galley antenna))
    (cargo    . (airlock))
    (antenna  . (cupola))))
```

> **Slow down**
>
> An edge is written **twice** if the hatch is two-way. You added `(airlock . (corridor cargo))` and forgot `(cargo . (airlock))` — you can't leave the hold. That isn't a Lisp bug. That's you drawing a one-way door and being surprised. Check `(neighbors 'cargo)` right away, not after a plot arc.

A room with no line in `*flavor*` — `assoc` returns `nil`, `cdr` of `nil` in `look` gives `nil`, `format` prints `NIL`. Honest and ugly. Add a description in the same moment as the edge. `*at*` too: otherwise `stuff-here` will fall over or eat someone else's pocket.

```
(setf *flavor*
      (append *flavor*
              '((cargo   . "dust. a box that says this side up. they already tumbled
it.")
                (antenna . "cold. the connector is frosted. earth is far and in no
hurry."))))

(setf *at*
      (append *at*
              '((cargo   . (solder))
                (antenna . (frost-note)))))
```

`solder` — solder. Useful when frost on the antenna won't take duct tape. `frost-note` — a note from ЩЦ.: "don't blow a hair dryer. energy. patience."

`append` on a global doesn't change the old list in place — it **builds a new one**. That's why `setf`. If you write a bare `append` and don't assign, the world stays five rooms, and you'll yell at the REPL. A classic.

Update `reset-world` too. Otherwise after a reset the new compartments vanish, as if ЩЦ's pencil broke again. Reset is a contract with yourself: the **whole** world in one place.

In the REPL after fixing the doors:

```
(neighbors 'airlock)  ; (CORRIDOR CARGO)
(walk 'cargo)
(take 'solder)
(walk 'airlock)
(walk 'corridor)
; ... to the cupola, then:
(walk 'antenna)
```

If `walk` yells "no hatch," you're either in the wrong room, or the edge is one-way, or `*here*` isn't what you think. `(look)` before panic. The panel lies less than memory.

Quest S.L8 · Lisp

Put `cargo` and `antenna` into `*doors*`, `*at*`, `*flavor*`, and into `reset-world`. Command `read-frost-note`: if `frost-note` is in the pocket — print Щ's warning about the hair dryer. Not in the pocket — honestly say where it lies (antenna). Check: airlock to cargo and back; cupola to antenna; reactor to antenna in one `walk` — no.

15.10 Watch 10. Two bugs that look like "Lisp broke"

It didn't break. You forgot a quote. Or you built nested dolls with no last doll. We'll take both apart, slowly, in this same world.

15.10.1 Bug 1. Forgot the apostrophe

You want to go to the reactor. You type like a human:

```
(walk reactor)
```

SBCL yells something like: variable `REACTOR` is unbound. Or it tries to **call** function `reactor`. Depends what's on that name. Either way you didn't walk.

> **Slow down**
>
> Lisp first **evaluates** arguments, then calls the function. `walk` wants a room symbol. `'reactor` — "put the tag, don't evaluate." Without the apostrophe `reactor` is "find this variable's value / call this name." No variable. No function. Fire.

Compare in the REPL, not in your head:

```
'reactor     ; REACTOR  — a tag
reactor      ; error, if no defun / defparameter
'corridor
(walk 'corridor)   ; a move
(walk corridor)    ; error again, unless you (defparameter corridor 'corridor) for a
joke. don't.
```

The same pit with `take`:

```
(take tape)     ; looks for variable TAPE
(take 'tape)    ; takes the item
```

And with the world's `defparameter`:

```
(defparameter *here* airlock)   ; evaluates airlock — no such thing
(defparameter *here* 'airlock)  ; standing in the airlock
```

> **What just happened**
>
> The apostrophe is not decoration. It's an order "don't cook, put it raw." The string `"reactor"` is another type. `eq` of a symbol and a string won't work. `walk` compares symbols. That's why in `play` we did `intern` and `string-upcase`: a bridge from human text to a tag.

If the error says `undefined variable` / `unbound` and the form has no apostrophe on a room or item name — put `'` first, don't rewrite the engine.

15.10.2 Bug 2. Infinite recursion

The station graph is **cyclic**: airlock ↔ corridor. A walk with no "already been" pocket goes in a circle until the stack runs out.
Here's the "obvious" function "every room I can reach":

```
(defun reachable-broken (from)
  (cons from
        (mapcan (lambda (n) (reachable-broken n))
                (neighbors from))))
```

Call `(reachable-broken 'airlock)`. Don't wait an hour. In a moment: `Control stack exhausted` / stack overflow. This is not "the computer is weak." This is nested dolls: airlock → corridor → airlock → corridor → …

> **Slow down**
>
> Recursion with no stop on **this same** node is infinity. A stop of "no neighbors" doesn't save here: there are always neighbors, they're just yours. You need a `seen` list. Watch 6 already wrote it. Break it on purpose, look at the bedsheet, **then** put `seen` back. Otherwise the word "stack" stays a sound.

Second classic — a function calls itself **with the same argument**:

```
(defun look ()
  (look)
  (format t "you're in ~a~%" *here*))
```

The first line of the body is `look` again. The queue never reaches `format`. The stack grows. Same error, even dumber cause: a typo "call myself instead of `look-at-doors`," or `walk` at the end calls `look`, and `look` out of kindness calls `walk` into the same room.

```
; ring walk → look → walk
(defun walk (to)
  (setf *here* to)
  (look))
(defun look ()
  (walk *here*))   ; "refresh," ha
```

Cure: draw arrows of **who calls whom**. If it's a ring with no change of argument — you didn't walk the graph, you sat on a fan.

How to read the error: from the top (or the bottom, whichever helps) look for **your** function, repeated many times in a row. `LOOK LOOK LOOK LOOK` — there it is. Not `SBCL internals` for two screens.

The `reachable` fix is the one already there: `seen`, `unless (member room seen)`, `push`, then neighbors. Check: `(reachable 'airlock)` returns seven compartments, doesn't die. Cut the hatch `cupola → antenna` and start from the airlock — `antenna` isn't in the list. That's a graph, not a picture.

Quest S.L9 · Lisp

Reproduce both bugs **on purpose**: (1) `(walk reactor)` without the apostrophe — write the exact error text in the log; (2) `reachable-broken` — see the stack overflow, don't leave it spinning. Then fix (2) via `seen`. Compare the list length with seven rooms. If fewer rooms — you forgot an edge one way.

15.11 Watch 11. Frost on the antenna — repair as a fight

Not orcs. Frost. Thickness in made-up centimeters. You hit with heat, frost hits the station's energy. Who runs out first.

The world already knows `*energy*`. Add an opponent:

```
(defparameter *frost* 12)
```

12 — thickness. 0 — the connector is dry, the antenna can `ping`. Each `blast` is a heat flash: −3 to frost, −5 to energy. Only in `antenna`. If energy is under 5 — the flash won't flash, print `'too-dark`.

```
(defun blast ()
  (cond
    ((not (eq *here* 'antenna))
     (format t "the frost isn't here. drag your feet to antenna.~%")
     'wrong-room)
    ((< *energy* 5)
     (format t "dark. nothing to heat with. energy ~a~%" *energy*)
     'too-dark)
    ((<= *frost* 0)
     (format t "already dry. don't be a hero, the connector hates it.~%")
     'already-clear)
    (t
     (setf *energy* (clamp-energy (- *energy* 5)))
```

```
    (setf *frost* (max 0 (- *frost* 3)))
    (format t "hisses. frost ~a, energy ~a~%" *frost* *energy*)
    (if (<= *frost* 0) 'clear 'frosted))))
```

> **What just happened**
>
> `max 0` — so frost doesn't go negative and become "even drier." No meaning, the plot spoils. `clamp-energy` is already on energy. Two guards, two resources. That's the whole "combat system": two numbers and a rule for when a move is legal.

Solder from the hold is a strong move. Costs 8 energy, strips 8 frost. No solder in the pocket — `'need-solder`. With solder — removes the item (once, this is not an infinite spool).

```
(defun solder-joint ()
  (cond
    ((not (eq *here* 'antenna)) 'wrong-room)
    ((not (member 'solder *pocket*)) 'need-solder)
    ((< *energy* 8) 'too-dark)
    (t
     (setf *energy* (clamp-energy (- *energy* 8)))
     (setf *frost* (max 0 (- *frost* 8)))
     (setf *pocket* (remove 'solder *pocket*))
     (format t "solder sat. frost ~a~%" *frost*)
     (if (<= *frost* 0) 'clear 'frosted))))
```

Frost is not a gentleman. If you aren't repairing, and just `walk` in the antenna (or stay — we'll make `wait`):

```
(defun wait ()
  (unless *fixed*
    (tick))
  (when (and (eq *here* 'antenna) (> *frost* 0))
    (setf *frost* (+ *frost* 1))
    (format t "frost grew. now ~a~%" *frost*))
  (status))
```

Losing: energy 0, frost still > 0. Winning: frost 0, you're in the antenna, you can blink:

```
(defun ping-earth ()
  (cond
    ((not (eq *here* 'antenna)) 'wrong-room)
    ((> *frost* 0) 'iced)
    ((< *energy* 15) 'too-dark)
    (t
     (setf *energy* (clamp-energy (- *energy* 15)))
     (format t "ping gone. earth doesn't have to answer. you did what you could.~%")
     'ping)))
```

A rehearsal round, not a novel:

```
(reset-world)
(walk 'cargo)
(take 'solder)
; get to antenna via corridor galley cupola, energy drips
(solder-joint)
(blast)      ; if not dry yet
(ping-earth)
```

Add `*frost*` to `reset-world` (12 again) and to `world-plist` / `apply-world`. Otherwise the save forgets the war with frost, and that's insulting after three flashes.
In `play` the commands: `blast`, `solder`, `wait`, `ping`. No arguments. A third word the station will still swallow. Live.

> **Bad idea**
> Don't add a "random crit" on the first watch. First the rule has to be repeatable: 12 frost, blast by 3, count on paper how many flashes. If paper and REPL disagree — the code is lying. Then `(random 3)`, if it itches.

Quest S.L10 · Lisp

`blast` / `solder-joint` / `ping-earth` as above. Win — `'ping` after a dry connector. Lose — `walk` or `blast` at energy 0. In `status` print frost. The save remembers `*frost*`. Check: without solder, clear it with flashes; with solder — fewer moves. Write in a comment how much energy each scenario cost.

15.12 Watch 12. What's in the save, syllable by syllable

The file `module-save.lisp-data` is not a novel and not JSON. It's what `print` wrote so `read` would chew it. Open it **with your eyes** after `(save-world "module-save.lisp-data")`.
Typical mash (your numbers will differ):

```
(:ENERGY 86 :LEAK 7 :HERE CORRIDOR :POCKET (TAPE) :AT ((AIRLOCK) (CORRIDOR) ...) :DOORS
((AIRLOCK CORRIDOR CARGO) ...) :FIXED NIL :FROST 12)
```

We go left to right, like a storekeeper.
A list. The outer parentheses — one form. `read` will eat it whole. Lose a closer — `read` will wait until end of file and get offended. Add an extra — the next form, `load-world` isn't waiting for it, the world will chew only the first, ignore the tail or fall over. Not "almost JSON." Parentheses count.
Keys with a colon. `:ENERGY` is a keyword. It's its own value, no quote needed. `getf` looks for exactly that. Write `ENERGY` without the colon — a different object, `getf` returns `nil`, energy becomes `nil`, `tick` explodes on subtraction. The colon is not decoration.
Numbers. `86` is a number. Not `"86"`. If you type quotes by hand, `*energy*` becomes a string, `(- *energy* 7)` will say type error. Hand-editing a save is fine for study, don't change the type.

Symbols. `CORRIDOR`, `TAPE`, `NIL`. Not strings. `NIL` is both false and the empty list, we already ate that. `:FIXED NIL` means "the crack still hisses," not "the string nil."
Nested parentheses. Pocket `(TAPE)` — a list of one symbol. Empty pocket — `NIL` or `()`. `*at*` and `*doors*` are alists, that is lists of pairs. If in a hand edit you break the pair `(AIRLOCK . (CORRIDOR CARGO))` into `(AIRLOCK CORRIDOR CARGO)` without the dot — `cdr` will return the wrong thing, `neighbors` will go mad. The dot in an alist is "here's the pair's tail." Not a period at the end of a sentence.

> **Slow down**
> `print` writes for `read`. `format` writes for you. A save is `print`. If "to make it pretty" you stick in commas like JSON, `read` will choke. A comma in Lisp is other syntax (not your friend here). Spaces are fine. Commas — no.

A useful hand sabotage:

```
(reset-world)
(save-world "module-save.lisp-data")
```

Open the file in an editor. Change `:ENERGY 100` to `:ENERGY 3`. Save. In the REPL:

```
(load-world "module-save.lisp-data")
(status)
```

Energy should be 3. If it's 100 — you loaded the wrong path, or after `load-world` you called `reset-world`. Order: game code → `load-world` → **no** reset.
Second sabotage: delete one closing parenthesis. `load-world` should fall over. Read the error. Put the parenthesis back. This is not "the file is mystically broken." The form isn't closed.
Third: write `:ENERGY "lots"`. The load may pass, the world becomes poisonous. The first `tick` will show the truth. There's no validation in `apply-world` — this is a teaching save, not a bank. That's why `read` from **your** disk is ok, from a disk on the internet — no. We already yelled, we're yelling again.

> **Quest S.L11** · Lisp
> Save the world, by hand set `:FROST 1` and `:HERE ANTENNA`. Load. One `blast` should make frost 0 (if energy is enough). Second file: break a parenthesis, catch the `read` error, fix it. In the watch log — one sentence: how a `print`-save differs from `energy=80` in the Java dashboard.

15.13 Why this was here

You made: state, a graph, items, a side effect, a save, a tiny command language, two new compartments, two bugs taken apart, a war with frost. That's the whole profession, only without a salary and without Jira.

In the main course the same ideas will arrive as `TaskStore`, HTTP, and a table. Here they smell of airlock metal. You can go back to watch 1 and add a second crack. You can not go back. The log doesn't grade you.

If you got hooked and you're writing a text station for a third night — set an alarm for Java. Station MODULE on a résumé doesn't feed you. Parentheses feed the brain. The brain then feeds Java. Barski might have nodded at that food chain. We still didn't steal his text.

> **Sunday sabotage**
>
> Draw seven circles and the hatches. Walk a token. Then walk in the REPL. If they disagree — either the paper is lying, or `*doors*`. Usually `*doors*`.

Chapter

16 Macros and CLOS: the station learns new words

This is not a month from the schedule. This is a Sunday hole for people who don't have enough parentheses: first macros — code that writes code, then CLOS — objects the Lisp way, not "like Java, only in parentheses."
If task-manager is down — bring task-manager up. If Hibernate has been red for three days — read `Caused by`, not this chapter. You come here when the main course is green, or when Sunday itches and Java already said "enough."

> **Station law**
> A macro is not an "advanced defun." A function computes values. A macro gets **forms** and returns another form, which Lisp then computes. Mix them up — you get either magic that isn't there, or mash you're ashamed to read in a week.

CLOS is the Common Lisp Object System. Not "classes like in Java." There, methods live **inside** the class. Here methods live outside: a generic function, and classes only say **which** method to pick. On the station that's handy: one `status` for the reactor, the antenna, and the kettle, without a ten-floor hierarchy.
Files: `lisp-experiments/macros-clos/`. Commit. Even a crooked macro.

16.1 Why a macro, if there's a function

A function sees values. A macro sees code **before** it was computed.

```
(+ 1 2)
```

`+` gets `1` and `2`. It doesn't care whether you wrote `(+ 1 2)` or `(+ x y)`, where `x` is already 1.
But `if` cannot be an ordinary function in the naive sense: if both arguments were computed ahead of time, the "else" branch would always run. In Common Lisp `if` is a **special operator**. Macros are a way to make such things for yourself, without asking the language committee.
The most honest teaching example is "make your own `when`." `when` already exists. We'll write our own to see the guts, then immediately forget it in favor of the built-in. Like taking apart a sensor that already works.
First — that code in Lisp is **already data**.

```
'(+ 1 2)
; (+ 1 2)

(first '(+ 1 2))
```

```
; +

(second '(+ 1 2))
; 1
```

You can build the list `(+ 1 2)` by hand:

```
(list '+ 1 2)
; (+ 1 2)
```

And **run** it:

```
(eval (list '+ 1 2))
; 3
```

`eval` in ordinary code smells of burnt duct tape. A macro is exactly so you **don't** call `eval` by hand: you return a form, and the compiler/interpreter inserts it where the macro was called.

> **Slow down**
>
> Function: numbers and list-values in, a value out.
> Macro: pieces of code in (not yet computed), a piece of code out.
> Then Lisp computes that piece **at the call site**. As if you added the parentheses yourself, only the station added them for you.

16.1.1 Backquote: a template with holes

Writing `(list 'if test (list 'progn ...))` by hand hurts. There's a template: the backquote ` and the comma `, `. lisp (let ((n 3)) `(+ 1 ,n))
; (+ 1 3)`

Everything
under ` is like quote, *except* what follows a comma: that's substitute the value. ,@
Comma-at
 – splice the *elements* of a list, not the list as a whole: ` lisp (let ((body '(a b c)))
`(progn ,@body))
; (PROGN A B C)`

Without `,@` it would be `(PROGN (A B C))` — one argument that is itself a list. The dog unpacks. On the station: duct tape **inside** the crate vs a crate of duct tape in another crate.

> **What just happened**
>
> An ordinary apostrophe `'x` — never compute. A backquote — compute only the holes. A comma is a hole. If you forget the comma, the template keeps the symbol `n`, not the number 3. Then `+` gets offended at a symbol. Read what the macro **returned** before you blame SBCL.

16.1.2 First macro: your own `when`

Built-in `when`: if the condition, do the body (may be several forms). Otherwise `nil`.

```
(defmacro module-when (test &body body)
  `(if ,test
       (progn ,@body)
       nil))
```

`&body` is the same as `&rest`, only polite for macros: "here's a body." Editors thank you with indent. The check is not "call it and believe," it's **expand**:

```
(macroexpand-1 '(module-when (> *energy* 10)
                  (format t "still alive~%")
                  (tick)))
```

Something like this should come out:

```
(IF (> *ENERGY* 10)
    (PROGN (FORMAT T "still alive~%") (TICK))
    NIL)
```

`macroexpand-1` — one layer. `macroexpand` — until macros run out. For study always `-1` first: otherwise you'll drown in the guts of `format`.

> **Slow down**
>
> 1. Type `defmacro` as above.
> 2. Call `macroexpand-1` on a form with two lines of body.
> 3. Make sure both lines are inside `progn`.
> 4. Only then call `module-when` live.
>
> If you go "live" first — you're testing luck, not a macro.

Why `progn`? `if` has one form per branch. The macro body is several. Glue them into one — `progn`. A function won't do that: by the time the function is called, the body would already have been computed.

> **Bad idea**
>
> Don't write a macro if a function is enough. A macro is justified when you need to **not compute** some argument, or introduce a new name (`let` inside), or so the call looks like new grammar. "So it's cool" is a bad reason. Cool is reading your `defun` in a week.

16.1.3 A macro that guards the reactor

You want to write:

```
(with-energy 15
  (blast)
  (ping-earth))
```

Meaning: if energy is under 15 — don't run the body, return `'too-low`. If there's enough — run it, charge the energy **once** at the start. Awkward as a function: the body would compute before entry again. A macro:

```
(defmacro with-energy (cost &body body)
  (let ((cost-name (gensym "COST")))
    `(let ((,cost-name ,cost))
       (if (< *energy* ,cost-name)
           'too-low
           (progn
             (decf *energy* ,cost-name)
             ,@body)))))
```

`gensym` — a fresh name that isn't in your code. Why: if we wrote `(let ((cost ,cost)) ...)` and a person called `(with-energy cost ...)`, where `cost` is their variable, you'd get mash: the name collided. That's **capture**. On a three-line teaching macro you might not meet it. On the fourth — you will. Habit: names the macro introduces itself go through `gensym`.

> **What just happened**
>
> `(gensym "COST")` will return something like `#:COST1234`. Ugly in the expand. But it won't collide with your `cost`. Ugly is better than quietly wrong.

Check:

```
(defparameter *energy* 10)
(macroexpand-1 '(with-energy 15 (print 'boom)))
(with-energy 15 (print 'boom))
; TOO-LOW
*energy*
; 10
```

Energy wasn't charged — the body didn't live. Now `*energy*` = 20, `with-energy` 15 again — the body lives, energy 5.

> **Quest M.L1** · Lisp
>
> `module-when` and `macroexpand-1` on a form with **two** expressions in the body. In a comment — what would happen if you forgot `progn`.

> **Quest M.L2** · Lisp
>
> `with-energy`: not enough energy — `'too-low` and `*energy*` doesn't change; enough — the body and the charge. Check both. The `cost` counter through `gensym`, not "eh my name is rare."

16.1.4 When a macro lies: computed twice

A classic:

```
(defmacro twice-wrong (x)
  `(+ ,x ,x))
```

A call `(twice-wrong (tick))` substitutes `(tick)` **twice**: two ticks, not one. If `tick` writes to `*energy*`, the station loses double. Expand — you see twins.
Right: first one name in `let`, then the name twice.

```
(defmacro twice-right (x)
  (let ((g (gensym "X")))
    `(let ((,g ,x))
       (+ ,g ,g))))
```

`(twice-right (tick))` → one `tick`, add the value to itself. Not two ticks.

> **Slow down**
> After every macro ask: **if the argument is a form with a side effect, how many times will it run?** If not once — you almost always need `let` + `gensym`. That isn't paranoia. That's a leak sensor.

16.1.5 Macros you've already eaten without knowing

`defun` is a macro (over something lower). `loop` is a macro, a whole language. `with-open-file` is a macro: open, do the body, close even if it fell over. `setf` is a macro. You've used them since week 1.
Java can't do this. It has annotations, codegen, lombok, processors — neighboring galaxies. That's why in Java you write more letters, and in Lisp you sometimes write a new word of the language. Don't drag macros into a Java-head as "I need this at work." At work you have Spring. A macro is so the brain knows grammar isn't a sacred cow.

> **SICP · open only if it itches**
> "Code as data" is the heart of SICP and Lisp. If it itches why `eval` and quote — go there, the pieces on quotation. Not instead of this chapter: first your own `module-when`, then Abelson.

16.2 CLOS: objects whose methods live outside

In Java: class `Task`, inside it methods `complete()`, `getTitle()`. The object carries both data and verbs.
In CLOS: the class carries **slots** (fields). Verbs live in **generic functions**. One function `status`, several methods: for the reactor, for the antenna, for "anything." When you call `(status x)`, Lisp looks at the class of `x` and picks a method.
This isn't "worse than Java" and isn't "better." It's a different center: the verb matters more than the object's passport. Handy on the station: you yell `status` all the time, and the gadgets are different.

16.2.1 A class — a blueprint of slots

```
(defclass module ()
  ((name :initarg :name
         :accessor module-name)
   (energy :initarg :energy
           :initform 0
           :accessor module-energy)))
```

`defclass` — declare a class. Name `module`. `()` — no parents (yet). Then slots:

- `:initarg :name` — at creation you can say `:name "reactor"`.
- `:accessor module-name` — Lisp will write a reader and a writer. Not a three-line getter.
- `:initform 0` — if you didn't say energy, it'll be 0.

Create:

```
(defparameter *reactor*
  (make-instance 'module :name "reactor" :energy 80))

(module-name *reactor*)
; "reactor"

(module-energy *reactor*)
; 80

(setf (module-energy *reactor*) 75)
```

`make-instance` — like `new`, only the word is more honest: an instance. `setf` with an accessor — like a setter, only without a separate `setEnergy`.

> **What just happened**
>
> `*reactor*` is now not a number and not an alist. It's an object. `describe` in the REPL will show the slots, if you forgot what's inside:
>
> ```
> (describe *reactor*)
> ```
>
> Useful when there are five slots and memory is already lying.

Without an accessor you can `(slot-value obj 'energy)` — crude, like opening a panel with a screwdriver. An accessor is a normal door. `slot-value` — when you're writing infrastructure. In this chapter's quests — accessor.

16.2.2 Generic function and method

```
(defgeneric module-status (m)
  (:documentation "A string for the panel. One verb, different gadgets."))

(defmethod module-status ((m module))
  (format nil "~a energy=~a"
```

```
          (module-name m)
          (module-energy m)))
```

`defgeneric` — "there will be a function `module-status` of one argument." You can skip it: the first `defmethod` sometimes starts the generic itself. Explicit is more honest, like a README.

`((m module))` — parameter `m`, and it is **specialized** on class `module`. Not a Java type in the compiler sense. Method choice at runtime: whichever class arrived, that method.

```
(module-status *reactor*)
; "reactor energy=80"
```

Now a child — an antenna, which also has a signal level:

```
(defclass antenna (module)
  ((signal :initarg :signal
           :initform 0
           :accessor antenna-signal)))

(defmethod module-status ((m antenna))
  (format nil "~a energy=~a signal=~a"
          (module-name m)
          (module-energy m)
          (antenna-signal m)))

(defparameter *dish*
  (make-instance 'antenna :name "dish" :energy 20 :signal 3))

(module-status *dish*)
; "dish energy=20 signal=3"

(module-status *reactor*)
; still the method for module
```

`antenna (module)` — an antenna **is** a module. It inherits name and energy slots. The method for `antenna` is **more specific**, so that's the one they take. For a bare `module` — the general one.

> **Slow down**
>
> In Java you'd write `Antenna extends Module` and `@Override status()`. In CLOS override lives not in the class but in a separate `defmethod`. You can leave the class alone and add methods in another file. That's weird if you came from Java. That's normal if you think in verbs: "one more way to show status."

16.2.3 Several arguments — not only "this object"

In Java a method belongs to one class. In CLOS you can specialize **several** arguments. A teaching gesture, not a dissertation:

```
(defgeneric dock (ship port))

(defmethod dock ((ship module) (port module))
  (format nil "~a -> ~a" (module-name ship) (module-name port)))
```

Then you'll add a method `(dock (ship shuttle) (port airlock))` — different text. Choice by a **pair** of classes. In Java that's visitor dances. Here — a second parameter in `defmethod`. Not required in the quest. Required to see once, so "the object model" doesn't look like a copy of Java.

16.2.4 `call-next-method`: don't copy the parent by hand

```
(defmethod module-status :around ((m module))
  (let ((s (call-next-method)))
    (if (<= (module-energy m) 0)
        (concatenate 'string s " ALARM")
        s)))
```

`:around` — a wrapper around the other methods. `call-next-method` — "do what they would have done anyway." Then we appended `ALARM` if energy is zero.

There's also `:before` and `:after` — before and after the primary. For study, `:around` or a simple method on the child is enough. Don't build a five-floor combinator on the first evening. The station holds on duct tape, not on the MOP.

16.2.5 Several parents at once. Java still can't

In Java a class has one `extends`. Then as many `implements` as you want — but those are **interfaces**: promises with (almost) no state. There are no two real dads with slots. You write `extends Module implements Powered, Radio` and then by hand sort out what goes where.

CLOS allows several parents **as classes**:

```
(defclass powered ()
  ((watts :initarg :watts
          :initform 0
          :accessor watts)))

(defclass radio ()
  ((freq :initarg :freq
         :initform 0
         :accessor radio-freq)))

(defclass comm-antenna (module powered radio)
  ())
```

`comm-antenna` is a module, and power, and radio. Slots `energy`, `watts`, `freq` all present. Not "pretend to be an interface." The ancestors are real.

```
(defparameter *comm*
  (make-instance 'comm-antenna
```

```
                :name "comm"
                :energy 40
                :watts 12
                :freq 437))

(list (module-name *comm*) (watts *comm*) (radio-freq *comm*))
; ("comm" 12 437)

(typep *comm* 'module)   ; T
(typep *comm* 'powered)  ; T
(typep *comm* 'radio)    ; T
```

The `module-status` method for `module` already works: an antenna **is** a module. You can add a method specially for `comm-antenna` or for `powered` — the generic will pick the most fitting.
Parent order matters. Lisp builds a **class precedence list** (CPL): who is more specific, whose slot `:initform` to take, whose method is closer. If two granddads share a slot name — there will be a rule, not a lottery. The algorithm is called C3, you don't have to memorize the name. You do have to: `(class-precedence-list (find-class 'comm-antenna))` in the REPL if it's suddenly unclear **whose** method got picked.

```
(mapcar #'class-name
        (class-precedence-list (find-class 'comm-antenna)))
; (COMM-ANTENNA MODULE POWERED RADIO STANDARD-OBJECT T)
```

`STANDARD-OBJECT` and `T` are ancestors of everyone. You didn't write them. If `POWERED` is before `RADIO` in the list — that's how you put them in `defclass`. Swap them — they swap. That isn't a bug. That's a lever.

> **Slow down**
>
> Java: one dad, many plaques "I can do this."\ CLOS: several dads, slots and methods from all of them.\ A diamond "both granddads → one grandchild" is forbidden for classes in Java precisely because it's scary. In CLOS it isn't forbidden: order in `defclass` and the CPL settle the fight. This isn't "better for Spring." This **exists**, and in many languages it still doesn't. Python can a little. Java — no. C# — no. Remember it as a sensor: "the object model" is not equal to "like in Java."

> **Bad idea**
>
> Don't build a six-floor pedigree "because you can" for study. Multiple inheritance is easy to turn into a hold where nobody remembers whose `status` this is. Two ancestors with different meanings (`powered` and `radio`) — ok. Two ancestors that both want to be "the main module" — a smell. Like microservices: possibility is not necessity.

Quest C.L5 · Lisp

A class with **two** parents, not counting `module` as one of them: for example `powered` + `radio`, or `module` + `powered`. Make an instance, check `typep` on each ancestor, read slots from both sides. In a comment one sentence: what Java doesn't have for this (not "an interface," but exactly **two parent classes with slots**).

Quest C.L6 · Lisp

`(class-precedence-list (find-class 'comm-antenna))` — or your class. Write down the order of names. Swap the parents in `defclass` and again. If the order in the CPL changed — there's the lever. If not — the parents are too simple, add a same-name slot conflict on purpose and see who won.

Bad idea

Don't learn "all of CLOS" like a multiplication table. `defclass`, `make-instance`, accessor, `defgeneric` / `defmethod`, one child — already more than you need so you don't go silent in a Lisp interview (rare), and so Java classes look like **one** way, not the only one.

16.2.6 A station of objects, not of globals

In the MODULE log the world lived in `*energy*` and `*here*`. That's fine for a three-screen game. When there are five entities, globals start lying: whose energy, which compartment.
A sketch, not the whole engine:

```
(defclass station ()
  ((modules :initarg :modules
            :initform nil
            :accessor station-modules)))

(defun station-report (st)
  (mapcar #'module-status (station-modules st)))

(defparameter *module*
  (make-instance 'station
                 :modules (list *reactor* *dish*)))

(station-report *module*)
```

A list of objects. `mapcar` over the verb `module-status`. Each object knows how to answer, because the method was chosen by class. In Java that's `for (Module m : list) m.status()` — the same gesture, a different address for the verb.

Quest C.L1 · Lisp

Class `module` with slots name and energy. `make-instance`, change energy through the accessor, `module-status` prints both. No need to put `describe` in a comment — live in the REPL.

Quest C.L2 · Lisp

Class `antenna` child of `module`, slot `signal`. Its own `module-status`. Two objects in a list, `mapcar #'module-status` — two different strings, one verb.

Quest C.L3 · Lisp

An `:around` method or a separate function `alarm-p`: energy 0 — the status has `ALARM`. A reactor at 0 and an antenna at 0 both yell. Not copy-paste of two `if`s in two methods, if you already know `call-next-method`. If you don't — two `if`s, but write in a comment how that's worse.

16.3 Macro + CLOS: don't blend them on night one

You can write a `define-module` macro that expands into `defclass` + a method. On the second evening. On the first — **don't**. First each tool separately. Otherwise a bug: unclear whether the macro, CLOS, or you is guilty.

An order that doesn't explode:

1. Functions and lists (you already).
2. A macro you expanded and **read** the expand.
3. One class, one method.
4. A child.
5. Only then "a macro writes defclass."

> **Station law**
>
> If you haven't run `macroexpand-1` — you don't have a macro, you have hope. If you haven't called `describe` on an instance — you don't have a class, you have a `defparameter` with an object you're afraid to open.

16.4 How this rhymes with Java, without a sermon

- A Java class ≈ a CLOS class + methods **nailed** to it. CLOS methods you can add in another file.
- A Java `interface` ≈ "a set of generics," but Java requires the class to **declare** that it can. CLOS doesn't require a declaration: a method for a class can be defined next door.
- Java has no macros. Repeating try/finally — you write by hand or live with lambdas. Lisp writes `with-open-file`.
- `record` in Java 21 — slots without ceremony. Closer to `defclass` with an accessor than to an old hundred-line JavaBean.
- Spring `@Service` is not a macro, though the **feeling** of "they extended the language with an annotation" is similar. Don't lie in an interview that an annotation = a macro. Neighboring galaxies.

They won't ask this in a Java interview. It will settle in the head: an object is not the only religion. When in Java you crawl into visitor or "duck" serialization, remember `defgeneric`. When you copy

the same try/catch for the tenth time — remember that in Lisp this would have been a macro, and in Java — a method with a lambda. And write the method with a lambda, don't wait for macros from Oracle.

16.5 Typical breakage

The macro doesn't expand, it's called as a function. You forgot `defmacro`, wrote `defun`. Then arguments get computed first. `when` becomes a trap. Look at `macro-function`:

```
(macro-function 'module-when)  ; a function or NIL
```

`NIL` — it isn't a macro.

`comma not inside backquote`. A comma in ordinary code. The backquote got lost.

Wrong slot. `:initarg :name`, and you create with `:title`. The slot is `nil`, because the initarg didn't match. Names are tags, not "come on you understood."

`there is no applicable method`. You called a generic on a number or on `nil`. There's a method for `module`, something else arrived. `(class-of x)` in the REPL.

The child doesn't see the slot. A typo in the parent's name. `(defclass antenna (modlue) ...)` — a new class with no ancestor `module`. Parentheses around the parent are required: `(module)`, not `module` alone in some layouts of the brain. Write it like the example.

Quest M.L3 · Lisp

Break `twice-wrong` via `(twice-wrong (tick))` (or a counter `incf`). Show in a comment: two side effects. Fix via `twice-right`. `macroexpand-1` of both.

Quest C.L4 · Lisp

Station: class `station`, a slot that is a list of modules, function `station-report`. At least two modules of different classes. Print the report. This is not the whole MODULE log — this is a panel of objects.

Hide it on GitHub

Folder `macros-clos/`: `when.lisp`, `energy-mac.lisp`, `modules.lisp`. README: how to load in SBCL (`(load ...)` in order). Paste one `macroexpand-1` into the README as proof you expanded, not only called.

Sunday sabotage

Write a macro `dountil-energy` (while energy is above N, do the body). Expand. Don't run without expand: easy to catch an infinite loop if the condition is wrong. `Ctrl+C` in SBCL is your spacesuit.

16.6 A program that doesn't ask for a restart

In Java the loop is: wrote → `javac` or Maven → built a jar → stopped the server → uploaded → started → you pray that `main` is the one. On a phone it's worse: APK, install, green arrow, the activity is born again. Hot swap in the IDE sometimes lies. "Rewrite a method on a live object" the language does not promise as a staff feature.

Common Lisp promises it as staff. The REPL is not a week-one toy. It's **the same** world the program is spinning in. `defun` and `defmethod` don't "compile the project." They replace a function **in an already live image**.

```
(defmethod module-status ((m module))
  (format nil "~a energy=~a"
          (module-name m)
          (module-energy m)))

(module-status *reactor*)
; "reactor energy=80"
```

Don't leave SBCL. Don't load the file "from scratch" if you don't want to. Just type a **different** method:

```
(defmethod module-status ((m module))
  (format nil "[~a] ~a%"
          (module-name m)
          (module-energy m)))

(module-status *reactor*)
; "[reactor] 80%"
```

The same object `*reactor*`. The same process. A new verb. No `mvnw`, no "wait, Gradle is downloading." The station didn't reboot. The sensor swapped firmware on the fly.

Same with an ordinary function: you redefined `tick` — the next call is already new, the globals are in place. Broke it — redefine again. The image remembers data. You change behavior.

> **What just happened**
>
> This is interactive programming, not "a console to poke pluses." You don't rebuild the world. You repair it while it's on. Emacs has lived like this for decades: you redefine a function — the editor is already different, the session is the same. Not an accident that this book was written inside Emacs. Recursion, yes. Again.

Almost nowhere else is like this anymore. Smalltalk could. Some Erlang/Elixir images hot-swap a module, but that's another religion. Python "rerun the notebook cell" is a cousin, plus rakes with objects already created. Java HotSwap in the debugger — sometimes, if the method signature didn't change, and so you don't relax. Lisp doesn't make this a debugger trick. This is a way to **write**.

> **Slow down**
>
> 1. Create `*reactor*`, call `module-status`. \

2. No `(quit)`. Type a new `defmethod`. \
3. `module-status` again on **the same** object. \
4. If you saw the old string — you loaded the file into another image or recreated the class so the objects became the "old" class. `(class-of *reactor*)` and the method once more.

Quest I.L1 · Lisp

A live object, two `defmethod`s in a row without leaving SBCL. In a comment: what stayed the same (data) and what changed (the verb). If it only changed after `make-instance` again — you did the wrong exercise, you restarted the station.

16.7 A hundred million miles and one REPL

Sometimes this story is told as "a satellite left orbit, they connected, they patched it." Closer to the truth — even stranger.

1999. NASA probe **Deep Space 1**, far from Earth (tens of millions of miles, round-trip delay — tens of minutes, not "a ping from the next room"). On board, the Remote Agent experiment: a piece of autonomy, written mostly in Common Lisp (Harlequin Lisp on VxWorks). Not a teaching REPL. A real one. On hardware worth over a hundred million dollars.

The software was supposed to run the probe for several days on its own. On the ground they ran it for months, a piece they even "proved" with a model checker. On the second day an expected event didn't happen. A race that had **never** shown up in tests: a deadlock. You can't flash firmware like a phone app: mail doesn't go to Mars, and the ceremony "compile — put in a jar — upload — restart `main`" here costs light and the risk of losing the machine.

There was a REPL on board. From Earth, through Deep Space Network antennas, they asked for a backtrace: who is waiting for whom. They found the hang. Not "they rebuilt the project." They manually poked the event the code was waiting for — and the agent went on. Ron Garret, who tells this, says it straight: without a "read — compute — print" loop on the ship, they wouldn't have patched this thing.

Station law

That's why in chapter one the black window wasn't a toy. The same gesture as `(+ 2 3)`: a live language next to live data. On DS1 the data was a station priced like a small town. Yours is `*reactor*`. Same ritual. Different scale.

Politically, Lisp at JPL almost died after this: the experiment was officially a success, and it mostly scared NASA's autonomy career. This is not a moral of "Lisp won." This is a moral of "a language where a program is not a sacred binary but a conversation sometimes saves hardware you can't reach with your hands." Station MODULE is made up. DS1 is not. Source of the tale: Garret's text **Lisping at JPL** and his talk on debugging from sixty million miles.

Don't lie in an interview that "I patched a satellite." Lie less: "in Lisp you can redefine a method without killing the process; that's how they once debugged a probe." That's enough for the person across the table to either smile or ask about `HotSwap`. Both outcomes are good.

16.8 Lisp and artificial intelligence, the original one

John McCarthy invented Lisp in 1958 not to feed Java juniors. To talk to a machine about **symbols**: not only numbers, but lists, rules, "if this — then that." For twenty or thirty years Lisp was the native language of artificial intelligence. MIT, Stanford, expert systems, Symbolics "Lisp machines" — separate computers, because ordinary ones were gasping then.

The idea was beautiful: intelligence as the manipulation of symbols. Code = data, a macro extends the language for the task, a REPL lets you sharpen a thought without rebuilding the world. On paper and in the lab it worked. In life they hit hardware. Little memory, a garbage collector on machines of that day — a luxury, symbolic search explodes combinatorially faster than you can buy another kilobyte. They promised "AI" by the end of the decade. The decade ended, the kilobyte wasn't enough.

Then winter came: the money left, Lisp machines went bankrupt, industry was overfed on promises. Not because parentheses are stupid. Because the task was bigger than a 1985 box.

About the box separately, because this isn't a metaphor. A Symbolics Lisp machine — a separate computer, so the garbage collector and lists wouldn't be ashamed of household hardware. Cost as much as a decent car. An ordinary office PC of the same year ate Lisp slowly and with resentment. When they promised "artificial intelligence by Monday," on Monday it turned out there wasn't enough memory for rules about the whole world, and if-then combinatorics grows faster than the bill. You could buy another Lisp machine. You couldn't buy another world in kilobytes. Winter arrived on electricity bills and grants, not on the aesthetics of parentheses.

Now "AI" is on posters again, only the box is different: video cards, tensors, statistics, not a rule-based expert system.

Hardware caught up. The approach isn't the same — and that's fine. Lisp from this doesn't have to become a job-post language again. It already did the main thing: showed that a program can be **plastic**. A macro, CLOS with several parents, a method you change while the object is alive, a probe you don't reboot for one race — branches of one tree. McCarthy planted the tree, AI labs watered it until the sockets gave up.

> **Slow down**
>
> If they ask "Lisp is dead, right?" — don't defend job posts. Say: the first AI grew on it, the hardware of the day ate it, and the ideas (REPL, code as data, a live image) crawled into other languages in pieces. Java took objects and a garbage collector, didn't take macros, didn't take several parents, didn't take hot-swapping a method. Pieces, not the tree. That's why forty minutes of parentheses in this book aren't nostalgia for a résumé. It's looking at the tree, not only at IKEA furniture.

Emacs, where emagent sits, is almost entirely Lisp. An editor you don't restart to add a command. You've already touched that, even if you thought you were "just writing a textbook." The station loves those loops.

16.9 Why Lisp then. A wrap-up, not a sermon

Put it on one panel before it scatters across chapters:

- **Code is data.** You can build a list, take it apart, feed it to a macro. The language isn't closed by a committee. You extend the grammar when a gesture repeats. In Java for that you have annotations, codegen, lombok — neighboring galaxies, plus a build ceremony.
- **A macro computes forms, not values.** That's why `when` and `with-open-file` exist. That's why `(twice-wrong (tick))` is dangerous. That's why `macroexpand-1` is a sensor, not scholarship.
- **CLOS: the verb on the outside.** A method isn't nailed to a class forever. Several parents with slots — yes, in 2026 Java still doesn't. Several arguments on a generic — yes. `call-next-method` — not copy-paste of dad.
- **Live image.** You redefined `defmethod` — the station is already different, the objects are the same. Interactive programming, not "a console for Hello." Almost nowhere else still spoon-feeds you this.
- **AI.** Lisp was the native language of artificial intelligence until the hardware gave up. Winter didn't come because of parentheses. Parentheses survive winters better than grants.
- **DS1.** A REPL on a probe. A race. A backtrace at light delay. A patch without "upload the APK." Not a fairy tale about a falling satellite — a talk from JPL.

Why does a Java junior need this? Not to write Spring in CLOS. To see the **edge** of a language: what's a choice in Java, and what's industry habit. When you copy try/finally for the tenth time — remember a macro. When one dad's `extends` isn't enough — remember that's a Java limit, not physics. When Gradle spends five minutes building to change a status string — remember `defmethod` in the same REPL. When they say in an interview "a real language is Java" — you can nod: it feeds you. And have in your pocket forty minutes that feed curiosity.

Land of Lisp shows this with games. Barski has his circus. We have a leaky station and someone else's probe. We didn't steal his text. The mood — that a language can be weird **and** useful for a head — we stole honestly.

> **Hide it on GitHub**
>
> In the README of folder `macros-clos/` three proofs, not slogans: `macroexpand-1` output; `typep` on an object with two parents; two different `module-status`es on the same instance without `(quit)`.

16.10 What not to take to work tomorrow

Don't rewrite Spring in CLOS. Don't macro every sneeze in teaching Lisp. Do:

- be able to read `macroexpand-1`;
- tell "don't compute the argument" from "compute and pass";
- create a class with slots and two methods of one verb;
- know that several parents in CLOS are normal, in Java they aren't;
- once redefine a method **without** killing the process;
- not lie that a Java class and a CLOS class are the same word;
- if AI comes up — don't mix 1958-with-parentheses and 2026-with-video-cards, but remember whose first workshop this was.

That's enough so Land of Lisp (if you read as far as Barski's macro chapters) doesn't look like outer space, and Java objects don't look like a religion. Station MODULE after this is still leaky. The leaks just got **types of gadget**, and the language got **new words**. The duct tape didn't go anywhere. The probe, just in case, also once held on more than duct tape. Also on a REPL.

16.11 If they ask at dinner why you need parentheses

A short answer, no lecture and no shame about Java.

Lisp is a language where a program isn't nailed to a file on disk. A macro extends the grammar. CLOS allows several parents, which Java still doesn't do with classes. You can change a method while the object is alive. AI once stood on that, until the kilobyte ran out. A probe once stood on that, until a backtrace arrived from Earth.

You don't need this to pass Spring. You need this so you don't take Spring for physics. Physics — a computer obeys exact instructions. Spring is one way to pack those instructions. Parentheses are another. Both honest. One feeds you. The second doesn't let you get bored. The book promised that from the title page.

If after this chapter you want to throw out Java — don't. The huskies will still wake you at seven, and the job post will still be Java. If after this chapter you want to throw out Lisp — don't either. The forty minutes didn't go anywhere. You just now know **where** they go: into the tree, not into one more annotation.

Chapter

17 Lab: more Java by hand

Spring does not enter this chapter. Anyone who entered — escort them out. Here: a console, files, text pretending to be data, and one letter into the network with no framework.
The lab doesn't replace months 1–2. It's for an evening when `TaskStore` is already green and you want to touch the stove with bare hands again. Folder: `java-basics/lab/`. Commands — like in the book: Mac or Ubuntu in WSL. `javac`, `java`, JDK 21.

> **Station law**
> Break every program yourself first. Then fix it from the error text. A lab where everything worked on the first copy-paste is a tour, not a lab.

17.1 Lab 1. Station dashboard, with errors like people have

We want a console:

```
MODULE dashboard. energy=80 oxygen=40
> status
energy 80
oxygen 40
> set energy 55
ok
> save
wrote station.txt
> quit
```

After a new launch — `load` brings the numbers up from disk. So far it sounds like lesson 3. Now we'll get there **through the pits**.

17.1.1 Pit 0. An empty main, so there's something to run

`StationDash.java`:

```java
public class StationDash {
    public static void main(String[] args) {
        System.out.println("MODULE dashboard");
    }
}
```

```
javac StationDash.java
java StationDash
```

If `javac` "is not recognized as a command" — wrong window or PATH. That's the computer chapter, the PATH compartment. Not this lab. First the stove.

17.1.2 Pit 1. Scanner and "it ate the line"

We add input. The typical first sin:

```
import java.util.Scanner;

public class StationDash {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        System.out.print("energy: ");
        int energy = in.nextInt();
        System.out.print("command: ");
        String cmd = in.nextLine();
        System.out.println("you said [" + cmd + "]");
        System.out.println("energy=" + energy);
    }
}
```

Run it. Type `80`, Enter. The cursor does **not** wait for a command. It prints `you said []`. Magic? No.

> **Slow down**
>
> `nextInt` ate the number and **left** the end of the line in stdin. `nextLine` saw that end and was delighted: an empty command. That isn't a Java bug "on my machine." That's two ways to eat a stream: by tokens and by lines. Mix them — get emptiness.

The fix, a teaching one, no religion:

```
int energy = Integer.parseInt(in.nextLine().trim());
```

Everything through strings. Then parse. Then Enter belongs to you, not to a ghost.
Second fix: after `nextInt` call an extra `nextLine()` to throw away the tail. It works. Easy to forget.
For the dashboard, take `nextLine` always.

> **Quest J.L1** · Java
>
> Reproduce the pit: `nextInt` + `nextLine`, see the empty brackets. Then rewrite to `parseInt(nextLine())`. In a comment above `parseInt`, one line: **why**. Not "that's how it's done." Why.

17.1.3 Pit 2. A loop that knows quit

```
int energy = 80;
int oxygen = 40;
Scanner in = new Scanner(System.in);

System.out.println("MODULE dashboard. energy=" + energy + " oxygen=" + oxygen);
```

```
while (true) {
    System.out.print("> ");
    String line = in.nextLine().trim();
    if (line.isEmpty()) {
        continue;
    }
    if (line.equals("quit")) {
        break;
    }
    if (line.equals("status")) {
        System.out.println("energy " + energy);
        System.out.println("oxygen " + oxygen);
        continue;
    }
    System.out.println("unknown: " + line);
}
```

An empty line isn't an error. A person hits Enter out of boredom. `continue`. `quit` is `break`, not `System.exit(0)` like a cannon at a seagull.

Compare strings with `equals`, not `==`. Otherwise in some runs it "works" (the string pool), in others it doesn't, and you'll write in chat "Java broke." Java didn't break. You compared bowl addresses.

17.1.4 Pit 3. set energy 55 — splitting the line

We want `set energy 55`. Three pieces.

```
String[] parts = line.split(" ");
```

Pit: two spaces in a row. `set  energy 55` will give an empty piece in the array. Pit: `set energy`. No number. Pit: `set energy lots`.

Write it honestly:

```
if (parts.length == 3 && parts[0].equals("set")) {
    String what = parts[1];
    int value;
    try {
        value = Integer.parseInt(parts[2]);
    } catch (NumberFormatException e) {
        System.out.println("not a number: " + parts[2]);
        continue;
    }
    if (value < 0 || value > 100) {
        System.out.println("range 0..100");
        continue;
    }
    if (what.equals("energy")) {
        energy = value;
        System.out.println("ok");
    } else if (what.equals("oxygen")) {
```

```
            oxygen = value;
            System.out.println("ok");
        } else {
            System.out.println("unknown sensor: " + what);
        }
        continue;
    }
```

> **What just happened**
>
> This isn't a Lisp REPL, but the gesture is the same: read a line, decide, answer, `>` again. The dashboard is a REPL for the poor. We're poor today.

Don't swallow `NumberFormatException` in silence. Print the piece that isn't a number. Future you will say thanks when you paste a non-breaking space from a messenger.

> **Quest J.L2** · Java
>
> A `set` command for `energy` and `oxygen`, bounds 0..100, bad numbers don't kill the process. Add a `temp` sensor (corridor temperature, let it be −20..40). Three checks by hand: ok, not a number, out of range.

17.1.5 Pit 4. A file. The disk says no

```
import java.nio.file.Files;
import java.nio.file.Path;
import java.io.IOException;
import java.util.List;
```

Save in two lines — human, not JSON yet:

```
energy=80
oxygen=40
```

```
static void save(int energy, int oxygen, Path path) throws IOException {
    String text = "energy=" + energy + "\n" + "oxygen=" + oxygen + "\n";
    Files.writeString(path, text);
}
```

`throws IOException` — honest. The disk is sometimes busy, the path is sometimes fantasy.
Load:

```
static int[] load(Path path) throws IOException {
    int energy = 80;
    int oxygen = 40;
    if (!Files.exists(path)) {
        return new int[] {energy, oxygen};
    }
    List<String> lines = Files.readAllLines(path);
```

```
    for (String raw : lines) {
        String s = raw.trim();
        if (s.startsWith("energy=")) {
            energy = Integer.parseInt(s.substring("energy=".length()));
        } else if (s.startsWith("oxygen=")) {
            oxygen = Integer.parseInt(s.substring("oxygen=".length()));
        }
    }
    return new int[] {energy, oxygen};
}
```

Pit: no file — **don't crash**. Defaults. Pit: the file is there, inside `energy=abc` — we'll die on `parseInt`. Catch it:

```
try {
    energy = Integer.parseInt(...);
} catch (NumberFormatException e) {
    System.err.println("bad line: " + s);
}
```

In `main`, commands `save` / `load`, path `Path.of("station.txt")`. That's the process's **current** folder. Not "next to the source in your head." `pwd` before `java StationDash`. If the file showed up "not there" — you were standing not there.

> **Bad idea**
>
> `C:\` and folder names with spaces in a study save don't participate. `station.txt` in the current one. Enough.

A typical miss: you saved, opened the file in an editor, you see everything, the program loads defaults. Cause: you launched from IDEA with the working directory somewhere else. Either print `Path.of("station.txt").toAbsolutePath()` on save — and stop guessing.

> **Quest J.L3** · Java
>
> `save` / `load` for three sensors (with temperature, if you did it). No file — live defaults, the message `no station.txt, using defaults`. A broken number — a line on stderr, the other sensors live. Not "the whole load in one catch Exception."

17.1.6 Pit 5. Cut main while it isn't embarrassing yet

When `main` is longer than the screen, pull out:

- state fields — a small class `Station` with `energy`, `oxygen`, methods `statusText()`, `set(String, int)`;
- the file — `StationFile.save(Station, Path)`;
- the command loop — `main` or `run()`.

This isn't Spring. This is "so you don't get lost." The same instinct as service and repository, only without annotations and without a résumé.

Break `set` on purpose: forget the upper bound. Type 10000. Watch the dashboard lie. Put the bound back. That's lab work. The report is a git diff, not an essay.

17.2 Lab 2. JSON by hand and a mention of Jackson

The server in the main course will spit JSON. Before Spring it's useful to **build a string yourself** once and **get scared** once.
Given: energy 80, oxygen 40. We want:

```
{"energy":80,"oxygen":40}
```

```
static String toJson(int energy, int oxygen) {
    return "{\"energy\":" + energy + ",\"oxygen\":" + oxygen + "}";
}
```

In Java a quote inside a string is `\"`. It won't be pretty. But you can see: JSON is text.
Pit: a compartment name.

```
static String roomJson(String room, int energy) {
    return "{\"room\":\"" + room + "\",\"energy\":" + energy + "}";
}
```

Enter room `corridor`. Fine. Enter room `cor"idor` with a quote. JSON breaks: the string ends early, the eater on the other side chokes. Enter a backslash. Same.

> **Slow down**
>
> You can do JSON by hand while the data is numbers and words with no quotes. The moment a string comes in from the street — you write escaping or you take a library. Otherwise it isn't JSON, it's a Halloween costume.

A mini-parser for **our own** format, no pretensions. Only a flat object with two ints that **we** just wrote:

```
static int[] fromJsonEnergyOxygen(String json) {
    int e = grabInt(json, "energy");
    int o = grabInt(json, "oxygen");
    return new int[] {e, o};
}

static int grabInt(String json, String key) {
    String needle = "\"" + key + "\":";
    int i = json.indexOf(needle);
    if (i < 0) {
        throw new IllegalArgumentException("no key " + key);
    }
    int start = i + needle.length();
    int end = start;
    while (end < json.length() && Character.isDigit(json.charAt(end))) {
        end++;
    }
```

```
    return Integer.parseInt(json.substring(start, end));
}
```

This is a teaching knife. It will die on a space after `:`, on a negative number, on nesting. **Let it die**. You'll see the knife's edge.

> **Bad idea**
>
> Don't drag this `grabInt` into task-manager. There it's Jackson / Gson / whatever Spring gives. Here it's a knife, so you understand **why** they exist.

Jackson is a library: object ↔ JSON, escaping, lists, dates. In Maven that's a dependency. In the lab it's enough to **know the name** and the reason: quotes, unicode, nesting, don't invent. When you get to Spring, `@RestController` often hides Jackson under a blanket entirely. The blanket doesn't cancel that under it there's text.
Try by hand in the dashboard a `json` command — it prints `toJson`. A `json` command after `load` — the same numbers, a different costume. That isn't a database. That's an envelope.

Quest J.L4 · Java

`toJson` for three sensors. Check with your eyes: paste the output into https://jsonlint.com or any validator. Then break it: add a quote to the **name** of a made-up room field with no escape — the validator should be offended. Write one sentence in the README about why Jackson exists.

17.3 Lab 3. HTTP without Spring: HttpClient and someone else's kitchen

The network is letters. You can send a letter **from** Java, not only from a browser.
The package `java.net.http` is in JDK 11+ itself. Not Maven. Not Spring. The stove already can.
The service httpbin.org (or `https://httpbin.org/get`) answers GET with JSON: which URL arrived, which headers, where they knocked from. A teaching echo wall. If httpbin is down (happens) — any simple GET to `https://example.com`: that's HTML, not JSON, but a letter is still a letter.

```
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.io.IOException;

public class StationPing {
    public static void main(String[] args) throws IOException, InterruptedException {
        HttpClient client = HttpClient.newHttpClient();
        HttpRequest req = HttpRequest.newBuilder()
                .uri(URI.create("https://httpbin.org/get"))
                .GET()
                .build();
        HttpResponse<String> res = client.send(req,
```

```
HttpResponse.BodyHandlers.ofString());
        System.out.println("status " + res.statusCode());
        System.out.println(res.body());
    }
}
```

> **Slow down**
>
> `send` is synchronous: we stand at the door, wait for the envelope. The Java thread is busy for that time. For a dashboard, fine. For a thousand letters a second — already a different watch. `InterruptedException` is "they poked us while we waited." `IOException` is envelopes, DNS, Wi-Fi dropping. Don't catch them with an empty catch. Print `e.getMessage()`.

Pit: no network. There will be an exception, not JSON. That's correct behavior. Catch it, print "network didn't pick up," don't crash with no text.
Pit: a typo in the URL. `httpbin.org/get` with a Cyrillic "e" — a masterpiece. Copy Latin.
Pit: http vs https. httpbin often wants https. `301` / a redirect. HttpClient can follow a redirect, but you have to look at the status. If 301 and an empty body — you didn't "not get JSON," you got a pointer "look over there."
A tiny parse, no full JSON parser: find `"url"` in the body with your eyes. Then `indexOf`. Then understand that by hand is embarrassing, and stop. The lab is about a **letter**, not a second Jackson.
Add a `ping` command to the dashboard: hits httpbin, prints only `statusCode`. If it isn't 200 — print the code anyway. Someone else's kitchen's 503 is not your bug. 404 — wrong door. You won't get 0: a status always arrived **if** `send` didn't throw. It threw — there was no network.

> **Mac, Windows, WSL**
>
> **Mac / WSL.** Usually just works.
> **Corporate proxy.** `send` will die on a timeout. Don't treat that with ten workarounds the first night. Do the lab on a home network.
> **Offline.** A normal outcome is a caught IOException. It counts if the message is human.

Quest J.L5 · Java

A `StationPing` class or a `ping` command in the dashboard. Print the status and the **first 200 characters** of the body, not a novel. Second request: `https://httpbin.org/status/404` — see 404 and don't call it "it broke on my machine." Third: a bad host `https://no-such-module-xxxx.example` — catch the exception, print the type (class name) and the message.

17.4 Lab 4. Glue it: the dashboard can save and send a letter

Not required. If you have energy left: a `report` command builds JSON by hand and… doesn't POST it anywhere, if you don't want to register for other people's APIs. Print the JSON. Next to it, a `ping` command. Two worlds: disk and network. Neither is magic.

If you want POST (not required to close the log):

```
HttpRequest req = HttpRequest.newBuilder()
        .uri(URI.create("https://httpbin.org/post"))
        .header("Content-Type", "application/json")
        .POST(HttpRequest.BodyPublishers.ofString(toJson(energy, oxygen)))
        .build();
```

httpbin will return your JSON inside its JSON. A mirror. You'll see **what** flew, including the quotes. If it flew crooked — `toJson` is guilty, not "HTTP."

> **Bad idea**
>
> Don't POST your dashboard at random other people's URLs "to check." httpbin is for that. Someone else's prod is not. Station MODULE is already on duct tape; let's skip incidents.

17.5 Assembling the dashboard whole, without heroics

Below is a skeleton you can type **after** the pits, not instead of them. If you copy it right away — the pits won't happen, the fingers will get nothing.

`Station.java`:

```
public class Station {
    int energy = 80;
    int oxygen = 40;
    int temp = 21;

    String statusText() {
        return "energy " + energy + "\n"
             + "oxygen " + oxygen + "\n"
             + "temp " + temp;
    }

    String set(String what, int value) {
        if (what.equals("energy") || what.equals("oxygen")) {
            if (value < 0 || value > 100) {
                return "range 0..100";
            }
        } else if (what.equals("temp")) {
            if (value < -20 || value > 40) {
                return "range -20..40";
            }
        } else {
            return "unknown sensor: " + what;
        }
```

```
        if (what.equals("energy")) energy = value;
        else if (what.equals("oxygen")) oxygen = value;
        else temp = value;
        return "ok";
    }

    String toJson() {
        return "{\"energy\":" + energy
             + ",\"oxygen\":" + oxygen
             + ",\"temp\":" + temp + "}";
    }
}
```

`StationFile.java` — `save`/`load` as in pit 4, only three keys. Field names match the file: less fantasy, fewer bugs.

`StationDash.java` — a `while (true)` loop, `split`, `quit` / `status` / `set` / `save` / `load` / `json` / `ping`.

Compiling two or three files in one folder:

```
javac Station.java StationFile.java StationDash.java
java StationDash
```

IDEA does this with a button. The terminal does it explicitly. Explicit is more useful for one evening. Error `cannot find symbol Station` — you're compiling from the wrong folder or you forgot a file in the list. `pwd`. `ls *.java`.

> **Slow down**
>
> Class `Station` is state on the counter. `StationFile` is the pantry. `HttpClient` in `ping` is letters. If in a month someone says "controller, service, repository," you've already smelled it. Only without annotations and without a paycheck.

17.5.1 One more pit: IDEA's working folder

Run → Edit Configurations → Working directory. If that's the project root and you thought "next to the java file," `station.txt` will be born at the root. Git will see an extra file. You won't see it in `ls` of the study folder. Both are right. The paths are different.

Print the absolute path on the first `save`. Once. Then either live with it, or fix the configuration. Don't fix this by copying the file by hand "well there it is."

17.5.2 One more pit: line separators

`Files.writeString` on Windows may write `\r\n`. `readAllLines` usually chews both. If you parse bytes yourself — don't parse bytes yourself. If you opened the file in old Notepad and it's one line — `\n` and Notepad are guilty, not the reactor.

17.5.3 HttpClient a bit more carefully

A timeout, so you don't hang forever:

```
HttpRequest req = HttpRequest.newBuilder()
        .uri(URI.create("https://httpbin.org/get"))
        .timeout(java.time.Duration.ofSeconds(10))
        .GET()
        .build();
```

A `User-Agent` header is sometimes asked for by polite servers. httpbin doesn't fuss. Someone else's prod fusses. For the lab you don't need to pretend to be a browser.
The body can be huge. `substring(0, Math.min(200, body.length()))` — the first 200. Without `Math.min` you'll catch `StringIndexOutOfBounds`, and that'll be funny exactly once.

Quest J.L6 · Java

Pull state into `Station`, the file into `StationFile`, leave the loop in `StationDash`. Three files compile from the terminal. README: three run commands and a five-line sample dialogue. Without a README the lab isn't closed — like in week 4, only shorter.

17.6 What should stay in the fingers

- Input: don't mix `nextInt` and `nextLine`.
- Commands: split the line, check the array length.
- File: no file — a default; a broken line — a message, not a silent crash.
- JSON: text; quotes are dangerous; a library exists not out of snobbery.
- HTTP: a client, a request, a status, a body, an exception on a drop.

That's the same galley as in the computer chapter: cook, counter, pantry, letters. Only now you're writing the recipes too.

> **Sunday sabotage**
>
> Launch the dashboard from the terminal, not from the arrow. Save. Find `station.txt` via `ls` in **the** folder where `pwd` is. If the file isn't in Finder "next to the source" — congratulate yourself: you just felt a process.

The lab is closed when: (1) the `Scanner` pit is reproduced with your own hands; (2) `station.txt` is found via `pwd`, not via prayer; (3) JSON was checked in a validator at least once; (4) HTTP returned a non-200 at least once and you didn't panic. Spring can wait. It isn't going anywhere, unfortunately.